Noise, uncertainty, and feature selection in molecular property prediction



Adelaide Punt

St Catherine's College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

Trinity Term 2026

In memory of my grandfather, Frederick Rebarber.

Statement of Contribution

Chapter 3 is adapted from a manuscript by Punt, A., Hanser, T., Werner, S., and Morris, G.M., *NoiseInject: A Benchmarking Framework to Study Noise Robustness in QSAR Models*, which will be released as a preprint and submitted to the *Journal of Cheminformatics*. Most of the figures, tables, and text in that chapter are identical or similar to the manuscript. I implemented the NoiseInject framework, conducted all experiments, and drafted the manuscript. T.H., S.W., and G.M. supervised the project, contributed to the experimental design, and revised the manuscript. NoiseInject is available at `https://github.com/adpunt/noise_inject`.

Chapters 5 to 7 involve the adaptive hybrid representation and its downstream applications in molecular similarity measures and methods intended for label noise detection and mitigation. We intend to submit this work as two manuscripts with G.M., S.W., and T.H. as co-authors. One will be on the adaptive hybrid representation and its direct use in similarity computation, as described in Chapters 5 and 6. The other will be on detecting and mitigating label noise. While the content of this is outlined in Chapter 7, not all of the experiments were run at their intended capacities. This will be completed in the final manuscript. I developed the adaptive hybrid search and general concept, conducted all the experiments, and wrote these chapters. The code was implemented with Claude (Anthropic) within that design, as described in the Statement on the Use of Generative AI. G.M., S.W., and T.H. supervised the work. The code for the adaptive hybrid search is available at `https://github.com/adpunt/KIRBy`.

Acknowledgments

Most importantly, I would like to thank my supervisors, Prof. Garrett M. Morris at the University of Oxford, and Dr. Thierry Hanser and Stéphane Werner at Lhasa Limited, for their guidance and support throughout my doctoral research.

I would like to extend my deepest gratitude to Garrett, who has supported me throughout the course of my DPhil. I am extremely grateful for his patience and for the encouragement he extends towards me and all of my occasionally insane ideas. I have grown so much as a researcher thanks to his guidance and I am very lucky to have been his student.

I would also like to extend an extra thanks to Stéphane and Thierry for continuing to support me after leaving Lhasa and stepping down from their formal roles as my supervisors. Thierry in particular showed up to regular meetings throughout the entirety of my DPhil and provided invaluable feedback on my thesis.

I am grateful to Lhasa Limited for supporting this project, and to Lhasa Limited and the UK EPSRC Centre for Doctoral Training in Sustainable Approaches to Biomedical Science: Responsible and Reproducible Research (EP/S024093/1) for their generous financial support. I thank everyone at Lhasa Limited, and Adrian Fowkes, my current supervisor at Lhasa.

I would like to thank Prof. Charlotte M. Deane and the members of the Oxford Protein Informatics Group, past and present, and the Department of Statistics. I would also like to thank my former undergraduate professors, Prof. Elizabeth Orwin and Prof. Ran Libeskind-Hadas.

I thank the Oxford Protein Informatics Group for pointing me to the antibody and protein datasets used in Chapters 5 to 7.

I would also like to thank my parents, Beth Rebarber and Douglas Punt, for their unending love and encouragement. My grandmother, Annette Rebarber, gave me a love of mathematics from a young age. And to my cat, Rhea, for putting up with me.

Contents

Statement of Contribution ii

Acknowledgments iii

Abstract viii

Statement on the Use of Generative AI ix

List of Figures x

List of Tables xii

List of Abbreviations xvi

1 Introduction 1

1.1 Outline of the thesis and contributions 4

1.2 Collaborations 7

2 QSAR modelling and label noise 8

2.1 Molecular representations in QSAR 8

2.2 Models for molecular property prediction 12

2.2.1 Tree ensembles 14

2.2.2 Support vector machines 14

2.2.3 Gaussian processes 15

2.2.4 Bayesian neural networks 15

2.3 Molecular similarity and activity cliffs 16

2.4 Noise in molecular data 18

2.4.1 Artificial noise injection 20

2.4.2 Learning with noisy labels 22

2.5 Implementation of the noise conditions 25

2.5.1 The NoiseInject package 25

2.6 Evaluation methodology 27
2.7 Conclusions 28

3 Noise robustness across molecular representations and model architectures 29
3.1 Introduction 29
3.2 Methods 30
3.2.1 Datasets 30
3.2.2 Representations 31
3.2.3 Models 32
3.2.4 Noise conditions and levels 34
3.2.5 QM9 data pipeline 35
3.2.6 Uncertainty estimation 35
3.2.7 Evaluation 36
3.3 Results 39
3.3.1 Variance decomposition 39
3.3.2 Predictive performance and noise robustness 42
3.3.3 Noise conditions on the HOMO–LUMO gap 45
3.3.4 Robustness on the three assay datasets 47
3.3.5 Probabilistic and deterministic counterparts 55
3.3.6 Uncertainty under increasing label noise 57
3.4 Conclusions 61

4 Limitations of the similar property principle 66
4.1 Introduction 66
4.1.1 Distance metrics for continuous molecular descriptors 68
4.2 Results 69
4.2.1 Flaws in standard representations 69
4.2.2 Flaws in similarity and other pairwise computations 74
4.2.3 Consequences for similarity-based methods 77
4.3 Conclusions 84

5 Building adaptive hybrid representations 86
5.1 Introduction 86
5.1.1 Combining representations 87
5.1.2 The adaptive hybrid representation 89
5.1.3 Blueprints and their chemical interpretation 90

5.1.4 Applications beyond small molecules . . . 91
5.2 Methods . . . 92
5.2.1 From single representations to a hybrid . . . 93
5.2.2 The search space . . . 96
5.2.3 Handling small and specialized feature sets . . . 101
5.2.4 Pruning the search space . . . 102
5.2.5 Search grid per dataset . . . 103
5.2.6 Evaluation protocol . . . 103
5.2.7 Datasets and splitting . . . 105
5.2.8 Representations by domain . . . 108
5.2.9 Models and metrics . . . 115
5.2.10 Comparators, controls, and ablations . . . 116
5.2.11 Transfer, collaboration studies, and separate sites . . . 120
5.3 Results . . . 122
5.3.1 Performance of the hybrid . . . 122
5.3.2 Sources of the gain . . . 132
5.3.3 Blueprints on specific datasets . . . 139
5.3.4 Extension studies . . . 142
5.3.5 Features chosen across separate sites . . . 143
5.4 Conclusions . . . 144

6 Similarity in an adaptive hybrid representation 146
6.1 Introduction . . . 146
6.2 Methods . . . 147
6.2.1 Feature spaces compared . . . 147
6.2.2 Similarity in a hybrid . . . 148
6.2.3 Nearest-neighbor settings . . . 149
6.2.4 Experiments . . . 150
6.3 Results . . . 154
6.3.1 Distance against label difference . . . 154
6.3.2 Prediction from nearest neighbors . . . 156
6.3.3 Activity cliffs . . . 162
6.3.4 Prediction error, retrieval, and clustering . . . 163
6.4 Conclusions . . . 165

7 Detecting and handling label noise **169**
7.1 Introduction . . . 169
7.2 Methods . . . 170
7.2.1 Noise conditions . . . 170
7.2.2 Noise detection . . . 172
7.2.3 Noise mitigation . . . 175
7.2.4 Noise-robust training . . . 180
7.2.5 Evaluation and controls . . . 182
7.3 Results . . . 183
7.3.1 The clean-label test: where noise is removed . . . 184
7.3.2 Changing a label against deleting a molecule . . . 185
7.3.3 Detection of the suspect labels . . . 196
7.3.4 Effect by noise type . . . 200
7.3.5 Effect by dataset . . . 208
7.3.6 Cleaning inside the feature search . . . 210
7.3.7 Changing the model instead of the labels . . . 212
7.3.8 Effect by model and representation . . . 217
7.4 Conclusions . . . 219

8 Conclusions and further thoughts **221**
8.1 Summary of findings . . . 221
8.2 Choosing a noise-handling method . . . 223
8.3 Limitations . . . 223
8.4 Future work . . . 225
8.5 Concluding remarks . . . 226

Summary of Research Contributions **228**

Bibliography **229**

Abstract

This thesis explores the impact of label noise in molecular modeling. Much of the data in this domain is experimental, and the labels inherit noise from measurement error and differences between laboratories. Most research on label noise assumes Gaussian noise and considers few molecular representations and models. We show that non-Gaussian noise matters, both for predictive modeling and for the techniques meant to detect and mitigate it.

First, we investigate the effects of seven types of label noise across 13 models and six representations. In quantitative structure–activity relationship (QSAR) models, robustness to label noise depended more on the model than the representation. Random forests were most consistently robust. Systematic noise, such as labels clipped at an assay limit, did the most damage. Estimated uncertainty rose with noise but, outside censoring, did not identify corrupted labels.

Published methods for learning with noisy labels did worst when they relied on a distance between molecules. Most features of a standard representation barely vary or do not relate to the task, yet all of them enter that distance.

We therefore developed the adaptive hybrid representation, which searches, for each dataset and model, for the best feature selection settings across several molecular representations. On every small-molecule dataset, the hybrid predicted better than every single representation, and its distances tracked activity more closely. On antibodies the gain depended on the dataset, and on reactions there was none.

Finally, we compare single representations and hybrids under noise-aware training and noise mitigation. Which methods worked depended on the task, the data, the model, and the noise. Many gains came from better training rather than from removing noise. On small molecules, the hybrids stayed ahead of the best single representation in almost every condition. Noise in a molecular dataset should thus be considered before modeling, not assumed Gaussian.

Statement on the Use of Generative AI

I declare that my use of generative AI in this thesis complies with the University of Oxford's policy on generative AI in research and summative assessment. It also complies with the guidance of the Mathematical, Physical and Life Sciences Division for postgraduate research students and of the Department of Statistics.

I developed the research questions, methods, experimental design, and results analysis independently, except where a citation says otherwise. I used generative AI to help structure my writing and within the review process; however, I did not have it generate direct text. I used generative AI to discuss research ideas and guide me through literature reviews; however, the contributions, specifically the adaptive hybrid search and the design of its search space, and the cliff-aware gap, a score for finding noisy labels, are my own.

In terms of the specific generative AI used throughout my DPhil, I have primarily used Claude (Anthropic) through the Claude web application and Claude Code, from February 2026 to October 2026. The models were Claude Opus 4.5, 4.6, 4.7, 4.8, 5, and 5.5, Claude Sonnet 5 and 5.5, and Claude Fable 5 and 5.1. Before that, I used ChatGPT (OpenAI), with GPT-5.1 and GPT-5.2, from November 2025 to January 2026.

I wrote the original Python and Rust data pipelines by hand. I then used AI to extend the methods. I also used it to write plotting scripts, which run on code from recorded experimental outputs. That code is publicly available and the results and plots are reproducible. Generative AI was not used to make any figures directly from a prompt, nor was it used to generate or change any result. I drew all schematic diagrams using Lucidchart.

List of Figures

2.1 Nodes of a deterministic and a Bayesian neural network. 16
2.2 Label distributions under the seven noise conditions. 21
2.3 Evaluation of a noise mitigation method. 24

3.1 Variance decomposition of robustness and predictive performance on QM9. 40
3.2 Variance shares of clean performance, noisy performance, and robustness on all four datasets. 42
3.3 Predictive performance against Gaussian label noise on QM9. 44
3.4 Clean predictive performance against robustness on QM9. 44
3.5 Robustness of each base model on each representation on QM9. . . . 45
3.6 Robustness under every noise condition on a subset of models on QM9. 47
3.7 Random forest under each noise condition on QM9. 49
3.8 Rank of each base model at each noise level on QM9. 50
3.9 Predictive performance against noise level on every dataset. 54
3.10 Robustness on the three assay datasets. 55
3.11 Robustness rank of each model on QM9 and the three assay datasets. 56
3.12 Change from replacing a model with its probabilistic counterpart. . . 57
3.13 Aleatoric and epistemic uncertainty against label noise on QM9. . . . 60
3.14 Aleatoric uncertainty against the added noise. 61

4.1 Shares of constant, near-constant, and varying features per representation. 70
4.2 Two hERG K_i stereoisomers with identical ECFP4 vectors. 73
4.3 Absolute pK_i difference against pair distance on hERG K_i. 78

5.1 Schematic overview of the hybrid search. 92
5.2 The three levels of splitting. 94
5.3 Held-out score against the number of features: small molecules and reactions. 135

5.4 Held-out score against the number of features: antibodies. 136
5.5 Features taken from each representation. 140
5.6 Features of the LogD hybrid picked on every fold. 141

7.1 How a noise method is run and scored. 171
7.2 Held-out score against the share flagged and the share clipped. 187
7.3 Did the cleaning work: molecules selected by a detector. 190
7.4 Did the cleaning work: molecules selected by a known flag. 191
7.5 Did the cleaning work: noise-robust training. 192
7.6 Precision of the deleted molecules against the score after cleaning. . . 201
7.7 Correcting clipped training labels. 202

List of Tables

2.1 Sources of label noise in bioactivity data. 20
2.2 The seven noise conditions and their parameters. 23
2.3 Terms for methods that handle noisy labels. 24

3.1 Variance decomposition by noise condition on QM9. 41
3.2 Robustness under censoring and under Gaussian noise on QM9. . . . 48
3.3 Change in robustness from Gaussian to grouped-shifted noise on QM9. 48
3.4 Robustness under the noise distributions on the three assay datasets. 51
3.5 Rise in mean predicted uncertainty under Gaussian noise. 58

4.1 Molecules that share a feature vector with another molecule. 74
4.2 All-pairs correlation, five-nearest-neighbor prediction, and activity cliffs per dataset. 78
4.3 Agreement between representations per dataset. 79
4.4 Methods of the exploratory study of noise-handling methods. 81
4.5 Recovery with neighbor-based sample weights by representation. . . . 83
4.6 Flaws of standard representations and similarity, and the requirement each sets. 85

5.1 Methods for combining or reducing representations. 89
5.2 Settings of the importance methods. 98
5.3 The five allocation methods searched. 100
5.4 Worked example of the four fixed-share allocation methods. 100
5.5 Values the hybrid search can take for each setting. 103
5.6 Search grid of each dataset. 104
5.7 Datasets of the hybrid experiments. 106
5.8 Short names of the datasets, used in the figures. 107
5.9 Representations used for small molecules. 110
5.10 Representations used for chemical reactions. 111
5.11 Representations used for antibodies. 111

5.12 Base and augmentation representations per dataset. 113
5.13 Experiment register for the hybrid experiments. 117
5.14 Held-out score of the best single representation and the hybrid, per dataset and model, mean over outer folds, in each dataset's own metric. 124
5.15 Hybrid against ensembles of one model per representation. 125
5.16 Search's hybrid against the top representation alone and an even split. 127
5.17 Hybrid on molecules from frequent and rare scaffolds. 129
5.18 The search's own score of its pick against the held-out score. 130
5.19 Search with one fold fewer, and stability of the selected features. . . . 131
5.20 Share of the variation in score from the downstream model and the budget. 137
5.21 Importance scores used again during training. 137
5.22 Hybrid on three activity-cliff targets. 142
5.23 Graph kernel on hERG K_i with and without added features. 143
5.24 Search settings found on LogD reused on hERG K_i. 143
5.25 Feature selection when each site holds one family of representations. . 144

6.1 Nearest-neighbor settings 150
6.2 Similarity experiments 151
6.3 Distance against label difference in single representations 155
6.4 Spearman correlation between the distance between two samples and the difference in their labels, per dataset, mean over outer folds. . . . 156
6.5 Distance against label difference in the predictive hybrids 157
6.6 Distance against label difference in each feature space 158
6.7 Prediction score and distance against label difference by number of features 159
6.8 Held-out score of predicting each molecule from its five nearest training molecules, per dataset, mean over outer folds, in each dataset's own metric. 160
6.9 Prediction from nearest neighbors with the setting picked again for each fold 161
6.10 Importance weighting in an SVM 162
6.11 Importance weighting on all representations joined 163
6.12 Importance weighting in prediction from five nearest neighbors 164
6.13 Prediction on activity-cliff molecules 165
6.14 Distance to the nearest training molecules against prediction error . . 166

6.15 Label gap to the nearest neighbors with the setting picked again . . . 167
6.16 Label coherence of k-means clusters 168

7.1 Damage kinds on a regression dataset. 172
7.2 Damage levels. 172
7.3 Detectors. 173
7.4 Noise mitigation methods. 176
7.5 The expanded LNL sweep. 181
7.6 How each group of methods is tested. 183
7.7 Actions and detectors under hERG label flips. 186
7.8 Deleting flagged molecules against random deletion. 186
7.9 Actions and detectors on LogD with outliers. 188
7.10 Down-weighting flags of known content. 188
7.11 Weighting flagged molecules. 189
7.12 Deleting flagged molecules on QM9 with no noise added. 193
7.13 Cleaning the labels as delivered against shuffled detector scores. . . . 193
7.14 Neighbor-mean smoothing on hERG K_i by fold. 195
7.15 Putting cliff molecules back on hERG K_i and QM9. 196
7.16 Detection by the Forest residual and the Cliff-aware gap. 197
7.17 Detection by distance family. 199
7.18 Scaling features by importance before finding neighbors. 200
7.19 LightGBM residual precision against break-even precision. 203
7.20 Censored EM on further outer folds. 204
7.21 Three ways to raise clipped labels. 204
7.22 Per-molecule lift against shuffled lifts. 205
7.23 Removing a shared offset per scaffold family. 206
7.24 Removing family offsets from the labels as delivered. 207
7.25 Moving every label toward a Huber-fitted prediction. 208
7.26 Neighbor-agreement weights on SMAC colloidal stability. 209
7.27 Repeat measurements on hERG IC_{50}. 210
7.28 Cleaning before the feature search. 211
7.29 Searching on clean against damaged labels. 213
7.30 Changing the model instead of the labels: damaged labels. 214
7.31 Changing the model instead of the labels: labels as delivered. 215
7.32 The expanded LNL sweep, by domain. 216
7.33 Two results of the expanded sweep. 216

7.34 The expanded sweep on QM9 with no noise added. 217
7.35 Neighbors found in the single representation or in the hybrid. 218

List of Abbreviations

AC Activity cliff

AC-SINS Affinity-capture self-interaction nanoparticle spectroscopy

ANOVA Analysis of variance

AUROC Area under the receiver operating characteristic curve

BLOSUM Blocks substitution matrix

BNN Bayesian neural network

BNN-α, BNN-β Bayesian versions of NN-α and NN-β

CDDD Continuous and data-driven descriptors

CDR Complementarity-determining region

CGR Condensed graph of reaction

CHO Chinese hamster ovary

CL_{int} Intrinsic clearance

CV Cross-validation

DFT Density functional theory

DNN Deep neural network

DRFP Differential reaction fingerprint

EC_{50} Half-maximal effective concentration

ECE Expected calibration error

ECFC4 Extended-connectivity fingerprint with counts, diameter 4

ECFP Extended-connectivity fingerprint

ECFP4 Extended-connectivity fingerprint with radius 2, diameter 4

ECFP6 Extended-connectivity fingerprint with radius 3, diameter 6

EGFR Epidermal growth factor receptor

EM Expectation maximization

F1 F1 score (harmonic mean of precision and recall)

FcRn Neonatal Fc receptor

GCN Graph convolutional network

GIN Graph isomorphism network

GNN Graph neural network

GP Gaussian process

hERG Human ether-à-go-go-related gene (a cardiac potassium channel)

HIC Hydrophobic interaction chromatography

HLM Human liver microsome

HOMO Highest occupied molecular orbital

IC_{50} Half-maximal inhibitory concentration

IMGT The international ImMunoGeneTics information system

KL Kullback–Leibler

kNN k-nearest neighbors

LGBM LightGBM (short form used in tables)

LightGBM Light Gradient Boosting Machine

LM Language model

LNL Learning with noisy labels (the problem of training on labels that are partly wrong, not a group of methods)

LUMO Lowest unoccupied molecular orbital

MACCS Molecular ACCess System

MAE Mean absolute error

MAP4 MinHashed atom-pair fingerprint

MCC Matthews correlation coefficient

MCS Maximum common substructure

MDCK Madin–Darby canine kidney

MDR1 Multidrug resistance protein 1 (P-glycoprotein)

MHG-GNN Molecular Hypergraph Grammar Graph Neural Network

ML Machine learning

MLP Multi-layer perceptron

mRMR Minimum redundancy, maximum relevance

NGBoost Natural gradient boosting

NLL Negative log-likelihood

NLP Natural language processing

NN Neural network

NN-α Neural network architecture α ([128, 64], dropout after each layer)

NN-β Neural network architecture β ([128, 128], dropout before output)

PCA Principal component analysis

PDV Physicochemical descriptor vector

PLS Partial least squares

QRF Quantile regression forest

QSAR Quantitative structure–activity relationship

QSPR Quantitative structure–property relationship

RBF Radial basis function

ReLU Rectified linear unit

RF Random forest

RLM Rat liver microsome

RMSE Root mean squared error

SALI Structure–activity landscape index

SD Standard deviation

SELFIES Self-referencing embedded strings

SHAP SHapley Additive exPlanations

SMAC Standup monolayer adsorption chromatography

SMARTS SMILES arbitrary target specification

SMILES Simplified molecular input line entry system

SNS Sort & Slice

SPP Similar property principle

SVM Support vector machine

USR Ultrafast shape recognition

USRCAT Ultrafast shape recognition with CREDO atom types

VAE Variational autoencoder

VBLL Variational Bayesian Last Layer

VBLL-α, VBLL-β VBLL versions of NN-α and NN-β

VEGF Vascular endothelial growth factor

WL Weisfeiler–Lehman

XGB XGBoost (short form used in tables)

XGBoost eXtreme Gradient Boosting

Chapter 1

Introduction

New avenues in the drug discovery pipeline have opened up thanks to advances in large dataset collection, analysis, and computation. Therapeutic targets are identified, potential drug candidates are screened and optimized, and the most promising of these go on to pre-clinical and clinical trials [1]. However, attrition is high, and roughly 90% of drugs that enter clinical trials are never approved, often due to poor efficacy or toxicity [2]. Computational methods can remove poor candidates early on in the drug discovery pipeline, long before clinical trials. This is done through virtual screening, which allows for a large-scale analysis of libraries of compounds [3]. One standard screening method is the prediction of bioactivities and pharmacological properties of such candidates. There is a long history of using Quantitative Structure–Activity Relationship (QSAR) models to make these predictions, to act as a filter for compounds with undesirable properties, and thus reduce the time and cost associated with experimental screening [4]. QSAR models aim to predict the relationship between a molecule's structural and chemical features as encoded by a particular molecular representation and a desired experimental molecular property [4] of interest. A QSAR model requires a molecular representation and some form of machine learning (ML) model. An ideal molecular representation will allow a model to learn only the relevant information about each molecule with respect to the target property. Representations are either 'static' handcrafted descriptors or circular fingerprints that encode structural and physicochemical features [5–8]; or learned representations, where the encoding is derived from a predictive model, often taken from the activations of the final layer of a neural network, or a latent space embedding [4, 9].

QSAR models are, of course, sensitive to the quality of the input data on which they are trained. Experimental noise can arise from measurement errors, variability under experimental conditions, varying provenance of the data, intrinsic biological, chemical

variability, and human error. Even the very measures of model performance may be flawed if test and validation sets are laden with noise. Kolmar and Grulke [10] showed that error in the test labels can make a model look worse than it is: a model trained on noisy data can predict the noise-free values more accurately than an evaluation on the noisy test set indicates. There is also a limit to how much noise the model can tolerate; Jorner et al. [11] argued that once their model's error approached the experimental error, more training data would not be expected to improve accuracy much. In the context of activity prediction, the inherent variability within experimental setups and assays, batch effects, measurement errors, and biological variability introduce additional layers of complexity [10, 12, 13]. All of this increases the likelihood of encountering noise in the data. Even the molecular representations in the training data themselves may be incorrect or misleading [14, 15], and chemical datasets contain too few repeat measurements of each compound to characterize the distribution the label is drawn from. Thus, models treat a single label as the population mean [10]. Here pIC_{50} and $\mathrm{p}K_i$ are the negative base-10 logarithms of the half-maximal inhibitory concentration (IC_{50}) and K_i in molar units, so a difference of one log unit is a tenfold difference in concentration. Repeat measurements of the same compound-target pair have a standard deviation (SD) of about 0.68 log units for pIC_{50} and 0.54 log units for $\mathrm{p}K_i$ [12, 13], which is about a 4.8-fold and a 3.5-fold difference in concentration. Restricting to measurements from a single assay lowers the mean disagreement from 0.50 to 0.27 log units [16], or from about a 3.2-fold to a 1.9-fold difference.

This thesis addresses the challenge of conducting molecular property prediction when the underlying experimental data, or "labels," are noisy. Most work on noise in QSAR modeling implicitly assumes Gaussian label noise and evaluates either representations or model architectures in isolation. Cortés-Ciriano et al. [17] examined 12 algorithms across multiple noise levels but used fixed representations, and studies that compare representations typically use clean data. No work has systematically compared the relative importance of representation versus architecture choice for label noise robustness, asked whether probabilistic methods justify their computational overhead under noise, or tested whether findings generalize across different types of injected noise. This thesis fills that gap. We will also discuss noise mitigation, using the results of our noise robustness study. We explore why many noise mitigation techniques underperform in cheminformatics data and methods to address this problem in the form of novel molecular representations.

The first half of this thesis therefore asks four questions. First, how does the choice of molecular representation and model architecture explain the variance in predictive

performance, and does that division change when you instead consider how much performance a QSAR model is able to retain under increasing label noise? Second, what does the choice of model provide under label noise? Within this question we compare the effects of deterministic and probabilistic modeling. Third, does the type of label noise matter, or just the quantity of it? We explore different techniques of generating and adding label noise to both clean data, as well as data with inherent experimental noise. Fourth, what is the impact of artificial label noise on a model's uncertainty estimation? Does the estimated uncertainty rise with increased label noise and can these uncertainty values point to which labels are corrupted? Together, these questions aim to identify what, if anything, makes a QSAR model robust to noise. To answer them we introduce NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty quantification, described in Section 2.5.1.

We ran a sweep of methods for learning with noisy labels (LNL) across clean labels with injected Gaussian noise. Some of the worst-performing methods within this sweep relied on molecular distance measures. We posited that this distance measure was at least partially to blame for the poor performance. On a given dataset, most features of a general-purpose representation either barely vary or do not relate to the property. Yet each one that varies adds a term to the distance, so with enough irrelevant features, the distance becomes irrelevant as well, as described in Chapter 4.

We propose an alternative: adaptive feature selection, where a fixed number of features are selected from several input feature sets, or molecular representations in the case of small molecules. We refer to the resulting fixed feature set as the hybrid representation. These hybrid representations work as ordinary inputs to prediction models as well. Chapter 5 explains how this search takes place and Chapter 6 tests whether the hybrids do in fact improve distance measures. We return back to noisy labels in Chapter 7, where for a given dataset, we compare the representation with the highest predictive performance against the top-performing hybrid across a wider range of methods designed for noisy labels. We extend these methodologies to antibody and chemical reaction prediction tasks.

The second half of this thesis asks four further questions. First, can a search find the best set of features for a given dataset and model, and does it carry over from small molecules to antibodies and reactions? Second, does distance in a hybrid follow the label more closely than distance in a single representation, and do methods built on that distance gain? Third, can noisy labels be found and corrected, and does a hybrid help more than the best single representation? Fourth, does feature importance help, both in building the hybrid and when reused during training?

1.1 Outline of the thesis and contributions

Chapter 2 introduces molecular representations, model families, molecular similarity and activity cliffs (ACs), and the sources of noise in molecular data. It defines the seven noise conditions used in Chapter 3 and the three groups of methods for noisy labels (noise detection, noise mitigation, and noise-robust training), and closes with artificial noise injection, and evaluation metrics.

Chapter 3 looks into what makes a QSAR model robust to label noise. The primary prediction target is the gap between the highest occupied and lowest unoccupied molecular orbitals (HOMO–LUMO gap) in QM9, whose computed labels carry negligible measurement noise. The same experiments were also run on three assay datasets: LogD, Caco-2, and human ether-à-go-go-related gene (hERG) K_i. We compare seven different types of label noise, referred to as noise conditions. We matched the quantity of noise to the training set label spread for all but one of the noise conditions. We measure noise robustness as the proportion of predictive performance retained as artificial label noise increases. We found that model family was the main factor in noise robustness, and choice of molecular representation matters more for predictive performance than noise robustness. Strong predictive performance on clean labels does not imply robustness. Light Gradient Boosting Machine (LightGBM) and eXtreme Gradient Boosting (XGBoost) predicted well on clean labels but lost the most with increasing label noise, while random forests (RFs) were the most consistent model across datasets, representations, and noise conditions. We replicated systematic noise by giving every scaffold family its own offset and by clipping labels at a made-up assay limit. Models struggled the most when training on this systematic noise. Label noise raised mean predicted uncertainty for the probabilistic models which estimate it; however, only the Bayesian neural networks (BNNs) with a variance head, the heteroscedastic Gaussian process (GP), and the heteroscedastic Variational Bayesian Last Layer (VBLL) networks correctly attributed the rise in uncertainty to the data rather than to the model itself, though not on every dataset and representation. To run these experiments, we built NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty in QSAR models. Earlier studies compared models under noise with fixed representations, or compared representations on clean labels [17]. To our knowledge, this chapter is the first to measure how much of the robustness to label noise the representation and the model each decide. It also extends earlier comparisons of noise distributions [18] to noise shared within a scaffold family and to labels clipped at an assay limit.

Chapter 4 takes a turn to look at another topic: molecular similarity. The natural next step to Chapter 3 is noise mitigation. However, a small sweep of twelve LNL methods, four that remove molecules and retrain and eight that change the training, showed disappointing results. None of the methods that incorporated the concept of molecular similarity showed consistently promising results. On a given dataset, standard representations carry many features that do not vary, and some map different molecules to the same vector. Similarity computed over these representations inherits both of these problems, and methods that rely on similarity suffer as a result. The chapter ends with what a representation would need to mitigate these problems, which Chapter 5 builds.

Chapter 5 presents the adaptive hybrid representation. The adaptive hybrid search customizes the settings of a feature selection process to a given dataset, model, and budget. It takes in several single representations and selects the most important features for that task by a fixed budget. This produces a tailor-made feature set, referred to as the hybrid representation. The actual output of the hybrid search is not the set of features, but rather the settings that define the feature selection method used to generate that hybrid representation. A newly published embedding can be added to the search as one more representation. For antibodies, we also built representations that read only the complementarity-determining regions (CDRs), each region, or the mutations from the parent antibody. Finally, we test the search on data held at separate sites.

We compare the hybrid representations in a variety of scenarios, like their performance compared against principal component analysis (PCA), recursive feature elimination, and ensembles trained on different single representations. We pay attention to the selected features and test methods across a variety of datasets and representations. The hybrid demonstrated higher predictive performance across all small-molecule datasets when compared to the single representations, with both of each dataset's downstream models. On antibodies the change in performance depended on the data, however, the larger datasets tended to benefit the most. On most antibody datasets, an ensemble of one model per representation did better. The reactions gained nothing from the hybrids. Ranking features by importance helped to build the hybrid, but reusing the scores to weight features during training added little. Earlier work joined the most important bits of several fingerprints by a fixed rule [19], and the group lasso keeps or drops whole groups of features [20]. To our knowledge, no earlier method searches, for each dataset and model, how features are scored, how the budget is split among representations, and how many features are kept. Each kept feature also

traces back to its single representation. Chapters 5 and 6 are being prepared as one paper.

Chapter 6 sits in contrast to Chapter 4, asking if the flaws identified with using insufficient representations in similarity and distance measures are improved by the use of hybrid representations. We build hybrids whose features are specifically optimized for distance, ranking each candidate by how well each molecule's five nearest neighbors predict its label. On every small-molecule dataset, distance in the hybrid tracked label differences more closely, and the nearest neighbors' predictions improved. Zheng and Tropsha [21] and Itskowitz and Tropsha [22] chose descriptors for a nearest-neighbor model of one endpoint. Here the features are chosen across several representations, and the resulting distance is tested on six methods that use it, among them ACs, Bayesian optimization, and clustering.

Chapter 7 returns to label noise, expanding on the noise detection, noise mitigation, and noise-robust training explored in Chapter 4. We adapt published methods to QSAR data, such as censored expectation maximization (EM) for labels clipped at an assay limit, a random intercept per scaffold family, and Gap over neighbor spread, a detector of noisy labels borrowed from the quality control of weather-station readings. We also introduce three methods we found no earlier use of. These are the Cliff-aware gap, a detector that allows for ACs; cleaning the labels before the hybrid search; and Cliff put-back, which returns AC molecules after a cleanup. The methods are tested on single representations and on hybrids, and most of them against clean QM9 labels, random deletion, or shuffled detector scores. These correction techniques work best when a known flag, such as an assay limit or a scaffold family, indicates which labels are wrong. For practical applications, this is possible if enough is known about the source of the data. On small molecules, hybrids stayed ahead of the best single representation in almost every condition, before and after treatment. QM9, whose labels are computed, shows that many gains on measured labels come from better training rather than from removing noise. The chapter is being prepared as a separate paper. Chapter 8 answers the research questions. It argues that the noise in a dataset should be modeled before a method is chosen. No single method fits every dataset, and thus simple methods, such as RFs or raising every clipped label by one shared amount, are often as good as, if not better than, complex ones. The chapter ends with the limitations and future work.

1.2 Collaborations

This work was supervised by Garrett Morris and co-supervised by Thierry Hanser and Stéphane Werner at Lhasa Limited, which runs a federated learning platform [23]. Lhasa also supplied the hERG blocking data used in Chapters 4 to 7; in Chapter 5, data held at separate sites, as in federated learning [23, 24], are simulated from two public datasets, Caco-2 and LogD.

Chapter 2

QSAR modelling and label noise

2.1 Molecular representations in QSAR

The success of ML hinges on the quality of input data. Although the quality can refer to noise, which the majority of our work is centered around, in this case we are talking about the type of input. Molecular representations provide different features with which QSAR models and noise detection techniques can be learned. However, a potential bottleneck on larger datasets should be noted; the more complex the data structure, the greater the computational cost. For smaller datasets, the number of parameters is limited by the size of the training set, to prevent overfitting.

One of the most common and intuitive molecular representations is the Simplified Molecular-Input Line-Entry System (SMILES) string, which consists of a sequence of ASCII characters generated by a fixed set of rules to specify a given chemical identity [25]. A common way to use SMILES strings as inputs to QSAR models is to tokenize and one-hot-encode them rather than to use the strings themselves [26]. The strings are tokenized into atomic branches and symbols and mapped to indices of a vocabulary, and finally converted into binary vectors using one-hot encodings [26]. This representation, referred to as one-hot-encoded SMILES, allows models to process molecular structures as bit-vector-based fingerprints or as sequences of vectors. Sequence-based models such as recurrent neural networks and transformers have proven to be particularly successful in capturing text semantics within a sequence. When applied to raw or one-hot-encoded SMILES strings, these models can effectively capture the structure of the molecule in its textual representation [26].

Another common representation is the molecular fingerprint, a vector containing information about the properties of the molecule in question. In a typical binary fingerprint, each bit accounts for the presence or absence of a given chemical feature [27], and

in the case of Morgan fingerprints, a given substructure. One common fingerprint is the Molecular ACCess System (MACCS) key, a substructure key-based fingerprint that encodes predefined atom symbols and properties, as well as bond types [28]. Another is the Extended Connectivity Fingerprint (ECFP), or Morgan fingerprint, which encodes local neighborhoods around each atom and bond connectivity in molecules [28]. These existing molecular representations are well established and have been quite successful when coupled with QSAR models [4, 29]. They are relatively simple, and as the bits can be mapped back to one or more substructures, they are human-interpretable, and furthermore easy-to-compute. However, hashed fingerprints are subject to information loss due to bit collisions, which happens when two different substructures are encoded at the same bit position [27]. They are also based on a fixed procedure and are not trainable or adaptable to a particular dataset. Alternative representations that capture different structural, chemical, or physical features are able to overcome these limitations.

Recent advances have led to the development of new molecular fingerprints. One such method is the Sort & Slice (SNS) technique, an alternative to hash-based folding for pooling ECFP substructures [7]. SNS begins by sorting all circular substructures in the training set according to their frequency of occurrence. It then "slices" away all but the N most frequent substructures, which are subsequently used to generate an N-dimensional, bit-collision-free binary fingerprint. This approach solves the ambiguity problem of hashed fingerprints, where multiple substructures might be encoded at the same bit position, making it very interpretable [7]. SNS fingerprints outperform ECFP of diameter four (ECFP4) with standard hash-based folding methods across various molecular property prediction tasks. These experiments consisted of different data splitting techniques, ML models, fingerprint diameters, fingerprint dimensions, and atomic invariants [7]. Furthermore, SNS outperformed advanced substructure selection schemes such as filtered fingerprints and mutual information maximization [7].

Neural fingerprints are a category of learned representations that overcome the fixed, hard-coded nature of traditional fingerprints. Unlike ECFP or MACCS keys, which rely on predefined hashing schemes or substructure catalogs, neural fingerprints train on molecular data to capture chemical information [30, 31]. These representations are the result of deep learning architectures that have learned dense, continuous embeddings which can be used for downstream tasks.

Mol2vec, which adapts the Word2vec algorithm from natural language processing (NLP) to molecular substructures [30], is a common neural fingerprint. Mol2vec treats molecules like "sentences" and circular substructures derived from Morgan fingerprints

like “words”. The model learns vector embeddings for each substructure through unsupervised training, such that chemically similar substructures are mapped to nearby points in the embedding space [30]. The embeddings of all present substructures are summed to obtain a full representation. This approach produces 300-dimensional dense vectors that avoid the sparsity and bit collisions inherent in hashed fingerprints [30]. Mol2vec has performed strongly on various QSAR benchmarks, matching or exceeding Morgan fingerprints on solubility and bioactivity prediction tasks [30].

Transformer-based architectures have also been applied to molecular representation learning, similarly stemming from NLP ideas. MolBERT applies a bidirectional transformer to learn molecular representations from SMILES strings through self-supervised pre-training [32]. The model has domain-relevant auxiliary adaptations, such as the prediction of calculated physicochemical properties alongside the masked token prediction, to improve the chemical fidelity of the learned embeddings [32]. MolBERT outperformed count-based ECFP4 (ECFC4), RDKit descriptors, and the continuous and data-driven descriptors (CDDD) neural representation on virtual screening and QSAR benchmarks [32]. ChemBERTa is another transformer-based approach which pre-trains masked language models on SMILES data from PubChem [31]. ChemBERTa uses a byte-pair-encoding tokenizer by default and compares it with a SMILES-specific tokenizer [31]. Attention visualization shows neurons selective for chemically relevant functional groups and aromatic rings, and others that track bracket closures in the SMILES string [31]. ChemBERTa-2, pre-trained on up to 77 million PubChem compounds, is competitive with existing architectures on MoleculeNet [33].

Neural fingerprints provide several advantages over traditional fingerprints. They produce dense continuous vectors that avoid sparseness and information loss from bit collisions [30]. They can also be pre-trained once on large unlabeled datasets and then fine-tuned for specific downstream tasks, opening the door to transfer learning [31, 32]. However, pre-training them is computationally expensive, and they are less directly interpretable than fingerprints, although attention visualization offers some insight [31].

Beyond fingerprints, physicochemical-descriptor vectors (PDVs) are a class of continuous-value molecular descriptors that encode structural and chemical properties of molecules [34]. Unlike binary fingerprints or graph-based representations, PDVs aggregate atom and bond-level quantities into fixed-length feature vectors that capture a wide variety of information about a molecule and its structure [35]. PDVs typically consist of hundreds of properties including constitutional descriptors such as molecular weight, atom and bond counts, and heteroatom ratios; topological indices encoding

molecular connectivity patterns; physicochemical properties such as logP (lipophilicity), molar refractivity, and topological polar surface area; and electronic descriptors derived from partial charges and polarizabilities [34, 36]. The `RDKit` cheminformatics toolkit provides a comprehensive suite of more than 200 molecular descriptors that cover these categories [37]. Many PDV descriptors, such as logP, molecular weight, and partial charges, have a direct physicochemical meaning, although many of the thousands available have no clear interpretation [36]. They are also less computationally expensive than some 3D methods or neural fingerprints, with many descriptors simply requiring a 2D molecular structure without conformer generation or extensive pre-training. This efficiency makes PDVs particularly appropriate for high-throughput applications. Descriptors based on 3D structure can also be used [35].

However, PDVs have limitations. The dimensionality of PDV feature spaces can lead to problems with multicollinearity [34]. This can be mitigated by feature selection or dimensionality reduction techniques [38, 39]. Additionally, PDVs encode molecular properties as aggregated global features, which may not capture fine-grained structural details such as specific substructure arrangements or local chemical environments, information important for particular endpoints.

One of the more intuitive representations is the molecular graph. These graphs can be categorized into 2D and 3D graphs. 2D graphs rely on adjacency matrices that encode topological atom- and bond-level features but leave out geometric properties such as bond angles or torsion, meaning they may fail to capture conformation. 3D graphs incorporate these spatial features, but are often limited by remaining in a single conformation [40]. A standard molecular graph is constructed such that each molecule is represented as $G = (V, E)$ with node features $\mathbf{x}_i$ and edge features $\mathbf{e}_{ij}$. In one common featurization, nodes encode atom type, chirality, hybridization, and aromaticity, and edges encode bond type, ring membership, direction, and stereochemistry [40].

There are also alternative molecular graph representations such as graphs with quantum 3D embeddings, weight alignment graphs, association graphs containing pairs of molecules, and geometric interaction graph neural networks that incorporate 3D structures and physical interactions [41–43]. Du et al. [44] developed hybrid 2D and 3D graphs, combining 3D data, guaranteeing precision and completeness by providing unabridged information, and 2D data, providing chemical intuitions for guidance, in a zipper-like arrangement. Voxel-based 3D representations, in which each atom occupies one grid cell, are too sparse for convolutional networks to perform well; Kuzminykh et al. [45] instead spread each atom over nearby voxels with a wave transform.

These graph representations do not highlight specific substructures or specific ar-

rangements of atoms and bonds within each molecule. The substructures contain functional groups that are related to a range of chemical properties and are potentially involved in downstream tasks related to the function of molecules. Bouritsas et al. [46] developed graph substructure networks, a topologically aware message passing scheme based on substructure encoding. The method involves encoding structural roles based on the appearance of specific substructures and using them in message passing updates. Since message-passing graph neural networks (GNNs) are at most as powerful as the Weisfeiler–Lehman (WL) test, they cannot isolate sufficient features from the graph structure [46]. Graph substructure networks, on the other hand, capture these substructure-related features. Encoders pretrained on unlabeled molecules, whether graph-based [9] or transformers on SMILES [47], can also produce embeddings that capture structural features.

2.2 Models for molecular property prediction

There is a wide variety of model architectures that can be used for QSAR modeling. Starting off with simpler and more explainable models, we have tree-based methods. Decision trees are non-parametric supervised learning algorithms that perform recursive binary partitioning of the feature space [48]. A decision tree builds a predictive model by learning a series of if-then rules that recursively split the data based on given feature values, creating a tree where internal nodes represent decision rules, branches represent the results of those decisions, and leaf nodes contain predicted values or class labels [48]. Each recursive split is chosen to maximize homogeneity within the resulting partitions. For classification, Gini impurity is commonly used and for regression, mean squared error reduction [48].

There are several properties that make decision trees a good foundation for QSAR modeling. First, they handle non-linear relationships between molecular descriptors and target properties without requiring explicit feature engineering or transformation [48]. They also do not make assumptions about the underlying distribution of the data, making them robust to violations of parametric assumptions that can limit linear regression or other classical methods [48].

Decision trees handle data in a “contributory” rather than “consensus” manner. In consensus methods such as linear regression or neural networks (NNs), all training points fit one shared set of parameters, so a training point can affect predictions far from it in the feature space. Tree-based methods make predictions based on the subset of training samples that fall into the same leaf node as the test point [48]. This

mechanism essentially quarantines outliers in their respective leaf nodes, limiting their impact on predictions for samples in other regions or chemical domains.

Individual decision trees tend to have high variance and often overfit, particularly at large depths [48]. Small modifications made to the training data can result in large differences in the structure of the tree, resulting in unstable predictions [49]. Ensemble methods are used to counteract this instability, aggregating predictions from many trees. RFs aggregate predictions from trees trained on bootstrap samples of data with random subsets of features at each split [50]. Bootstrapping ensures that each tree is trained on a slightly different subset of data, which effectively reduces the influence of any individual sample [50]. This is especially beneficial with noisy samples. The selection of random features at every node decorrelates the trees, which lowers the generalization error of the ensemble [50]. The aggregation of trees further prevents overfitting to noise by smoothing out prediction errors from individual noisy labels.

NNs, a very popular family of models, can also be used for QSAR. An NN is a system of interconnected nodes which learn by passing information through its layers. There are weights and biases attached to the nodes and various other parameters which the model optimizes to minimize the difference between the predicted output and the actual target value. During training, the network parameters are updated using backpropagation, in which gradients of the loss function with respect to each weight are propagated backward through the network [51]. The structure of these networks and the hyperparameters used depend on the type of NN. In QSAR, one of the most commonly used architectures is the multilayer perceptron (MLP), an NN containing fully-connected layers with non-linear activation functions which transforms molecular feature vectors (i.e., fingerprints, descriptors) into a property or activity prediction [52].

A typical MLP for molecular property prediction has a few fully connected hidden layers with rectified linear unit (ReLU) activations and dropout regularization to prevent overfitting. Training is performed using backpropagation with gradient descent optimizers such as Adam, to minimize a loss function (mean squared error for regression, cross-entropy for classification) on batches of training data.

However, NN performance on QSAR-related tasks has been very mixed. NNs have a long history in QSAR [53], and tuned deep networks have outperformed shallow methods such as RFs and support vector machines (SVMs) across diverse activity classes [54]. Their complexity and use of gradients make them prone to label memorization [55], which in the context of noise is problematic, as it means that models may memorize the noise themselves. Although models that are memorizing labels may initially appear to

have strong predictive performance, they cannot generalize to out-of-distribution samples properly and often provide inaccurate uncertainty estimates. Various approaches have been proposed to improve NN robustness to label noise, including noise-robust loss functions [56], confident learning to identify mislabeled examples [57], and regularization techniques such as mixup [58] or label smoothing [59].

All the models up to this point have assumed a vector-based input. Molecular graphs, on the other hand, require different types of predictive modeling. Two standard ML methods on molecular graphs include Graph Convolutional Networks (GCNs) and Graph Isomorphism Networks (GINs). GCNs apply the concept of convolution to graph-structured data. Instead of operating on grid-based data, such as images, GCNs aggregate information from neighbors of a node to update its representation [60]. In contrast, GINs address the limitations in GCNs' ability to differentiate between non-isomorphic graphs. Unlike GCNs, which normalize and weight the sum of neighboring node features, GINs use a simple sum followed by a trainable MLP, giving the model flexibility to learn complex transformations of node features. The GIN model's design is informed by the WL test for graph isomorphism, enabling it to distinguish between molecular graphs that are structurally similar but differ in subtle ways [26, 61]. GIN features have been competitive with or better than fingerprints and descriptors at classifying pairs of molecules as activity cliffs (ACs), described in Section 2.3. In ACs, small variations in molecular structure can lead to significant changes in chemical or biological activity [62]. Descriptor-based models still predicted the potency of cliff molecules better than deep learning methods [63].

2.2.1 Tree ensembles

Gradient boosting methods such as eXtreme Gradient Boosting (XGBoost) sequentially fit trees to the residuals of previous iterations, progressively reducing the prediction error [64–66]. We used a handful of gradient-boosted tree ensembles in this thesis including Light Gradient Boosting Machine (LightGBM) [67], and Natural Gradient Boosting (NGBoost) [68].

2.2.2 Support vector machines

An SVM fits a function that is as flat as possible while keeping training errors small [69, 70]. It uses the ϵ-insensitive loss. Under this loss, errors smaller than ϵ cost nothing and larger errors cost their size minus ϵ. A constant C trades flatness against tolerance of large errors [70, 71]. The features enter the model only through the kernel. With the

radial basis function (RBF) kernel, every feature therefore enters the Euclidean distance between two molecules. In Chapter 3 the continuous representations are standardized and the binary fingerprints are used as built. A wrong label has limited effect. An error inside the ϵ tube changes nothing, and outside it the loss grows only linearly. The weight of each molecule is also capped at C [70]. Under simulated experimental error, the RBF-kernel SVM degraded smoothly and linearly [17]. Its main limitation is computational cost. Training operates on a kernel matrix that grows with the square of the number of training molecules [72].

2.2.3 Gaussian processes

We employed GP models using the `Gauche` framework [73], which is optimized for cheminformatics tasks. Standard kernels for GPs are not optimized for the chemical landscape. `Gauche` defines such kernels for strings, fingerprints, and graphs that operate on a range of widely used molecular representations [73]. We used the Tanimoto kernel [74, 75], which is defined for binary vectors $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$ for $d \geq 1$ as:

$$k_{\text{Tanimoto}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - \langle \mathbf{x}, \mathbf{x}' \rangle}, \tag{2.1}$$

where $\| \cdot \|$ represents the Euclidean norm [73].

The RBF kernel is defined for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ as:

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \tag{2.2}$$

where ℓ is the lengthscale and σ_f^2 the signal variance, which scales both kernels.

2.2.4 Bayesian neural networks

Standard feed-forward networks are deterministic; however, Figure 2.1 shows how they can be transformed into probabilistic models.

Unlike standard deterministic ML methods, Bayesian methods model distributions over predictions using Bayes' rule, allowing for the quantification of uncertainty within parameters and labels. Bayesian methods treat the model parameters as random variables with distributions. Uncertainty can also be estimated from other non-deterministic methods, including deep ensembles [76] and bootstrap modeling [77].

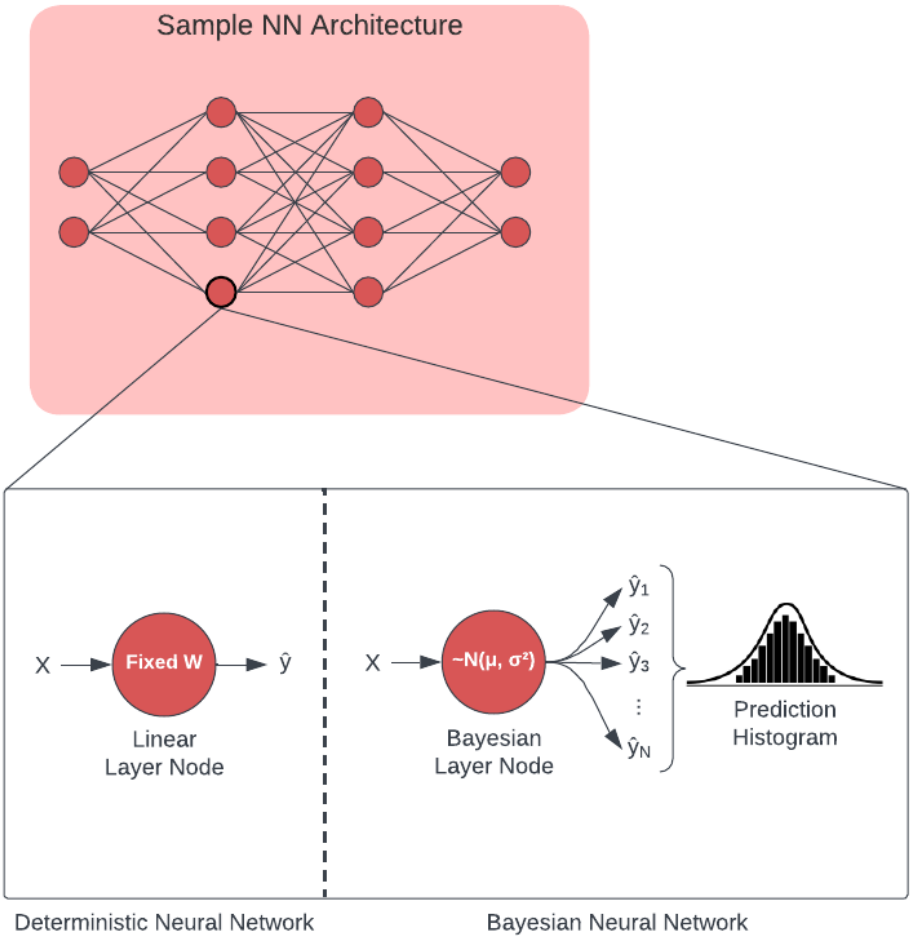


Figure 2.1: One node of a deterministic and of a Bayesian neural network. The deterministic node (bottom left) applies a fixed weight W and returns one prediction $\hat{y}$. The Bayesian node (bottom right) draws its weight from $\mathcal{N}(\mu, \sigma^2)$, so repeated predictions $\hat{y}_1, \ldots, \hat{y}_N$ form a histogram.

2.3 Molecular similarity and activity cliffs

Much of cheminformatics rests on the similar property principle (SPP): structurally similar molecules tend to have similar properties, including biological activity [63, 78]. Nearest-neighbor prediction and GP kernels rely on this principle. So do many noise detection methods, which flag a label that differs from the labels of the molecule's nearest neighbors. However, similarity is only defined relative to a representation and a coefficient, the formula that compares two feature vectors [79]. Two molecules that are near-identical under a substructure fingerprint may be far apart under a PDV. A computed similarity value is therefore easily over-interpreted [79]. The representation and the coefficient decide which molecules count as neighbors. They therefore also decide which labels such a noise detector flags.

The most common measure for binary fingerprints is the Tanimoto coefficient [80]. For two molecules with fingerprints $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$,

$$T(\mathbf{x}, \mathbf{x}') = \frac{\sum_{j=1}^{d} x_j x'_j}{\sum_{j=1}^{d} x_j + \sum_{j=1}^{d} x'_j - \sum_{j=1}^{d} x_j x'_j}, \tag{2.3}$$

which is the number of bits set in both molecules divided by the number of bits set in either. Continuous representations are instead compared with a distance, most often the Euclidean distance $\sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2}$; Chapter 4 uses the Manhattan distance $\sum_{j=1}^{d} |x_j - x'_j|$. Both measures are built from sums over the features, and neither weighs a feature by its relation to the property. A bit that is off in both molecules contributes nothing to the Tanimoto coefficient, whereas every bit set in either molecule counts; in the Euclidean distance, every feature that differs between the two molecules counts. Chapter 4 returns to this point in Equation 4.3.

Pairwise distance comparisons rely on the assumption that the underlying features contain relevant structural information. When the important features are scarce, similarity scores end up dependent on irrelevant information. When the labels are corrupted, the neighbors' labels are wrong as well. Either way, the assumption that similar molecules have similar properties breaks down. Bajusz et al. [80] found that the Tanimoto, Dice, and Cosine coefficients and the Soergel distance rank molecules almost identically on binary fingerprints. We take this to mean that changing the formula cannot compensate for a weak representation.

There are many downstream tasks which rely on the concept of similarity. The second half of this thesis relies on many such tasks, and as such focuses on identifying representations that agree with this principle.

One of the challenges to handling noise is distinguishing it from ACs, defined as a pair of structurally similar molecules whose chemical properties differ by a large amount [81, 82]. While there is no universal definition of an AC [63], by definition they rely on a molecular similarity measure. The MoleculeACE benchmark [63] uses three measures: the Tanimoto coefficient on ECFPs, the Tanimoto coefficient on scaffold fingerprints, and a scaled Levenshtein distance on SMILES strings. A pair forms an AC if its similarity exceeds 90% under at least one of them and the two molecules differ tenfold or more in K_i or half-maximal effective concentration (EC_{50}) [63]. Across the thirty MoleculeACE targets, molecules in at least one cliff pair make up between 7% and 52% of a dataset [63]. These ACs, which potentially contain the most important information for model prediction, can be confused with noise. Another barrier to distance-based reasoning is persistent noise across labels. The task of trying to deter-

mine if a molecule's value matches that of its neighbors becomes a lot harder when all of the neighbors have some form of noise as well.

As the number of dimensions grows, the distances to the nearest and to the furthest point become increasingly similar [83]. Molecular representations are routinely high-dimensional and sparse. When many of their features carry no information about the property, distances computed over them are dominated by those features.

2.4 Noise in molecular data

Real-world cheminformatics datasets contain label noise arising from multiple sources across the drug discovery pipeline. Although noise is often modeled as being random, or drawn from a Gaussian distribution, other varieties of noise must be considered when working with experimental data. To properly model experimental noise, we must first understand where the noise is coming from.

One key issue with QSAR models is the standard ML assumption that labels represent true values, as most chemical datasets lack sufficient replications to provide a robust statistical representation of the underlying distributions [10]. A single measurement, or even multiple measurements for a given label, will not guarantee that this label accurately reflects the population mean [10]. Most ML models rely on the assumption that the training data is, in fact, an accurate reflection of the population. Therefore, they treat these potentially noisy labels as point values rather than distributions, which opens the door to overfitting to noise. However, Bayesian methods such as GPs do not make this assumption, and neither does conformal prediction [10].

Studies involving noise, and specifically the quantification of noise robustness, typically approach label noise by adding independent, zero-mean, Gaussian error to the labels. However, there's evidence to suggest that noise from experimental assays takes on different forms. Kruger and Overington [84] rejected normality of the differences in binding affinity between orthologous targets by Anderson–Darling test, found them approximately Laplace, and found that differences between assays of the same ligand and target closely resemble them. Kalliokoski et al. [13] examined the widest disagreements among 16,844 pairs of repeat pIC_{50} measurements and found that in most cases, the wide gap between repeat values was due to annotation error rather than to an imprecise assay.

Although numerous studies have explored the effects of noise on ML algorithms for molecular property prediction, comparisons between different molecular representations remain limited. Kolmar and Grulke [10] examined the predictive limits of QSAR in the

presence of noise, showing that by learning the underlying trends rather than fitting to noise, the models can achieve predictions more accurate than those of their training data. They assumed experimental noise could be modeled as Gaussian-distributed, using the Central Limit Theorem to argue that aggregated experimental measurements from multiple independent error sources exhibit a Gaussian distribution. However, Heid et al. [18] distinguish random noise, which affects all molecules alike, from systematic noise that differs between regions of chemical space. Using synthetic noise applied only to nitrogen-containing molecules, they showed that mean-variance estimation can detect it.

Beyond just the distribution, the assumptions of independence and zero mean may fail as well [85]. Bentz et al. [86] examined P-glycoprotein transport measurements from 23 laboratories and found that, among the laboratories using cell lines, 62% of the variance in log digoxin efflux ratio was due to differences between laboratories. When you pool data from different sources, each source has the potential to carry a shared offset. These sources usually contain families of related compounds, breaking the independence assumption. The zero-mean assumption is not assured either, especially when true values fall above or below an assay's detection limit, called censoring. Svensson et al. [87] count censored labels in 15 industrial assays, of which 13 carry censored labels and eight sit between a quarter and two thirds censored. Chapter 3 tests noise conditions that vary the distribution, the correlation, and the bias of injected error separately, at a matched quantity of noise for every condition except censoring.

Label noise enters bioactivity data at every step from the assay plate to the database. Table 2.1 lists these sources, with examples of each and the kind of error each tends to produce. The kind of error describes how the error is distributed across molecules, which is the property that the noise conditions in Section 2.4.1 vary.

Although the general assumption in much of the noise analysis and mitigation literature is that noise, specifically label noise, will have a negative effect on the model, this is not necessarily the case. There are multiple instances of using noise as a method of regularization. In linear regression, appending observations of pure noise shrinks the coefficient estimates, an implicit regularization, although not an optimal one [99]. Furthermore, adding artificial noise drawn from a Gaussian distribution to the activation layers of an NN is considered another form of regularization [100]. These studies indicate that the presence of noise may act as a form of regularization in some contexts. Neither corrupts the labels of real observations, so whether label noise regularizes QSAR and quantitative structure–property relationship (QSPR) models remains open.

Source	Example	Kind of Error
Assay	Edge, row, and column effects from evaporation or temperature gradients, and errors in liquid handling [88]; carryover between wells [89]	Error that varies with plate position
Instrument	Higher relative error near a detection limit [90]; autofluorescence of the test compound [91]	Larger error for weak binders; a systematic bias; labels clipped at the limit
Biology	Seeding density, growth rate, cell cycle, and serum properties [92]; cell passage number [93]	Variation between repeats of the same experiment
Aggregation across sources	ChEMBL, BindingDB, and PubChem pool hundreds of laboratories [94, 95]; independent K_i measurements of one compound–target pair differing by 0.44 log units on average [12]	An offset shared by each laboratory or source, hard to separate from the true value without metadata [96]
Federated learning	Teacher models label a shared public set, and a student model learns from those labels [23]	The teachers' prediction errors on top of the measurement error
Annotation	Mis-registered structures, wrong stereochemistry, and salt forms [14]; 0.1–3.4% of structures incorrect, depending on the database [97]; ambiguous protonation states and counterions [98]	A label attached to the wrong molecule; distinct molecules merged

Table 2.1: Sources of label noise in bioactivity data, with reported examples and the kind of error each source produces.

2.4.1 Artificial noise injection

Our objective is to evaluate the robustness of QSAR models against label noise. To do so, we "inject" artificial noise into the labels during training while test data remain free of noise. Validation labels are given noise of the same type and quantity as the training labels, drawn independently. This artificial noise simulates real-world experimental noise. The clean target label y_i is replaced by a noisy label $\tilde{y}_i = y_i + \epsilon_i$, where ϵ_i is the artificial noise injected at the index i, the value of which is determined by a specified noise strategy. As we increased the quantity of noise, we measured how much predictive performance fell.

We explored seven different ways of adding noise, which we refer to as noise conditions and which are illustrated in Figure 2.2 and summarized in Table 2.2. Three conditions vary the shape of the distribution from which the error is drawn, and three concentrate the error on some molecules or share it within Bemis–Murcko scaffold families. The seventh condition, censoring, clips labels at a limit.

We define a scaffold family as the set of molecules that share a Bemis–Murcko scaffold, the ring systems and linking chains left once all other side chains are removed [101].

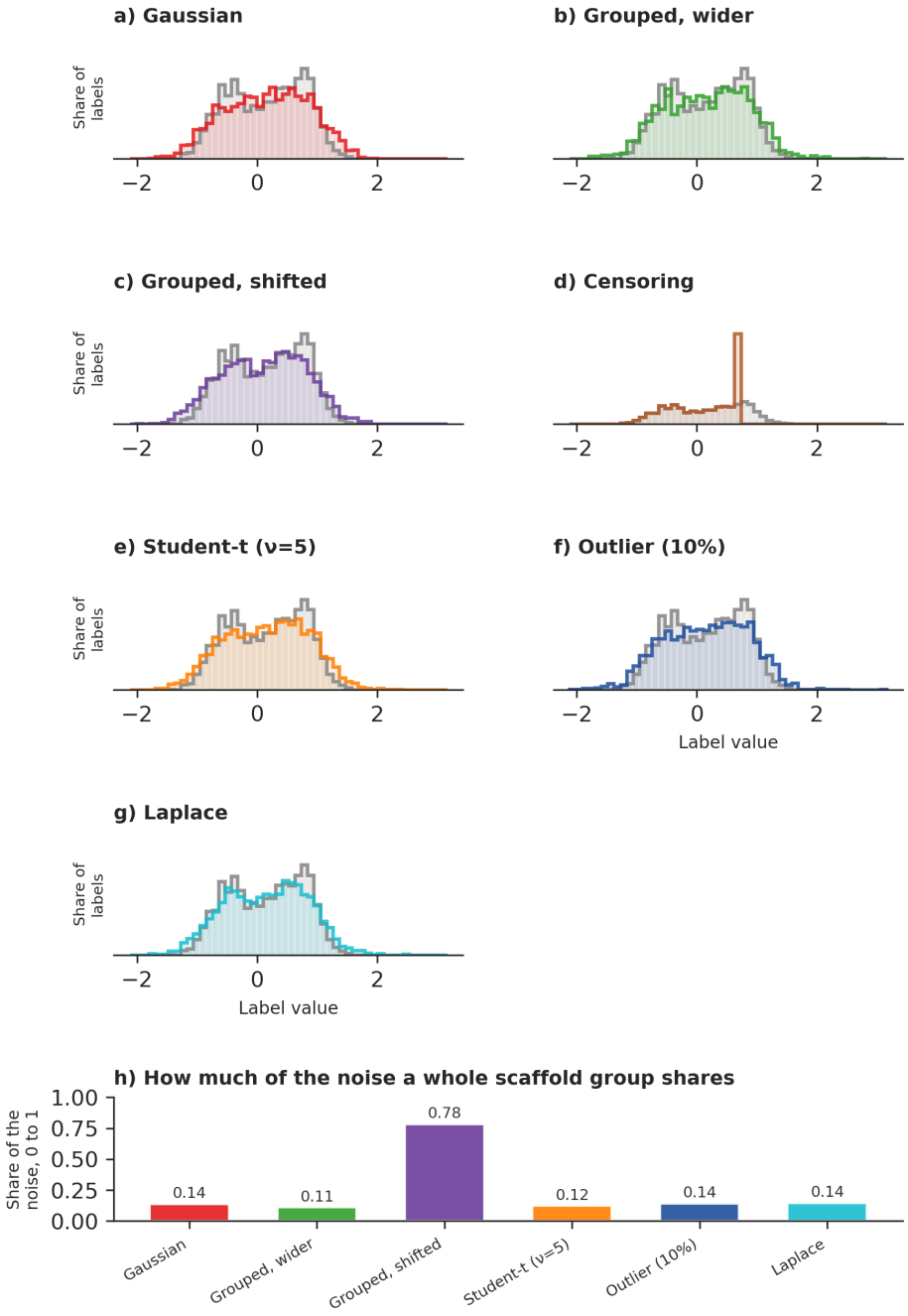


Figure 2.2: Label distributions under the seven noise conditions, for 2,000 synthetic labels in 40 random groups that stand in for scaffold families. a)–g) Clean (gray) and noised (color) labels at a noise level of 0.5 times the SD of the clean labels (25% of labels clipped under censoring). h) SD of the mean error within a group divided by that of all errors, for every condition except censoring; a value near 1 means a group's labels move together, and noise that ignores the groups gives about 0.14 with about 50 labels per group.

For a given label, every condition except censoring delivers the same expected quantity of noise at a set noise level. These levels are defined as a fraction of the SD of clean training labels. The noise added to the label of molecule i is

$$\epsilon_i = \frac{\tau}{G}\, s_i z_i, \qquad G = \sqrt{\frac{1}{n}\sum_{j=1}^{n} s_j^2}\ \sigma_z, \tag{2.4}$$

where τ is the target SD of the noise, in the units of the label, equal to the noise level times the SD of the clean training labels, and z_i is drawn from the condition's distribution at unit scale. The multiplier s_i is 1 for most molecules and $\lambda = 3$ for those the condition affects, which are a random 10% of molecules under outlier and whole scaffold families covering 20% of the split under grouped-wider. The constant σ_z is the SD of z, equal to 1 for the Gaussian, $\sqrt{\nu/(\nu-2)}$ for the Student-t, and $\sqrt{2}$ for the Laplace. The factor G divides both of these out, so that ϵ has SD τ in expectation whatever the distribution and whichever molecules are affected.

Grouped-shifted widens no molecule's error, and instead gives every scaffold family g an offset of its own,

$$\epsilon_i = \frac{\tau}{G}\left(\sqrt{\rho}\, b_{g(i)} + \sqrt{1-\rho}\, w_i\right), \tag{2.5}$$

with one draw b per family and one draw w per molecule. The two variances sum to τ^2, and the family offset carries a share ρ of that total.

We selected the parameter values from published measurement errors wherever possible, and Table 2.2 notes where no published estimate exists. We found no published estimate of how large a share of a dataset is affected by a difference between laboratories, so under grouped-wider we chose to draw scaffold families until they covered 20% of the training molecules.

2.4.2 Learning with noisy labels

In this thesis, methods for LNL fall into three groups, listed in Table 2.3: noise detection, noise mitigation, and noise-robust training. The groups are defined by the general way in which noisy labels are detected and then by what is done with them. Song et al. [56] sort deep learning methods for noisy labels differently, into five groups by the part of training they change: robust architectures, robust regularization, robust loss functions, loss adjustment, and sample selection. Figure 2.3 shows how a noise mitigation method is evaluated.

No single method is expected to win. Averaged over all problems, no learning

Condition	What It Changes	Value	Source for the Value
Gaussian	Shape: every label's error is Gaussian	–	The standard assumption in noise-robustness studies
Student-t	Shape: heavier tails than Gaussian	$\nu = 5$	Differences in binding affinity between related targets are non-normal, and differences between assays of the same ligand and target resemble them [84]; no published estimate of ν, so we chose $\nu = 5$
Laplace	Shape: sharper peak and heavier tails than Gaussian	–	Approximately Laplace for differences in pK_i between related targets, which resemble differences between repeated measurements [84]
Outlier	Error λ times wider on a random share of molecules (Huber's contamination model [102])	10% of molecules; $\lambda = 3$	1–10% of routine data contain gross errors, upper bound taken [103]; for one solubility method, inter-laboratory error is about three times intra-laboratory error [104]
Grouped-wider	Error λ times wider on whole scaffold families	Families covering 20% of the training molecules; $\lambda = 3$	No published estimate of the share; λ as for outlier [104]
Grouped-shifted	Every scaffold family gets its own offset; no error is widened	$\rho = 0.62$	Laboratory identity explains 62% of the variance in log digoxin efflux ratio across the laboratories using cell lines [86]
Censoring	Labels past a limit are set to the limit; the level is the fraction clipped	Set by the noise level	Censored labels in industrial assays [87]

Table 2.2: The seven noise conditions and their parameter values. ν is the number of degrees of freedom of the Student-t distribution, λ the widening factor (Equation 2.4), and ρ the share of the noise variance carried by the scaffold-family offset (Equation 2.5). For censoring, the noise level is the fraction of labels clipped.

Table 2.3: Terms used throughout this thesis for methods that handle noisy labels. A noise mitigation method is named by both halves: what selects the molecules, and what is then done to them. The group follows from the two.

What selects the molecules	
Detector	A score inferred from the features and the labels, saying how wrong each label looks. The score is that molecule's suspicion score.
Known flag	Already recorded in the data, with nothing inferred: a label at the assay limit, how many times a molecule was measured, which source reported it, which scaffold family it belongs to.
Every molecule	No selection. The method acts on all the labels.
What is done to them	
Delete	The molecule leaves the training set.
Down-weight	The molecules and labels are kept and each molecule carries a weight.
Move the label	The label shifts toward a target by some distance. The target is a neighbors' mean, a predicted value, a group mean, or an assay limit.
The groups	
Noise detection	A detector alone, with nothing done afterwards.
Noise mitigation	A detector, a known flag, or every molecule, and then one of the three actions.
Noise-robust training	Nothing is selected and the data is untouched. The loss, the kernel, a noise term, the training loop, or the model changes.

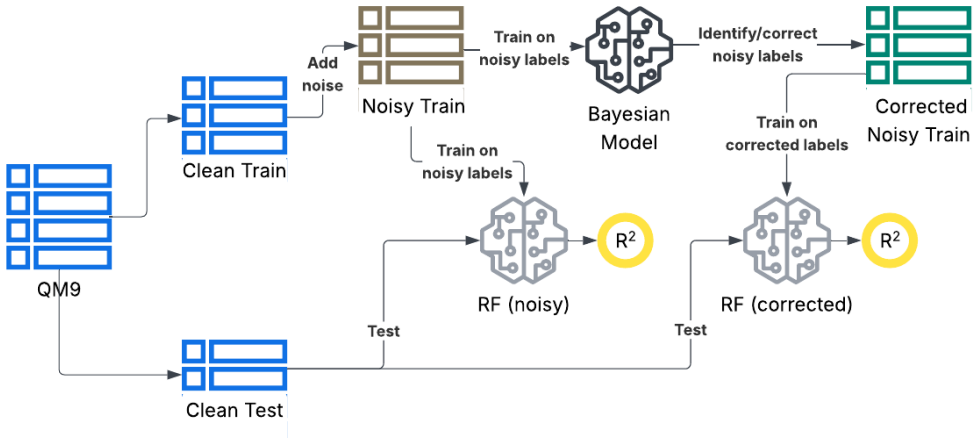


Figure 2.3: Evaluation of a noise mitigation method, with QM9 as the example. An RF trained on noisy training labels is compared with one trained on labels corrected by a Bayesian model; the test labels stay clean, and both forests are scored on them by R^2 (higher is better).

algorithm outperforms another [105]. Neighbor-based cleaning relies on the SPP. It dates back to editing a training set by its nearest neighbors [106]. However, ACs break that principle, and QSAR models frequently fail to predict ACs [62]. Chapter 4 tests the principle, and Chapter 7 tests neighbor-based noise mitigation with the adaptive hybrid representation as the feature space.

2.5 Implementation of the noise conditions

The noise conditions used in this thesis are primarily implemented by NoiseInject, an open-source Python package that helps the user inject noise onto labels. Experiments done with QM9 in Chapter 3 use a separate Python and Rust pipeline with its own Rust implementation of the same conditions. The two implementations have been confirmed to deliver the same noise using uniform seeds.

2.5.1 The NoiseInject package

NoiseInject (version 0.3.0) evaluates model robustness and uncertainty calibration under artificial label noise, for both regression and classification. It operates on arrays of labels and on the predictions of any externally trained model, which makes it model-agnostic, and its core depends only on NumPy, pandas, SciPy, and scikit-learn. Results are returned as `pandas` data frames with one row per noise level and one aggregate summary row. The software is available under an MIT license and archived on Zenodo [107].

Regression noise

The regression injector sets a noise shape (Gaussian, Student-t, or Laplace) and a targeting rule (uniform, grouped-wider, grouped-shifted, outlier, or censoring) separately. Each named condition is one combination of the two with its parameters, and the name is recorded with every result. The requested noise τ is the noise level times the standard deviation of the clean training labels. The injector solves for the scale τ/G in closed form with Equation 2.4, and a level of zero returns the labels unchanged. The affected molecules are selected by a separately seeded random number generator, so they are the same at every noise level while the noise draws change. Under grouped-wider, whole scaffold families are added in random order until they cover as close as possible to 20% of the training molecules.

Censoring is parameterized by the fraction f of labels clipped, with its limit at the

$(1 - f)$ quantile of the clean training labels. On the training split the censored set is the $k = \text{round}(fn)$ largest labels; on any other split it is whichever molecules lie past the training limit.

Each injection also returns a record of the target and delivered root-mean-square noise, the factor G, the solved scale, the mean of ϵ, the fraction of molecules affected, the group sizes, and the random seeds. The noise is checked against a tolerance band, described in Additional file 1, and a draw outside it raises a warning.

Under uniform and grouped-shifted targeting every molecule gets the same scale, so the package flags these scales as degenerate instead of reporting a rank correlation of zero. A separate injector sizes the noise against a unit supplied by the user. In Chapter 7 this unit is set to be the standard deviation of a model's out-of-fold residuals on the clean training labels. It either adds Gaussian noise or shifts 2%, 5%, or 10% of labels. The Gaussian noise has variance the level times the square of the unit. Here the level scales the variance, not the standard deviation as in Equation 2.4. Each shift is the level times the unit, with a random sign or upward only. It can also add Gaussian noise whose standard deviation follows a per-molecule covariate or a hidden log-normal draw, scaled to the same mean variance.

Classification noise

The package provides six label-flipping strategies: uniform flips, flip rates set by class frequency, asymmetric flip rates for binary tasks, a random flip rate for each sample, a flip rate for each class, and flips directed by a confusion matrix. Because the realized flip rate usually differs from the nominal probability, a binary search finds the probability that changes a target fraction of labels.

Robustness metrics

For regression, the package computes R^2, the root mean squared error (RMSE), and the mean absolute error (MAE) at every noise level. For classification, it computes accuracy, together with macro- and weighted-averaged precision, recall, and F1 score score and a per-class breakdown. Robustness is read from the retention curve $\text{ret}(x) = m(x)/m(0)$, where x is the noise level. Here m is a metric bounded above by 1 for which higher values are better: R^2 for regression, and accuracy, F1, precision, or recall for classification. The primary summary is the normalized area under this curve, AUC_{norm}, defined in Equation 3.2 of Chapter 3 and evaluated by the trapezoid rule. It is the mean fraction of clean performance retained across the noise levels and lies approximately

within $[0, 1]$. It is also less directly correlated with clean-label R^2 than other robustness metrics we tested. A secondary summary fits $\mathrm{ret}(x) \approx \exp\bigl(-(x/\tau_W)^\beta\bigr)$ to the curve. The shape parameter β separates curves that hold and then collapse ($\beta > 1$) from curves that fall early and then plateau ($\beta < 1$). The parameter τ_W sets the scale of the decay, and the fit requires at least four noise levels. The package also reports the clean baseline and the retention at the highest noise level, flags any curve that falls below zero retention, and can withhold the curve summaries for models whose clean baseline is too low.

Uncertainty metrics

For a model that returns a predicted standard deviation u_i for each sample, the package computes, at each noise level, the Spearman correlation between u_i and the absolute error $|y_i - \hat{y}_i|$, the empirical coverage $\frac{1}{n}\sum_i \mathbf{1}\left[|y_i - \hat{y}_i| \le k u_i\right]$ at $k = 1$ and $k = 2$, against Gaussian targets of 68.3% and 95.4%, the mean interval width $\frac{1}{n}\sum_i 2u_i$, and the expected calibration error (ECE),

$$\mathrm{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} \left| \bar{u}_b - \overline{|y - \hat{y}|}_b \right|, \tag{2.6}$$

computed over ten equal-count bins B_b of u; all of these use the clean test labels. Under Gaussian errors a calibrated model has an MAE of $\sqrt{2/\pi}\,u \approx 0.80\,u$, so its ECE is about $0.2\,\bar{u}$, not zero. The Spearman correlation between u_i and the injected noise $|\epsilon_i|$ can only be computed on the corrupted molecules, where the noise is known, and is reported within each noise level. A correlation pooled across levels is also returned, but it mostly reflects that uncertainty and noise both rise with the level, so it does not measure whether a model can identify which molecules are corrupted.

Three optional wrappers produce u_i from a point predictor: split-conformal prediction around any scikit-learn-compatible model, optionally scaled by the label spread of each molecule's nearest training neighbors; an exact GP with a Tanimoto or RBF kernel through `Gauche` [73]; and Monte Carlo dropout on any `PyTorch` network [108].

2.6 Evaluation methodology

Chapter 3 reports predictive performance as R^2 on held-out molecules and robustness as $\mathrm{AUC}_{\mathrm{norm}}$, defined in Equation 3.2. Chapters 4 to 7 state the metric used for each dataset where it is introduced, and report results as per-fold scores and their spread

across folds, with no significance tests.

No split or fold in this thesis is random, and no group of molecules is split between training and test. Random splitting inflates performance estimates for molecular data, because it places structurally similar molecules on both sides of the train/test boundary [109–111]. For small molecules the group is the Bemis–Murcko scaffold family described in Section 2.4.1. Section 3.2 gives the splits of Chapter 3, and Section 5.2.7 gives the folds of the later chapters and the groups used for datasets that are not small molecules.

2.7 Conclusions

A QSAR model consists of two parts, the molecular representation and the model itself. Measures of similarity depend on the representation as well, and a representation with uninformative features generates negative effects across the board. Label noise coming from experimental assays, while often assumed to be Gaussian in the literature, can take on a variety of forms. For this reason, we defined seven artificial noise conditions. Methods for noisy labels fall into noise detection, noise mitigation, and noise-robust training. No method works on every dataset; all of them show promise conditionally.

Chapter 3

Noise robustness across molecular representations and model architectures

The work in this chapter is adapted from Punt, A., Hanser, T., Werner, S., and Morris, G.M., NoiseInject: A Benchmarking Framework to Study Noise Robustness in QSAR Models, to be submitted to the Journal of Cheminformatics. Most of the figures, tables, and text in this chapter are therefore either identical or similar to the content of that manuscript. The authors' contributions are given in the Statement of Contribution.

3.1 Introduction

In this chapter, we inject the noise conditions of Chapter 2 into training labels and measure how much predictive performance each pairing of representation and model retains. While there have been many investigations into the effect of noise on molecular property prediction, direct comparisons between how different molecular representations respond to label noise remain limited. The choice of molecular representation certainly affects predictive performance. Deng et al. [112] evaluated 62,820 models and found that fixed representations, in most datasets RDKit 2D descriptors, outperformed representations learned from SMILES and molecular graphs. Whether these conclusions translate to noise robustness is our question. Between different types of label noise, Heid et al. [18] found no difference in model performance between error distributions with the same mean and standard deviation. We expand on this by considering alternative patterns of noise, namely noise concentrated in structural groups or on a sparse subset of samples.

This chapter addresses the questions set out in Chapter 1, specifically how the choice of representation and model architecture explain the variance in predictive performance and in noise robustness, what is the impact of the type of label noise, and whether uncertainty estimates track label noise.

3.2 Methods

We paired six representations with thirteen base models, as well as several variant models, across four prediction tasks. The hyperparameters of every model are given in Additional file 2.

3.2.1 Datasets

Although we are investigating the effects of label noise stemming from experimental settings, for the purposes of testing we inject controlled amounts of artificial noise into clean labels. Data were extracted from the QM9 molecular property dataset distributed by PyTorch Geometric [113], which contains 130,831 small organic molecules with pre-computed quantum-mechanical properties obtained from density functional theory (DFT) calculations at the B3LYP/6-31G(2df,p) level of theory. We selected the gap between the highest occupied and the lowest unoccupied molecular orbital (HOMO–LUMO gap) as the prediction target in QM9. We assumed that QM9 labels were free of noise, notwithstanding approximations in the level of theory. Each experiment was replicated 10 times with a distinct random seed and a randomly selected subset of $N = 10{,}000$ molecules, split 80/10/10 into 8,000 training, 1,000 validation, and 1,000 test molecules. All models were fitted on those 8,000 training molecules, except the GPs, which were fitted on a random subset of 5,000 of them; the validation and test molecules were unchanged.

We repeated these experiments on three molecular property datasets with experimentally determined endpoints. From the OpenADMET initiative [114], we used LogD and Caco-2 efflux permeability, with $N = 5{,}039$ and $N = 2{,}161$ respectively. We also selected hERG K_i data from ChEMBL [115], following a protocol inspired by Landrum and Riniker [16]: filter for binding assays, deduplicate by median pChEMBL value, and remove compounds with an inter-assay standard deviation greater than 1.0 log unit, resulting in $N = 1{,}415$ compounds. Each of the three was partitioned into five folds by grouped k-fold cross-validation (CV), and each fold's training block gave up a further grouped fifth as a validation set for early stopping. A model therefore fits

between 3,194 and 3,364 of the 5,039 LogD molecules, depending on the fold, so the GP subsample never applies to these datasets. Unlike QM9's 10 replicates, which are 10 independent draws from one pool, each assay dataset is a single fixed set in which every molecule is tested exactly once and nothing is redrawn.

Log D is the logarithm of the distribution coefficient of a molecule between octanol and water, usually measured at pH 7.4. It counts every charge state, whereas log P counts only the neutral form, and a basic amine that is protonated at that pH therefore has a log D below its log P [116]. Caco-2 cells come from a human colorectal carcinoma and grow as a layer that models absorption in the gut. Permeability across the layer is measured in both directions, and the efflux ratio is the permeability from the basolateral to the apical side divided by that in the other direction [117]. In this chapter the Caco-2 label is the efflux ratio on a logarithmic scale. A ratio above about 2 suggests the compound is pumped out by a transporter such as P-glycoprotein. The hERG gene encodes a cardiac potassium channel. Drugs that block it can prolong the QT interval and cause arrhythmia, so hERG binding is screened early in drug discovery [118]. The hERG K_i dataset holds binding affinities for this channel, as pK_i values.

All splits and CV folds were grouped by the chirality-blind Bemis–Murcko scaffold, implemented such that each distinct acyclic molecule forms its own group, and groups are filled in random order. Prior to modeling, all molecular structures were sanitized and canonicalized using `RDKit` to remove invalid valence states and standardize atom and bond typing [37]. Prediction targets were standardized to zero mean and unit variance. Experiments that track uncertainty were run on 10 replicates on QM9 and five folds on each assay dataset.

3.2.2 Representations

In this study, we evaluated six molecular representations: extended-connectivity fingerprints of diameter four (ECFP4), Avalon, Sort & Slice (SNS), physicochemical descriptors (PDV), ChemBERTa, and Molecular Hypergraph Grammar GNNs (MHG-GNNs). The same six were used on QM9 and on the three assay datasets. We used the standard ECFP with radius $r = 2$ with $d = 2048$ bits [5, 37]. SNS takes the same substructures as ECFP4, but it ranks them by prevalence and selects the top L ($L = 1024$) [7]. We fitted the SNS vocabulary on the training molecules alone, refitting it for each QM9 replicate and for each CV fold. Avalon is a 2,048-bit fingerprint, generated with the Avalon toolkit through RDKit [8, 37]. The PDV representation is a vector of 200 physicochemical descriptors computed with RDKit's

`MolecularDescriptorCalculator` [36, 37]; the 200 are listed in Additional file 2, and the list is fixed in our code rather than taken from RDKit's default descriptor set, which grows between releases. ChemBERTa embeddings are the mean over the non-padding token embeddings of the `DeepChem/ChemBERTa-77M-MTR` checkpoint, producing 384 features [33]. MHG-GNNs are a GIN-based autoencoder pre-trained on about 1.38 million PubChem molecules using β-variational autoencoder (β-VAE) loss, producing 1024-dimensional embeddings through iterative message passing and found in the published `mhggnn_pretrained_model_0724_2023` checkpoint [9]. We z-score normalized PDV, ChemBERTa, and MHG-GNN per feature, using the training molecules' mean and standard deviation; ECFP4, Avalon, and SNS reach the model as built.

3.2.3 Models

To understand model behavior in the presence of label noise, we explored a range of model families. We fitted tree ensembles, an SVM, GPs, and NNs. Several are variants of one another, differing in the probabilistic machinery they add, and Additional file 2 gives every configuration with its settings.

Random Forests (RFs) are one of the most common choices for QSAR modeling thanks to their robustness and interpretability [119]. Quantile Regression Forests (QRFs) hold the distribution of training labels in each leaf and compute quantiles over all trees [120]. While we used 100 trees for RF, we observed a noisy quantile estimate with 100 trees for QRF, prompting us to use 300 instead. To observe the effect of the quantile mechanism, we then fit a second RF at 300 trees so the two models were identical in all other settings.

We also used eXtreme Gradient Boosting (XGBoost) [64], Light Gradient Boosting Machine (LightGBM) [67], and Natural Gradient Boosting (NGBoost) [68]. NGBoost extends deterministic gradient boosting by treating the parameters of a chosen parametric distribution as regression targets and learns them through boosting with a natural gradient update rule [68]. NGBoost relies on scoring off validation data, which differs by dataset. On QM9 the validation split is already held out of every fit. However, on the three assay datasets NGBoost holds out its own scaffold-grouped fifth of the rows it is handed, which already exclude the validation molecules.

SVMs are a well-established baseline in QSAR modeling [17, 69]. We used a radial basis function (RBF) with $C = 1.0$ and $\gamma = 1/(d\,\mathrm{Var}(X))$, where d is the number of features and $\mathrm{Var}(X)$ the variance of all training feature values, as the kernel, as it accommodates continuous vectors.

Gaussian processes (GPs) are a non-parametric class of models that use a kernel to produce a Gaussian predictive distribution over every data point [121, 122]. We fitted three variants of GPs, each with a constant mean and a scaled kernel. The first uses an RBF kernel and runs on all six representations. The second uses the Tanimoto kernel [74, 75] and was run on ECFP4 alone; it is not reported in this chapter. The third is a heteroscedastic variant in which the RBF process is given a second network that predicts the observation noise of each molecule. One of the major limitations of using GPs in practice is computational scaling; inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training points [122]. This can be mitigated with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [123]. As such, we used a standard GP and only fit on at most 5,000 training molecules, drawn by a seeded sample. Due to the size of the datasets, this cap only affected QM9. We implemented these GP models in `GPyTorch` [124], with the Tanimoto kernel taken from the `Gauche` framework [73].

We tested two deterministic feed-forward architectures, both fully connected and both implemented in PyTorch [125]: (i) NN-α with two hidden layers of sizes 128 and 64, and dropout after each hidden layer and (ii) NN-β with two hidden layers, each of size 128, with dropout applied before the output layer. Each model was trained using the Adam optimizer with early stopping on validation loss. We used Bayesian transformations to convert NNs into Bayesian neural networks (BNNs), and implemented them using `PyTorch` [125] with ReLU activations and dropout regularization ($p = 0.2$). Additional file 2 gives their full widths, dropout fractions, learning rates, and batch size.

To get these networks producing uncertainty estimates, we applied four probabilistic transformations to each architecture, producing eight probabilistic networks. The first replaces every linear layer with a Bayesian layer carrying a Gaussian prior $\mathcal{N}(0, 0.1^2)$ on its weights, producing BNN-α and BNN-β. They are trained on an evidence lower bound whose Kullback–Leibler (KL) divergence term is divided by the number of training molecules. While this is a straightforward approach, it is computationally expensive. We explored alternative techniques like our second variant, where we add an output head that predicts a variance to the BNN and fit the negative log-likelihood (NLL) of a Gaussian with that variance [126]. The third, Variational Bayesian Last Layer (VBLL), replaces the output layer with a Bayesian last layer that produces mean-field variational posteriors $q(\mathbf{W}) = \mathcal{N}(\boldsymbol{\mu}_W, \mathrm{diag}(\boldsymbol{\sigma}_W^2))$ over the last-layer weights and trains by maximizing the evidence lower bound. This can be thought of as minimizing the reconstruction loss plus KL divergence $D_{\mathrm{KL}}(q(\mathbf{W}) \| p(\mathbf{W}))$ from a standard normal

prior, scaled by $1/N$ [127]. Because only its output layer keeps a learned observation-noise parameter, VBLL networks report only epistemic uncertainty per molecule, with one aleatoric variance for the whole fit. The fourth makes observation noise a function of the input. All eight probabilistic networks estimate their predictive distributions from 100 stochastic forward passes. Each network and its Bayesian counterpart were tuned to the same set of parameters so that NN-α against BNN-α and NN-β against BNN-β differ by the Bayesian layers alone.

The thirteen base models are RF, QRF, XGBoost, LightGBM, NGBoost, SVM, the RBF-kernel GP, and NN, BNN, and VBLL in each of the two architectures. The six variant models are RF (300 trees); GP (het.), the heteroscedastic GP; BNN-α (var. head) and BNN-β (var. head), the two Bayesian networks with a variance head; and VBLL-α (het.) and VBLL-β (het.), the two heteroscedastic VBLL networks.

3.2.4 Noise conditions and levels

The seven noise conditions are defined in Section 2.4.1.

We controlled noise quantity using noise levels, representing a fraction of the standard deviation of clean training labels. We ran experiments across seven levels, 0, 0.2, 0.3, 0.5, 0.75, 1.0, and 1.5. We added the noise to the raw label and standardized the target afterwards, using the clean training mean and spread. To control the quantity of noise for censoring, we recorded every label above the kth largest clean training label as that limit and used this level as the fraction clipped, at 0, 0.1, 0.2, 0.25, 0.3, 0.4, and 0.5. Censoring is common in pharmaceutical assays, where a third or more of labels can be censored [87].

We ran Gaussian, grouped-wider, and grouped-shifted noise on every model and representation pairing. The remaining four conditions ran on selected pairs.

On QM9, Laplace, Student-t, and outlier noise were run on eight models, namely RF, NGBoost, SVM, the GP, GP (het.), BNN-α (var. head), VBLL-α (het.), and VBLL-β (het.), each on ECFP4, PDV, and ChemBERTa. On the three assay datasets they were run on the same models and representations except VBLL-α (het.). Censoring was run on five pairings, namely RF and GP (het.) on ECFP4, and NGBoost, the GP, and BNN-α (var. head) on PDV. On the three assay datasets, $\mathrm{AUC}_{\mathrm{norm}}$ under censoring was computed for the same five pairings, except BNN-α (var. head) on PDV for hERG K_i, whose clean R^2 was below 0.3 in every fold. The uncertainty statistics under censoring on the assay datasets cover seven models, namely QRF, NGBoost, the GP, GP (het.), BNN-α (var. head), BNN-β (var. head), and VBLL-α, each on ECFP4,

PDV, and ChemBERTa.

3.2.5 QM9 data pipeline

QM9 [113], loaded through PyTorch Geometric, was processed by a separate pipeline where the Python code loads and splits the data, and computes several representations. The data is then passed to the Rust portion of the pipeline which injects the label noise, standardizes the labels, computes the remaining representations, and sends the processed data back to Python which trains and scores the models.

Each replicate, using its own random seed, randomly selects a subset from QM9, which is split by a new scaffold split, and the same grouping defines the scaffold families targeted by the grouped noise conditions.

Python and the Rust program exchange the data through memory-mapped binary files on disk, one record per molecule, as described in Additional file 11.

We compute the quantity of noise, the censoring limit, and the affected scaffold families using only the clean training labels. Validation labels get all three, with the amount in label units, from an independent draw seeded from the training seed. The program checks that validation received as much noise as training. Test labels are never corrupted, but every split records the noise scale each molecule would have received, so that held-out molecules can be scored against the pattern applied in training.

For every molecule, the program records its canonical SMILES, clean and noisy labels, standardized value, injected noise, noise scale, and noise pattern. Each run also writes a summary of the noise parameters and the amount of noise delivered. The run halts if a record holds a different molecule when it is written than when its noise was drawn, or if any molecule has an all-zero ECFP4 or Avalon fingerprint, which marks a failed featurization. A self-test mode runs the same noise checks on a label file alone.

3.2.6 Uncertainty estimation

Up to this point we have been concerned with how noise affects the predictive performance of QSAR models. However, label noise should also affect the uncertainty these models report. Some models return only a point prediction, while others also return an estimate of uncertainty, either as a predicted distribution or as the spread across an ensemble [128]. Furthermore, some report a single value of uncertainty while others report one that can be decomposed into an aleatoric and an epistemic component [126]. The aleatoric component describes underlying noise the model associates with the data, while the epistemic component represents the uncertainty within the model's

fit. We sought to determine if, as expected, the aleatoric component rises with injected label noise while the epistemic component does not.

For the BNN and VBLL transformations, we took the two components over $T = 100$ stochastic forward passes [126]:

$$\hat{u}^2_{\text{ale}}(x) = \frac{1}{T}\sum_{t=1}^{T} \hat{v}_t(x), \qquad \hat{u}^2_{\text{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T} \big(\hat{\mu}_t(x) - \bar{\mu}(x)\big)^2, \tag{3.1}$$

where $\hat{\mu}_t(x)$ and $\hat{v}_t(x)$ are the mean and the variance returned for molecule x on pass t, and $\bar{\mu}(x)$ is the mean over passes. Here $\hat{v}_t(x)$ is the predicted observation noise variance. It varies per molecule in the variance-head and heteroscedastic variants, and is one number for the whole fit in the VBLL transformation. A full-BNN without a variance head returns no variance, so its uncertainty is epistemic alone.

The other models follow the same split. For GPs, the epistemic component is the posterior variance of the latent function and the aleatoric component is the learned observation noise [122]. All variants of the GP produce one number for the whole fit except heteroscedastic. For QRF models, each tree holds multiple training molecules. So for a given molecule, the aleatoric component is the variance of the labels in the molecule's given leaf. The total aleatoric uncertainty is those variances averaged over the trees. The epistemic component is the variance of the leaf means across the trees. NGBoost predicts one variance for each molecule from a single fit, which is aleatoric alone [68]. In every model, the total uncertainty is the sum of the aleatoric and epistemic components.

No held-out label carries injected noise, so every comparison against the injected noise was made on training molecules. Each training set was divided into five folds grouped on Bemis–Murcko scaffolds, and each molecule was scored by a refit that excluded its own fold. For NGBoost on QM9, three of the five folds were scored to limit its cost. No uncertainty was calibrated after fitting.

3.2.7 Evaluation

We compute statistics on the held-out molecules, for QM9 the test split and for the three assay datasets the held-out fold of the scaffold CV. Different combinations of dataset, model, representation, noise condition, and noise level are examined in each scenario. We used the coefficient of determination (R^2) throughout. For molecule i, let y_i be the clean label, $\hat{y}_i$ the prediction, ϵ_i the noise injected into its label, and u_i the predicted uncertainty.

To evaluate the effect of label noise, we tracked predictive performance retention under increasing artificial noise. For each QSAR pairing we recorded $R^2(\tau)$ at each noise level τ, and normalized it to obtain the retention ratio $R^2(\tau)/R^2(0)$. To quantify robustness, we used the normalized area under this retention curve,

$$\mathrm{AUC}_{\mathrm{norm}} = \frac{1}{\tau_{\max}} \int_0^{\tau_{\max}} \frac{R^2(\tau)}{R^2(0)}\, d\tau, \tag{3.2}$$

evaluated using the trapezoidal rule over the seven noise levels, $\tau \in \{0, 0.2, 0.3, 0.5, 0.75, 1.0, 1.5\}$, each a fraction of the standard deviation of the clean training labels. While we tested other metrics of robustness, we found $\mathrm{AUC}_{\mathrm{norm}}$ to be less directly correlated to clean-label R^2 predictions. For censoring, we sweep over a set of fractions to clip the labels prior to normalization.

An $\mathrm{AUC}_{\mathrm{norm}}$ value near 1 indicates that the QSAR model in question has retained predictive performance with artificial noise, meaning it is more robust to noise. Since we normalize the curve by its own clean-label predictive performance, a model with a low clean R^2 has less to lose, so $\mathrm{AUC}_{\mathrm{norm}}$ is reported beside clean R^2. We excluded replicates with a clean $R^2 < 0.3$ from the robustness analysis, as performance retention ratios become unstable when the clean-label denominator approaches zero. The excluded configurations are listed in Additional file 3. On QM9 one $\mathrm{AUC}_{\mathrm{norm}}$ is computed per replicate, and we report the median over the 10 replicates. On the three assay datasets one value is computed per scaffold fold, and the median over the five folds is reported.

To compare the relative effect of the 13 base models across datasets and noise conditions, we compared their rankings using Spearman's rank correlation coefficient (ρ) between clean R^2 and $\mathrm{AUC}_{\mathrm{norm}}$. We compared probabilistic models with their deterministic counterparts using a two-sided Wilcoxon signed-rank test paired on the replicate, at $\alpha = 0.05$, separately for each representation and noise condition. This test was run only on QM9, as with five scaffold folds the smallest attainable p is 0.0625.

We conducted a separate analysis of variance (ANOVA) for each noise condition to identify the relative contributions of molecular representation and model architecture to both predictive performance and noise robustness. For a given metric y, either R^2 at a noise level of 1.0 or $\mathrm{AUC}_{\mathrm{norm}}$:

$$y_{fmjr} = \mu + s_r + \alpha_f + \gamma_{m(f)} + \beta_j + (\alpha\beta)_{fj} + (\gamma\beta)_{m(f)j} + e_{fmjr}, \tag{3.3}$$

where s_r represents the effect of replicate r, α_f represents the effect of model family f, $\gamma_{m(f)}$ represents the effect of model architecture m within its family, β_j represents the

effect of molecular representation j, $(\alpha\beta)_{fj}$ and $(\gamma\beta)_{m(f)j}$ are the interaction terms, and e_{fmjr} is the residual. The proportion of variance explained by each term was calculated as the effect size

$$\eta^2_{\text{term}} = \frac{SS_{\text{term}}}{SS_{\text{total}}}, \tag{3.4}$$

using Type I (sequential) sums of squares, entered in the order replicate, model family, model architecture within family, molecular representation, interaction. The 10 QM9 replicates provide the residual. The uncertainty on each share is the 95% interval of η^2 over 1,000 bootstrap resamples of whole replicates. These shares describe the models and representations tested here, not the choice of model or representation in general.

ANOVA relies on the assumption that observations both within and across groups are independent. Two features of this study break that assumption. First, each QM9 replicate uses the same data split and the same noise draw for every model and representation, so observations from one replicate are not independent of each other; we therefore entered the replicate as a block. Second, across the whole set of models in this study, some pairs produced similar predictions that are not independent, so a method present in several near-identical versions would count several times towards the share of model architecture. To handle this issue, we grouped the models into families. We considered grouping pairs of models that share a method or an architecture, and compared each pair on clean R^2 and on AUC_{norm} under the three noise conditions run on every representation, separately on each of the six representations, which gives 24 comparisons per pair. A pair was grouped when the mean difference between the two models was smaller than their standard deviation across replicates in at least two thirds of these comparisons. Additional file 4 gives the details of this grouping. Five families met this criterion: RF with QRF (16 of 24), XGBoost with LightGBM (23 of 24), NN-α with BNN-α (20 of 24), NN-β with BNN-β (18 of 24), and VBLL-α with VBLL-β (20 of 24). VBLL did not group with the plain or Bayesian network of its own architecture (4 to 10 of 24), as its clean R^2 was lower by 0.02 to 0.09, and NGBoost did not group with XGBoost or LightGBM (2 and 3 of 24). The two architectures of the plain networks, and of the Bayesian networks, also met the criterion (18 and 17 of 24); we grouped by architecture instead, which the evidence supports equally well. The 13 base models therefore form eight families. Within these families, the differences between models accounted for at most 1.0% of the variance in any outcome and noise condition on QM9, and up to 15.1% of predictive performance on clean labels on the three assay datasets. Additional file 5 reports Tukey's honestly significant difference test, which we also ran between the members of each family, separately for each repre-

sentation. With 10 replicates it detects differences smaller than the variation between replicates, so we report it beside the size of each difference rather than use it to form the families.

Across representations, we considered grouping SNS with the ECFP4 fingerprint, as both encode overlapping circular substructure information. The two differed by more than their variation across replicates on clean R^2 for every model (0 of 13), so all six representations were kept as separate levels. The comparison is given in Additional file 6.

To assess the interaction between uncertainty estimation and label noise, we posed three questions. First, whether uncertainty rises with label noise, as measured by the mean u_i over held-out molecules at the highest noise level divided by its value with no added noise, in each replicate or fold. The second pertains to models that report separate aleatoric and epistemic components. We divided the slope of the mean epistemic component against noise level by the slope of the mean aleatoric component to determine how each responds to label noise. A ratio near 0 means the added noise was attributed to the labels while a ratio of 1 or above means the epistemic component grew as much or more. We also compared the mean aleatoric component with the standard deviation of the injected noise. The third question we had was whether per-sample uncertainty values track injected label noise, as measured by the Spearman correlation between u_i and $|\epsilon_i|$ over out-of-fold training molecules, per fold and per noise level. We also report the Spearman correlation between u_i and the absolute error of the prediction. Finally, we asked whether dividing a molecule's out-of-fold absolute error by u_i ranks the corrupted labels better than the error alone, and report the difference between the two Spearman correlations with $|\epsilon_i|$.

3.3 Results

3.3.1 Variance decomposition

Considering the two main components of a QSAR model, the molecular representation and the model architecture, we sought to understand how each contributes to both predictive performance and noise robustness. ANOVA decomposition divides the variance between model family, model architecture within its family, molecular representation, the pairing of the two, the replicate, and a residual. We ran this decomposition for Gaussian noise, the standard model, as well as methods whose injected noise is related to the feature space, grouped-wider and grouped-shifted. Figure 3.1 and Table 3.1 give

the results. For predictive performance measured in R^2 at a noise level of 1.0, model family and molecular representation account for comparable shares of the variance, 26.6% and 29.0% under Gaussian noise. However, for noise robustness, quantified by the normalized area under the R^2 retention curve (AUC_{norm}; higher values indicate greater robustness), model family accounts for 47.2% of the variance while molecular representation for 8.1%. This pattern is observed across noise conditions and makes sense, as label noise does not impact the features. The interaction between model and molecular representation explains 19.9% of the variance in AUC_{norm}, more than twice the 8.1% explained by representation alone, meaning representation acts on noise robustness mainly through the model it is paired with. As such, we primarily compare representations within each model.

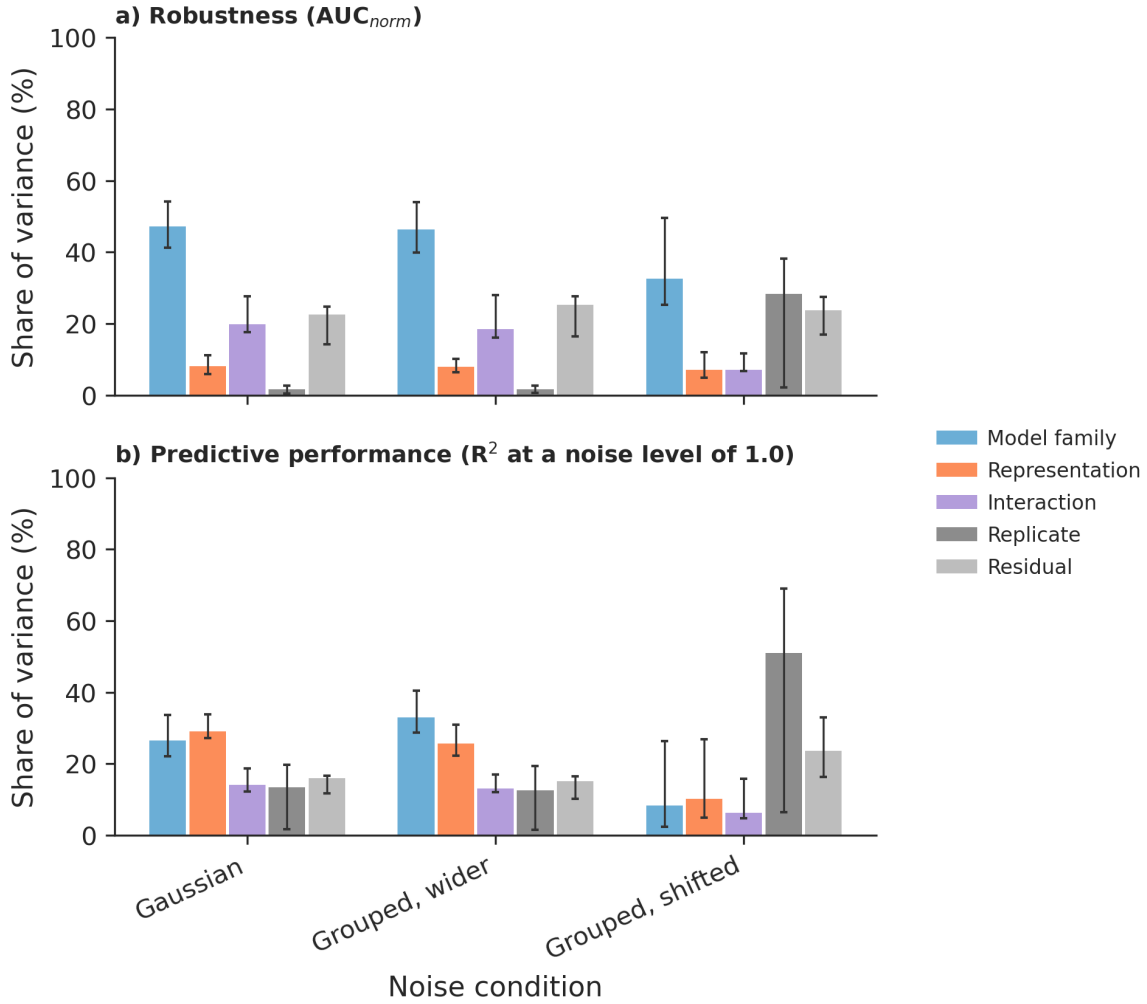


Figure 3.1: Share of the variance explained by each term, on the QM9 HOMO–LUMO gap: a) robustness (AUC_{norm}); b) predictive performance (R^2 at a noise level of 1.0). Whiskers are 95% bootstrap intervals over replicates. Model within family, at most 1.0% in every decomposition on QM9, is not drawn.

Figure 3.2a compares the variance decomposition on both clean and noisy labels. For predictive performance, the artificial label noise lowers the impact of the choice of model family, while the replicate and residual take more. This means that the variance is left more up to chance and to the underlying variation within the data itself with label noise, which is the expected behavior. Robustness divides the variance a bit differently. Representation explains 8.1% of the variance in $\mathrm{AUC}_{\mathrm{norm}}$, against 31.1% of clean R^2, while model family rises to 47.2%. The replicate nearly disappears from $\mathrm{AUC}_{\mathrm{norm}}$, likely because each replicate's curve is divided by that replicate's own clean R^2, so a difficult split lowers both and cancels out.

We include two terms which stem from the data rather than the model, the residual and replicate terms. The replicate measures how much of the variance can be attributed to the train and test split, which in this case is a scaffold split. On QM9 we replicated each experiment 10 times with different random draws of $N = 10{,}000$, each time giving a new scaffold split. On the assay datasets, a replicate refers to one of the five scaffold CV folds. A large replicate term in ANOVA indicates that the folds differ from one another. The residual is variance that no other term accounts for.

While we observe similar patterns between noise conditions, the predictive performance of grouped-shifted noise in particular varies significantly between replicates, resulting in a larger share for the replicate. We find that 28.4% of the variance in $\mathrm{AUC}_{\mathrm{norm}}$ and 51.0% of the variance in R^2 are due to the replicate, with model family falling to 32.7% and 8.4%. This indicates that the offset applied to each scaffold may account for more of the variance in both R^2 and $\mathrm{AUC}_{\mathrm{norm}}$.

Condition	Outcome	Replicate	Model family	Representation	Interaction	Residual
None	Clean R^2	10.6	35.4	31.1	12.1	10.2
Gaussian	$\mathrm{AUC}_{\mathrm{norm}}$	1.7	47.2	8.1	19.9	22.6
Grouped, shifted	$\mathrm{AUC}_{\mathrm{norm}}$	28.4	32.7	7.0	7.2	23.7
Grouped, wider	$\mathrm{AUC}_{\mathrm{norm}}$	1.7	46.3	7.9	18.5	25.3
Gaussian	R^2 at level 1.0	13.4	26.6	29.0	14.2	16.0
Grouped, shifted	R^2 at level 1.0	51.0	8.4	10.3	6.2	23.7
Grouped, wider	R^2 at level 1.0	12.7	33.0	25.7	13.1	15.1

Table 3.1: Share of the variance (%) explained by each term on the QM9 HOMO–LUMO gap, for 13 base models and six representations over 10 replicates. Model within family, at most 1.0% in every row, is left out, so a row does not sum to 100%. Intervals are in Additional file 7.

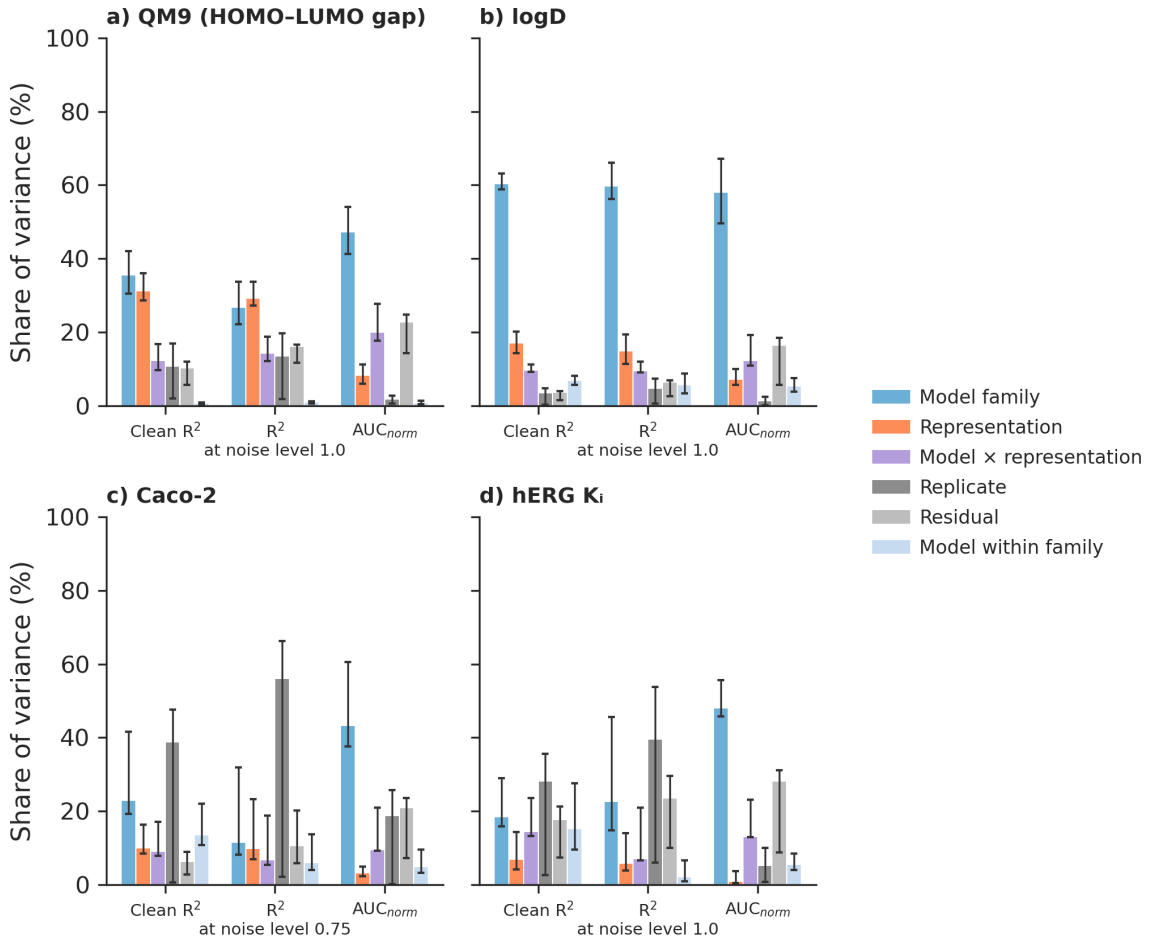


Figure 3.2: Share of the variance explained by each term on a) the QM9 HOMO–LUMO gap, b) LogD, c) Caco-2, and d) hERG K_i, for clean R^2, and for R^2 at a noise level of 1.0 (0.75 on Caco-2, whose clean R^2 is only about 0.5, so little is left to lose at 1.0) and $\mathrm{AUC}_{\mathrm{norm}}$ under Gaussian noise. Whiskers are 95% intervals from 1,000 bootstrap resamples of replicates: 10 on QM9 and five scaffold folds on each assay dataset. The values and their intervals are in Additional file 7.

3.3.2 Predictive performance and noise robustness

The variance decomposition tells us what aspect of QSAR modeling affects predictive performance and robustness; however, it does not tell us about the characteristics of individual models. Figure 3.3 shows that all models' performance degrades as the labels get increasingly corrupted with noise. The loss remains small until the artificial noise reaches about half the spread of the clean training labels, and then it begins to drop. The rate it drops and the quantity of noise retained are not necessarily correlated to the model's predictive performance on clean labels. Figure 3.4 shows that across the 13 base models under Gaussian noise at ECFP4, clean R^2 and $\mathrm{AUC}_{\mathrm{norm}}$ are barely correlated. The Spearman correlation is -0.10 ($p = 0.73$). If anything, the lower a model's predictive performance on clean labels, the less it has to lose as the labels get

noisier.

Figure 3.4 also lets us examine the link between predictive performance and noise robustness by model family. The forests, boosted trees, SVM, and GP all retain relatively high predictive performance on clean labels while maintaining relatively strong robustness to noise. NN-β and BNN-β are among the highest in predictive performance on clean labels, and the plain NNs and BNNs are the four least robust. The two variational networks sit between these groups, with lower predictive performance on clean labels and middling noise robustness. Standard feed-forward networks can make strong predictions under clean labels; however, they are not a safe bet under noisy ones.

NGBoost is a standout case in the sense it is highly dependent on its paired molecular representation. Under Gaussian noise it has the highest AUC_{norm} on every one of the six representations, yet also the lowest in predictive performance on four of the six representations. It is one of the worst performing models with respect to R^2 on clean data. Its high robustness is partially due to the fact that when you start with a low predictive performance, you don't have as much to lose. However, it maintains shallow robustness curves across representations. On ECFP4 NGBoost's predictive performance ranks lowest up to a noise level of 0.75 and stays in the lower half at every noise level. However, on PDV it starts second lowest and ends highest, surpassing the other models between noise levels 1.0 and 1.5. So when paired with the right features, NGBoost has the potential to make better predictions under higher quantities of label noise. On grouped-shifted noise, RF has higher AUC_{norm} values than NGBoost on ECFP4, Avalon, and ChemBERTa. Even then, NGBoost reports the highest noise robustness for PDV, MHG-GNN, and SNS. While NGBoost performs well under higher label noise, LightGBM performs well under smaller quantities of noise. Its performance drops sharply after noise level 1.0.

As we've seen, the choice of representation can have an impact on noise robustness, but that impact is determined by model family, represented by the interaction term between model and representation in Figure 3.5. Some models are more impacted by representation than others, though. In general, networks span a wider range of AUC_{norm} across the six representations than any of the other models. Between ECFP4 and PDV under Gaussian noise, the four plain and Bayesian networks differ by between 0.037 and 0.065 of AUC_{norm}, while no tree or kernel model shifts by more than 0.011. VBLLs are an exception, as no two variational networks differ by more than 0.019.

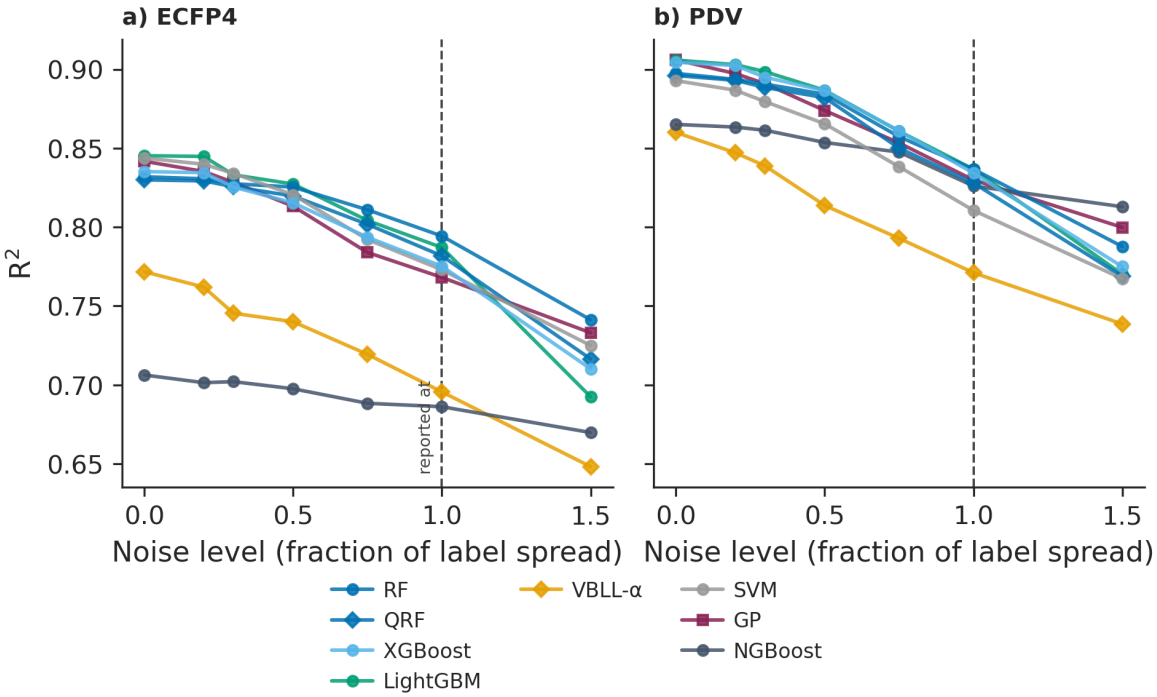


Figure 3.3: R^2 against noise level on the QM9 HOMO–LUMO gap under Gaussian noise, for the eight base models with the highest $\mathrm{AUC}_{\mathrm{norm}}$ on ECFP4: a) ECFP4; b) PDV. Medians over 10 replicates. The dashed line marks the noise level of 1.0 at which R^2 is reported.

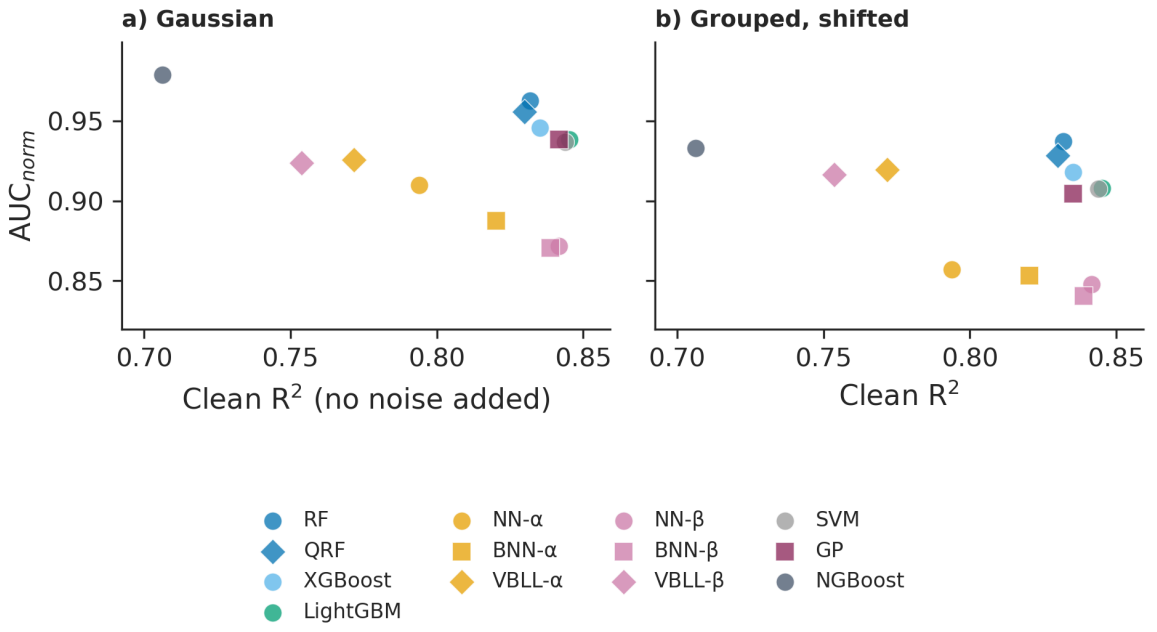


Figure 3.4: Clean R^2 against $\mathrm{AUC}_{\mathrm{norm}}$ for each base model on the QM9 HOMO–LUMO gap with ECFP4, under a) Gaussian and b) grouped-shifted noise.

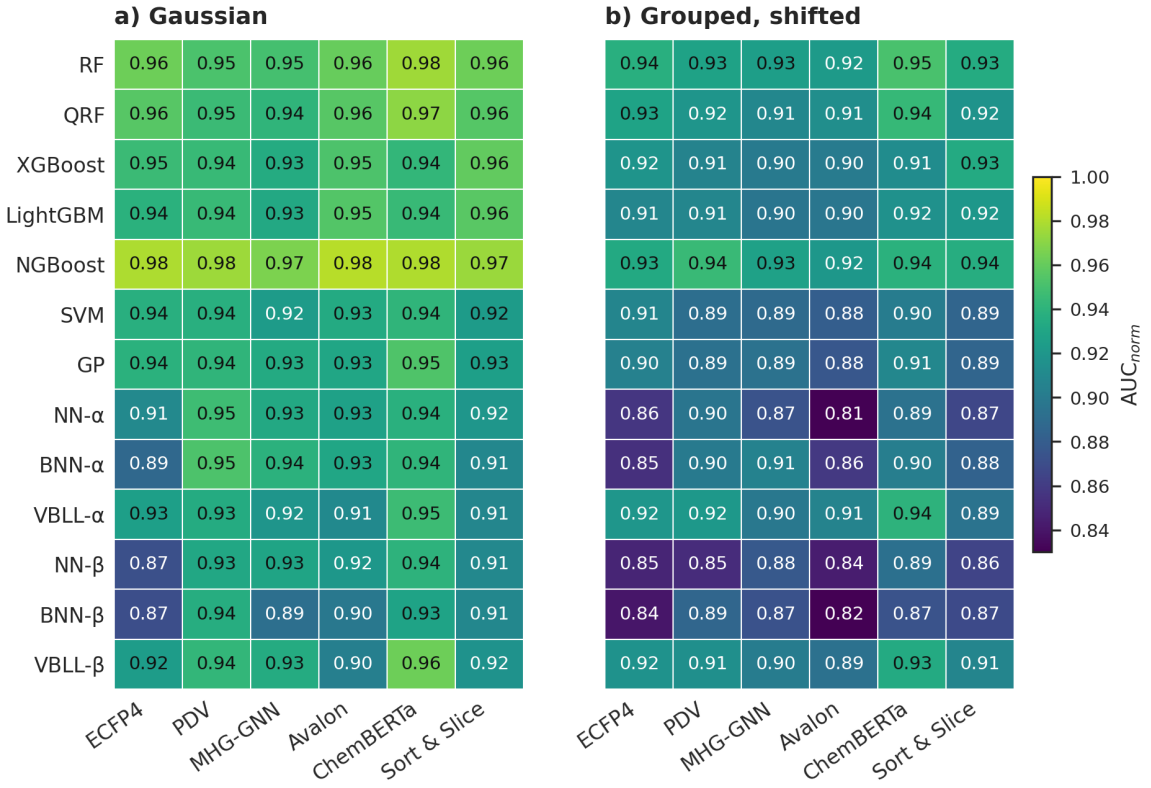


Figure 3.5: AUC_{norm} of the 13 base models on each representation, on the QM9 HOMO–LUMO gap under a) Gaussian and b) grouped-shifted noise. Medians over 10 replicates, on one color scale. No QM9 replicate was excluded (Additional file 3); grouped-wider noise is in Additional file 8.

3.3.3 Noise conditions on the HOMO–LUMO gap

Many studies involving experimental label noise rely on the assumption that noise can be modeled by a Gaussian distribution. We decided to vary the distribution which noise is drawn from, as well as the methodology by which it's injected into specific labels. Five of the seven conditions draw each label's error on its own, so the error is random. Grouped-shifted gives a whole scaffold family a shared offset, and censoring pulls every label above a limit down to that limit, so in both the error is systematic. We controlled the quantity of noise by adding it in proportion to the spread of the clean labels on the training set, so a difference between two conditions is a difference in pattern rather than amount. One condition, censoring, could not be held to a set amount as its levels count the fraction of labels clipped.

We compare Gaussian noise on the HOMO–LUMO gap to Laplace, Student-t, and outlier noise in Figure 3.6. The shape of the distribution from which the noise was drawn made little difference. SVM reported higher AUC_{norm} values on non-Gaussian label noise, 0.937 under Gaussian noise against 0.947, 0.952, and 0.955 under Student-

t, outlier, and Laplace noise on PDV, while NGBoost lost AUC_{norm} under every one. NGBoost's relatively stronger performance on Gaussian makes sense as it fits a per-molecule scale under a Gaussian likelihood, which the other distributions violate. Table 3.4 shows that on the three assay datasets with ECFP4, SVM gains AUC_{norm} under all three distributions, and NGBoost loses it in all but one case. The size of the shift stays small, which is why these results coincide with Heid et al. [18], who found no difference between error distributions drawn at a matched mean and SD. The shape of the noise does not change how much performance a model keeps, but it does change which model keeps the most.

While alternative shapes of noise may be accurately modeled by the Gaussian distribution, we do not find that is the case for systematic label noise. Table 3.3 shows that, relative to Gaussian, grouped-shifted noise lowered the AUC_{norm} across all models and representations. The models struggle to make predictions when the noise is isolated on molecules of specific scaffold families instead of being applied randomly across every sample. The variational networks retained the most predictive performance, while the plain, deterministic NNs lost the most. The predictive abilities of BNN-α degraded at a slower rate compared to its deterministic counterpart on every representation except PDV, while BNN-β did so only on MHG-GNN, PDV, Avalon, and Sort & Slice. Grouped-wider noise changed no model beyond its replicate spread except QRF on Avalon and MHG-GNN, which makes predictions particularly well under this noise condition. Censoring created the largest drop in predictive performance. In Table 3.2, AUC_{norm} was 0.12 to 0.18 lower for the same QSAR models than under Gaussian noise. Censoring levels count clipped labels, though, so the two are on different scales.

Most of these observations are being made at the upper end of the noise range. With ECFP4, six of the seven most robust models start within 0.015 of one another in R^2 for clean labels, stay together up to a noise level of 0.5, and separate only above 1.0. Figure 3.3 shows them finishing between 0.692 for LightGBM and 0.741 for RF. We ranked the models by predictive performance at each noise level. Of the 13 base models, RF started with a median rank of 6, but by level 1.0 it was ranked first. Figure 3.8 shows that SVM, on the other hand, falls from first to a median rank of 4.5 at levels 0.75 and 1.0. Because AUC_{norm} weights the whole range from 0 to 1.5 equally, and the first half of it separates little, it has the potential to understate just how much the pattern of the noise costs. However, choosing to look at performance at a set level has its drawbacks, as the ranks shift at a different pace between noise conditions.

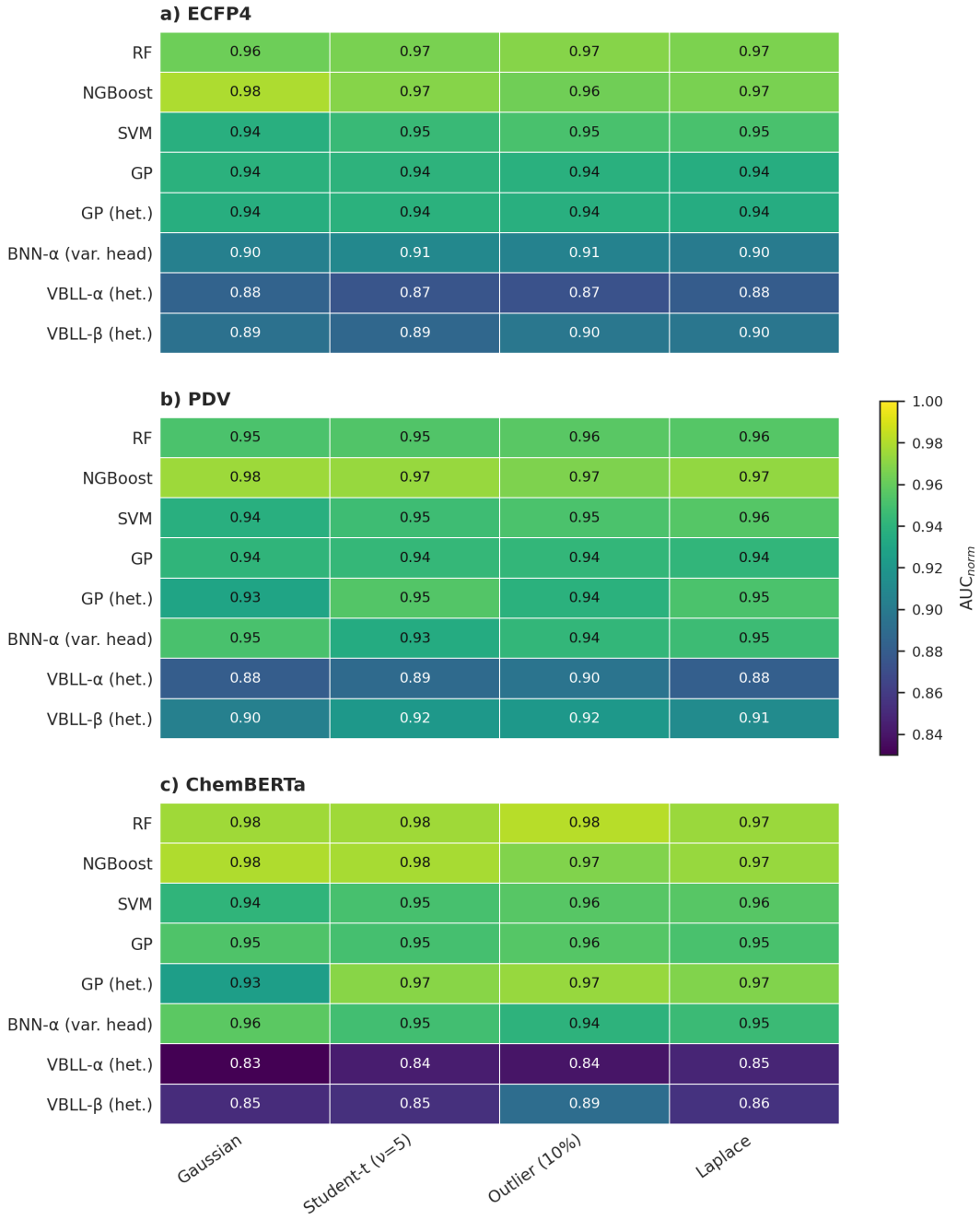


Figure 3.6: $\mathrm{AUC}_{\mathrm{norm}}$ under the noise conditions run on a subset of models, on the QM9 HOMO–LUMO gap: a) ECFP4; b) PDV; c) ChemBERTa. Medians over 10 replicates, on one color scale. Censoring is in Table 3.2.

3.3.4 Robustness on the three assay datasets

So far, we have only discussed results on the HOMO–LUMO gap from QM9, a computed property whose labels carry no measurement error. We repeated these experiments on three experimentally obtained datasets, LogD, Caco-2 [114], and hERG K_i [115], replicating our process of adding artificial noise. Unlike QM9, the experimental endpoints carry their own measurement noise in both training and test labels; the

Model	Representation	Clean R^2	Gaussian	Censoring
RF	ECFP4	0.832	0.963	0.817
GP (het.)	ECFP4	0.840	0.937	0.813
NGBoost	PDV	0.865	0.975	0.809
GP	PDV	0.906	0.941	0.815
BNN-α (var. head)	PDV	0.890	0.951	0.771

Table 3.2: AUC_{norm} under censoring and under Gaussian noise on the QM9 HOMO–LUMO gap, for the five model and representation pairs run under censoring. Medians over 10 replicates. A censoring level is the fraction of labels clipped, so the two AUC_{norm} columns are on different scales.

Model	ECFP4	PDV	MHG-GNN	Avalon	ChemBERTa	Sort & Slice	Exceeds spread
RF	-0.025	-0.024	-0.025	-0.038	-0.025	-0.028	6 of 6
QRF	-0.027	-0.027	-0.032	-0.042	-0.027	-0.038	5 of 6
XGBoost	-0.028	-0.039	-0.035	-0.052	-0.030	-0.025	5 of 6
LightGBM	-0.030	-0.037	-0.037	-0.058	-0.028	-0.037	5 of 6
NGBoost	-0.046	-0.030	-0.039	-0.062	-0.041	-0.039	6 of 6
SVM	-0.029	-0.045	-0.034	-0.053	-0.040	-0.038	5 of 6
GP	-0.034	-0.048	-0.040	-0.054	-0.045	-0.041	6 of 6
NN-α	-0.053	-0.051	-0.060	-0.115	-0.044	-0.053	4 of 6
BNN-α	-0.034	-0.053	-0.034	-0.073	-0.039	-0.025	2 of 6
VBLL-α	-0.006	-0.014	-0.020	-0.005	-0.007	-0.013	0 of 6
NN-β	-0.024	-0.080	-0.052	-0.076	-0.047	-0.045	4 of 6
BNN-β	-0.030	-0.050	-0.019	-0.075	-0.055	-0.041	2 of 6
VBLL-β	-0.007	-0.028	-0.033	-0.010	-0.028	-0.009	0 of 6

Table 3.3: Change in AUC_{norm} from Gaussian to grouped-shifted noise on the QM9 HOMO–LUMO gap, per base model and representation; negative is a loss. The last column counts the representations on which the loss exceeds the replicate spread, the highest minus the lowest AUC_{norm} over 10 replicates under Gaussian noise.

injected artificial noise therefore adds on top of an unknown noise floor. Repeat measurements of the same compound have an SD of about 0.54 log units for pK_i [12, 13]. The hERG K_i labels have a spread of 0.915 log units, so a noise level of 0.6 of that spread, between the tested levels 0.5 and 0.75, is about one unit of that estimated laboratory error. Figure 3.10 gives each model's AUC_{norm} under each of the three noise conditions run on every model on the three endpoints, with its clean R^2 beside it.

Table 3.4 shows that, as on QM9, the noise distribution does not change the predictive capabilities of a model. Laplace, Student-t, and outlier noise were run on seven models, RF, the two GPs, SVM, NGBoost, BNN-α (var. head), and VBLL-β (het.), on ECFP4, PDV, and ChemBERTa. Each delta below is measured from the same model's AUC_{norm} under Gaussian noise. RF and the two GPs never lose more than

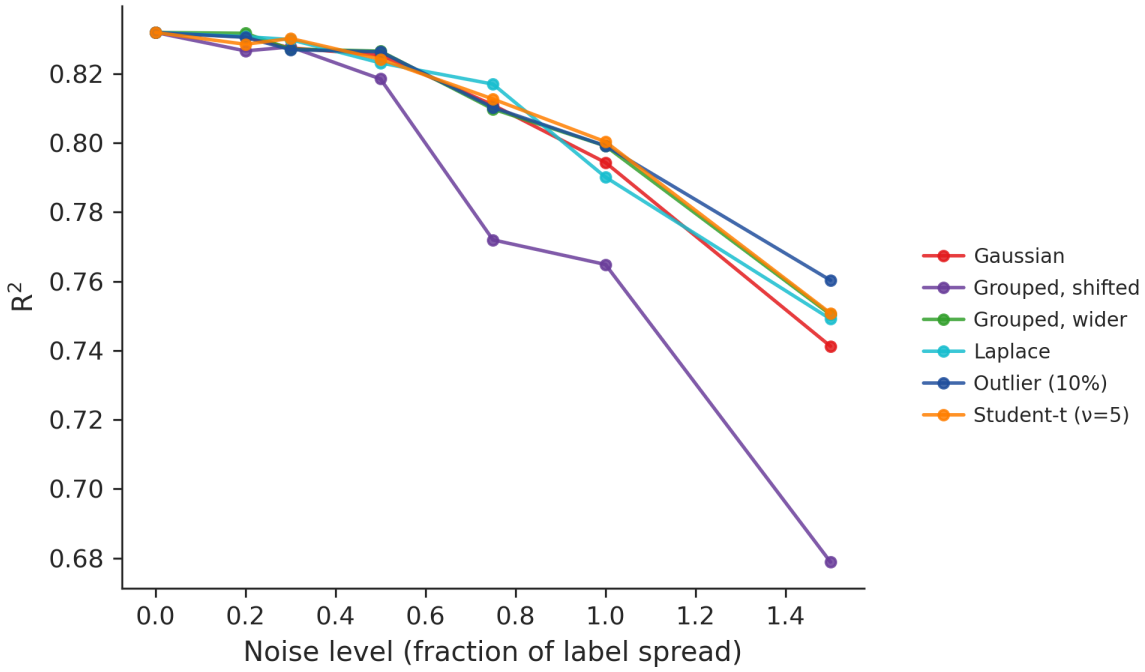


Figure 3.7: R^2 of RF against noise level under each noise condition, on the QM9 HOMO–LUMO gap with ECFP4. Medians over 10 replicates. Censoring is not shown, as its level is a fraction of labels clipped.

0.03 of AUC_{norm} under these three conditions or under grouped-wider noise, on any dataset or representation. SVM and NGBoost again move in opposite directions; SVM never loses AUC_{norm} under any of the four, while NGBoost loses under all four on Caco-2 and hERG K_i. The two networks move furthest in both directions. It's worth remembering that while we are injecting artificial noise, the assay measurements carry their own source of noise whose properties are unknown, so no "clean" label is truly clean.

In Figure 3.10, of the three noise conditions run on every model, grouped-shifted noise gives the lowest AUC_{norm} values on every assay dataset. Under it, RF ranks first or second for AUC_{norm} with every representation on LogD and with four of six on hERG K_i. R^2 at a noise level of 1.0 depends more on clean performance. By that measure, SVM ranks first to third on LogD, while RF ranks fifth to ninth across datasets.

LightGBM has particularly varied results across the datasets. On LogD, it has relatively high predictive performance and middle-of-the-pack noise robustness. However, on hERG and Caco-2, it loses the most with increased noise, particularly on Gaussian and grouped-shifted noise. The effects are not as pronounced on grouped-wider, although it still ranks among the least robust there. XGBoost degrades in the same way on hERG and Caco-2.

While the results are not identical between the datasets, Figure 3.11 shows some

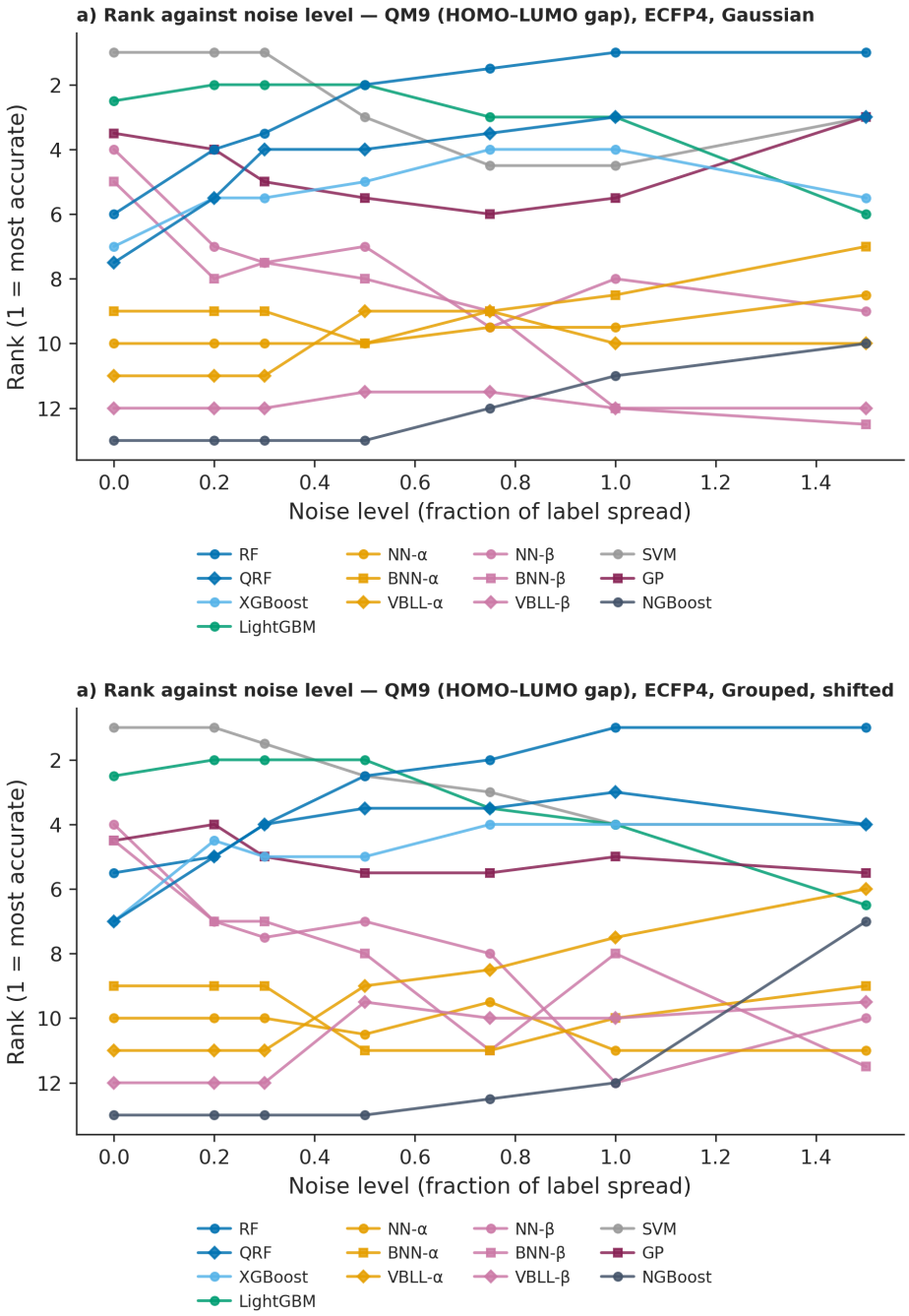


Figure 3.8: Median rank of each base model by R^2 at each noise level, on the QM9 HOMO–LUMO gap with ECFP4, under a) Gaussian and b) grouped-shifted noise. Models are ranked within each replicate; tied medians give half ranks.

Dataset	Model	Gaussian	Laplace	Student-t (ν=5)	Outlier (10%)	Grouped, wider
LogD	RF	0.935	+0.018	+0.006	+0.007	+0.021
LogD	NGBoost	0.924	0.000	-0.028	-0.032	-0.009
LogD	SVM	0.879	+0.044	+0.023	+0.027	+0.044
LogD	GP	0.891	+0.004	+0.003	-0.001	+0.006
LogD	GP (het.)	0.896	0.000	-0.002	-0.007	-0.003
LogD	BNN-α (var. head)	0.881	-0.013	-0.013	-0.024	-0.029
LogD	VBLL-β (het.)	0.796	+0.010	+0.012	+0.020	+0.068
Caco-2	RF	0.865	+0.035	+0.028	+0.026	+0.053
Caco-2	NGBoost	0.878	-0.069	-0.038	-0.060	-0.147
Caco-2	SVM	0.841	+0.049	+0.051	+0.031	+0.018
Caco-2	GP	0.809	+0.039	+0.034	+0.020	+0.023
Caco-2	GP (het.)	0.815	+0.035	+0.032	+0.011	+0.019
Caco-2	BNN-α (var. head)	0.787	-0.110	+0.092	-0.043	-0.017
Caco-2	VBLL-β (het.)	0.707	+0.036	+0.025	+0.015	+0.032
hERG K_i	RF	0.898	+0.015	-0.006	+0.016	-0.012
hERG K_i	NGBoost	0.842	-0.031	-0.027	-0.042	-0.093
hERG K_i	SVM	0.859	+0.051	+0.035	+0.025	+0.055
hERG K_i	GP	0.831	+0.022	+0.034	+0.025	+0.030
hERG K_i	GP (het.)	0.843	+0.010	+0.036	+0.015	+0.021
hERG K_i	BNN-α (var. head)	0.809	-0.034	-0.032	-0.063	-0.009
hERG K_i	VBLL-β (het.)	0.836	+0.007	-0.047	+0.005	-0.002

Table 3.4: AUC_{norm} under Gaussian noise, and its change under each other noise condition, for a subset of models on the three assay datasets with ECFP4; negative is a loss. Medians over five scaffold folds.

shared patterns. The ranks in this paragraph are among the 13 base models, while the figure ranks among 20, the 19 models of this chapter and a GP with a Tanimoto kernel. In general, the relative AUC_{norm} model rankings match between QM9 and LogD, as well as between Caco-2 and hERG K_i with every representation except ChemBERTa. RF is the most consistent; among the 13 base models under Gaussian noise it ranks between first and seventh for AUC_{norm} on every dataset and representation, second or third on QM9, and first on LogD at five of six. The GP has the highest predictive performance on clean labels on all three assay datasets with nearly every representation, and ranks third to ninth for AUC_{norm}. NGBoost, the most robust model on QM9, ranks first to third on LogD, second to tenth on Caco-2, and sixth to ninth on hERG K_i. LightGBM falls furthest, ranking 10th to 13th on Caco-2 and hERG K_i at every representation while being among the highest in predictive performance on clean labels. VBLL-α is the opposite, ranking first or second for AUC_{norm} on hERG K_i and last on LogD at every representation, from one of the lowest clean R^2 values. This fits the pattern of the NNs being exceptionally varied.

In Figure 3.10, the forests, RF and QRF, sit in the middle for predictive performance on clean labels, but they are the most reliably robust to noise across all datasets. Their rank barely changes between noise conditions, so no pattern of noise suits them better than another. Caco-2 is the exception, where RF ranks third to seventh rather than first or second.

Other models perform conditionally well, and in some cases that condition is the representation they are paired with. The GP has some of the higher predictive capabilities on clean labels and under Gaussian noise across assay datasets. It is the highest on LogD and Caco-2 under grouped-wider noise as well, but under grouped-shifted noise it falls. The choice of representation has little impact on GP, moving its AUC_{norm} less than repeating the split does on every dataset.

There is a slight consideration with using both AUC_{norm} and predictive performance side by side. AUC_{norm} measures the fraction of its own clean R^2 that a model keeps, so predictive performance on clean labels can work against it. A model that already predicts poorly has little to lose and often scores a higher AUC_{norm}, while a model that starts with strong performance has more to lose. The GP and NGBoost are the clearest cases. The GP has among the highest predictive performance on clean labels and a middling AUC_{norm}, while NGBoost has the highest AUC_{norm} on QM9 from one of the lowest clean R^2 values. While NGBoost's predictive performance is highly dependent on its representation, its robustness is not. On QM9 it is both the most robust model and the lowest ranked model for predictive performance up to a noise level of 0.75 with ECFP4. But with PDV it's able to retain some predictive performance under noise, finishing highest across all the models at noise level 1.5.

Upon first glance, SVM does not seem like a standout; however, it is the model whose standing is better with non-Gaussian noise. RF, the GP, NGBoost, and SVM are the four models run under every condition except censoring. In Table 3.4, SVM has the lowest AUC_{norm} of the four under Gaussian noise on LogD and the second lowest on Caco-2 with ECFP4. It rises to first or second under Laplace, Student-t, and outlier noise and under grouped-wider noise in all but one case. Gaussian noise is actually the one condition it is worst at. Its AUC_{norm} stays in the middle, partially because it starts high, and it ranks among the five highest for predictive performance at a noise level on every assay dataset under every noise condition.

The feed-forward networks are the most variable, although in general they have some of the highest predictive performance under clean labels and are generally not robust to noise. They are also dependent on the choice of representation. VBLL networks perform poorly on clean labels, are the least robust to noise on LogD, yet are

the most robust on Caco-2 and hERG K_i. The gradient-boosted trees, LightGBM and XGBoost, were also quite variable, with strong predictive performance on clean labels and poor noise robustness on Caco-2 and hERG K_i.

The quantity of noise is an important factor as well. At a noise level of 0.5 the models still rank by predictive performance much as they do on clean labels, while by 1.0 the order has changed, most of all on hERG K_i and then QM9. The order of models by $\mathrm{AUC}_{\mathrm{norm}}$ is broadly similar across noise conditions, with the forests near the top under each.

While it is clear that QSAR model performance drops with label noise, different models drop faster than others. How much noise a dataset tolerates before anything happens is itself a property of the dataset. Up to a noise level of 0.3, three tenths of the spread of the clean labels, the models are largely unaffected by the noise on the HOMO–LUMO gap and on LogD, where every base model with ECFP4 still keeps at least 93% of its clean R^2; on Caco-2 one of the 13 and on hERG K_i four of the 13 are already below 90% at that level. By a noise level of 0.5 the lowest retained fraction is 90% on the HOMO–LUMO gap against 76% on hERG K_i, as Figure 3.9 shows. Boosted tree ensembles are some of the first to fall. On Caco-2 with ECFP4, LightGBM has among the highest predictive performance on clean labels. It is also the first tree ensemble to fall below 90% of that performance, at a noise level of 0.5. By level 1.5 it retains 26% of its original R^2, compared to 63% for RF on ECFP4. XGBoost drops off at the next recorded noise level. However, NGBoost is the opposite, having some of the lowest predictive performance on clean labels and the flattest curve with injected label noise across all models. On the HOMO–LUMO gap it never falls below 90% of its clean R^2 at any level tested. The networks drop at an inconsistent rate. VBLL-α is the earliest to fall on LogD, dropping below 90% of its clean R^2 at a level of 0.5 and reaching 38% by 1.5. Yet on Caco-2 and hERG K_i it is among the flattest models.

The variance decomposition of predictive performance in Figure 3.2 and Additional file 7 tells us how much the choice of model and representation matters under clean labels. Extending that concept, we are also interested in how these shares change under increasing label noise and compare to the variance decomposition of $\mathrm{AUC}_{\mathrm{norm}}$. The replicate and residuals account for more variance across all datasets. On Caco-2 and hERG K_i the replicate, which on these datasets is one of five scaffold folds, is the largest share of predictive performance before and after noise is injected. Conversely, it falls well below the choice of model family for robustness. On every dataset the interaction between model and representation explains more of the variance in $\mathrm{AUC}_{\mathrm{norm}}$ than

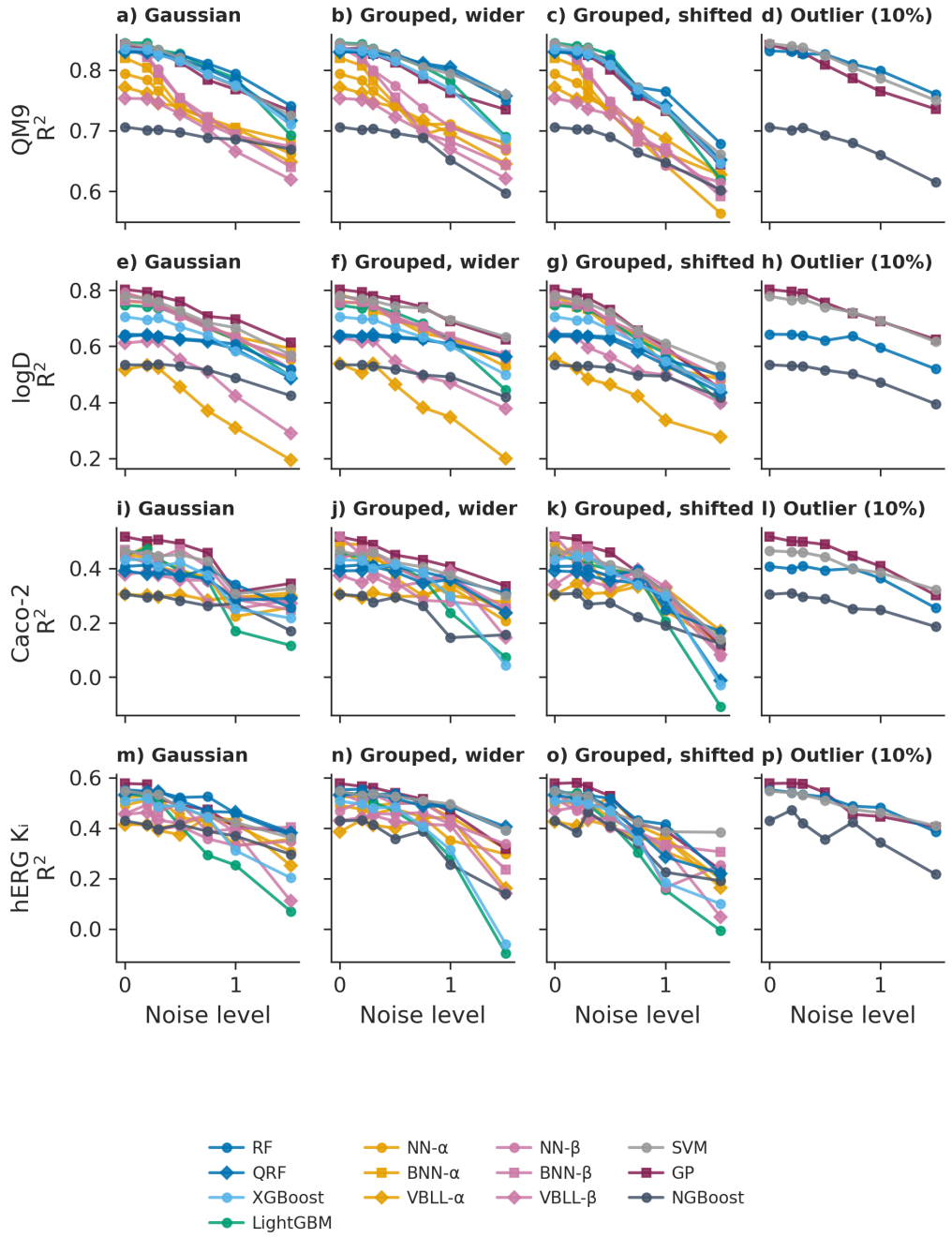


Figure 3.9: R^2 against noise level for the 13 base models on every dataset with ECFP4. Rows are datasets and columns noise conditions: a)–d) QM9 HOMO–LUMO gap; e)–h) LogD; i)–l) Caco-2; m)–p) hERG K_i. The outlier column (d, h, l, p) draws only RF, SVM, GP and NGBoost. Medians over 10 replicates on QM9 and five scaffold folds on the assay datasets.

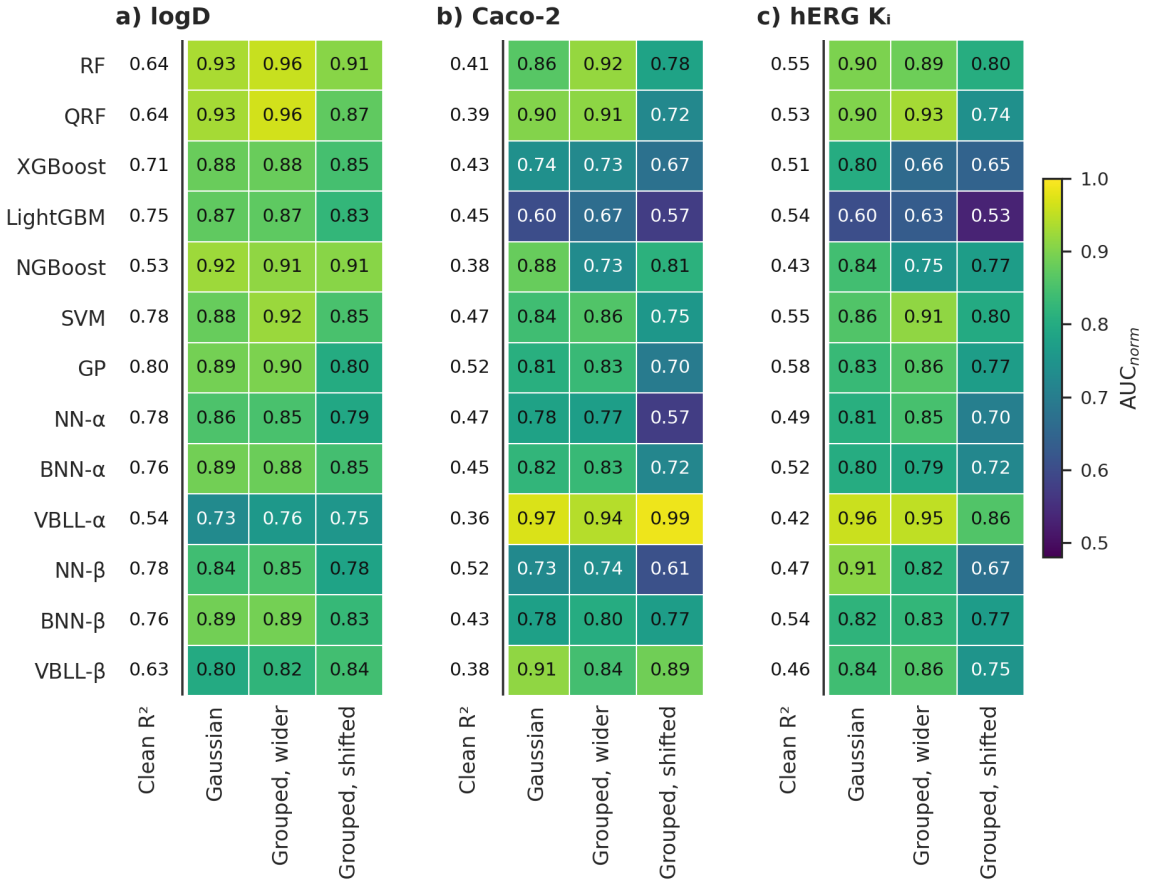


Figure 3.10: AUC_{norm} of the 13 base models with ECFP4 on a) LogD, b) Caco-2 and c) hERG K_i, under the three noise conditions run on every model. The first column is clean R^2. Medians over the scaffold folds that pass the clean-R^2 threshold (Additional file 3), on one color scale. The variant models are in Additional file 9.

representation alone.

3.3.5 Probabilistic and deterministic counterparts

To further explore the impact of model architecture on noise robustness, we examined deterministic and probabilistic variants of the same core model architecture. The hypothesis was that a model that reports a distribution rather than a point value may be more robust to small amounts of injected label noise. We compared variants across four sets of base models, shown in Figure 3.12: two NNs labeled NN-α and NN-β, RF, and the GP. For each NN, we transform a standard deterministic network into a BNN. The BNNs with a variance head run at different settings from their Bayesian bases, so they are compared only for uncertainty. We compared RF to its probabilistic counterpart, QRF, with the same baseline architecture and hyperparameters. Comparisons of this nature are only possible when the variants carry the same underlying settings, which

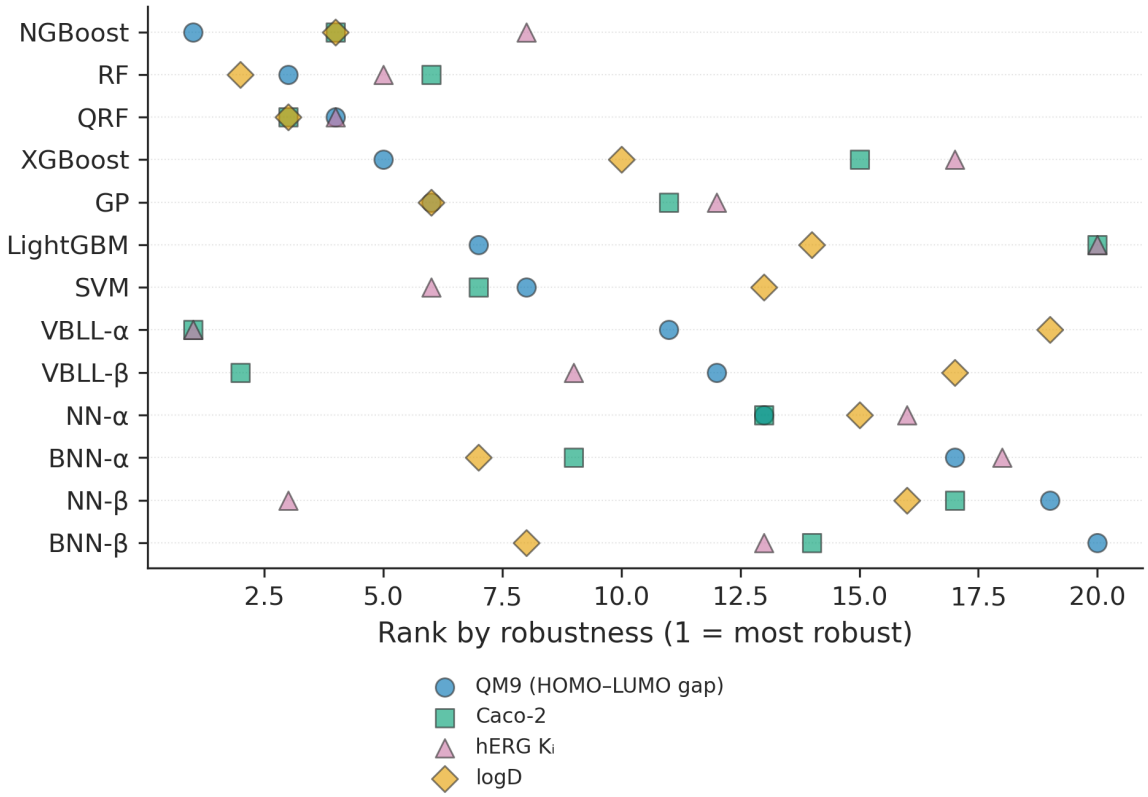


Figure 3.11: Rank by $\mathrm{AUC}_{\mathrm{norm}}$ of each base model on every dataset, with ECFP4 under Gaussian noise. Models are ordered by their rank on the QM9 HOMO–LUMO gap. Ranks are among 20 models, the 19 of this chapter and a GP with a Tanimoto kernel; only the 13 base models are drawn.

Additional file 2 lists. NN-α and BNN-α therefore share one setting, as do NN-β and BNN-β, and QRF is compared with an RF of the same 300 trees.

We compare RF and QRF directly with the same underlying architecture. QRF had lower $\mathrm{AUC}_{\mathrm{norm}}$ on QM9 under Gaussian and grouped-shifted noise on every representation, and changed it little under grouped-wider noise, with almost no change in clean R^2. Additional file 10 gives the assay datasets. There, QRF was less robust than RF under grouped-shifted noise on every dataset and representation, and under Gaussian noise on LogD and hERG K_i. Under grouped-wider noise it was more robust on Caco-2 and mixed elsewhere.

The Bayesian transformations that we used to transform feed-forward deterministic NNs into probabilistic ones produced a slightly larger $\mathrm{AUC}_{\mathrm{norm}}$, alongside lower predictive performance on clean labels. BNN-β kept more $\mathrm{AUC}_{\mathrm{norm}}$ than NN-β in most combinations on every assay dataset, and BNN-α on all assay datasets. On QM9 the Bayesian transformations made no significant change on either predictive performance or $\mathrm{AUC}_{\mathrm{norm}}$ for either network.

Replacing the GP with GP (het.) changed $\mathrm{AUC}_{\mathrm{norm}}$ by at most 0.055 on every dataset and representation. It left clean R^2 almost unchanged on the assay datasets, and on QM9 lowered it by up to about 0.06.

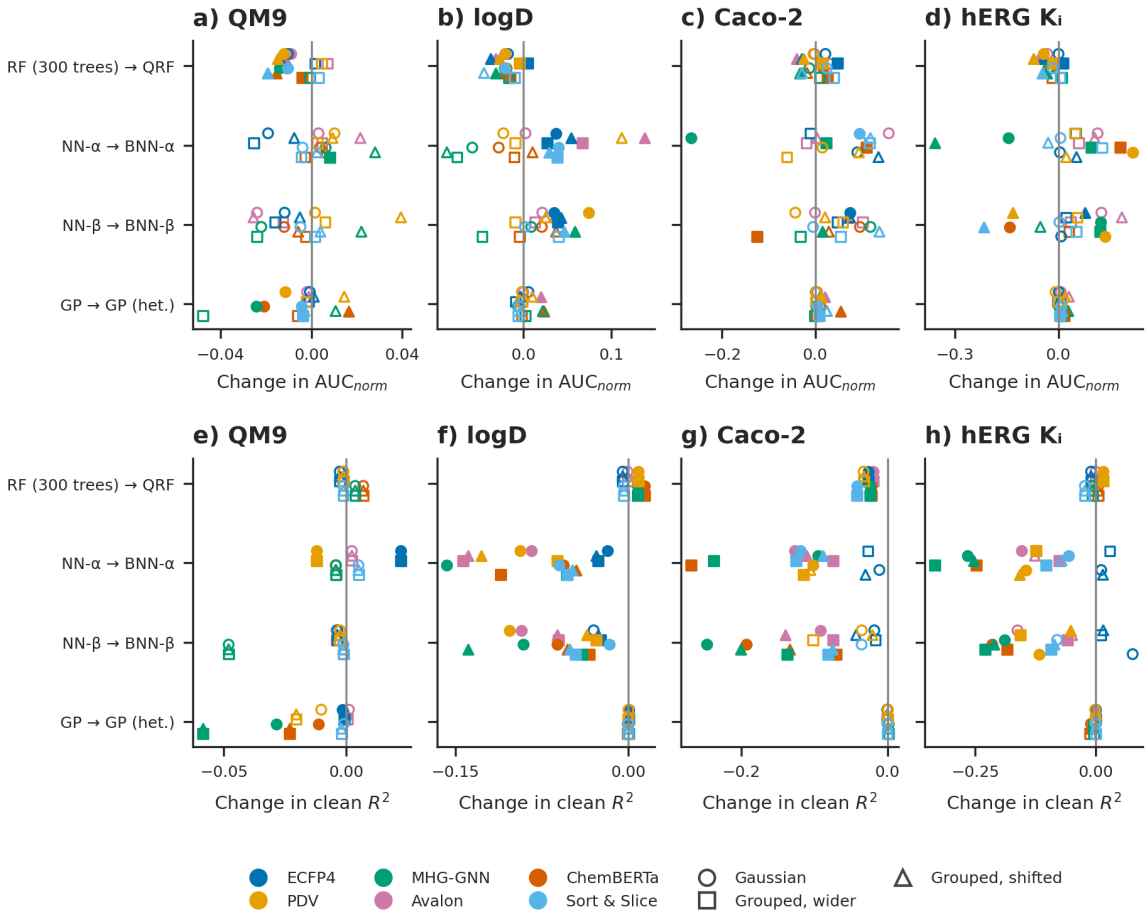


Figure 3.12: Change in $\mathrm{AUC}_{\mathrm{norm}}$ (top) and clean R^2 (bottom) from each model to its probabilistic counterpart at the same settings, on every dataset: counterpart minus base, the median of paired differences over 10 replicates on QM9 and five scaffold folds on the assay datasets. Color is the representation and shape the noise condition. A filled mark changed in the same direction in every replicate or fold.

3.3.6 Uncertainty under increasing label noise

Until now we've discussed how much predictive capability a model is able to retain while its training labels are corrupted with noise. Now, we consider how confident the model reports its own predictions are in the presence of noise, and whether or not those estimates match the increased label noise. Kolmar and Grulke [10] found that the mean predicted uncertainty derived from the GPs increases with the amount of label noise in the training data. We go beyond the population level and instead ask if a model's per-sample uncertainty tracks label noise. One key sign that a model is

able to handle noise is its ability to track it, represented by the per-sample Spearman correlation between predicted uncertainty and noise magnitude.

Table 3.5 shows that, as expected, mean predicted uncertainty is positively correlated with Gaussian label noise for most models and representations. There are a few exceptions, namely BNN-β on PDV and VBLL-β (het.) on Avalon for QM9, and BNN-β on PDV for LogD. BNN-α fails to rise in at least one fold on every assay dataset, though with PDV only on LogD and Caco-2, and both full-BNNs with a variance head fail on MHG-GNN for hERG K_i, and BNN-β with a variance head also fails on ChemBERTa and on Caco-2. Kolmar and Grulke [10]'s finding therefore holds for almost every probabilistic model on the computed property, and for most of them on measured labels.

Model	QM9	LogD	Caco-2	hERG K_i	Aleatoric	Epistemic
QRF	4.9	2.7	2.4	2.3	per molecule	per molecule
NGBoost	4.8	3.2	2.6	2.6	per molecule	none
GP	5.3	4.1	2.5	2.6	one per fit	per molecule
GP (het.)	—	4.3	2.7	2.8	per molecule	per molecule
BNN-α	3.4	1.4†	1.0†	1.4	none	per molecule
BNN-α (var. head)	4.1	2.7	1.9	2.1	per molecule	per molecule
VBLL-α	3.5	2.3	1.4	1.2	one per fit	per molecule
VBLL-α (het.)	3.0	1.9	2.8	3.5	per molecule	per molecule
BNN-β	2.7†	1.2†	1.1	1.2	none	per molecule
BNN-β (var. head)	4.3	2.5	2.4	1.7	per molecule	per molecule
VBLL-β	3.3	2.4	1.3	1.2	one per fit	per molecule
VBLL-β (het.)	2.6	2.0	2.7	3.1	per molecule	per molecule

Table 3.5: Mean predicted uncertainty on held-out molecules at a noise level of 1.5, divided by its value with no added noise, with PDV under Gaussian noise. †: the uncertainty fails to rise in at least one of the 10 replicates on QM9 or one of the five folds on an assay dataset. A dash: no uncertainty result under Gaussian noise; its AUC_{norm} was computed. Aleatoric and Epistemic: whether the model reports that component per molecule, as one value per fit, or not at all.

Aleatoric and epistemic attribution

While this tells us that the models are reportedly less certain under noisy data, we don't know what the model is attributing this uncertainty to. To answer this question, we decompose uncertainty estimates into their aleatoric and epistemic components wherever possible. We expect to see aleatoric uncertainty rise with injected label noise, while the epistemic component should remain relatively constant, as it is the aleatoric component that describes uncertainty in the underlying data. The models in Table 3.5 whose frameworks allowed for this decomposition include full-BNNs with a variance

head, the heteroscedastic GP, and QRF. The two heteroscedastic VBLL networks also report both components, and they separate them more often than any other model, though not on every dataset and representation. They are not drawn in the figures, as they were not scored out of fold.

Figure 3.13 shows that the full-BNN with a variance head, BNN-α (var. head), behaves as expected. The epistemic component grows by at most 0.36 of the aleatoric growth in every configuration under every noise condition except censoring. Despite QRF's noise robustness, both of its aleatoric and epistemic components rise with noise and on some representations, the epistemic component actually rises at a faster rate.

Unlike the predictive performance and robustness, the representation actually dictates whether decomposition works. For QRF, the epistemic growth is 0.61 to 1.14 times the aleatoric growth on the three fingerprints, 1.02 to 1.44 times on PDV, and 1.29 to 2.04 times on ChemBERTa and MHG-GNN. Under Gaussian noise the heteroscedastic GP keeps its epistemic growth below a quarter of its aleatoric growth on PDV, ChemBERTa, and MHG-GNN except MHG-GNN on QM9, but not on ECFP4 under grouped-shifted noise, where the two grow by similar amounts.

The noise conditions follow the pattern found in robustness: the three conditions which alternate the noise distribution show similar decompositions to those under Gaussian noise, while grouped-shifted raises the epistemic share of the heteroscedastic GP and of QRF in every pairing of dataset and representation where they were run. This scaffold family-based offset may be appearing as structure in the data rather than label noise to the models.

The aleatoric component indicates that label noise increased, but Figure 3.14 shows that it doesn't tell us by how much. For each unit of added noise it rises by roughly 0.4 to 1 unit, depending on model, dataset, and representation. The aleatoric component can therefore compare noise between datasets or noise levels, but it does not estimate a dataset's internal error.

Uncertainty estimation breaks under censoring as the process of clipping the labels narrows their spread, which in turn shrinks the total uncertainty. None of the models tested treat a clipped label as a noisy one, so they actually grow more confident while losing information.

Per-sample uncertainty and label noise

Finally, we ask whether a model's uncertainty on one molecule tracks the noise on that molecule's label. We assess this using the Spearman correlation between predicted uncertainty and the size of the injected noise over out-of-fold training molecules.

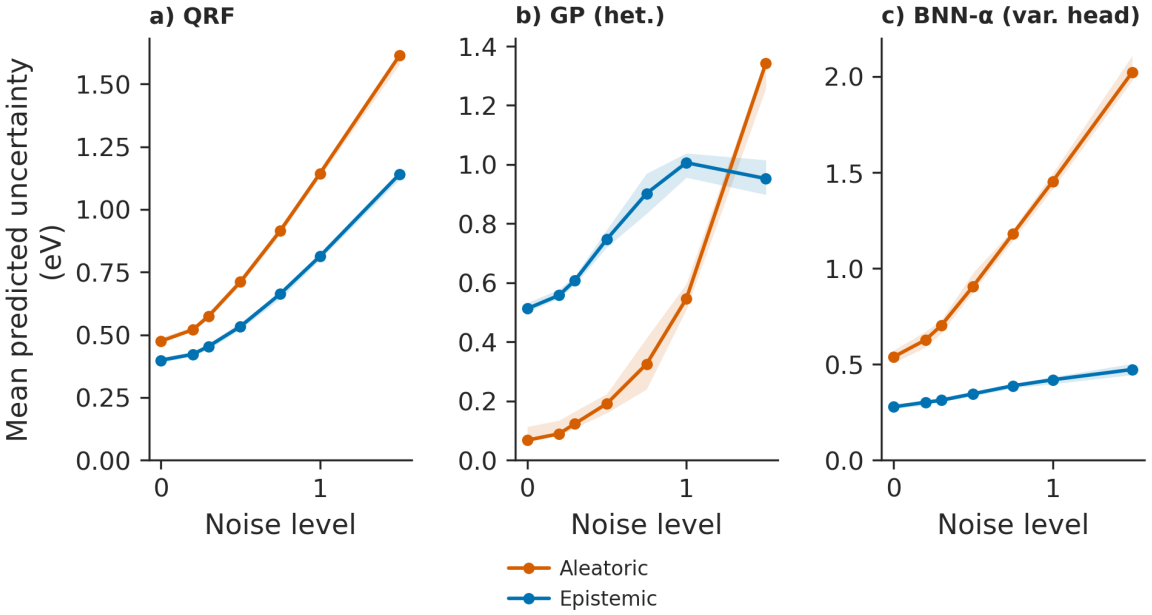


Figure 3.13: Aleatoric and epistemic components of the predicted uncertainty against noise level, one panel per model, on the QM9 HOMO–LUMO gap with ECFP4 under Gaussian noise. Each value is the mean over the molecules of one out-of-fold pass; lines are medians over folds and bands their range. A component that is one value per fit is not drawn (Table 3.5).

Since Gaussian, Laplace, Student-t, and grouped-shifted conditions add noise from the same scale to every label, there is not much for uncertainty to find. It's the same situation with outlier noise, under which a tenth of the data is chosen at random for extra noise, and with grouped-wider in which each scaffold family falls in a single fold, so a model makes predictions based on unseen scaffolds and has no basis for estimating uncertainty. The only case where per-sample uncertainty tracking is possible is censoring, as the noise is dependent on the value of the label itself. The correlations are then large and mostly negative, meaning the model is least uncertain on the labels that were clipped most. On QM9 with half the labels clipped, on PDV, they run from -0.84 to -0.80 across folds for NGBoost and from -0.47 to -0.41 for the GP, and on LogD with half the labels clipped every fold of every model except the two GPs is at -0.29 or lower. We divided each molecule's error by its uncertainty and found that the division ranked the clipped labels better than the error alone on QM9 and on most of LogD. Models are most confident on the clipped labels, probably because the clipping piles many labels into one value, which the model learns to tightly predict. Ironically, per-molecule uncertainty can be used in reverse to identify clipped labels as the labels a model is most certain about are the most likely to have been clipped.

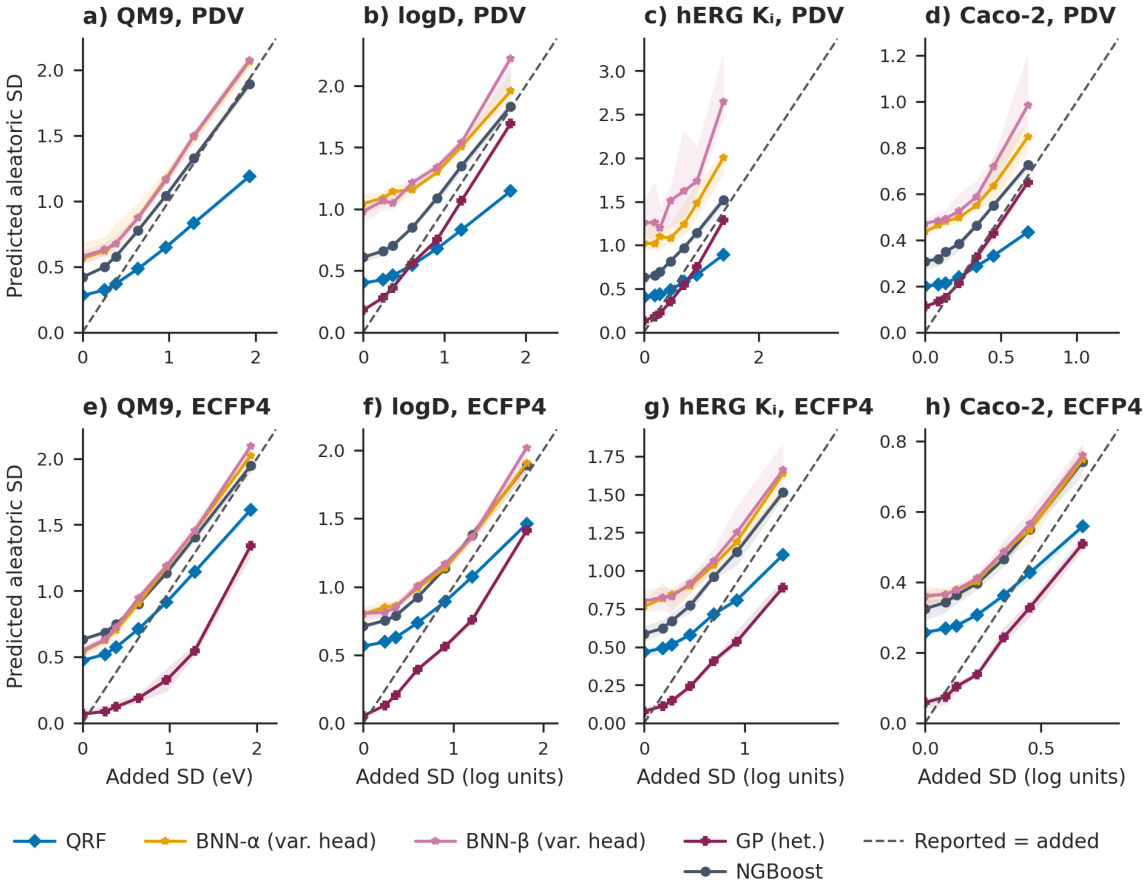


Figure 3.14: Predicted aleatoric standard deviation against the standard deviation of the added noise, under Gaussian noise. Rows are PDV (top) and ECFP4 (bottom); columns are the QM9 HOMO–LUMO gap, LogD, hERG K_i, and Caco-2. Each line is one model: the median over folds of the mean over the molecules of one out-of-fold pass, with a band spanning the folds. The dashed line is equality. The heteroscedastic GP has no uncertainty result on QM9 with PDV under Gaussian noise, and the two heteroscedastic variational networks were not scored out of fold.

3.4 Conclusions

We investigated how a variety of molecular representations and models perform under increasing label noise. We used ANOVA variance decomposition across three noise conditions on four datasets to show that molecular representation accounts for a larger share of the variance in predictive performance than in robustness. Robustness to label noise, as measured by $\mathrm{AUC}_{\mathrm{norm}}$, the normalized area under the R^2 retention curve, is more impacted by the choice of model. The representation each model is paired with explains more variance than the representation alone. As such, a representation should not be chosen independently of the model it will be used with, and some models benefit from alternative molecular representations across both predictive performance and noise robustness. Model family was the primary factor in every decomposition of

$\mathrm{AUC}_{\mathrm{norm}}$ except one, Caco-2 under grouped-shifted noise, where the scaffold fold took the larger share.

Grouped-shifted and censored noise, the two conditions in which the error is systematic, corrupt QSAR model predictions the most. Changing the shape of the distribution from which noise is drawn barely has an impact on model performance, a conclusion which is in agreement with Heid et al. [18]. Figure 3.7 shows that low quantities of noise do not make much of an impact. On the HOMO–LUMO gap every base model keeps at least 93% of its clean R^2 up to a noise level of 0.3. Grouped-shifted noise separates from the other conditions only from a level of 0.75. Models' performance begins to degrade more rapidly on the assay datasets than on the HOMO–LUMO gap, which makes sense as assay data contain inherent experimental noise. Tree and kernel models maintained consistent $\mathrm{AUC}_{\mathrm{norm}}$ across all representations, while the plain NNs and BNNs depended on the representation they were given, as did the predictive performance of NGBoost. Across datasets and noise conditions, RF and QRF were reliably robust, although their predictive performance on clean labels was middling and on Caco-2 they struggled under Gaussian noise. QRF was the slightly less robust of the two, falling below RF under Gaussian and grouped-shifted noise. Other models performed well only under some conditions. The GP had the highest predictive performance on clean labels and under independent noise but fell behind when the noise was shared within a scaffold family, NGBoost predicted well only with the right representation, and SVM led in predictive performance under scaffold-shared noise on some datasets. Transforming a feed-forward network into a Bayesian one increased $\mathrm{AUC}_{\mathrm{norm}}$ on the assay datasets alongside a lower clean R^2, and barely changed it on QM9, so switching to a tree-based model is the stronger move under label noise.

With regard to uncertainty estimation, mean predicted uncertainty rose with label noise for almost every probabilistic model on QM9 and for the majority of models on the assay datasets. Of the models whose aleatoric and epistemic components can be split per molecule, the BNNs with a variance head, the heteroscedastic GP, and the heteroscedastic VBLL networks correctly attributed error from the injected noise as aleatoric, though not on every dataset and representation, while QRF read that error as epistemic. The success of a split also depends on the representation. The aleatoric component rises by roughly 0.4 to 1 unit for each unit of noise added, so it can compare noise between datasets or between noise levels, but it does not measure a dataset's internal error. Outside censoring, no model's per-sample uncertainty identified which labels had been corrupted. Under censoring the models' most confident predictions are the clipped labels themselves, and dividing a molecule's error by its uncertainty picked

out the clipped labels better than the error alone.

QM9 served as a relatively clean baseline as its properties are computed. The assay datasets containing experimental measurements on the endpoints LogD, Caco-2 [114], and hERG K_i [115] are smaller and cover a more narrow range of the chemical space. As the labels in these data are experimentally derived and pooled from different laboratories, they contain some unknown amount of noise. We are then adding artificial noise on top of the existing experimental noise. These validation datasets supported the QM9 findings for the forests, with the boosted trees XGBoost and LightGBM the notable exceptions, degrading on the external datasets. Future work should extend this type of noise robustness benchmarking to classification tasks and larger experimental datasets with better-known uncertainty estimates.

When training on noisy experimental data in the context of QSAR, it is important to keep in mind that a model's robustness to noise is primarily driven by its own mechanism of training, and to an extent the molecular representation the input data comes in, oftentimes with respect to the model. Of the models tested, RF was the most consistent choice across the four datasets, ranking between first and eighth for $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset, representation, and noise condition. Where a predictive distribution is wanted, its probabilistic counterpart QRF maintains RF's predictive performance and nearly all of its noise robustness. What QRF cannot do is say where that uncertainty came from. Beyond the forests, the better choice depends on how much noise there is. LightGBM and XGBoost predict well on clean labels and lose the most as noise rises, so they suit data you trust, while NGBoost and the VBLL networks start lower and hold what they have, except the VBLL networks on LogD.

In this research, we define a noise level as a fraction of the spread of the clean training labels. It is therefore relative to the dataset rather than an absolute measurement error, and cannot be mapped onto a given dataset exactly. There are other mechanisms used to estimate the noise in a dataset directly. When an assay's controls are available, the Z′-factor can be used to estimate noise [90]. You can also use screening data to test for row, column, and plate effects, which produce systematic rather than independent error [129]. When you lack controls, replicates, and metadata, a label can be checked against the data itself. Confident learning, developed for classification, flags labels by comparing each label to the class probabilities a model predicts for that molecule when it was left out of training [57]. You can use the point at which the model error matches the data's error to generate a predictive limit [10, 130]. Beyond formal methods, you should consider how a dataset was assembled. How many independent replicates were done, how many laboratories contributed, whether assay protocols were consistently

applied, and how varied are the compounds in chemical space?

The first limitation of this study is data availability. While HOMO–LUMO gap from QM9 is a good baseline as its computed labels carry no measurement error, it is only a single task. As the noise is meant to replicate experimental data, this analysis would be improved by more quality-controlled, relatively clean, and well-replicated experimental data across different targets. However, data of that nature are hard to find publicly.

Second, $\mathrm{AUC}_{\mathrm{norm}}$ is a ratio to clean R^2, so a model that starts from a low clean baseline has less to lose, and a few replicates scored above 1.05. We tried other measures of noise robustness, but they were either too strongly correlated with clean R^2 or did not capture the right quantity. The change in R^2 at a single noise level, for example, misses the shape of the curve. $\mathrm{AUC}_{\mathrm{norm}}$ is not a perfect metric, but it describes the whole curve, and across the 13 base models it is barely correlated with clean R^2.

Third, it is hard to draw conclusions that hold across so many conditions. No model or representation wins everywhere, although RF comes closest. When working with cheminformatics data, you don't know in advance how a model and representation will perform on the given prediction task. The underlying data is probably noisy, but you don't know how noisy it is or what shape that noise takes.

Given that, our recommendation is to test a handful of models against a handful of representations rather than tuning a single pairing, since on every dataset the pairing explains more of the variance in robustness than the representation does alone. The pattern of the noise should also guide that choice. Under Laplace, Student-t, and outlier noise, RF was usually the most robust model and SVM usually ranked second on the assay datasets. NGBoost lost $\mathrm{AUC}_{\mathrm{norm}}$ under all three, except Laplace noise on LogD. Under grouped-wider noise, QRF was the most robust model and SVM was close behind. NGBoost lost $\mathrm{AUC}_{\mathrm{norm}}$ on every assay dataset. Grouped-shifted noise was the hardest condition. Every model lost $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset and representation. RF lost the least of the models that also predicted well, and on the HOMO–LUMO gap the plain networks lost the most. Gaussian noise was the only condition with a clear specialist. NGBoost was the most robust model on the HOMO–LUMO gap under Gaussian noise with every representation, but not on Caco-2 or hERG K_i.

These results follow from the loss function each model fits. NGBoost fits a per-molecule scale under a Gaussian likelihood, so it loses ground when the noise is not Gaussian. SVM's loss grows only linearly outside its margin, so large errors cost it less, and it gains under the same conditions. Each boosting stage fits the residuals of the stage before it, and those residuals are largest in the noisy families. The GP is the

most affected by error that is correlated with structure, because its kernel measures structure. None of this matters if the data are clean, as on the HOMO–LUMO gap and LogD little separated the models below a noise level of 0.3.

In this chapter each model was given a standard molecular representation, and the choice of representation explained relatively little of the variance in noise robustness. We observe that this choice does impact the performance of different models to varying degrees, and the representations seem to capture different signals on each dataset that we tested. Chapter 4 asks how well these representations compare molecules on a single dataset.

Chapter 4

Limitations of the similar property principle

4.1 Introduction

The previous chapters have been focused on the impact of label noise. Before we delve into techniques that are meant to train on, identify, and mitigate such label noise, we first focus on a tangential point, molecular similarity. All of the aforementioned noise-adjacent methods, as well as the QSAR modeling itself, rest on the SPP of Section 2.3. The SPP posits that structurally similar compounds display similar chemical properties [78]. This serves as the conceptual foundation for QSAR modeling. The concept of molecular similarity is also used in various contexts to compare neighboring molecules. This comes into play when identifying noisy molecules that in theory stick out from their neighbors. A label is judged against the labels of its nearest molecules, so the features that decide which molecules are nearest also decide which labels look noisy. There are some important exceptions to the SPP, namely activity cliffs (ACs) [63, 82]. Furthermore, there is no single way to define similarity between compounds. This is where the principle begins to break down in practice. When defining similarity between two molecules, we typically rely on some pairwise mathematical function, or coefficient, that takes in two molecular representations as input. Two factors vary here, the representation and the coefficient. Bajusz et al. [80] compared eight coefficients and found Tanimoto, Dice, Cosine, and Soergel the best and largely equivalent, while similarities derived from Euclidean and Manhattan distances were not recommended on their own. In this chapter we ask if the standard methods of comparing molecules satisfy the SPP, that is, whether they quantify how similar two molecules actually are.

General-purpose molecular representations are designed to work across chemical

domains. Consequently, when considering data from a specific domain, many of their features show little to no variance. Some representations assign separate molecules identical feature vectors. Since common fingerprints do not encode stereochemistry in their default settings [131], they are often unable to differentiate stereoisomers, which can have substantially different measured activity [132]. As these coefficients compare every feature, irrelevant features dilute the result. Different representations also disagree about which molecules are close based on the information they carry.

The problem of whether neighboring molecules can predict chemical activity has been measured before. The modelability index of Golbraikh et al. [133] scores a binary dataset by a class-weighted share of nearest-neighbor pairs that share an activity class, producing a single value for how strongly the similarity assumption holds on that particular combination of representation and data. It is not clear whether a low value is due to the data being intrinsically hard to model or to the representation not capturing adequate features for the task. Other work in this area relates to ACs. Sheridan et al. [134] compared experimental error, activity distribution, and AC metrics as predictors of how well a dataset can be modeled, and found the cliff metrics the strongest of the three. However, they found that these metrics cannot separate real cliffs from those produced by uncertain activity measurements. Medina-Franco [135] points out that some of these ACs are artifacts of the chosen representation, similarity threshold, or noise in the data. ACs are distinct from noise, as they carry some of the most valuable information about the interaction between structure and activity. None of these measures ask specifically about whether the specific features within these representations matter for the property at hand.

We first consider how much the representation impacts similarity measures. Riniker and Landrum [136] benchmarked 12 fingerprints and two simple baseline fingerprints for similarity-based virtual screening on 88 targets from three public collections, and found that the screening performance of the 12 fingerprints was highly correlated. Only the two baselines differed significantly from the rest. Nearly all of 25 pretrained neural embeddings were found to have a negligible improvement over ECFP across 25 datasets; only CLAMP was significantly better [137]. These comparisons, like this thesis, use representations fixed before the task is known, such as ECFP or a frozen pretrained embedding. Such a representation was never tuned to the property, so it need not give a smooth property landscape. We would expect a representation learned on the task's labels, such as a network's last hidden layer, to give a smoother one, in which the remaining cliffs are more likely real. However, when looking into the differences between the structure of fingerprints, Huber and Pollmann [138] found that fingerprints

which store raw counts are more effective at defining similarity than folded fingerprints when compared against a graph-based structural reference. Folding introduces bit collisions, which disproportionately distort similarity for molecules with many set bits [138]. ACs follow the same pattern: across 15 configurations of representation and similarity measures on 3 datasets, the representations disagreed about which pairs form cliffs [139], supporting the point Medina-Franco [135] makes about ACs being artifacts of their representation.

Any method that incorporates similarity measures, such as AC detection, or cleaning a training set by comparing each sample's label with its nearest neighbors' labels, whether by removing the samples that disagree [106] or by smoothing their labels, inherits this choice, as do the noise estimation and mitigation methods of this thesis. Coupry and Pogány [140] learn a molecular similarity from graph embeddings, but independently of any endpoint. Zheng and Tropsha [21] did choose the similarity for the task. They predict each molecule's activity as the average over its k most similar molecules by Euclidean distance over descriptors, and choose the descriptors by simulated annealing to maximize the leave-one-out q^2. Itskowitz and Tropsha [22] also optimize the number of neighbors and their weighting. Both select features for a nearest-neighbor model of one endpoint, whereas this thesis selects features across several representations.

The purpose of this chapter is to establish the problem that Chapter 5 sets out to solve. Section 4.2.1 asks, for a given dataset, how much the features of standard representations vary and how many molecules map to identical feature vectors. Section 4.2.2 asks what happens when these features are used to quantify similarity, and Section 4.2.3 follows the consequences for methods built on these similarity measures, including a small sweep of LNL methods.

4.1.1 Distance metrics for continuous molecular descriptors

While binary fingerprints are often compared with the Tanimoto coefficient introduced in Chapter 2, we need separate distance metrics for continuous feature vectors. The small sweep of LNL methods in Section 4.2.3 used the following four:

- Euclidean (L_2): standard geometric distance, $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$
- Manhattan (L_1): sum of absolute differences, $d(x, y) = \sum_i |x_i - y_i|$, more robust to outlier features
- Cosine: angle-based, $d(x, y) = 1 - \frac{x \cdot y}{\|x\| \|y\|}$, invariant to magnitude

- Mahalanobis: covariance-adjusted distance, $d(x, y) = \sqrt{(x-y)^\top S^{-1}(x-y)}$, where S is the feature covariance estimated on the training molecules [141]

All features are standardized to zero mean and unit variance prior to distance computation. Each value x of a feature is replaced by $(x - \mu)/s$, where μ and s are that feature's mean and SD over the molecules.

4.2 Results

4.2.1 Flaws in standard representations

General-purpose representations are built to span a broad range of chemical domains. When a dataset covers only a small region of the chemical space, a lot of the features within these generalized representations become irrelevant, describing chemistry which is absent from the data at hand. Pairwise comparisons of these representations rely on these irrelevant features.

Constant and near-constant features

To investigate similarity estimation, we first look at a simple measure: how many features vary within a dataset. A single dataset often contains a set of relatively similar molecules. Many features within a generalized representation contain information that describes all, or almost all, of the molecules in the set, failing to capture the variation between them. We define a constant feature to be one that takes a single value in every molecule of a dataset and a near-constant feature to be one that carries the same value in at least 95% of the molecules. We counted constant and near-constant features from a set of representations described in Table 5.12 across hERG K_i, Caco-2, LogD, QM9, MDR1–MDCK (multidrug resistance protein 1 in Madin–Darby canine kidney cells) efflux, human liver microsome (HLM) clearance, rat liver microsome (RLM) clearance, and the Lhasa hERG blocker training set. MDR1–MDCK efflux and the two clearance datasets come from Biogen [142]. HLM clearance is not in Table 5.12, and the Lhasa hERG set appears there as hERG blocking; for both, the representations counted are those with counts in Additional file 12.

The substructure-based fingerprints have the most near-constant features. Figure 4.1 and Additional file 12 show that the learned embeddings, which are taken from the last layers of a NN, have none except MHG-GNN. ECFP4 is the worst offender. On hERG K_i (1,412 molecules) 1,815 of the 2,048 features are near-constant. That

jumps to 1,997 out of 2,048 near-constant features for the HOMO–LUMO gap on QM9. Almost all of these bits are near-constant because they are absent. On hERG K_i, 1,812 of the 1,815 near-constant bits are off in at least 95% of molecules, and the other 3 are on in at least 95%. 292 of MHG-GNN's 1,024 features are zero in every molecule across all seven datasets that used it. They come directly from the saved encoder output rather than the data, and the cause was not investigated.

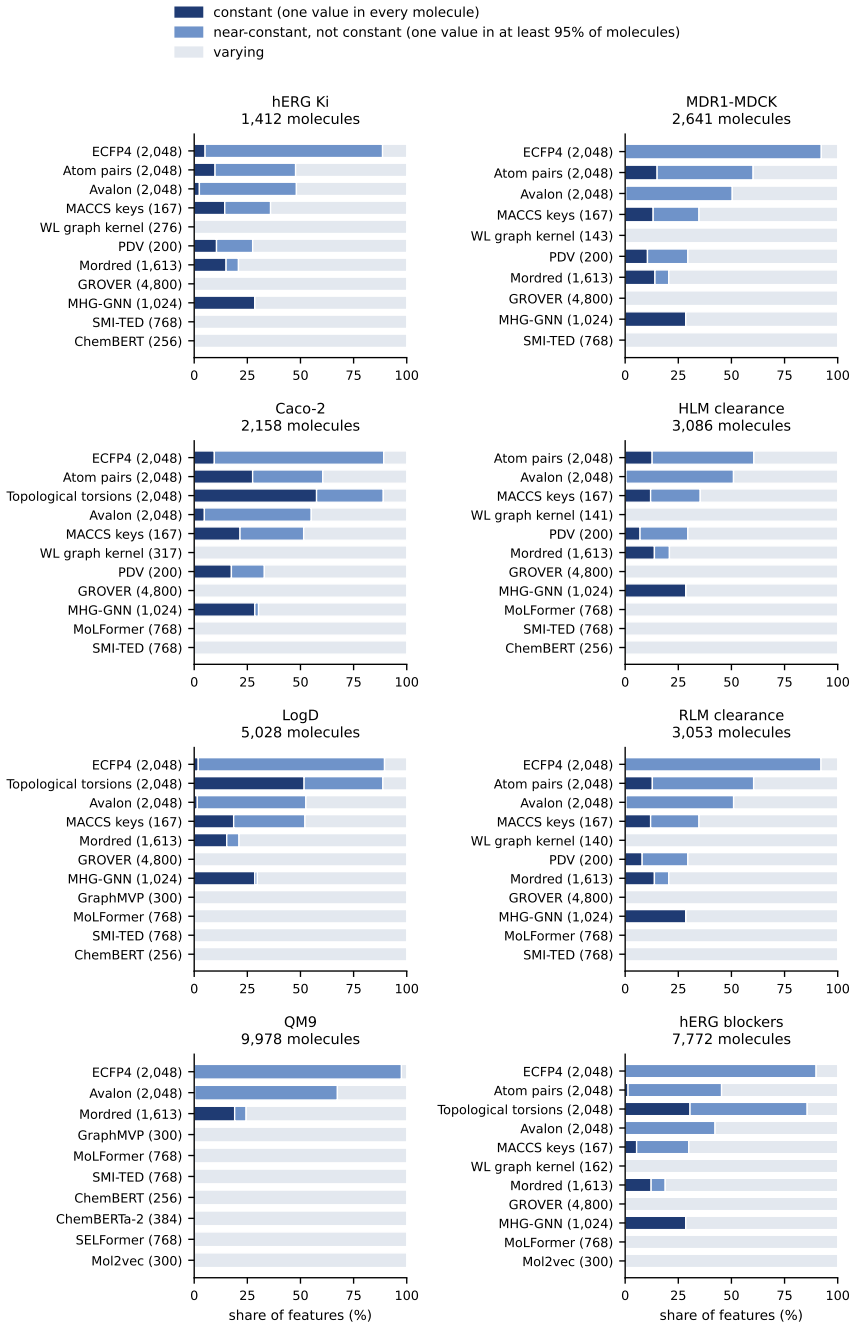


Figure 4.1: Shares of constant (dark blue), near-constant (mid blue), and varying (gray) features of each representation on eight datasets. The number of features of each representation is in parentheses.

Which count matters depends on the distance metric, and some metrics ignore constant features. Tanimoto similarity, defined in Equation 2.3 with distance $1 - T$, ignores bits off in both molecules, but a bit on in every molecule raises every molecule's measured similarity. It does not use the 105 constant ECFP4 bits on hERG K_i, which are all off. Constant features add zero to a Euclidean or Manhattan distance; however, near-constant features are amplified. Standardization, described in Section 4.1.1, turns a bit that is on in a share p of the molecules into $\sqrt{(1-p)/p}$ where it is on. A bit on in half of the molecules becomes 1, a bit on in 5% becomes 4.36, and a bit on in one of the 1,412 hERG K_i molecules becomes 37.56. Two molecules that differ only in that rare bit are therefore far apart, although they differ in one substructure. That is useful if those features define an AC. If not, they are contributing disproportionately to the measure of similarity. In short, the features that vary least between molecules carry the most weight in a standardized distance.

Molecules with identical feature vectors

A standard fingerprint- or descriptor-based representation can produce the same feature vector for two separate molecules. In turn, models using this representation will produce a single prediction for the molecules with identical features. Identical feature vectors do not always mean that the representation has merged two different molecules, since a dataset pooled from several sources can hold two measurements of the same molecule.

Many fingerprints are computed without stereochemistry [131]. In this research, ECFP4 is built with stereochemistry off, and the 2D descriptor sets (PDV, Mordred) also ignore it. So, as Table 4.1 shows, almost every molecule they produce identical representations for is a stereoisomer of another. A 2D graph can store stereochemistry as labels on atoms and bonds, but these representations do not read those labels, so they merge stereoisomers. Additionally, methods using the WL relabeling scheme are bounded by the 1-WL test described in Chapter 2, so some non-isomorphic graphs also receive the same vector. This is seen in the graph-kernel features which tie 105 of 1,412 hERG K_i molecules, 56 of which differ only in stereochemistry and 49 of which are different 2D structures. These WL features label each atom by its element only and ignore bond orders, and, as Table 5.9 records, keep only labels found in at least 5% of molecules.

Whether or not a learned embedding is able to differentiate stereoisomers is dependent on its input as well as its individual implementation. SMI-TED, a pretrained SMILES encoder, ties 56 hERG K_i, 633 LogD, and 177 Caco-2 molecules, all stereoiso-

mers. A tokenizer can drop stereochemistry prior to training. Additional file 12 calls the ChemBERTa-77M-MTR checkpoint ChemBERTa-2, and Chapter 3 calls it ChemBERTa. Its tokenizer gives 72 of the 1,415 hERG K_i molecules of Chapter 3 the same token sequence as another molecule. Since stereoisomers can differ in measured activity [132], these representations lack potentially useful chemical information.

Two molecules are considered to be tied when a representation produces the same feature vector for both. Table 4.1 counts ties on hERG K_i, LogD, and Caco-2 for each dataset's single representations and for its augmentation features. These are the small feature sets added to the hybrid after it is built, described in Section 5.2.3. A tie has one of four causes. It can be a repeat of one compound, with the same full InChIKey, which includes stereochemistry. It can be a failed featurization, giving all zeros. It can be a merge of stereoisomers, which share the first InChIKey block, encoding connectivity only. Otherwise it is a merge of different structures. No tie was a repeat or a row of zeros.

Apart from the MACCS keys, most tied molecules are stereoisomers. Atom pairs, PDV, Mordred, and SMI-TED tie only stereoisomers, and ECFP4, Avalon, and topological torsions add a few molecules that differ in more than their stereochemistry. On hERG K_i, ECFP4, atom pairs, PDV, Mordred, and SMI-TED tie the same 56 molecules, so joining any of these five representations leaves all 56 tied. The WL graph kernel also ties 49 hERG K_i and 26 Caco-2 molecules that are not stereoisomers, and the MACCS keys tie the most molecules on every dataset, mostly not stereoisomers. The 3D augmentation features and the learned embeddings ChemBERT, GROVER, MHG-GNN, MoLFormer, and GraphMVP tie none, and joining all of a dataset's representations leaves no molecule tied.

A model has to give every molecule in a tied group the same prediction. The best single prediction for a group is its mean label, so ties set an error that no model can go below,

$$\mathrm{RMSE}_{\min} = \sqrt{\frac{1}{n}\sum_{G}\sum_{i\in G}\left(y_i - \bar{y}_G\right)^2}, \tag{4.1}$$

where n is the number of molecules, G runs over the tied groups, and $\bar{y}_G$ is the mean label of group G. The matching best R^2 is $1-\mathrm{RMSE}^2_{\min}/\mathrm{Var}(y)$. Both use the measured labels of all molecules in the dataset, with no split into training and test molecules. They therefore bound the error over the whole dataset, not on one held-out fold. On hERG K_i this lowest error is at most 0.134 pK_i (MACCS keys), small next to the label SD of 0.915 pK_i.

Single groups can still disagree by a lot. Independent pK_i measurements of the

same compound against the same target differ by 0.44 log units on average, with a standard deviation of 0.54 [12], so a gap larger than 0.54 inside a tied group is larger than typical measurement noise. On hERG K_i, 6 of the 26 ECFP4 groups and 38 of the 153 MACCS groups differ by more than 0.54. Figure 4.2 shows the widest ECFP4 group, a pair of stereoisomers 1.14 pK_i apart. An ECFP4 model can at best predict their mean, 5.59 pK_i, which leaves each molecule 0.57 pK_i from its measured value.

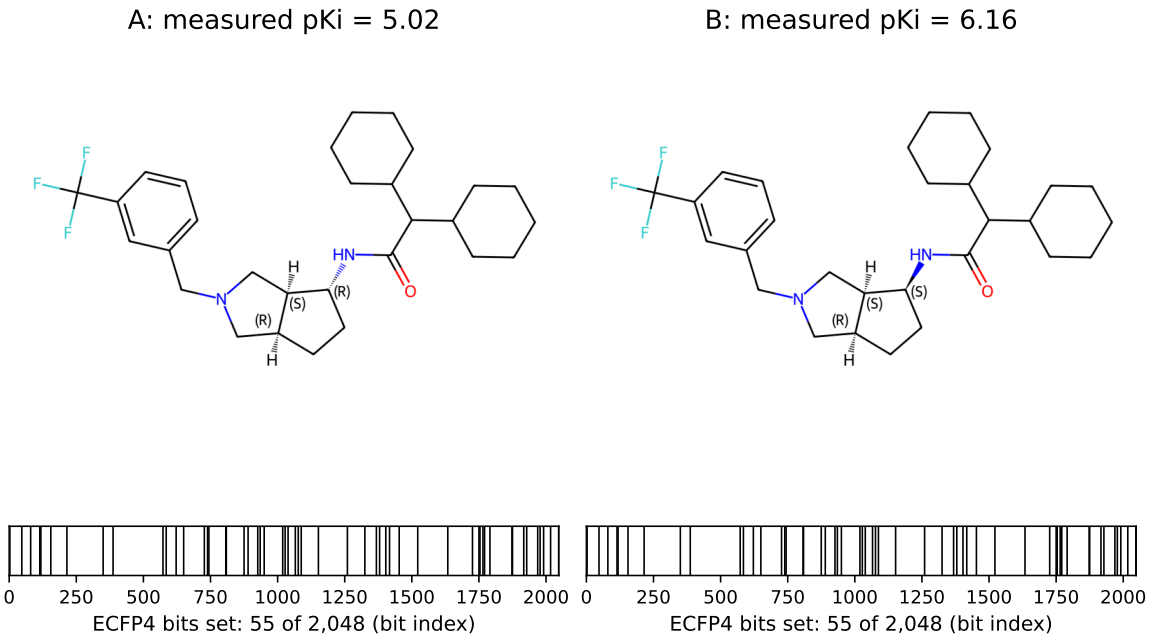


Figure 4.2: Two hERG K_i stereoisomers with identical ECFP4 vectors, which differ at the carbon bearing the amide nitrogen (R in A, S in B). The top row shows the structures with their measured pK_i values, and the bottom row shows the 55 of 2,048 ECFP4 bits set in both.

A tied pair with different labels is also the most extreme AC possible. The structure–activity landscape index (SALI) of molecules i and j,

$$\mathrm{SALI}_{ij} = \frac{|y_i - y_j|}{1 - \mathrm{sim}(i, j)} \tag{4.2}$$

[143], is infinite when their similarity is 1 and their labels differ, and here the infinite index is made by the representation. The label gap itself may be a real effect of stereochemistry, a gap between repeat measurements of the size given above, or both, and the index cannot separate them. A method that judges a label by its nearest neighbors sees a tied neighbor at distance 0 with a different label, and may flag a correct label as noise. Section 4.2.3, “Noise detection by neighbors”, returns to this case.

Dataset	Representation	Molecules	Tied	Stereo-only	Best RMSE	Best R^2	Largest spread
hERG K_i (pK_i)	ECFP4, Atom pairs, PDV, Mordred, SMI-TED	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Avalon	1,412	66 (4.7%)	56	0.049	0.997	1.20
	WL graph kernel	1,412	105 (7.4%)	56	0.071	0.994	1.49
	MACCS keys (aug.)	1,412	377 (26.7%)	30	0.134	0.978	1.80
LogD (log units)	ECFP4	5,028	641 (12.7%)	633	0.044	0.999	1.00
	Mordred, SMI-TED	5,028	633 (12.6%)	633	0.043	0.999	1.00
	Avalon	5,028	647 (12.9%)	629	0.047	0.998	1.40
	Topological torsions	5,028	675 (13.4%)	631	0.047	0.998	1.10
	MACCS keys (aug.)	5,028	2,343 (46.6%)	437	0.220	0.965	2.80
Caco-2 (log units)	ECFP4	2,158	179 (8.3%)	177	0.027	0.996	0.74
	Atom pairs, PDV, SMI-TED	2,158	177 (8.2%)	177	0.027	0.996	0.74
	Avalon	2,158	181 (8.4%)	177	0.027	0.996	0.74
	Topological torsions	2,158	186 (8.6%)	177	0.029	0.996	0.74
	WL graph kernel	2,158	203 (9.4%)	177	0.028	0.996	0.74
	MACCS keys (aug.)	2,158	804 (37.3%)	139	0.111	0.935	1.72

Table 4.1: Molecules that share their feature vector with at least one other molecule, one row per representation that ties any molecules; representations with identical entries share a row. Tied gives the number and share of such molecules, and Stereo-only those tied only with their own stereoisomers. Best RMSE and best R^2 are the best any model can reach given the ties. Best RMSE and Largest spread, the largest label range within one tied group, are in the units beside the dataset name, and (aug.) marks an augmentation representation.

4.2.2 Flaws in similarity and other pairwise computations

Section 4.2.1 described some of the limitations of standard representations. Here we ask what happens when two molecules are compared through one of them, as the pairwise comparisons described in Chapter 2 can violate the SPP.

The Tanimoto coefficient is the most popular similarity metric in cheminformatics [144], and is commonly computed on ECFPs, although the coefficient applies to any bit vector. It has been described as the "coefficient of choice" for molecular similarity [145], while the Morgan fingerprint, ECFP4 in this research, ranks among the strongest fingerprints for virtual screening and target prediction [146]. ECFP4 and ECFP6 specifically, alongside topological torsion, rank among the best of 28 fingerprints tested for ordering diverse structures by similarity [147]. However, it is not a one-size-fits-all solution. Across 96,456 human metabolites, 72.9% had an ECFP4 nearest neighbor at Tanimoto distance 0 [146]. Stereochemistry had been removed before that comparison, so these are constitutionally different molecules that ECFP4 cannot tell apart.

A similarity coefficient cannot tell you which features matter to the prediction task at hand, unless the representation itself was trained on that task's labels. And the SPP holds only relative to such a measure. To understand the impact of irrelevant features

on similarity measures, we make the following computation. First, we categorize a representation's features into those relevant to the task, R, and those irrelevant to the task, I, and take the squared Euclidean distance as a sum over both:

$$d^2(\mathbf{x}, \mathbf{y}) = \sum_{j \in R} (x_j - y_j)^2 + \sum_{j \in I} (x_j - y_j)^2. \tag{4.3}$$

Every non-constant irrelevant feature adds one further term, so the irrelevant portion grows linearly with $|I|$. After standardization each feature contributes the same amount on average over all pairs, so the share of the distance carrying signal is about $|R|/(|R|+|I|)$. When we use a general-purpose representation like ECFP4 on a dataset spanning a small portion of the chemical space, $|I|$ can be a lot larger than $|R|$. Constant features contribute nothing, while near-constant features have a strong impact. The worst-case scenario involves a single rare bit set in only a handful of molecules. After standardization, uncommon bits take on larger values, outweighing common features that may be more relevant. If this bit is irrelevant to the task at hand, the similarity measure suffers.

Larger representations compound the problem. As the number of features grows, the nearest and farthest molecules end up at nearly the same distance, meaning that these pairwise distances are less able to discriminate, as explained in Chapter 2. Two measures of this effect are the relative contrast, defined as the difference between the largest and smallest measured distance divided by the smallest [148], and hubness, a measure of how unevenly the molecules appear in other molecules' nearest neighbors [149]. These errors propagate along larger data pipelines, starting from the representation into similarity, from similarity into the set of neighbors a molecule is compared against, and from there into a prediction of some kind.

The four measurements below use the six small-molecule regression datasets of Table 5.12 and HLM clearance. Binary representations were compared by Tanimoto distance, and all others by Manhattan distance on their standardized non-constant features (see Section 4.1.1).

Activity cliffs

Chapter 2 gives the MoleculeACE definition of an AC, which combines three similarity measures. Here we use one. Two molecules are similar if their ECFP4 Tanimoto similarity is at least 0.55 [150], and a molecule is a cliff if a similar molecule has a label that differs from its own by more than a set gap. We chose the gap on each dataset so that 15% of the molecules are cliffs, so the gap follows the spread of the labels.

Table 4.2 gives the cliffs for each dataset. On hERG K_i the gap is 1.76 pK_i, and 211 of the 1,412 molecules are cliffs. On QM9 only 1,093 pairs of the 9,978 molecules are similar, and the gap is 0.05 eV, 0.06 times the median absolute deviation of the labels. Because the gap is set by a share of molecules, on QM9 a cliff marks a small label difference, not a cliff in the usual sense. On the three Biogen datasets (RLM clearance, MDR1–MDCK efflux, and HLM clearance) it is 0.20 to 0.34 log units, 0.50 to 0.60 times the median absolute deviation.

The count of cliffs depends on the representation. We kept each dataset's gap and replaced ECFP4 with each other single representation, calling similar the same number of closest pairs as ECFP4 does. On hERG K_i the number of cliff molecules then ranges from 138 (atom pairs) to 267 (WL graph kernel). Of the pairs that ECFP4 calls similar, only 57% to 74% are also among another representation's closest pairs, and on QM9 only 1% to 24%. Section S.3 of the Additional files gives these overlaps. So which molecules count as cliffs depends on the representation, not only on the labels.

Distance and label difference

We asked whether distance tracks label difference in two ways. Over all pairs of molecules, we took the Spearman correlation between the distance of a pair and the absolute difference of its labels, a one-number summary of the neighborhood-behavior plot of Patterson et al. [151]; datasets above 2,000 molecules were sampled down to 2,000. Over each molecule's closest pairs only, we predicted each label as the mean label of the molecule's five nearest neighbors, taken from other scaffold groups, and took the Spearman correlation between predicted and measured labels.

Across the 70 combinations of dataset and representation (seven datasets, ten single representations each), the all-pairs correlation ranged from -0.04 to 0.23 and was below 0.10 in 44 of them. The five-neighbor prediction is a correlation between predicted and measured labels. Table 4.2 shows that it ranged from 0.36 to 0.88 for the three representations drawn there, and from 0.29 to 0.89 over all 70, which Section S.3 of the Additional files gives for every representation. On hERG K_i with ECFP4, the all-pairs correlation is 0.06, while the five-neighbor prediction reaches 0.72, with an RMSE of 0.63 pK_i against a label SD of 0.92 pK_i.

Figure 4.3 shows why the two differ. Of the 996,166 hERG K_i pairs, 93% have an ECFP4 Tanimoto similarity below 0.20, and only 1.1% reach 0.55. Over all pairs the median label difference is 0.81 pK_i, but above a similarity of about 0.4 it is 0.20 to 0.30 pK_i, although 8.3% of the 774 pairs at 0.8 or above still differ by more than 1 pK_i. Mordred shows the same pattern: the closest 5% of pairs have a median difference of

0.44 pK_i, and every other 5% has 0.78 to 1.11 pK_i. The useful information sits in the few closest pairs. The all-pairs correlation is set by the other 95%, whereas the five-neighbor prediction uses only the closest pairs. So the SPP holds for each molecule's closest neighbors but not across all pairs. Neither ECFP4 nor the descriptor set follows it better on every dataset. Both reach 0.72 on hERG K_i. The descriptors are higher on LogD, Caco-2, and the three Biogen datasets, and ECFP4 is higher on QM9.

Agreement on neighbors

We compared the five nearest neighbors of each molecule, from other scaffold groups, under ECFP4, SMI-TED, and Mordred (PDV on Caco-2, where Mordred is not a single representation; on HLM clearance ECFP4 is not a single representation and its stored features were used). On hERG K_i any two of the three share on average 0.47 to 0.52 of each molecule's five neighbors. On the three Biogen datasets and QM9 they share only 0.13 to 0.21, as Table 4.3 shows. So a neighbor-based method that switches representation compares each label with a largely different set of neighbors.

Agreement with shared structure

For 500 random pairs per dataset we found the maximum common substructure (MCS) with RDKit [37], allowing 2 seconds per pair and leaving out pairs that ran out of time. The MCS distance is $1 - n/(a+b-n)$, where n is the number of atoms in the common substructure and a and b are the numbers of heavy atoms in the two molecules. Its Spearman correlation with ECFP4 Tanimoto distance ranged from 0.20 (hERG K_i and HLM clearance) to 0.51 (Caco-2 and LogD). ECFP4 distance therefore agrees only partly with how much structure two molecules share.

In short, close molecules do tend to have close labels, but over all pairs distance barely follows label difference. Representations also disagree on which molecules are close and which are cliffs. Every method in Section 4.2.3 that takes a similarity as input inherits the dilution described here.

4.2.3 Consequences for similarity-based methods

So far this chapter has shown three flaws, each with its own origin. Most features of a general-purpose representation barely vary on one dataset, because the representation was built for all of chemistry. Several representations give different molecules, mostly stereoisomers, the same feature vector, because they ignore stereochemistry. A distance sums over every feature, so over all pairs it barely follows label difference.

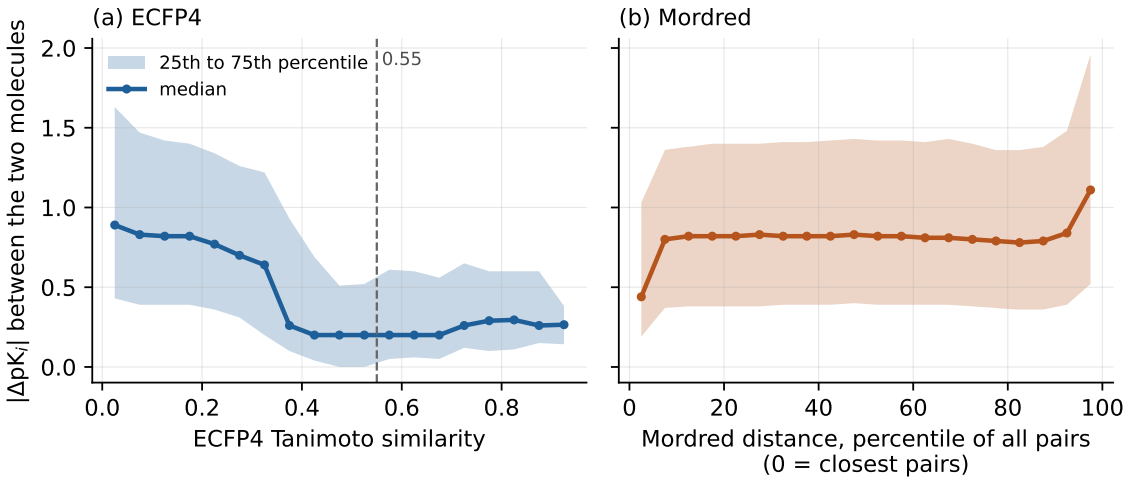


Figure 4.3: Absolute pK_i difference against similarity or distance for all pairs of hERG K_i molecules, with the median in each bin (markers) and the interquartile range (shaded). a) ECFP4 Tanimoto similarity in bins of 0.05 (bins with fewer than 20 pairs omitted), with the AC threshold of 0.55 dashed. b) Manhattan distance on standardized Mordred features, as a percentile of all pairs.

		All pairs			Five neighbors			Cliffs	
Dataset	Molecules	ECFP4	Desc.	SMI-TED	ECFP4	Desc.	SMI-TED	Molecules	Gap
hERG K_i	1,412	0.06	0.07	0.09	0.72	0.72	0.69	211	1.76 pK_i
LogD	5,028	0.12	0.06	0.10	0.76	0.80	0.76	731	2.90 log
Caco-2	2,158	0.08	0.18	0.14	0.59	0.62	0.57	324	1.21 log
RLM clearance	3,053	0.02	0.07	0.04	0.36	0.49	0.46	458	0.34 log
MDR1–MDCK efflux	2,641	0.02	0.16	0.10	0.51	0.58	0.59	397	0.20 log
HLM clearance	3,086	0.00†	0.07	0.03	0.37†	0.49	0.45	462	0.27 log
QM9	9,978	0.19	0.18	0.13	0.88	0.84	0.85	1,495	0.05 eV

Table 4.2: All-pairs correlation, five-nearest-neighbor prediction, and ACs for each dataset. All pairs gives the Spearman correlation between pair distance and absolute label difference, and Five neighbors the Spearman correlation between each label and the mean label of its five nearest neighbors. Cliffs gives the number of cliff molecules and the label gap that defines a cliff. Desc. is Mordred, or PDV on Caco-2, and † marks ECFP4 where it is not a single representation of the dataset.

Representations also disagree about which molecules are neighbors. While this calls into question the predictive limits of QSAR and other structure-based modeling, it also calls into question downstream applications of molecular similarity. Nearest-neighbor methods, where a molecule's neighbors, as measured by similarity, are the inputs to several other processes, are by definition sensitive to the quality of similarity measurements. They are used within other processes like predicting a molecule's activity using its neighbors' labels, detecting outliers, either ACs or noise, defining an applicability

Dataset	Desc.	Five-neighbor overlap			ECFP4 against MCS		
		ECFP4–Desc.	ECFP4–SMI-TED	Desc.–SMI-TED	Spearman	Pairs	Timed out
hERG K_i	Mordred	0.52	0.50	0.47	0.20	495	5
LogD	Mordred	0.38	0.41	0.41	0.51	494	6
Caco-2	PDV	0.38	0.41	0.38	0.51	492	8
RLM clearance	Mordred	0.15	0.21	0.18	0.28	500	0
MDR1–MDCK efflux	Mordred	0.15	0.20	0.17	0.26	500	0
HLM clearance	Mordred	0.15	0.21	0.18	0.20	500	0
QM9	Mordred	0.13	0.18	0.21	0.46	500	0

Table 4.3: Agreement between representations for each dataset. Five-neighbor overlap is the mean fraction of a molecule's five nearest neighbors that two representations share. ECFP4 against MCS gives the Spearman correlation between ECFP4 Tanimoto distance and MCS distance, the number of pairs used, and the number left out because the MCS search timed out.

domain by distance to the training set [152], grouping molecules into clusters for data splits [153], identifying ACs, and, in federated learning, weighting each site's vote on a molecule by that molecule's distance to the site's own data [154]. If this measure of similarity, or equivalently distance, is flawed, then all of these downstream applications will feel the effects of it.

Below, we report a preliminary sweep we ran over 12 LNL methods. Those that relied on pairwise distance measurements reported the lowest retention of predictive performance under increasing label noise.

A small sweep of methods for learning with noisy labels

It is established in Chapter 3 that model family explains the largest share of variance in noise robustness, while representation explains at most 8.1% on QM9.

Here, we review a small sweep of LNL methods, many of which were originally built for image classification with flipped labels [56], so they do not transfer automatically to continuous labels or cheminformatics domains. We explored various adaptations to get them to fit the tasks at hand and swept various settings relating to distance computation, loss functions, and use of uncertainty estimates. We originally sought to ask whether training methods built for noisy labels improve robustness and whether representation still matters under them.

The study tested 12 LNL methods on the QM9 HOMO–LUMO gap (10,000 molecules, scaffold splits), adding Gaussian noise with σ from 0 to 1.0 in steps of 0.1 to the training labels only. The noise was added to the raw gap in eV before the

labels were standardized. Its σ is therefore in eV, not the noise level of Section 2.4.1. The representations were ECFP4, Sort & Slice [7], PDV, canonical SMILES, and randomized SMILES. Depending on the method, a configuration can also add the model's own uncertainty estimate or a pairwise distance measure such as Tanimoto, Euclidean, Manhattan, cosine, or Mahalanobis. Other methods added an alternative GP kernel or loss function, for 221 configurations in all. Four of the methods attempt to identify noisy labels, smooth or remove them, and retrain on the updated set. These methods include Small-loss selection, Confident learning, Distance-based selection, and Multistage cleaning. The other eight involve training methodologies that take noise into account (noise-robust training). Table 4.4 lists the 12 methods and how each uses a distance between molecules.

We measured noise robustness by retention, R^2 under noise divided by R^2 with clean labels, as a percentage, so a retention of 86.7% means the model kept 86.7% of the R^2 it reached at $\sigma = 0$. While later research in this thesis uses $\mathrm{AUC}_{\mathrm{norm}}$, the area under the retention curve, this sweep predates that change in evaluation metric. Retention correlates too closely with clean R^2, so it struggles to separate the change in predictive performance from its magnitude. In an ANOVA over the adapted components, using a distance explained the most variance in R^2. All 20 configurations with the lowest retention used a distance, and none of these 20 reached both a clean R^2 above 0.6 and a retention above 80%.

The top-performing methods relied on estimating noise from the model's own loss or uncertainty. DivideMix [155] retained the most, 86.7% (SD 0.4% across its configurations), followed by Small-loss selection [156] at 84.2% (2.1%) and the Uncertainty curriculum, which adds molecules in order of the model's own predicted uncertainty [157], at 82.7% (3.5%). The Heteroscedastic GP, a GP whose noise is predicted per molecule by a second model [158], retained 79.0% at $\sigma = 0.6$, averaged over 15 configurations, with R^2 of 0.71 on clean labels and 0.56 at that level. The exploratory study does not record the noise level for the other methods, so this value cannot be ranked against theirs. Regularization alone retained less, 73.0% for Mixup and 74.0% for sharpness-aware minimization. Under these methods, the choice of representation explained even less variance than in Chapter 3.

We now outline which of the 12 methods use a distance between molecules, as listed in Table 4.4. Distance-based selection always does, as it removes the molecules that are the strongest outliers by molecular similarity. Four more methods add a distance only in some configurations. Small-loss selection uses it to remove molecules far from the selected ones, and Confident learning uses it to flag further outliers. MentorNet takes

Method	Category	Key adaptation	Use of a distance
Meta-Weight-Net	Meta-learning	Input extended to the loss and the uncertainty; weights updated on the validation set every 5 epochs	None
Small-loss selection	Sample selection	One-time selection of the 70% of molecules with the smallest squared error	Optional: removes molecules far from the selected ones
Confident learning	Sample selection	Flags molecules with high error (above the 85th percentile) and low uncertainty (below the 25th)	Optional: also flags outliers by distance
Distance-based selection	Sample selection	Removes the 15% of molecules that are the strongest outliers by molecular similarity	Always: the selection is by distance
MentorNet	Curriculum	Online mentor, not pre-trained; input is the loss, its last 5 values and, under a loss that predicts one, the uncertainty	Optional: distance to the 10 nearest neighbors as a further input
Uncertainty curriculum	Curriculum	Staged inclusion by uncertainty, 25%, 50%, 75% then 100%, after a 20-epoch warmup	None
DivideMix	Iterative refinement	Squared-error loss, uncertainty-weighted mixture separation, simplified co-teaching	None
Multistage cleaning	Iterative refinement	Removes 10% after each of the first 2 of 3 stages, scored by normalized loss plus normalized uncertainty	None
Early learning	Iterative refinement	Weights molecules by their loss in the early epochs and, optionally, their uncertainty, with geometric decay	None
Contrast-to-divide	Iterative refinement	Selects molecules whose predictions vary least when the input is perturbed	Optional: similarity between molecules in the consistency score
Heteroscedastic GP	GP extension	A GP (Tanimoto, RBF, or Matérn kernel) for the mean and a network for the noise variance, trained jointly	Through the kernel only
Evidential kernel	GP extension	Multitask GP predicting the four normal-inverse-gamma parameters	Through the kernel only

Table 4.4: The 12 LNL methods of the exploratory study, adapted for regression on the QM9 HOMO–LUMO gap. "Optional" marks a distance that some configurations added and others did not.

the distance to the 10 nearest neighbors as an extra input, and Contrast-to-divide adds the similarity between molecules to its consistency score. The two GP extensions, the Heteroscedastic GP and the evidential kernel, compare molecules only through their kernel. The remaining five methods use no distance at all. As stated above, every one of the 20 configurations with the lowest retention used a distance, while the three methods that retained the most do not need one.

There are various reasons why the methods relying on distance failed. First, a single representation may lack the information a distance needs, with and possibly even without noise. PDVs encode global molecular properties and ECFP4 contains substructure information, and any one of these sources alone may not be sufficient for robust similarity estimation. Second, some representations are sparse, with many features irrelevant to the molecules of a given dataset, as explained in Section 4.2.2. We sought to address the second with the adaptive hybrid representations in Chapter 5.

Noise detection by neighbors

Neighbor-based noise detection uses a sample's neighbors to determine whether or not that sample's label is an outlier, an idea that goes back to editing nearest-neighbor training sets [106]. As discussed in Chapter 2, ACs are outliers in the context of neighbor-based detection, but are not noise. This methodology does not distinguish between the two. The wider literature on filtering noisy labels concerns classification [159, 160], but Martín et al. [161] adapt fourteen classification noise filters to regression, seven of them nearest-neighbor filters that use the neighbors' mean label. They find that the filters improve regression methods on 20 general regression benchmarks with injected label noise, none of them QSAR datasets.

The neighbor-based noise scores, sample removal, and sample weighting are defined in Sections 7.2.2 and 7.2.3 of Chapter 7, and the recovery rate in Section 7.2.5, Equation 7.6. Label smoothing here moves each label toward a target by the share in Equation 7.2, or by two variants that set the share from the score differently.

A noise detection method requires a representation and a distance metric that together pick the right neighbors, in terms of what is going on structurally. Ideally, those neighbors would also have clean labels. Earlier sections of this chapter argue that the first condition fails, and the second fails whenever noise is spread over the whole training set. A large gap from the neighbors' labels can indicate a noisy label, a noisy neighbor, the wrong neighbors, or an AC. And the score cannot tell you which it is.

We also used neighbors to clean labels, on 5,000 randomly chosen QM9 molecules (HOMO–LUMO gap, scaffold split) with Gaussian noise on the training labels at $\sigma =$

0.3 and $\sigma = 0.6$. Each cleaning method was judged by retraining an RF on ECFP4 with clean, noisy, and cleaned labels, giving the recovery rate, the share of the R^2 lost to noise that cleaning recovers. Neighbor-based scores compared each label with those of the molecule's k nearest training molecules under Euclidean, Manhattan, or cosine distance. Table 4.5 holds only the configurations built on a single representation. This study is separate from the sample-removal study of Chapter 7.

Table 4.5 shows that with the neighbor-based scores used as sample weights at strength $\alpha = 0.1$, mean recovery was negative for every representation at both noise levels. Of all 860 cleaning configurations, 8 had positive recovery at both noise levels; 6 of these used a single representation, and 4 of those 6 were neighbor-based. Label smoothing toward neighbor or model estimates gave positive recovery in 5.0% of configurations at $\sigma = 0.3$ and 0.8% at $\sigma = 0.6$. Mean recovery was negative at every weighting and smoothing strength tried, from 0.1 to 0.7.

	Mean recovery (%)		Positive (% of configurations)	
Representation	$\sigma = 0.3$	$\sigma = 0.6$	$\sigma = 0.3$	$\sigma = 0.6$
ECFP4	−3.0	−7.3	33.3	16.7
PDV	−2.4	−3.4	50.0	27.8
MHG-GNN	−0.1	−7.2	38.9	5.6

Table 4.5: Recovery rate in R^2 on the QM9 HOMO–LUMO gap when neighbor-based noise scores are used as sample weights, for each representation used to find the neighbors, at two levels σ of Gaussian label noise. Recovery is the share of the R^2 lost to noise that cleaning regains, so negative values mean cleaning made the model worse. Positive is the share of configurations with recovery above zero.

Judging a label by its neighbors usually made the model worse and rarely helped at both noise levels, but it did not always fail. The best configuration at $\sigma = 0.6$ was neighbor-based (PDV, Euclidean distance, and a score equal to the gap from the neighbors' mean label divided by the standard deviation of their labels) and recovered +17.2%. Because noise was added to every training label, the neighbors' labels are also noisy.

Uncertainty-based methods, which use no neighbors, showed potential at lower noise but degraded at higher noise. Sample weighting showed more promise than label smoothing, but it still usually had an overall negative effect. Over all sample-weighting configurations, not only those in Table 4.5, recovery was positive in 14.2% of configurations at $\sigma = 0.3$ and 4.7% at $\sigma = 0.6$. The mean recoveries were −22.9% and −20.9%.

4.3 Conclusions

It is not uncommon that very few features of a standard representation vary across a dataset, especially a smaller dataset. On hERG K_i, 1,815 of the 2,048 ECFP4 features take one value in at least 95% of the 1,412 molecules. Additionally, several representations give different molecules, mostly stereoisomers, the same feature vector, and no model can separate those molecules, whatever their labels. ECFP4 gives 56 of the 1,412 hERG K_i molecules a vector shared with another molecule, all of them stereoisomers.

A pairwise distance metric sums over every feature, so when only a few features matter for the property, the rest dominate the sum. Across 70 combinations of dataset and representation, the Spearman correlation between pair distance and label difference was below 0.10 in 44. Using the five nearest neighbors to predict labels produced Spearman correlations of 0.29 to 0.89 with the measured labels, indicating that the useful information sits in a few close pairs rather than in the distance as a whole.

In the exploratory sweep of LNL methods, all 20 configurations with the lowest retention used a pairwise distance measure, and judging a label by its neighbors' labels usually made the model worse, as discussed in Section 4.2.3. The sweep added a distance to some methods designed without one, so poor adaptation may explain part of this.

Table 4.6 lists each flaw found in this chapter and the requirement it sets for a representation. Chapter 5 outlines the development of the adaptive hybrid representation and asks: what is the best way to select features for a given prediction task?

Flaw (where shown)	Requirement	Where handled
Features that do not vary ("Constant and near-constant features")	Features chosen for the task, within a budget	"Feature importance scoring"
Molecules merged into one vector ("Molecules with identical feature vectors")	Features drawn from several representations	"Combining representations"
Distance dominated by irrelevant features ("Flaws in similarity and other pairwise computations")	Compact, with the number of features searched	"The search space"
Poor performance of neighbor-based methods ("Consequences for similarity-based methods")	Usable as a space for similarity	"Similarity in a hybrid" (Chapter 6)
Features that are hard to read chemically	Traceable, so that each chosen feature leads back to its representation and can be read chemically wherever the base feature can	"Blueprints and their chemical interpretation"

Table 4.6: Flaws of standard representations and similarity, the requirement each sets for a representation, and where the requirement is addressed, in Chapter 5 unless stated.

Chapter 5

Building adaptive hybrid representations

5.1 Introduction

So far, Chapter 4 explained how the standard methods of estimating molecular similarity, or conversely distance, are flawed and potentially break the SPP. Specifically, the representations used in these similarity calculations contain irrelevant features that either do not vary or more generally do not pertain to the prediction task at hand. This chapter explores a solution. With the understanding that each representation contains a set of features which contain a certain type of information about that data, we aim to select the most informative features across a larger set of representations, and so leave out the irrelevant ones. We create an adaptive hybrid representation, a set of features bounded by a budget that is selected from several single representations by a search using one dataset and downstream model. The goal is to select a set of features that is customized for a given model and prediction task. As discussed in Chapter 3, performance can vary significantly across datasets and models. Rather than hope the features you give a model will be useful, this process lets you prescreen them.

In this chapter we build the adaptive hybrid representation, referred to as the hybrid representation, first reporting on when and why it helps. This chapter asks how to select the best set of features from several representations for a given dataset and model. It also asks whether the method carries over from small molecules to antibodies and reactions. Chapter 6 asks whether distances computed in a hybrid are better than in a single representation. This chapter measures predictive performance on the labels as published, apart from one separate-site experiment with flipped labels, reported in Additional file 30. Distance measures between hybrids are covered in Chapter 6, and

noise detection and LNL in Chapter 7. Chapter 7 asks whether noise detection and noise mitigation work better in a hybrid than in the best single representation. It also cleans the labels that the search itself uses, as described in Section 7.3.6.

5.1.1 Combining representations

Not every molecular representation is equal. In fact, they hold very different types of information. RDKit2D descriptors matched or outperformed features derived from GNN embeddings and pretrained language models on many datasets across 62,820 models on a variety of molecular property prediction benchmarks [112]. Descriptor-based models held 73% of top-three positions on 11 benchmarks [162]. In 1,310 ChEMBL assays, feedforward networks on ECFP fingerprints outperformed graph convolutions and recurrent networks trained on SMILES [163]. We know that the choice of representation matters, as is discussed in Chapter 3, yet the combinations of representation and model are hard to explore by hand [164]. Beyond just molecules, general protein language models predict antibody expression better than antibody-specific ones, while the antibody-specific models are better on binding [165], and no pathology foundation model is best on every clinical task [166]. We turn our attention to combining features from these different representations.

Most research in this field relies on a single representation, and few combine features from different representations beyond concatenation. One of the reasons just pooling features from different representations together doesn't work is dimensionality. If you add a standard ECFP4 ($d = 2,048$) with Mordred descriptors ($d = 1,613$) and an MHG-GNN embedding ($d = 1,024$), you end up with 4,685 features, which is far too many considering many datasets from experimental assays contain only a few hundred labeled samples. With such ratios, more features mean weaker predictions [167, 168]. Joining representations also adds the irrelevant features of every representation along with the useful ones. So a method that joins representations has to combine and filter at the same time. The hybrid is built to do both. Therefore, we consider ways of combining representations.

The most straightforward method of combining representations is concatenation. This, however, degrades with non-homogeneous feature sets and obscures which representation is responsible for the score [169]. Deng et al. [112] found that appending RDKit2D descriptors to GROVER lowered RMSE on only 9 of 24 datasets tested. Late fusion trains one model per source and aggregates the predictions [170]. MOGONET is of this kind, combining per-source predictions in label space [171]. While late fusion

is effective, it loses interactions between the source representations, an issue which is addressed by intermediate fusion, for example cross-attention between sources [172]. Multiple kernel learning combines the sources as kernels rather than as whole models [173, 174], and produces no interpretable feature sets. Laufkötter et al. [175] found that combining bioactivity-based fingerprints with structure-based ones like ECFP4 improved both predictive performance and scaffold-hopping. Similarly, Turkina et al. [19] kept, from each of six non-hashed fingerprints, the bits that carried most of an RF's importance on it, and joined them. The joined bits consistently improved predictive accuracy over the original fingerprints across six datasets. Most of this work involves limited combinations, adding together standard fingerprints and linear representations [28, 175]. Although the features derived from embeddings are not as conceptually straightforward, they contain another axis of information. The cost is interpretability. A selected embedding feature can be traced to its representation, but not to a substructure or a descriptor, as discussed for blueprints below.

When you want to filter a feature set for the features most important to the prediction task, you can use a dimensionality reduction technique or feature selection methodology. Principal component analysis (PCA) uses linear combinations of the original features, ordered by the variance they explain [39]. Their combined features are dependent on the original feature units unless each feature is z-scored first, which matters when data from different modalities like fingerprint bits, descriptors, and embeddings are joined [39]. PCA never sees the labels, and components with small variance can still be important in prediction [176]. Partial least squares (PLS), the supervised counterpart in chemometrics, builds components that covary with the property [177]. Neither gives components that can be traced to one substructure, descriptor, or representation, making them difficult to interpret.

Feature selection keeps a subset of the original features, unlike PCA or PLS which build new ones. Of its three families [38], filter methods score features against the labels before any model is fitted, as minimum-redundancy maximum-relevance selection (mRMR) ranks features by their mutual information with the label minus their redundancy with already-chosen features [178]. Wrapper methods score subsets of features by fitting the model [179]. Embedded methods select features while the model is fitted, as the lasso does [180]. Recursive feature elimination repeatedly fits a model and drops the lowest-ranked features [181].

When selecting features, the question is not only what features are most important, but also, what features does the model prefer? One of the ways to answer this is to consider how many features relative to the size of a dataset a given model works best

with. As discussed in Chapter 4, distance-based models such as k-nearest neighbors (kNN) and SVM use every feature in every distance computation. Tree ensembles, by contrast, choose one feature per split and can essentially ignore irrelevant ones. Dablander et al. [7], who created a substructure-based fingerprint called Sort & Slice, suggest that using labels to select features may result in overfitting. Table 5.1 compares these alternatives with the adaptive hybrid methodology developed in this thesis.

Method	Uses the labels	Keeps original features	Several representations	Retrain to add one
Concatenation [169]	no	yes	yes	yes
Late fusion and ensembling [170, 182]	only if pooling weights are learned	yes	no	no, one more model
Intermediate fusion [172]	yes	no	yes	yes
Multiple kernel learning [173, 174]	yes	no, kernels only	yes	yes
PCA and PLS [39, 177]	PCA no, PLS yes	no	yes, on a concatenation	yes
Flat feature selection [38]	yes	yes	yes, on a concatenation	yes
Group lasso [20]	yes	yes, whole groups	yes	yes
Adaptive hybrid representation	yes	yes	yes	search rerun, method unchanged

Table 5.1: Methods for combining or reducing representations. The columns give whether each method uses the labels, keeps the original features, lets one model see several representations, and must refit to add a representation.

5.1.2 The adaptive hybrid representation

The task of selecting the best features from a set of representations and combining them is not simple. How do you measure what features are "best"? Are these features "best" relative to the representation they came from or to the whole pool of features? Should you select more features from representations that seem to have more signal to begin with, or take a set number of features from every representation? How many single representations should you select features from? How many features should you select? When scoring potential combinations of features, what type of evaluator should you use?

In a dream world, there would be a single answer to each one of these questions

that applies to every dataset. Unfortunately we live in reality, and as we discovered, the answer to every single one of these questions is "depends on the data." The task of selecting the best features then became a search space optimization challenge. The question actually became for a given prediction task, defined by the dataset, prediction target, and model, what is the best methodology for selecting features? The hybrid search does not output just a list of features, rather it outputs an ideal search configuration to find features for your data. It tells you how to score features (feature importance score), what rules you should use to select them (allocation method), how to score potential hybrid feature combinations (internal evaluator). In this research, we run this hybrid search multiple times across different maximum numbers of features (budgets), numbers of input representations, and models (downstream evaluators) on each dataset. While it is not formal "training" in the ML sense, the hybrid search is essentially a feature selection method that is fit on the training data. The representations used in this thesis are listed in Tables 5.9, 5.10 and 5.11.

Definition. Take $K \geq 2$ single representations of the same samples, their labels, one downstream model, and a budget B. An adaptive hybrid representation is a set of at most B of the original features of these single representations. Its features are chosen by a hybrid search on the training samples, for that dataset and downstream model. Each kept feature stays linked to the single representation it came from.

The output of a hybrid search is itself a feature selection method that when run on the given dataset, produces the hybrid representation. This search is run on a single dataset and downstream model, and the resulting hybrid differs by model, dataset, and even split. Its intended use case is when there are multiple single representations that describe different aspects of a molecule, and you want to combine these different features for a single training task. The hybrid ranks features by a fitted model's importance scores, as embedded methods do, and keeps a fixed number of the top-ranked ones, as a filter does. Classical methods treat all features as one flat list. The nearest classical relative, the group lasso, keeps or drops predefined groups of variables whole [20], whereas the hybrid splits the budget among representations as set out in Section 5.2.2, "Feature allocation methods". Because only original features are kept, each traces back to one representation, which makes blueprints possible.

5.1.3 Blueprints and their chemical interpretation

As explained above, the adaptive hybrid representation relies on a search. A completed search is saved as a blueprint, which records the settings needed to select the features

that make the hybrid representation. Because each selected feature traces back to its original single representation and relative importance rank, the search can tell you about your own prediction task. The best configuration is saved for each downstream model and budget, giving one blueprint per pair, allowing us to compare blueprints and see if different models prefer different features.

Depending on the origin of the feature, some are chemically interpretable. By definition, the PDV contains values that relate back to chemical properties like molecular weight or logP. An ECFP4 bit can be shown as the atom environment that sets it, and a MACCS key stands for a known substructure. Beyond small molecules, the readable unit of an antibody is a sequence position and its residue. This feature-level explainability fails when features are derived from learned embeddings. That will only tell you if the learned embedding itself is useful as a representation, but nothing about the selected feature.

While a set of thousands of features may seem overwhelming and irrelevant, identifying substructures or descriptors that recur among the selected features can point to parts of the molecule that may be involved in binding or activity. If the selected features make sense chemically for that endpoint, it's a good sign that the selection mechanism is finding real signal instead of fitting noise. Conversely, if the hybrid search is done over a range of budgets or models, and the blueprints are showing a vastly different set of features, that may be an indication that the features at hand are struggling to provide information about the prediction task. Explainability is a recognized need in drug discovery [183], especially in drug safety assessment when predicting and interpreting toxicity. A predictor that scores proposed molecules in generative design [184] could also take a hybrid as input, which we leave to future work.

5.1.4 Applications beyond small molecules

The hybrid search is a generalized search that takes in multiple feature matrices from the same samples and a supervised prediction task, and outputs a feature selection method that produces a filtered set of features from those original vectors. The default search objective scores prediction only, meaning the methodology applies to tabular tasks or cases where you have different measurements of the same inputs. Although this was developed with small molecules and QSAR modeling in mind, it is by no means limited to that use case. While there are some functionalities in this research that are tailored to chemical tasks, such as blueprints mapping features back to their chemical interpretations and the use of nearest-neighbor objectives to aid in the computation of

molecular similarity, the core functionality of the hybrid process is domain-independent.

One caveat concerns sparse representations such as one-hot encoded sequences, where each feature is set in only a few samples and so scores low importance on its own. Here, the only one-hot encoding used in the experiments is a single representation on three antibody datasets: nanobody thermostability, change in binding free energy ($\Delta\Delta G$), and Chinese hamster ovary (CHO) polyreactivity. The blocks substitution matrix (BLOSUM) encoding is a single representation on humanness.

While this setup was initially intended for small molecules, we extend this research to antibodies. Twelve antibody datasets are used in this chapter, listed in Table 5.7. They range from deep mutational scans of one antibody to panels of distinct antibodies. One Buchwald–Hartwig yield dataset tests the method on reactions, a separate domain.

Another caveat of this setup, whose sample-size half is discussed in Section 5.4, is that the hybrid can help only on datasets with enough samples and with single representations whose features carry importance. When all the representations score poorly, there are not enough useful features to pull from. Additionally, if there is only a single representation that carries signal, there is nothing for the hybrid to improve upon, which is shown in Section 5.3.1.

5.2 Methods

Figure 5.1 outlines the hybrid search.

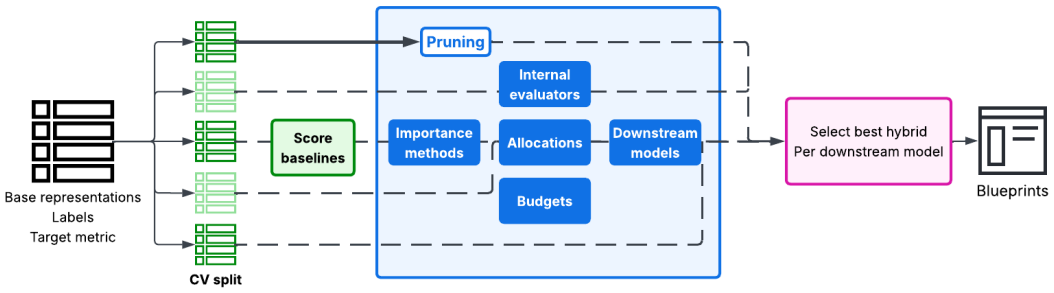


Figure 5.1: Schematic overview of the hybrid search on the training part of one outer fold. The outer test fold is set aside before the search starts and is not drawn (Figure 5.2). The inputs (left) are split into five CV folds, and the single representations are scored as baselines. Each fold is searched over importance methods, allocation methods, budgets, internal evaluators, and downstream models (center). The best hybrid for each downstream model is stored as a blueprint (right). Pruning is drawn but was not used in this chapter's searches (Section 5.2.4).

5.2.1 From single representations to a hybrid

The input to the search is K frozen representation matrices $\mathbf{X}^{(k)} \in \mathbb{R}^{n \times d_k}$, $k = 1, \ldots, K$, for the same n samples, whose widths d_k range from tens to thousands of features, as well as a label vector $\mathbf{y}$, one group label per sample for CV (for example, its scaffold group), the evaluation metric, the task type, one or more downstream models, and a list of budgets. It returns a hybrid of at most B features before any augmentation features added outside the budget, which Section 5.2.3 describes. The budget B is a maximum rather than a target. It also returns a blueprint that records the settings necessary to rebuild the hybrid representation on another set of samples, such as another subset of the same dataset.

When combining representations, we are careful to avoid data leakage. Because the feature selection process trains on the features and their labels, selecting features on all samples before the CV split would leak validation-level information into the training, which is a well-documented bias in feature selection [185, 186]. As such, we run each hybrid search within a CV fold. When a single representation is scored, we impute missing values with training-fold medians. A representation wider than 5,000 features is scored on its own using only its top 500 features (200 on datasets under 1,000 samples). These are the features that a LightGBM model of 50 trees of depth 5, fitted on the training part of each fold, splits on most often. This affects only that representation's score, not the features the hybrid can select from it.

The data are split at three levels within CV runs. Every split keeps each group whole, a scaffold for small molecules, a sequence cluster for antibodies, and some predefined grouping for datasets from other domains, as Section 5.2.7 describes. With fewer than five groups, the inner and internal splits use as many folds as there are groups, as listed in Additional file 16. Figure 5.2 shows the three levels as nested boxes.

For each representation k, the importance method gives every feature a score $s_j^{(k)} \geq 0$, computed on the 80% part of the internal split. These scores rank features only within their own single representation. The standalone score r_k is the score of the internal evaluator, trained on representation k in the 80% part and scored on the 20% part, in the dataset's metric. These scores cannot be compared across metrics as they stand: a lower RMSE is better and it has no fixed upper limit, whereas a Spearman correlation is higher when better and lies between -1 and 1. Before the budget is split, each r_k is therefore converted to a score $\tilde{r}_k$ on which 0 means the representation predicts no better than a trivial predictor and 1 means it predicts perfectly. The trivial

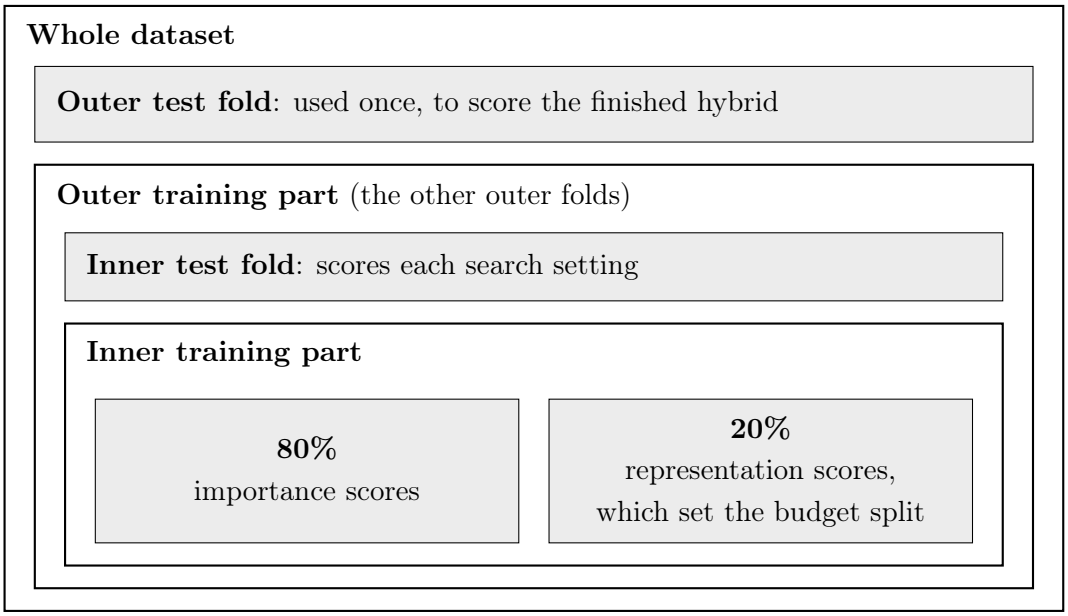


Figure 5.2: The three levels of splitting, each inside the one before. Every split keeps each group whole.

predictor depends on the metric. For Spearman, Pearson, the Matthews correlation coefficient (MCC), and R^2, $\tilde{r}_k = \max(0, r_k)$. For the area under the receiver operating characteristic curve (AUROC) and the concordance index, $\tilde{r}_k = \max(0, 2(r_k - 0.5))$. For RMSE, $\tilde{r}_k = \max(0, 1 - \mathrm{RMSE}_k/\mathrm{RMSE}_0)$, where RMSE_0 is the error of predicting the training mean, and mean absolute error is rescaled likewise against the training median. For accuracy and weighted F1, $\tilde{r}_k = \max(0, (r_k - r_0)/(1 - r_0))$, where r_0 is the score of always predicting the majority training class.

An optional representation count N keeps the N representations with the highest standalone scores on the fold and discards the rest for that iteration of the search. Each remaining representation receives a weight $w_k = g(\tilde{r}_k)$, with g set by the allocation method, and a share $b_k^* = B\, w_k / \sum_l w_l$ features, rounded by largest remainder and capped at its width. A representation left out by the representation count, or whose standalone fit fails, receives no features. Thus $\sum_k b_k = \min(B, \sum_k d_k)$ and $b_k \leq d_k$, and a representation with $w_k > 0$ keeps at least one feature. The global pool does not use these shares (see below).

Let S_k be the top b_k features of representation k by $s_j^{(k)}$, among those with $s_j^{(k)} > 0$; if none scores above 0, its full b_k is taken. The cut exists because tree importance gives exactly 0 to every feature no tree split on, and filling the budget with those would pick them by their position in the matrix. The hybrid is $\mathbf{H} = [\mathbf{X}^{(1)}_{:,S_1}, \ldots, \mathbf{X}^{(K)}_{:,S_K}]$, with $\sum_k |S_k| \leq B$, so it can be narrower than its budget.

The hybrid search returns a list of settings for feature selection, referred to here as

a configuration. It is scored by fitting the downstream model on the hybrid in each fold of the search and averaging the held-out scores. Each pair of downstream model and budget has its own search, and thus its own winner. Because this score is chosen over a range of selected configurations, it is optimistic [187, 188]. However, so is the score of the highest-performing single representation it is compared with, which is also the best of several. The whole search runs inside each outer training fold, and the winning settings are rebuilt there and scored once on the untouched outer test fold, both for the winner at each budget and for each downstream model's best across budgets.

The inner search score and the outer test score are reported side by side, with the budget each fold picked, in Section 5.3.1, "Overfitting of the search". Two variants of this search are used later. In Chapter 7, under label noise, training quantities use noisy labels while held-out scoring uses clean labels. Under the nearest-neighbor objective, each sample's label is predicted from its five nearest neighbors in the feature space, as in Chapter 6.

As explained above, a hybrid search returns a list of settings that function as a feature selection method to generate the hybrid representation itself. These settings are recorded in a blueprint that can be used on new folds of the same dataset or even new data. The blueprint contains additional information, including some fields for analysis, meta-data, and scores. These scores are used only for comparative purposes as in this research, the hybrids are rebuilt per-fold and are scored as such. Additional file 17 lists every field of a blueprint and its type. The blueprint does not record the specific features that are returned by the hybrid, as it is meant to be re-run using the same settings on new, labeled data. Alternatively, you can reuse the features of a labeled fit and apply them to data with no labels under the assumption that the same features matter between sets.

The blueprint is extended by an additional explainability layer. Since it does not store feature names, this additional step runs on a completed hybrid, records where each selected feature came from and, wherever possible, what that feature means. An ECFP4 bit is named by the atom environment that sets it, written as a SMILES fragment from a molecule with the bit set. MACCS keys are named by their SMILES arbitrary target specification (SMARTS) pattern and a short name, and descriptor features by descriptor name. Protein composition, descriptor and k-mer features, antibody composition, one-hot, BLOSUM, structural and inverse-folding features and PCA components are named too. MinHashed atom-pair fingerprint (MAP4) and differential reaction fingerprint (DRFP) features get only a bit index, and Avalon, Mordred, atom-pair, graph-kernel, and learned-embedding features only the representation name

and dimension index. The Mordred descriptor names in Figure 5.6 were added in a separate step.

5.2.2 The search space

To run a hybrid search, we first measure feature importance, select how many features to take from each contributing single representation, and choose how to score a candidate feature set. And for each of those steps, there's no one right way to do it. Different prediction tasks prefer different methods of scoring feature importance or candidate features, different sets of input representations, and so forth. The search tries to find the ideal settings for the hybrid feature selection process, tailored to a given dataset and downstream model. All combinations of settings values are explored, as the settings interact. The best budget depends on the downstream model, as shown below for nanobody thermostability. This search grid also tests the chapter's claim that no single setting suits every dataset.

Six settings in total are searched, namely the downstream model, the budget, the representation count, the importance method, the allocation method, and the internal evaluator. Table 5.5 gives the values each setting can take, and Table 5.6 the smaller set searched on each dataset in this chapter. The importance methods tested are LightGBM gain, TreeSHAP, or DeepLIFT (described below). Feature-level importance depends on the model as well as on the data [189], and as such we observe that within the search, a different ranking of features produces a different hybrid output.

The downstream model is trained on the hybrid representation. This model can be a supervised model of any form, classification or regression. We used RF, XGBoost, LightGBM, SVM, a deep neural network (DNN), and a GP. Each dataset in this chapter uses two of them.

The budget is the maximum number of features in the hybrid, taken from eight values (10, 25, 50, 100, 250, 500, 1,000, and 2,000 features). Every feature counts once against the budget, whether it is an ECFP4 bit or a PDV descriptor such as logP. Bits and descriptors from different representations do not compete for the budget directly. As Section 5.2.2, "Feature allocation methods", explains, each representation's share is set first, and its features are ranked only against each other. The one exception is the global pool, an allocation method that ranks features from different representations against each other. When the budget is too small, informative features can be left out. When it is too large, the hybrid picks up too many uninformative features, and on small datasets these give the model room to overfit.

We select a set of up to 10 single representations per dataset, chosen prior to the search as seen in Table 5.12. The selections were made based on each representation's relative performance on the prediction task. We took into account what type of information each representation contained, and tried to span a wide set across the selected 10. For example, while there are many high-performing graph-based embeddings in QSAR modeling, for the purposes of testing a feature selection process which takes features from different sources of information, it is better to have a mix of graph-based embeddings, as well as other types of molecular representations like substructure fingerprints, property vectors, sequential embeddings, etc. The representation count (2, 3, 5, or all) sets how many may contribute features, keeping those with the highest standalone scores on each fold. The number of representation counts to test is searched in the sweep as datasets differ in how many representations have features worth selecting, and pooled feature selection over all representations has no such setting. While we tested various methods of pooling the features, we found computing feature importance scores and concatenation techniques on a global pooled feature set was less often best in preliminary runs than when the feature scoring is kept on a per-representation basis. Additional file 15 reports these preliminary runs. Since the features contain different types of information at different scales, pooling the features too early makes comparisons within the pool challenging. We found that it's better to determine which features are important to each representation first, and go from there.

Every allocation method is paired with every count of input representations used. Some datasets performed best with 2 or 3 representations and others with all 10. Some prediction tasks had a very clear preference. For instance, on the HOMO–LUMO gap with LightGBM, which searched 3 and all representations, preferred 3 on every fold.

Feature importance scoring

We run a feature importance scoring method on each single representation, which gives each feature a single positive score. These scores remain relative within a given representation, so they are not compared or normalized between representations. Split-based tree importance is biased toward features with many possible split points [190], which matters here because we use binary fingerprint bits alongside continuous embeddings. No single importance method works best across datasets and downstream models. After testing a wider variety of methods found across literature, we settled on three methods whose rankings did not correlate strongly with each other and whose selected features performed well as model input. Additional file 15 gives these tests.

We score features using LightGBM gain importance, SHapley Additive exPlana-

tions (SHAP) values for tree ensembles (TreeSHAP), and DeepLIFT (Deep Learning Important FeaTures) [191]. LightGBM gain measures how much a feature reduced the training loss where trees split on it [67]. TreeSHAP gives each feature's exact Shapley value for each molecule's prediction from a tree model [192]. DeepLIFT measures how much a feature moves the output of a small network, trained with Adam [193], away from its output on an all-zero reference input, which after standardization places every feature at its training mean [191, 194, 195]. On Caco-2, the median share of features scoring above 0 was 30% for LightGBM gain, 70% for TreeSHAP, and 90% for DeepLIFT (preliminary, over 10 representations and 3 folds). Table 5.2 gives the settings of each method, including the pre-screen of representations wider than 2,000 features. Each dataset searches one, two, or all three importance methods depending on its individually decided search settings, given in Table 5.6.

Method	Model fitted per representation	Score of one feature	Classification labels	Representations over 2,000 features
LightGBM gain	LightGBM, 50 trees, depth 6	Summed loss reduction of its splits; 0 if never split on	One model on the class labels	2,000 highest-variance features fitted; the rest score 0
TreeSHAP	RF, 50 trees, depth 10	Mean absolute SHAP value over training molecules	Binary: positive class; multiclass: mean over classes	As for LightGBM gain
DeepLIFT	NN, ReLU layers of 64 and 32 units, 100 epochs of Adam, learning rate 0.001	Mean absolute attribution against an all-zero reference (Rescale rule)	Binary: positive class; multiclass: mean over classes	Every feature scored

Table 5.2: Settings of the importance methods, fitted per representation and fold. Unlisted settings are library defaults.

Feature allocation methods

The allocation method decides how many of the B features each representation contributes while the importance method decides which. Of the five methods searched, four give each representation a fixed share computed from its rescaled standalone score (linear, squared, softmax, and rank-decay allocation). The fifth, the global pool, lets the candidate features offered by every representation compete in one importance ranking.

The four fixed-share methods compute budget shares from the standalone score on internal validation. Only linear allocation makes the share proportional to that score. All four assume that representations with stronger predictive performance contribute more valuable features. This assumption fails when a strong representation relies on a few select features. In that case, a large share of features will be given to that representation, picking up on weaker features and potentially missing out on stronger ones from other representations. Fixed shares ignore complementarity, so two representations carrying the same information both receive a share. Only the global pool ranks features from different representations against each other; however, ranking feature importance on a global pool is dubious as the features are not necessarily appropriate to compare.

The first method of feature allocation we tried was a greedy forward selection, in which the hybrid is built one block of features at a time [38, 179]. In each round, the next block is added from representations in order of which improves the internal validation score. This in theory lets it choose features for how well they work together. In practice, this is very expensive. With blocks of 10 features and a budget of 500 and 10 representations, it needs up to 500 model fits per configuration and fold against 10 for a score-based method. It was also less accurate; in preliminary runs on Jain antibody neonatal Fc receptor (FcRn) retention it placed 16th of 17 allocation methods, with a Spearman correlation of 0.360 against 0.453 for linear allocation. Additional file 15 gives these runs, and we therefore removed it from the search.

The fixed-share methods differ only in how w_k is computed from $\tilde{r}_k$, as seen in Table 5.3 with a worked example in Table 5.4. Because every rescaled score lies between 0 and 1, softmax cannot give one representation most of the budget. Across all 10 single representations the best takes at most 23.2%. Rank decay uses only the order: the best representation receives 27.3% and the tenth 1.8%, whatever the gaps between their scores.

The global pool works in two steps. First, each representation offers a number of candidate features, set by its squared weight $\tilde{r}_k^2$ as a share of a pool of $2B$ candidates. Each offer is capped at the representation's width and holds only features that score above 0. Second, the same importance method is fitted once on all the offered features joined together, and the hybrid keeps the top B that score above 0. How many features a representation contributes is therefore decided by this joint ranking, and it can be none. A feature's rank also depends on what the other representations offered. In this joint ranking, fingerprint bits compete directly with continuous descriptors and embedding values. Split-based tree importance is biased toward features with many possible

Method	Weight w_k	Fixed setting	How features are chosen	When a representation receives 0 features
Linear	$\tilde{r}_k$	none	Top b_k of its own ranking	$\tilde{r}_k = 0$
Squared	$\tilde{r}_k^2$	power 2	Top b_k of its own ranking	$\tilde{r}_k = 0$
Softmax	$\exp\big((\tilde{r}_k - \max_l \tilde{r}_l)/T\big)$	$T = 1$	Top b_k of its own ranking	never from its score
Rank decay	$e^{-0.3\rho_k}$, ρ_k = rank by score, 0 for the best	decay 0.3	Top b_k of its own ranking	never from its score
Global pool	$\tilde{r}_k^2$, sizing its offer to a pool of $2B$ candidates	power 2; pool $2B$	Top B of one joint ranking over all offered candidates	$\tilde{r}_k = 0$, or none of its candidates in the joint top B

Table 5.3: The five allocation methods. w_k is the weight computed from representation k's rescaled standalone score $\tilde{r}_k$, B is the budget, b_k is the share of representation k, ρ_k is its rank by score, and T is the softmax temperature.

Method	$\tilde{r}_1 = 0.6$	$\tilde{r}_2 = 0.5$	$\tilde{r}_3 = 0.2$
Linear	46	39	15
Squared	55	39	6
Softmax	39	35	26
Rank decay	44	32	24

Table 5.4: Worked example of the four fixed-share allocation methods for a budget of 100 features and three representations. Each entry is the number of features a representation receives, with shares rounded by largest remainder.

split points [190]. With LightGBM gain or TreeSHAP, a binary bit can therefore rank below a continuous feature that carries no more information.

Search configuration and internal evaluation

The most useful features for an RF may not be the most useful for an SVM, and vice versa. The relative importance of a feature is dependent on the model [189]. Within the search grid, we use an internal evaluator to fit on one single representation at a time. This process is separate from the final downstream model evaluation. In a standard search, four internal evaluators are used: RF, DNN, LightGBM, and the matched evaluator. The matched evaluator scores each single representation with the same model type as the downstream model. Each internal evaluator is fitted on the internal training fold and scores on the internal validation fold within that CV iteration using the dataset's specified evaluation metric.

In the full grid, an RF, LightGBM, or DNN downstream model is searched with

three internal evaluators, because its matched evaluator is already one of the other three. An XGBoost, SVM, or GP downstream model is searched with four, because the matched evaluator adds one of its own family. Table 5.6 shows the five datasets that searched fewer. Epidermal growth factor receptor (EGFR) binding has no RF evaluator, expression titer and Buchwald–Hartwig yield have two evaluators, and affinity-capture self-interaction nanoparticle spectroscopy (AC-SINS) self-association has no GP evaluator. SMAC colloidal stability scores the SVM's hybrids without the DNN evaluator. Only the DNN evaluator shares its downstream model's architecture and training settings. Even it draws its 20% early-stopping samples at random, whereas the downstream DNN splits them by the outer split's groups. Additional file 13 gives every evaluator setting.

The internal evaluator's scores are the standalone scores r_k. Using the allocation methods in Table 5.3, they set each representation's share b_k of the budget and, through the representation count, which representations are kept. The internal evaluator does not rank features within a representation. Each finished hybrid is then scored by fitting the downstream model on it. Selecting a feature set by fitting a model on it, and keeping the set that scores best, is known as the wrapper method [179]. This is distinct from a filter method which scores features without fitting a model.

The search tests every combination of the six settings on a dataset under 5-fold CV. Over every value of Table 5.5, this is 10,080 configurations with all six downstream models, or 8,160 without the GP. Additional file 14 gives the search size. With each dataset's own values in Table 5.6, the searches reported in this chapter scored 108 to 3,360 configurations per outer fold.

Each dataset's search space is hand-selected from the full search, made to save compute, as Section 5.2.5 describes. At worst, it finds a less informative hybrid than the full search would have. This decision was made due to time limitations.

5.2.3 Handling small and specialized feature sets

Up to this point, we have considered features as coming from a whole representation of a molecule. However, you can compute features about a molecule that are not descriptive or expansive enough to be considered a full representation. Examples of these features include log-likelihood ratios, 2- and 3-D shape features, and pharmacophore features. Others, such as mutation features that record differences from a parent antibody, may carry more meaning beside a full representation. Therefore, on some datasets we add them to the hybrid as augmentation features. These are extra features for each sample,

not the extra samples of data augmentation. We call the finished hybrid **H** the base and the feature sets that supply extra features the donors. Donors are set per dataset in the Augmentation column of Table 5.12:

- small molecules: MACCS keys, 3D pharmacophore features, shape features (ultrafast shape recognition, USR, and USR with CREDO atom types, USRCAT), the Coulomb matrix, topological torsions and, on hERG blocking, MolCLR;
- antibodies: mutation features, log-likelihood ratios, amino-acid composition, and the paratope and structural features;
- Buchwald–Hartwig: MACCS keys and 3D pharmacophore features of the reactants and of the product.

Uncapped augmentation adds the extra features outside the budget, and capped augmentation puts them in place of the weakest base features. In an earlier search of 16 datasets, neither augmented condition beat no augmentation by more than 1% on any dataset, or 3% on datasets under about 1,000 samples. The searches in this chapter therefore ran without augmentation. Only the kNN search of Chapter 6, which uses a kNN model as the downstream model, on standup monolayer adsorption chromatography (SMAC) colloidal stability and hydrophobic interaction chromatography (HIC) retention, keeps uncapped augmentation. Additional file 18 gives how donor features are ranked and added, and the three conditions.

5.2.4 Pruning the search space

Adaptive pruning stops scoring a configuration that is far behind after two of the five folds, since the other three folds would only confirm its loss. The per-fold searches reported in this chapter ran without it and instead used the smaller grid per dataset given in Section 5.2.5.

Adaptive pruning: every configuration is first scored on the first two folds. At budget 100, within each (model, evaluator, representation count) group, (allocation, importance) pairs with no configuration in the better half are not searched at larger budgets. Each axis is then voted on within groups that fix every other axis: an allocation method is pruned if it is in the best 2 in fewer than 20% of groups, a representation count if it is best in fewer than 10%, an importance method if it is in the best 2 in fewer than 20%, and an internal evaluator if it is in the best 3 in fewer than 10%. Survivors are scored on the remaining folds.

5.2.5 Search grid per dataset

We identified such a large range of settings in the search space, as seen by every value in Table 5.5, that running every setting for every fold of every dataset would be costly and frankly wasteful. It is more efficient to adaptively prune the search space, or rely on hard-coded search settings for each dataset. Due to time constraints, we chose the latter as it gave us a bit more flexibility. From earlier searches across each dataset, we identified two downstream models with high scores and reasonable training times. Likewise, we kept the budgets whose ten best configurations scored within 1% of the best budget (3% on datasets under about 1,000 samples), widened to at least three consecutive budgets. Importance methods, allocation methods, internal evaluators, and representation counts that scored well below the best were dropped.

Two datasets, QM9 and hERG blocking, were too slow for the earlier search over every value. They were searched over a shorter fixed list, called the fast tier (LightGBM gain importance; linear, squared, and rank-decay allocation; three or all representations), and their grid stays within it. The search with the nearest-neighbor objective has its own grid, given in Chapter 6.

Setting	Values
Downstream models	RF, XGBoost, LightGBM, SVM[a], DNN, GP[b]
Internal evaluators	Matched evaluator, RF, LightGBM, DNN
Feature budgets (features)	10, 25, 50, 100, 250, 500, 1,000, 2,000
Allocation methods	Linear, squared, softmax, rank decay, global pool
Importance methods	LightGBM gain, DeepLIFT, TreeSHAP
Representation counts	2, 3, 5, all representations

[a]Not fitted above 10,000 molecules (5,000 when augmentation features are added outside the budget). [b]Not fitted above 1,000 molecules.

Table 5.5: Values the hybrid search can take for each setting. The matched evaluator belongs to the same model family as the downstream model.

Additional file 19 follows hERG K_i through these cuts, from the 8,220 configurations of its earlier search, which included kNN among six downstream models, to 224, and its final grid still contains the best configuration.

5.2.6 Evaluation protocol

Within each experiment, the dataset is split into outer folds, as set out in Section 5.2.7. The search sees only the training samples of each outer fold, and the selected features produced by the search are rebuilt on them and applied unchanged to the held-out

Dataset	Downstream models	Budgets (features)	Importance	Allocation	Internal evaluators	Representation counts	Configurations
Small molecules							
QM9†	DNN, LightGBM	500–2,000	G	L, Q, R	RF, LGBM, DNN	3, all	108
hERG K_i	SVM, DNN	250–2,000	G, T	L, Q, S, R	RF, LGBM, DNN, SVM	all	224
Caco-2 efflux	DNN, SVM	500–2,000	G, T, D	L, Q, S, R	RF, LGBM, DNN, SVM	all	252
LogD	DNN, SVM	500–2,000	G	L, Q, R	RF, LGBM, DNN, SVM	2, 3, 5, all*	252
Rat microsomal clearance	SVM, DNN	500–2,000	G, T, D	all five	RF, LGBM, DNN, SVM	all	315
MDR1–MDCK efflux	SVM, DNN	500–2,000	G, T, D	all five	RF, LGBM, DNN, SVM	all	315
hERG blocking†	LightGBM, XGBoost	100–2,000	G	L, Q, R	RF, LGBM, DNN, XGB	3, all	210
Antibodies							
Anti-VEGF scan expression	SVM, DNN	250–2,000	G, T, D	all five	RF, LGBM, DNN, SVM	all	420
Anti-VEGF scan binding	RF, SVM	250–1,000	G, T	all five	RF, LGBM, DNN, SVM	all	210
Nanobody thermostability	DNN, SVM	50–2,000	G, T	all five	RF, LGBM, DNN, SVM	2, 3, 5, all*	1,680
EGFR binding	DNN, SVM	250–2,000	G, T	all five	LGBM, DNN, SVM	2, 3, 5, all*	800
Angiopoietin-2 binding	GP, SVM	25–100	G, T	all five	RF, LGBM, DNN, GP, SVM	3, 5, all	720
Antibody humanness	XGBoost, RF	10–2,000	G, T, D	all five	RF, LGBM, DNN, XGB	2, 3, 5, all	3,360
Binding free energy	DNN, SVM	100–500	G, T	L, S, R	RF, LGBM, DNN, SVM	3, 5, all	378
SMAC colloidal stability	DNN, SVM	500–2,000	G, T, D	all five	RF, LGBM, DNN§, SVM	3, 5, all	810
HIC retention	DNN, GP	250–2,000	G, T, D	all five	RF, LGBM, DNN, GP	5, all	840
AC-SINS self-association	LightGBM, GP	50–2,000	G, T, D	L, Q, S, R	RF, LGBM, DNN	2, 5, all	1,296¶
Expression titer	DNN, RF	250–2,000	T, D	all five	RF, DNN	2, 3, 5, all	640
CHO polyreactivity	DNN, GP	250–2,000	T, D	all five	RF, LGBM, DNN, GP	2, 5, all	840
Chemical reactions							
Buchwald–Hartwig yield	RF, LightGBM	500–2,000	T, D	all five	RF, LGBM	2	120

†Fast tier. *The representation counts were reduced after the search ran. §The DNN evaluator scored only the DNN's hybrids, not the SVM's. ¶The GP evaluator was in the grid but failed on every configuration on four folds; on the third outer fold, 155 of its 216 configurations scored, giving 1,451.

Table 5.6: Search grid of each dataset, as run. One row is one dataset. Budgets are consecutive values of Table 5.5. Importance: G, LightGBM gain; T, TreeSHAP; D, DeepLIFT. Allocation: L, linear; Q, squared; S, softmax; R, rank decay; all five adds the global pool. Internal evaluators include the matched evaluator of each downstream model, except the Gaussian process on AC-SINS (RF, random forest; LGBM, LightGBM; XGB, XGBoost; GP, Gaussian process; VEGF, vascular endothelial growth factor). Configurations is the number scored in one outer fold's search, over both downstream models.

samples. Only three experiments limit each held-out feature to the range it takes on the training samples: the comparison with the best single representation, the frequent and rare scaffolds, and the comparison of many and few representations.

Because of time and computational constraints, not every experiment in Chapters 5 to 7 ran on every fold or every dataset. Some experiments here ran on the first three outer folds, most noise experiments in Chapter 7 ran on the first outer fold only, and some experiments have no result on some datasets. We report the results that exist, give the number of folds beside each one, and name the datasets where a result is missing.

We compare the hybrid with the best single representation, using all of its features, as this is what a user would otherwise run. This best representation is picked from the search's scores on the outer training samples. It is picked separately for each downstream model and each fold, so it can change between folds. It is then refit on the training samples and scored on the same held-out samples as the hybrid. The hybrid is held to its budget, so the single representation often has many times more features. We record both numbers of features, since part of any difference in score can come from the difference in width. The main result is the hybrid's score minus the score of the best single representation. A positive value means the hybrid scored better, and for RMSE we flip the sign so that this still holds.

No test set exists outside these outer folds. Each reported score is on an outer test fold that the search of that fold never saw. The hybrid's settings and the best single representation are both chosen from scores on the outer training samples only. One choice was made before these searches. Each dataset's grid, including its two downstream models, came from an earlier search of the whole dataset, described in Section 5.2.5. That earlier search included the samples that the outer folds later hold out.

5.2.7 Datasets and splitting

The experiments in this chapter were run on 20 datasets outlined in Table 5.7. Of those 20, seven are small molecules, 12 are antibodies, and one is a reaction dataset.

Table 5.7: The 20 datasets of the hybrid experiments after curation, grouped by domain. n is the number of samples, and the fold groups are given with the number of groups in parentheses. For classification tasks, the class counts are in parentheses in the label column. CL_{int} is the intrinsic clearance, CGR is the condensed graph of reaction, and arrows show whether lower or higher is better. The hERG blocking data were supplied by Lhasa Limited. The last column gives, for each of the dataset's two downstream models, the single representation the search ranked first on the training samples of at least three of the five outer folds.

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)	Best single representation
Small molecules					
QM9 [113, 196]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)	Mordred (both)
hERG K_i [115]	pK_i	regression, RMSE ↓	1,412	scaffold (769)	SVM: MHG-GNN; DNN: GROVER
Caco-2 efflux [114]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)	DNN: MHG-GNN; SVM: PDV
LogD [114]	logD at pH 7.4 (no unit)	regression, RMSE ↓	5,028	scaffold (2,413)	DNN: Mordred; SVM: MHG-GNN
Rat microsomal clearance [142, 197]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)	SVM: PDV; DNN: MHG-GNN
MDR1–MDCK efflux [142, 197]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)	PDV (both)
hERG blocking	blocker or not (3,630 : 4,142)	binary, AUROC ↑	7,772	scaffold (4,002)	MHG-GNN (both)
Antibodies					
Anti-VEGF scan expression [198, 199]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position (225)	ESM3 CDR (both)
Anti-VEGF scan binding [198, 199]	binding enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position (225)	RF: ESM3; SVM: varies
Nanobody thermostability [200]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)	SaProt (both)
EGFR binding [201]	mutational scan score	regression, Spearman ↑	659	mutated position (36)	SVM: Ab-RoBERTa CDR; DNN: varies
Angiopoietin-2 binding [201]	mutational scan score	regression, Spearman ↑	397	mutated position (20)	SaProt (both)
Antibody humanness [202]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)	RF: AntiBERTy CDR; XGBoost: varies
Binding free energy [203, 204]	$\Delta\Delta G$ (kcal/mol)	regression, Spearman ↑	429	complex (30)	SVM: Mutation features; DNN: ESM-2 shift from parent
SMAC colloidal stability [205, 206]	SMAC retention time	regression, Spearman ↑	242	competition fold (5)	DNN: ProtT5; SVM: varies
HIC retention [206]	HIC retention time	regression, Spearman ↑	242	competition fold (5)	ESM-2 (joined chains) (both)

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)	Best single representation
AC-SINS self-association [206]	AC-SINS shift at pH 7.4	regression, Spearman ↑	242	competition fold (5)	LightGBM: IgT5; GP: AbLang-2
Expression titer [206]	expression titer	regression, Spearman ↑	239	competition fold (5)	RF: IgT5; DNN: varies
CHO polyreactivity [206]	polyreactivity to CHO cell proteins	regression, Spearman ↑	197	competition fold (5)	ProtT5 (both)
Chemical reactions					
Buchwald–Hartwig yield [207]	yield (%)	regression, RMSE ↓	3,955	additive (22)	RF: CGR fingerprint; LightGBM: MHG-GNN (difference)

Small molecules and reactions		Antibodies	
Dataset	Short name	Dataset	Short name
QM9	QM9	Anti-VEGF scan expression	VEGF-E
hERG K_i	HK_i	Anti-VEGF scan binding	VEGF-B
hERG IC_{50}	HIC_{50}	Nanobody thermostability	Nb-Tm
hERG blocking	HB	EGFR binding	EGFR
Caco-2 efflux	Caco-2	Angiopoietin-2 binding	Ang2
LogD	LogD	Binding free energy	$\Delta\Delta G$
Rat microsomal clearance	RLM	SMAC colloidal stability	SMAC
MDR1–MDCK efflux	MDR1	HIC retention	HIC-R
Buchwald–Hartwig yield	BH	AC-SINS self-association	AC-SINS
		Expression titer	Titer
		CHO polyreactivity	PR-CHO
		Antibody humanness	Hum

Table 5.8: Short names of the datasets, used in the figures.

Section S.7 of the Additional files gives the shared and per-dataset pre-processing steps of the small-molecule and reaction datasets and of Koenig expression, nanobody thermostability, and humanness, with the counts after each step. The LogD and Caco-2 labels are clipped at their 1st and 99th percentiles, so on Caco-2, for example, 22 molecules share the highest label and 23 the lowest.

As in Chapter 3, no group is split between folds, and classification folds are also stratified with a fixed seed. Every dataset is split into five outer folds, though some experiments ran on fewer, as explained in Section 5.2.6. As set out in Section 2.6, no split is random. On a given dataset, every representation, model, and hybrid is scored on the same folds.

Groups by domain

The small-molecule datasets, QM9 included, are grouped by Bemis–Murcko scaffold, maintaining stereochemistry. On QM9 the largest group is the empty scaffold of acyclic molecules, with 1,041 of the 9,978 molecules. The two datasets from Biogen [142], rat microsomal clearance and MDR1–MDCK efflux, have 1.2 and 1.1 molecules per scaffold, respectively, so holding out a scaffold there is essentially like holding out single molecules, meaning in practice it is random. Chapter 3 used a different scaffold key and different curation for its versions of LogD, Caco-2, hERG K_i, and QM9, and Section S.7.2 of the Additional files lists the differences.

The reaction dataset is Buchwald–Hartwig C–N couplings from high-throughput plates. Following Ahneman et al. [207], it is grouped by additive: each held-out fold holds four or five additives unseen in training, as outlined in Section S.7 of the Additional files.

In the Koenig deep mutational scan of the anti-vascular endothelial growth factor (VEGF) antibody G6, each variant carries one mutation. Variants are grouped by their mutated position, producing 225 groups. With all mutations at one position on the same side of a split, a model cannot look up a per-position effect at test time [208]. The nanobody and humanness sets are grouped by single-linkage clusters at 95% sequence identity, the identity threshold Kenlay et al. [165] used to cluster their pretraining sequences.

The two AbAgym deep mutational scans [201], EGFR and angiopoietin-2 binding, are grouped by mutated position, as for the Koenig scans, giving 36 and 20 groups. The $\Delta\Delta G$ set joins antibody–antigen entries from SKEMPI 2.0 [203] and AB-Bind [204]. It is grouped by protein complex, producing 30 groups. The five Ginkgo developability assays are measured on the same panel of antibodies. Two are retention times, in hydrophobic interaction chromatography (HIC) and standup monolayer adsorption chromatography (SMAC). The others are affinity-capture self-interaction nanoparticle spectroscopy (AC-SINS), expression titer, and polyreactivity to Chinese hamster ovary (CHO) cell proteins. We use the five CV folds released with the competition [206] as the groups, which makes the outer folds the competition's own.

5.2.8 Representations by domain

Any representation that takes the form of a per-sample fixed-length vector can serve as an input representation in the adaptive hybrid search. The first step in the search is to rank how well each single representation does using the given evaluation. The

distinction of "best" is dependent on the dataset as well as the downstream model. These scores and relative rankings are later used in the concatenation step of the feature selection process.

If an input cannot be parsed or embedded, it gets an all-zero row, so that every representation of a dataset keeps the same rows. In computed descriptors, a missing or infinite value is set to 0, and values are stored unscaled. A pretrained model gives one vector per input, the mean of its last-layer token vectors with padding excluded, unless otherwise stated.

Small molecules

The small-molecule representations in Table 5.9 are the fingerprint, descriptor, graph, and language-model families described in Chapter 2. The only additions are four sets of 3D shape and pharmacophore features, used as augmentation only. All four are computed from a single conformer per molecule, embedded with a fixed seed and optimized with the Merck molecular force field. A single conformer may misdescribe flexible molecules.

Chemical reactions

Reaction datasets use the three kinds of representation listed in Table 5.10. The condensed graph of reaction (CGR) merges reactants and products into one graph in which the bonds that change are marked [209, 210]. Here it is hashed into a circular fingerprint after atom mapping [211]. Reaction language models read the reaction SMILES directly [212, 213]. Any molecular representation can also be applied to the reactants, to the product, or to their difference. A one-hot encoding of which aryl halide, ligand, base, and additive was used carries no chemistry. It is kept because on plate data such identity features can predict yield about as well as chemical descriptors [214].

Antibodies

Antibody representations, listed in Table 5.11, start from the heavy- and light-chain amino-acid sequences. Besides one-hot and BLOSUM62 encodings [215], most are frozen language models, either trained on all proteins or on antibody repertoires only. Most models embed each chain separately and join the two vectors, so their width doubles on datasets with both chains. Several representations read one model in different ways, for instance averaging over earlier layers or over the complementarity-determining

region (CDR) residues only, with CDRs found by international ImMunoGeneTics information system (IMGT) numbering [216] using ANARCI [217]. Datasets of point variants of one parent antibody also use features that describe each mutation.

Table 5.9: Representations used for small molecules. In the Role column, B marks a single representation, A an augmentation, and B/A a representation that is a single representation on some datasets and an augmentation on others.

Representation	What it encodes	Features	Source	Role
ECFP4	hashed circular substructures, radius 2	2048	[5]	B
Atom pairs	hashed atom-type pairs with bond distance	2048	[218]	B
Topological torsions	hashed paths of four bonded atoms	2048	[219]	B/A
Avalon	hashed paths, rings, and atom environments	2048	[8]	B
MACCS keys	166 predefined substructure keys	167	[220]	B/A
WL graph kernel	Weisfeiler–Lehman label counts, 5 rounds, labels present in at least 5% of molecules	140–317	[221]	B
PDV	RDKit 2D physicochemical descriptors	200	[37]	B
Mordred	2D molecular descriptors	1613	[222]	B
GROVER	frozen pretrained graph transformer	4800	[223]	B
MHG-GNN	frozen pretrained graph encoder	1024	[9]	B
GraphMVP	frozen 2D graph encoder pretrained with 3D views	300	[224]	B
MoLFormer	frozen SMILES LM; public checkpoint trained on 10% of its corpus	768	[47]	B
SMI-TED	frozen SMILES encoder–decoder	768	[225]	B
ChemBERT	frozen SMILES LM; community checkpoint trained on ChEMBL following the cited method	256	[226]	B
ChemBERTa-2	frozen SMILES LM	384	[33]	B
SELFormer	frozen SELFIES LM	768	[227]	B
Mol2vec	summed substructure word vectors	300	[30]	B
3D pharmacophore	counts of 7 pharmacophore types, and mean and spread of distances for the 21 type pairs, one conformer	49	[37]	A
USR	shape moments from 4 reference points, one conformer	12	[228]	A
USRCAT	USR per pharmacophore atom type, one conformer	60	[229]	A
Coulomb matrix	sorted eigenvalues of the Coulomb matrix, one conformer	50	[230]	A
MolCLR	frozen graph encoder pretrained by contrastive learning	512	[231]	A

Table 5.10: Representations used for chemical reactions. Role is read as in Table 5.9, a dash in the Role column marks a representation used on no dataset, and the suffix in parentheses states whether a molecular representation is computed on the reactants, on the product, or as their difference.

Representation	What it encodes	Features	Source	Role
CGR fingerprint	circular fingerprint, radius 2, hashed from the CGR	2048	[209, 210]	B
rxnfp	frozen reaction LM	256	[212]	B
ReactionT5	frozen reaction LM	768	[213]	B
Component one-hot	which aryl halide, ligand, base, and additive	45	[207]	B
GROVER (reactants)	GROVER averaged over reactants	4800	[223]	B
Mordred (reactants)	Mordred averaged over reactants	1613	[222]	B
Mol2vec (reactants)	Mol2vec averaged over reactants	300	[30]	B
ECFP4 (reactants)	ECFP4, bitwise OR over reactants	2048	[5]	–
Avalon (difference)	product minus reactant Avalon bits	2048	[8]	B
MHG-GNN (difference)	product minus reactant embedding	1024	[9]	B
MACCS (difference)	product minus reactant MACCS keys	167	[220]	B
Mol2vec (difference)	product minus reactant Mol2vec	300	[30]	–
MACCS (reactants)	MACCS keys, bitwise OR over reactants	167	[220]	A
MACCS (product)	MACCS keys of the product	167	[220]	A
3D pharmacophore (reactants)	3D pharmacophore averaged over reactants	49	[37]	A
3D pharmacophore (product)	3D pharmacophore of the product	49	[37]	A

Table 5.11: Representations used for antibodies. LM is a language model. Feature counts marked /chain double on the datasets with heavy and light chains. A dash in the Source column marks a standard encoding or one built for this thesis, and Role is read as in Table 5.9.

Representation	What it encodes	Features	Source	Role
One-hot	one-hot residues, both chains padded to fixed length	5600	–	B
BLOSUM62	BLOSUM62 row per residue, both chains padded	5600	[215]	B
Amino-acid composition	fraction of each amino acid, per chain	40	–	B/A
ESM-2	protein LM, last layer, mean over residues	1280/chain	[232]	B
ESM-2 (joined chains)	ESM-2 on heavy and light chain joined into one string	1280	[232]	B

Representation	What it encodes	Features	Source	Role
ESM-2 early layers	ESM-2, mean over layers 0–16	1280/chain	[232]	B
ESM-2 middle layers	ESM-2, mean over layers 20–33	1280/chain	[232]	B
ESM-2 CDR	ESM-2, mean over CDR residues only	1280/chain	[232]	B
ESM-2 per CDR	ESM-2, CDR1/2/3 pooled separately then joined	3840/chain	[232]	B
ESM-2 variable positions	ESM-2, mean over the 20 highest-entropy positions	1280/chain	[232]	B
ESM-2 shift from parent	ESM-2 variant minus parent embedding	1280/chain	[232]	B
ESM-2 log-likelihood ratio	masked mutant vs parent log-probability: sum, mean, min	3	[233]	A
ESM3	protein LM	1536/chain	[234]	B
ESM3 CDR	ESM3, mean over CDR residues	1536/chain	[234]	B
ESM C	protein LM	1152/chain	[235]	B
ProtT5	protein LM	1024/chain	[236]	B
ProtT5 CDR	ProtT5, mean over CDR residues	1024/chain	[236]	B
SaProt	structure-aware protein LM	1280/chain	[237]	B
AbLang-2	paired antibody LM, both chains read together	480	[238]	B
IgT5	paired antibody LM, both chains read together	1024	[165]	B
AntiBERTy	antibody LM	512/chain	[239]	B
AntiBERTy CDR	AntiBERTy, mean over CDR residues	512/chain	[239]	B
Ab-RoBERTa	antibody LM	768/chain	[240]	B
Ab-RoBERTa CDR	Ab-RoBERTa, mean over CDR residues	768/chain	[240]	B
Mutation features	wild-type and mutant residue, substitution score, changes in hydrophobicity, weight, and charge, position; averaged over mutations	48	–	B/A
ESM-2 layer 24	ESM-2, layer 24 of 33 only	1280/chain	[232]	B
ESM-2 per region	ESM-2, each of the seven IMGT regions (FR1–FR4, CDR1–CDR3) pooled separately, then joined	8960/chain	[232]	B
SaProt CDR	SaProt, mean over CDR residues	1280/chain	[237]	B
ESM C CDR	ESM C, mean over CDR residues	1152/chain	[235]	B
Structural	solvent-accessible surface area and model confidence from a predicted structure, plus the paratope features	28	[241]	A
Paratope	sequence-based summaries of the CDRs, from their boundaries	8	–	A

Representation	What it encodes	Features	Source	Role
AntiBERTy log-likelihood ratio	masked mutant vs parent log-probability under AntiBERTy	3	[239]	A

Table 5.12: Base and augmentation representations of each dataset, grouped by domain. The hybrid search selects features from the single representations. Augmentation features would be added after the hybrid is built, but no search in this chapter uses them (Section 5.2.3). A dagger marks datasets searched at the fast tier.

Dataset	Task	Single representations	Augmentation
Small molecules			
QM9† [113]	HOMO–LUMO gap, regression	Mordred, Avalon, SMI-TED, ECFP4, SELFormer, ChemBERTa-2, Mol2vec, ChemBERT, GraphMVP, MoLFormer	Topological torsions, 3D pharmacophore, USRCAT, Coulomb matrix
hERG K_i [115]	hERG binding, pK_i, regression	MHG-GNN, Mordred, WL graph kernel, Avalon, GROVER, PDV, ECFP4, Atom pairs, ChemBERT, SMI-TED	MACCS keys, 3D pharmacophore, Coulomb matrix, USR, USRCAT
Caco-2 efflux [114]	efflux ratio (log), regression	WL graph kernel, MHG-GNN, ECFP4, PDV, GROVER, Avalon, Topological torsions, SMI-TED, Atom pairs, MoLFormer	MACCS keys, 3D pharmacophore, USRCAT, Coulomb matrix, USR
LogD [114]	LogD, regression	MHG-GNN, Mordred, GROVER, ECFP4, SMI-TED, MoLFormer, Avalon, Topological torsions, ChemBERT, GraphMVP	MACCS keys, 3D pharmacophore, USRCAT
Rat microsomal clearance [142]	intrinsic clearance (log), regression	PDV, MHG-GNN, SMI-TED, Mordred, MoLFormer, GROVER, WL graph kernel, Atom pairs, Avalon, ECFP4	3D pharmacophore, MACCS keys, USRCAT, Coulomb matrix, USR
MDR1–MDCK efflux [142]	efflux ratio (log), regression	Mordred, PDV, MHG-GNN, Avalon, GROVER, SMI-TED, Atom pairs, WL graph kernel, MACCS keys, ECFP4	3D pharmacophore, Coulomb matrix
hERG blocking†	blocker vs non-blocker, binary	MHG-GNN, Mordred, GROVER, Atom pairs, Avalon, WL graph kernel, ECFP4, Mol2vec, Topological torsions, MoLFormer	USRCAT, Coulomb matrix, MolCLR
Antibodies			
Anti-VEGF scan expression [198]	expression enrichment of variants, regression	ProtT5, ESM3 CDR, SaProt, IgT5, ESM-2 early layers, ESM C, AbLang-2, ESM-2 (joined chains), ESM-2, ESM-2 middle layers	Mutation features, ESM-2 log-likelihood ratio, Amino-acid composition

Dataset	Task	Single representations	Augmentation
Anti-VEGF scan binding [198]	binding enrichment of variants, regression	ESM3, AbLang-2, ProtT5, IgT5, ESM C CDR, SaProt, ESM-2 early layers, ESM-2 variable positions, ESM-2, ESM-2 (joined chains)	ESM-2 log-likelihood ratio, Mutation features, AntiBERTy log-likelihood ratio, Amino-acid composition
Nanobody thermostability [200]	melting temperature, regression	SaProt, ESM3, ESM C, ESM-2 variable positions, ESM-2 CDR, ESM-2 early layers, ProtT5 CDR, IgT5, One-hot, ESM-2 shift from parent	Mutation features
EGFR binding	mutational scan, regression	ESM-2 (joined chains), ESM3, Ab-RoBERTa, Ab-RoBERTa CDR, AbLang-2, Amino-acid composition, SaProt CDR, ESM C, ESM-2 early layers, Mutation features	none
Angiopoietin-2 binding	mutational scan, regression	SaProt, ESM-2 middle layers, ESM-2 layer 24, AntiBERTy, ESM3, ProtT5, ESM C, Ab-RoBERTa CDR, ESM-2 CDR, Mutation features	none
Antibody humanness [202]	human vs non-human, binary	IgT5, AntiBERTy CDR, Ab-RoBERTa, AbLang-2, BLOSUM62, ESM C, ESM-2 per CDR, ESM-2 early layers, ProtT5, ESM-2 middle layers	none
Binding free energy	$\Delta\Delta G$, regression	One-hot, AntiBERTy CDR, ESM-2 layer 24, ProtT5 CDR, ESM-2 (joined chains), AntiBERTy, Mutation features, ESM-2 shift from parent	Amino-acid composition
SMAC colloidal stability [205]	retention time, regression	ProtT5, ESM-2 (joined chains), ESM-2 variable positions, ESM3, IgT5, ESM-2 early layers, AbLang-2, ESM C, SaProt, AntiBERTy	Paratope, Structural
HIC retention [206]	retention time, regression	ESM-2 (joined chains), Amino-acid composition, ESM3, ProtT5, ESM-2 early layers, IgT5, ESM-2 variable positions, ESM C, AbLang-2, ESM-2 middle layers	Paratope, Structural
AC-SINS self-association [206]	AC-SINS shift, regression	IgT5, AbLang-2, AntiBERTy CDR, ESM-2 (joined chains), Ab-RoBERTa, SaProt, Amino-acid composition, ESM C, ESM-2 early layers, ProtT5	Paratope, Structural

Dataset	Task	Single representations	Augmentation
Expression titer [206]	titer, regression	SaProt, IgT5, ESM-2, ESM C, ESM-2 variable positions, ESM-2 (joined chains), ESM-2 middle layers, AbLang-2, ProtT5, ESM3	Paratope, Structural
CHO polyreactivity [206]	polyreactivity, regression	ESM-2 (joined chains), ESM C, ProtT5, SaProt, One-hot, Amino-acid composition, ESM-2 per region, IgT5, ESM-2 variable positions, AbLang-2	Paratope, Structural
Chemical reactions			
Buchwald–Hartwig yield [207]	yield (%), regression	CGR fingerprint, GROVER (reactants), Avalon (difference), Mordred (reactants), MHG-GNN (difference), ReactionT5, Component one-hot, Mol2vec (reactants), rxnfp, MACCS (difference)	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)

5.2.9 Models and metrics

The downstream models across hybrid experiments include RF, XGBoost, LightGBM, an SVM, a DNN, and a GP [122]. Each dataset uses the two that scored best in the earlier search of the whole dataset, defined in Table 5.6. The hyperparameters are kept consistent across datasets so the differences between scores come from the input. We tested the effects of hyperparameter tuning to see what we were missing. Additional file 20 shows that on one outer fold of five datasets, tuning helped the SVM most and made the RF worse on Buchwald–Hartwig yield.

We do not use the kNN as a downstream evaluator; however, it is used across a variety of experiments using the hybrids. It takes $k = 5$ neighbors by default, lowered to the square root of the number of training samples when a fold has fewer than 25.

One of the inputs to the hybrid search is an evaluation metric, which the downstream model gets scored on. For this research, we used RMSE for regression on small molecules and reaction yields in the label's units. We use Spearman ρ for antibody regression as these assays rank candidates and their scales differ between laboratories. Binary classification uses AUROC, which does not depend on a decision threshold.

The tables below report the mean over 5 folds. We do not use significance testing here, as 5-fold searches with challenging splits are not sufficient for such analysis.

Summed over its 5 outer folds, the search of one dataset took 2.3 hours (hERG K_i) to 28.2 hours (MDR1–MDCK efflux) on small molecules, and 11.1 hours on Buchwald–

Hartwig yield. On antibodies it took 0.4 hours (CHO polyreactivity) to 38.6 hours (the anti-VEGF expression scan). Each fold ran as its own job on a CPU node of a shared university cluster with 8 cores and 64 GB of memory, except QM9, which needed 12 or more cores and at least 120 GB. The times count only each fold's last attempt, so a fold resumed from a stopped run is undercounted. One hERG K_i fold recorded only 90 seconds.

5.2.10 Comparators, controls, and ablations

Unless stated otherwise, every experiment below uses the dataset's grid in Table 5.6, its two downstream models, five outer folds, and the dataset's own metric. From each fold's search it takes, for each downstream model, the two single representations and the two hybrids with the best scores on the training samples. Table 5.13 lists every experiment of this chapter.

Comparisons against related methods

PCA We compare the hybrid against the two best single representations as well as a PCA variant. We pool together the features across every single representation, standardize them on the training samples, and reduce the set by PCA [39] to each budget of the dataset's grid. The downstream model is fit on the principal components as they are. On each fold, the hybrid is compared with PCA at the hybrid's dictated budget. This way, neither side's width is chosen on the held-out samples. PCA cannot have more components than training samples, and where the budget is larger the two widths differ.

Ensembles We also consider ensemble modeling of a particular form. We fit one model per representation and pool their predictions, a process referred to as late fusion [169]. As the hybrid fits a single model and a fused feature vector, it could be considered a form of early fusion. Every choice for the ensemble is made on an inner 3-fold group split of the outer training samples. The first ensemble uses the same downstream model on every representation, and the second uses each representation's own better model. Each picks its members, the top 3 representations or all of them, and one of seven ways to pool predictions, from the mean and median to trimmed, outlier-removed, and score-weighted means. The mean is the high-level fusion of Turkina et al. [19], which averages the predictions of one RF per fingerprint.

Question	Datasets	What varies	Held fixed	Reported in
Does the hybrid beat the best single representation, and PCA at the same width?	20	hybrid, single representation, PCA of all representations joined	folds, models, the search's grid	Section 5.3.1
Does one model on the hybrid beat an ensemble of one model per representation?	19	hybrid, two ensembles, their membership, and pooling	folds, models, representations	Section 5.3.1, "Against ensembles, and many representations against few"
At the same budget, is it better to draw on many representations or few?	20	the top representation alone, an even split over all, the search's split	budget, importance method, models	Section 5.3.1, "Against ensembles, and many representations against few"
Does the hybrid help on rare scaffolds as well as common ones?	5	the half of the data, and which samples the search saw	the 2,000-sample subset, models	Section 5.3.1, "Against ensembles, and many representations against few"
Does the search overfit as the budget grows, or with one fold fewer?	14 and 18	budget, up to 8,000 features; training samples	every other setting of the grid	Section 5.3.1, "Overfitting of the search"
Does the gain come from several representations or from fewer features?	15 regression	hybrid, mRMR, recursive elimination, random features	budget, models	Section 5.3.2
Must the internal evaluator match the downstream model?	17	internal evaluator	every other setting of the grid	Section 5.3.2
Does importance help during training?	20	feature and sample weights, above-median features	the hybrid, models	Section 5.3.2
Does a smaller hybrid replace tuning the model?	5	budget, the SVM's C or the DNN's hidden-layer width	the hybrid's settings	Section 5.3.2
Does the hybrid take in a new embedding?	12	which embedding is in the pool; random features in its place	models, grid	Section 5.3.2
Are the hybrid's prediction intervals calibrated?	18	hybrid or single representation	models, 90% conformal intervals	Section 5.3.2
Are the selected features the same on every fold?	18	fold	models, grid	Section 5.3.3

Table 5.13: The experiments of this chapter. One row is one experiment. Datasets is the number of the 20 datasets shown in the experiment's table, and every experiment uses five outer folds unless its section says otherwise.

Frequent and rare scaffolds

Datasets above 2,000 samples are first reduced to about 2,000 by drawing whole scaffold families with a fixed seed. Scaffolds at or above the 80th percentile of group size form the frequent half, and the rest form the rare half. Each half gets its own search on each outer fold. The hybrid of a search on all the subset's training samples is also rebuilt and scored on each half.

Overfitting with the budget

We use the standard list of budgets described in Table 5.5 plus 4,000 and 8,000 features. At every budget, we rebuild the winning configuration on the outer training samples and score it on the held-out fold. We measure the selection bias at a budget by looking at the search's own score minus the held-out score. A positive value indicates that the search promised more than the held-out fold gave. We report how the bias changes with the budget, and the budget the search picks on each fold from its training samples alone. We do not compare hybrids wider than the training set against narrower ones by their best held-out budgets, because that choice reads the held-out labels on both sides.

A second check repeats each outer fold's search with one more fold removed from its training samples, for each of the other four folds. The held-out fold is the same and the two searches differ by one fold of training samples. Each search, however, picks its own budget, so the two hybrids can differ in width as well.

Feature reduction at the same width

On regression datasets, every method receives B features from all single representations joined, fitted on the fold's training samples. The methods are the two best hybrids at budget B from the fold's search, mRMR [178], recursive feature elimination [181], and B random features. mRMR ranks features by mutual information and stops at 200 features for run time. It consequently has no result on any small-molecule dataset, where every budget is 250 features or more. Recursive elimination fits a 50-tree RF of depth 5 and removes 10% of the features at each step. Each of the dataset's two downstream models scores every method, and only the features differ.

Internal evaluator

One search per outer fold is run for each evaluator setting. The matched evaluator belongs to the downstream model's family, while a near-family evaluator pairs tree

models with each other, the SVM with a GP, and the DNN with a narrower DNN. A far-family evaluator pairs tree models with an SVM, the SVM with LightGBM, and the DNN with kNN. A fourth setting scores the DNN with a wider DNN, and a fifth searches the dataset's own internal evaluators, listed in Table 5.6. Each setting's search picks its own budget, and the number of features can thus differ between settings. On every dataset, the SVM's near-family evaluator is a GP whose length scale started at 1.

Importance during training

Importance is computed on the training samples by the method the search picked. Sample weighting is not run for the GP, which takes no per-sample weights, nor on multiclass tasks. The five conditions are:

1. No weighting: the features as selected.
2. Feature weighting: standardized features multiplied by the square root of their importance, with the weights normalized to average 1.
3. Sample weighting: samples weighted by how well their label agrees with their 10 nearest neighbors in the rescaled space.
4. Combined weighting: feature and sample weighting together.
5. Above-median selection: only the features strictly above the median importance.

The weight of sample i is $1 - |y_i - \bar{y}_{N(i)}|/(\max y - \min y)$, clipped to lie between 0.1 and 1, where $\bar{y}_{N(i)}$ is the mean label of its 10 nearest neighbors by Manhattan distance. Tree splits depend only on the order of values [242], so for tree models feature weighting differs from no weighting only in that zero-importance features drop out. The rescaling does change which samples are nearest neighbors, and so changes sample weighting. Sample weighting down-weights rather than removes a suspect label, unlike the neighbor-based filtering of Section 4.2.3, "Noise detection by neighbors". We found no QSAR study that does this, and learned per-sample weights for noisy labels come from general ML [243].

Calibration

Split-conformal 90% prediction intervals are built on the training samples and scored on the held-out samples by the fraction of held-out labels they cover and by their mean width.

Stability and blueprints

Selection on fold m, for $m = 1, \ldots, F$, gives a binary vector $\mathbf{z}_m \in \{0,1\}^d$. Here d is the number of features in all single representations joined, not the size of the union of the selected features. We measure the stability of the selection with the index of Nogueira et al. [244],

$$\hat{\Phi} = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\frac{\bar{k}}{d}\left(1 - \frac{\bar{k}}{d}\right)}, \qquad s_f^2 = \frac{F}{F-1}\hat{p}_f(1-\hat{p}_f).$$

Here $\hat{p}_f$ is the fraction of folds that selected feature f, and $\bar{k}$ is the mean number of features selected. $\hat{\Phi} = 1$ means the same features on every fold, and $\hat{\Phi} \approx 0$ is what random selection gives. Each fold picks its own budget, and the index allows for the resulting difference in set size through $\bar{k}$. To read a blueprint, each selected feature is traced to its single representation, and its importance in the fitted model is measured with SHAP: TreeSHAP for tree models [192], the DeepLIFT-based SHAP explainer for the DNN, and Kernel SHAP on 50 held-out samples for the SVM and the GP [245].

5.2.11 Transfer, collaboration studies, and separate sites

We explore various use cases of the adaptive hybrid searches and subsequent representations. First, we ask if the search settings produced by the hybrid on one dataset, contained in the blueprint, transfer to other datasets. Specifically, can you identify important features on a larger dataset and then transfer them to a smaller one? We test this using LogD, with 5,028 molecules, as the larger dataset and hERG pK_i, with 1,412 molecules, as the smaller dataset. Both use the seven representations the two datasets share: Avalon, ChemBERT, ECFP4, GROVER, MHG-GNN, Mordred, and SMI-TED. The source settings come from one search on every LogD molecule, none of which is in the target. The closest prior work, Meta-QSAR [246], chose the learning method and representation for a new QSAR dataset from results on over 2,700 others and, like this study, reused decisions rather than selected features.

We also looked at comparisons run inside every held-out fold of the target: the best single representation, a hybrid searched on the target's training molecules as the upper reference, the source settings with features re-selected on the target, and the source's selected features applied unchanged. The last two ask separately whether the search decisions and the chosen features transfer. We use five folds grouped by scaffold family, RMSE in pK_i units, and the two downstream models decided for both datasets, the DNN and SVM.

We then repeat existing data pipelines using the hybrids as the main representation,

comparing its performance against the original. First, we repeat the activity cliff (AC) QSAR setting of Dablander et al. [62] on three targets, with five scaffold folds. Dopamine D2 (6,333 compounds) and factor Xa (3,605) come from ChEMBL, and the SARS-CoV-2 main protease (1,924) comes from the COVID Moonshot project. Each target's search draws on eight representations, which do not include GROVER or SMI-TED. The downstream models are LightGBM and the DNN, and the metric is the mean absolute error (MAE) in log units of activity.

We also count the molecules that a representation cannot tell apart, meaning that two or more molecules get an identical feature row, on every representation of the datasets in Table 4.1, and ask whether a hybrid removes the problem. Leaving out the MACCS keys, which are added only as augmentation features (Section 5.2.3), the representation with the most such molecules on hERG K_i is the WL graph kernel, and it is the target. Two hybrids add features of two partner representations to it. One adds ECFP4 and PDV, which are also blind to stereochemistry. The other adds ECFP4 and MHG-GNN, which together give no two hERG K_i molecules the same row. We report the score of each hybrid against the graph kernel alone, and the number of molecules that still share a row in the hybrid.

Federated learning is a technique which trains one model across data held by several parties without moving the data [247]. It has been run across pharmaceutical companies [248], whose structures and assay results are commercial assets that cannot be pooled [23, 24]. Our collaborator, Lhasa Limited, uses the federated knowledge-distillation process of Table 2.1 in Chapter 2 [154]. We use their code for the fingerprints, site models, and site clustering, and re-implement from it the step that combines the site models' predictions.

We make a slight adjustment to the federated learning process, namely we run feature selection at each of these sites. A single search on the training molecules of all sites together chooses the settings (allocation method, importance method, and internal evaluator). Each site then selects features with these settings on its own molecules and reports only feature indices, importance values, and one score. A coordinator splits the total budget B across sites in proportion to their scores, with at least one feature for each site whose selection succeeded. The main experiment below instead gives each site an even share, and no experiment compares the two splits, so we cannot say what the site scores buy. We replace a feature taken by another site with the site's next-highest ranked feature, keeping all features in the hybrid distinct. In related work, each client runs mutual-information feature selection on its own data and a server aggregates the results [249].

We run all four separate-site experiments on Caco-2 and LogD, with five folds grouped by scaffold family and three seeds, the DNN and SVM, and budgets of 500 to 2,000 features. In the main one, each site holds the same molecules but one family of representations: substructure fingerprints, physicochemical descriptors, or pretrained embeddings. It compares the best single family, picked on the training molecules, with one feature selection over all families, and both with one selection per family at an even share of the budget. When a family is narrower than its share, the rest goes to the other families. The even split can, as a result, build fewer features than one selection over all families.

Three additional experiments, with flipped labels, with one representation per site, and with features selected per cluster, are described in Additional file 30.

5.3 Results

We begin with the simplest comparison: the hybrid against the representations whose features it's built from. Upon seeing it does in fact improve predictive performance, we then explore where the gain is coming from, looking at individual blueprints from hybrid searches. Finally, we explore some simple downstream applications of the hybrid as a standard molecular representation.

5.3.1 Performance of the hybrid

Every number in this section is a held-out score, in the dataset's own metric, over five outer folds unless stated otherwise.

Against the best single representation

Table 5.14 shows that on small molecules the hybrid scores better than the best single representation on every dataset, with both downstream models. On the six regression datasets it does so on every fold. On hERG blocking the hybrid is ahead on most folds with each model.

On every small-molecule dataset in Table 5.14, the hybrid also beats, with both models, the single representation that was best on the held-out molecules. No user could make that choice in advance.

It does not improve things globally. On Caco-2 with the DNN, PDV is the best single representation in hindsight, a choice possible only after seeing the held-out molecules. It beats the representation picked on the training molecules, but the hybrid

beats both, 0.2775 against 0.298 RMSE. Additional file 21 shows which representations the hybrid relies on, by shuffling each representation's features in turn on the held-out molecules. On hERG K_i, Caco-2, LogD, rat microsomal clearance, and MDR1–MDCK efflux, the drop in score spreads about equally over five to seven representations. None carries a third of it. On QM9 with LightGBM and on hERG blocking the hybrid leans on one representation. It carries most of the score on QM9 and about half on hERG blocking, yet the hybrid still beats every single representation there.

On antibodies the hybrid's gain differs from one dataset to the next, as Table 5.14 shows. On the anti-VEGF expression scan, one of the two largest antibody sets, the hybrid is ahead with the SVM on all five folds. On $\Delta\Delta G$ the mean gain is the largest in the chapter with both models, but it moves by more than its own size from fold to fold. On EGFR binding, expression titer, and AC-SINS self-association the hybrid's mean is higher with both models, from wins on only two to four of the five folds.

In an antibody representation, as in a molecular fingerprint, the features that matter are few and far between. The problem is even more complex, as an antibody is larger than a small molecule. Within a dataset, most of the structure also stays the same between antibodies. An LM's embedding of the whole sequence mixes the CDRs that vary with a framework that mostly does not. Many of the antibody representations in Table 5.11 were built for this thesis to separate the two. On four antibody datasets the hybrid beats even the single representation that was best in hindsight. On two of them, the anti-VEGF expression scan and $\Delta\Delta G$, that representation is one of ours.

The representations built for this thesis matter most where the antibodies differ by defined mutations. In the $\Delta\Delta G$ hybrid they carry most of the drop in score under shuffling, with both models. On the other antibody datasets they carry much less. On AC-SINS self-association, another of the four, the hybrid draws mainly on two antibody LMs, IgT5 and AbLang-2. Across the antibody datasets the hybrids draw on about one and a half to five representations by the same shuffling, also in Additional file 21. On angiopoietin-2 binding the best representation in hindsight with the SVM, AntiBERTy, was never ranked first on any fold's training antibodies.

On nanobody thermostability, SMAC colloidal stability, HIC retention, and CHO polyreactivity the hybrid adds nothing. On HIC retention it is behind with the GP, and here the search picked the same ESM-2 representation on every fold, so the single representation was not the problem. The thermostability label also pools five assays into one. On angiopoietin-2 binding, whose labels were read off a published heatmap, the hybrid is higher with both models on three of five folds. Antibody humanness is a task one representation already solves, so the hybrid has nothing to add.

Dataset	Model	Best single base representation	Hybrid	Best in hindsight	Its score
RMSE ↓					
Caco-2 efflux	SVM	0.3036	0.2862	PDV	0.300
	DNN	0.3042	0.2775	PDV	0.298
LogD	SVM	0.4930	0.4302	MHG-GNN	0.493
	DNN	0.4497	0.4089	MHG-GNN	0.447
MDR1–MDCK efflux	SVM	0.4678	0.4370	PDV	0.468
	DNN	0.4732	0.4438	PDV	0.473
QM9 HOMO–LUMO gap	LightGBM	0.3720	0.3463	Mordred	0.372
	DNN	0.3619	0.3155	Mordred	0.362
Rat microsomal clearance	SVM	0.5209	0.4818	PDV	0.521
	DNN	0.5318	0.4974	PDV	0.533
hERG K_i	SVM	0.5881	0.5590	MHG-GNN	0.588
	DNN	0.6031	0.5691	MHG-GNN	0.593
Buchwald–Hartwig yield	RF	14.4056	15.2383	Avalon (difference)	14.2
	LightGBM	16.1679	15.6211	CGR fingerprint	15.2
Spearman ↑					
AC-SINS self-association	LightGBM	0.4631	0.5242	IgT5	0.472
	GP	0.5357	0.5547	AbLang-2	0.542
Angiopoietin-2 binding	SVM	0.4543	0.5216	AntiBERTy	0.577
	GP	0.4550	0.5049	SaProt	0.601
Anti-VEGF scan binding	RF	0.4325	0.5112	SaProt	0.445
	SVM	0.4538	0.5214	ProtT5	0.490
Anti-VEGF scan expression	SVM	0.7029	0.7729	ESM3 CDR	0.725
	DNN	0.7163	0.7352	ESM3 CDR	0.724
Binding free energy	SVM	0.0819	**0.2178**	Mutation features	0.203
	DNN	0.1317	**0.3542**	ESM-2 shift from parent	0.222
CHO polyreactivity	GP	0.5009	0.4419	ProtT5	0.510
	DNN	0.4003	0.4133	ESM C	0.431
EGFR binding	SVM	0.2821	0.3138	Ab-RoBERTa CDR	0.357
	DNN	0.1810	**0.2160**	Ab-RoBERTa	0.290
Expression titer	RF	0.2504	0.2706	IgT5	0.362
	DNN	0.3083	**0.3668**	ProtT5	0.372
HIC retention	GP	0.6000	0.5515	ESM-2 (joined chains)	0.600
	DNN	0.5311	0.5293	ESM-2 (joined chains)	0.531
Nanobody thermostability	SVM	0.6901	0.6899	SaProt	0.6901
	DNN	0.6939	0.6764	SaProt	0.694
SMAC colloidal stability	SVM	0.3485	0.3480	ProtT5	0.394
	DNN	0.2766	0.2677	ESM-2 early layers	0.313
AUROC ↑					
hERG blocking	XGBoost	0.8532	0.8644	Mordred	0.855
	LightGBM	0.8555	0.8666	MHG-GNN	0.860
Antibody humanness	RF	0.9659	0.9614	AntiBERTy CDR	0.967
	XGBoost	0.9511	0.9555	IgT5	0.963

Table 5.14: Held-out score of the best single representation and the hybrid, per dataset and model, mean over outer folds, in each dataset's own metric. On each fold, the hybrid is the configuration with the best score in that fold's search on the training samples. It is rebuilt on those samples and scored on the held-out fold. The best single representation is picked and scored the same way. Bold: the four rows where the hybrid gains most over the best single representation, as a fraction of the single score. Best in hindsight is the single representation best on the held-out samples, with its score.

On the Buchwald–Hartwig reactions in Table 5.14, the hybrid lowers the RMSE with LightGBM and raises it with the RF, both by less than the spread across folds. Additional file 25 shows that the search drew on only two of the ten representations on every fold. These were the reaction fingerprint and the change in the MHG-GNN embedding from reactants to product.

Against ensembles, and many representations against few

On most antibody datasets tested, an ensemble of one model per representation beats the hybrid with both models. On small molecules the hybrid holds up better. Table 5.15 compares the hybrid with two such late fusion ensembles. On the anti-VEGF expression scan the DNN ensemble is ahead of the hybrid on every fold. On small molecules the same-model ensemble beats the hybrid with both models only on hERG blocking and on the Buchwald–Hartwig reactions. The hybrid is ahead of both ensembles on every fold of QM9 with the DNN and of hERG K_i with the SVM. Whether the hybrid beats an ensemble likely hinges on whether interactions between features from different representations matter to the task. For antibodies, most of the representations used are variants of LMs that contain similar information. Averaging them through late fusion likely smooths out the noise of each model on a small training set. Small-molecule representations differ in the information they contain, and one model on their selected features can use those interactions more effectively.

Table 5.15: Hybrid against ensembles that fit one model per single representation and pool the predictions by one of seven rules (Section 5.2.10). One row is one dataset and downstream model. The same-model ensemble uses the row's model on every representation. The best-model ensemble gives each representation its own better model and is the same for both rows of a dataset. Held-out means over five outer folds, in the dataset's own metric. Dataset short names as in Table 5.8.

Dataset	Model	Hybrid	Same-model ensemble	Best-model ensemble
RMSE ↓				
QM9	DNN	**0.317**	0.340	0.340
	LightGBM	0.346	0.360	
HK_i	SVM	**0.560**	0.582	0.583
	DNN	0.570	0.554	
Caco-2	DNN	0.278	0.271	0.281
	SVM	0.286	0.299	
LogD	DNN	0.409	0.4167	0.4175
	SVM	0.430	0.475	

Dataset	Model	Hybrid	Same-model ensemble	Best-model ensemble
RLM	SVM	0.482	0.509	0.508
	DNN	0.499	0.493	
MDR1	SVM	0.437	0.4550	0.4552
	DNN	0.444	0.442	
BH	RF	15.2	14.7	14.6
	LightGBM	15.6	15.1	
AUROC ↑				
HB	LightGBM	0.867	0.8719	0.8724
	XGBoost	0.864	0.874	
Hum	XGBoost	0.955	0.9684	0.9684
	RF	0.962	0.969	
Spearman ↑				
VEGF-E	SVM	0.772	0.781	0.783
	DNN	**0.735**	**0.785**	
Nb-Tm	DNN	0.6788	0.6785	0.6790
	SVM	0.687104	0.687096	
EGFR	SVM	0.316	0.369	0.408
	DNN	0.216	0.219	
Ang2	GP	0.498	0.521	0.569
	SVM	0.512	0.633	
$\Delta\Delta G$	SVM	0.213	0.233	0.260
	DNN	0.346	0.189	
SMAC	DNN	0.265	0.300	0.394
	SVM	0.345	0.402	
HIC-R	GP	0.551	0.534	0.535
	DNN	0.532	0.514	
AC-SINS	LightGBM	0.524	0.510	0.517
	GP	0.557	0.525	
Titer	DNN	0.368	0.390	0.387
	RF	0.271	0.331	
PR-CHO	GP	0.443	0.485	0.508
	DNN	0.413	0.449	

At a fixed budget, drawing from several representations beats filling the budget from the top-ranked representation alone. Table 5.16 gives the smallest budget, and Additional file 22 every budget. The top representation alone scores worse than the search's hybrid at every budget, with both models, on most datasets with a result. On the rest it scores worse at all but one budget. How the budget is split matters far less on small molecules, where an even split over the same representations comes close to the search's split. On antibodies the split matters more, and at the smallest budget each rule is well ahead on some datasets. The even split is also ahead on EGFR binding

at every budget. On small molecules, the gain comes more from which representations are drawn on, and from ranking features within each one, than from the allocation rule that sizes their shares.

Table 5.16: Search's hybrid against the budget filled from the top-ranked representation alone and against an even split over the same representations. One row is one dataset and downstream model, at the smallest budget (number of features) its search covers; every budget is in Additional file 22. Held-out means over five outer folds, in the dataset's own metric. Dataset short names as in Table 5.8.

Dataset	Model	Budget	Top representation	Search's hybrid	Even split
RMSE ↓					
QM9	DNN	500	0.357	**0.314**	**0.328**
	LightGBM	500	0.372	0.347	0.346
HK_i	SVM	250	0.599	0.564	0.562
	DNN	250	0.574	0.569	0.571
Caco-2	DNN	500	0.307	0.2827	0.2829
	SVM	500	0.302	0.285	0.283
LogD	DNN	500	0.455	0.4165	0.4175
	SVM	500	0.499	0.438	0.440
RLM	SVM	500	0.547	0.4876	0.4881
	DNN	500	0.560	0.506	0.511
MDR1	SVM	500	0.474	0.438	0.441
	DNN	500	0.495	0.452	0.460
BH	RF	500	15.5	15.4	14.9
	LightGBM	500	16.2	15.5	16.1
AUROC ↑					
HB	LightGBM	100	0.849	0.858	0.862
	XGBoost	100	0.848	0.863	0.856
Hum	XGBoost	10	0.894	0.941	0.936
	RF	10	0.932	0.952	0.944
Spearman ↑					
VEGF-E	SVM	250	0.661	0.762	0.763
	DNN	250	0.658	0.732	0.735
VEGF-B	RF	250	0.365	0.5119	0.5121
	SVM	250	0.371	0.516	0.504
Nb-Tm	DNN	50	0.553	**0.624**	**0.572**
	SVM	50	0.534	0.643	0.622
EGFR	SVM	250	0.254	0.321	0.357
	DNN	250	0.222	0.247	0.251
Ang2	GP	25	0.348	0.506	0.490
	SVM	25	0.397	0.549	0.477
$\Delta\Delta G$	SVM	100	0.125	0.240	0.358
	DNN	100	0.026	0.342	0.267

Dataset	Model	Budget	Top representation	Search's hybrid	Even split
SMAC	DNN	500	0.259	0.308	0.255
	SVM	500	0.282	0.346	0.296
HIC-R	GP	250	0.431	0.552	0.499
	DNN	250	0.421	0.447	0.446
AC-SINS	LightGBM	50	0.323	0.521	0.516
	GP	50	0.336	0.521	0.491
Titer	DNN	250	0.230	0.296	0.292
	RF	250	0.247	0.292	0.381
PR-CHO	GP	250	0.294	0.471	0.483
	DNN	250	0.304	0.406	0.448

Out of the four small-molecule regression tasks tested, the hybrid also gains on rare scaffolds, such that each half is searched separately, as seen in Table 5.17. On the rare-scaffold half, the hybrid's gain is larger than on the frequent half with both models on LogD as well as Caco-2 and QM9 with specifically the DNN. On QM9 with LightGBM the gain is smaller on the rare half, and QM9 has a rare-half result on only two folds. On hERG K_i the hybrid is ahead in both halves, by less on the rare half, and on hERG blocking it is ahead on the rare half with LightGBM only. On rare scaffolds few training molecules resemble the held-out ones, so a single representation has less to go on.

Against a standard PCA on all the pooled features, the hybrid is clearly ahead with the SVM, LightGBM, and XGBoost, and close with the DNN. With the SVM, the hybrid is ahead on every fold of hERG K_i, Caco-2, LogD, rat microsomal clearance, and MDR1–MDCK efflux. With LightGBM and XGBoost, it is ahead on every fold of QM9 and hERG blocking. With the DNN the hybrid is still ahead on every fold of QM9, but PCA is ahead on LogD, 0.392 against 0.409 RMSE. The hybrid wins only two of the five folds of Caco-2. On hERG K_i, MDR1–MDCK efflux, and rat microsomal clearance the two are within 0.0020 RMSE. Additional file 23 compares this with two other experiments, one of which disagrees on LogD and Caco-2.

On antibodies neither method is a clear winner. With the SVM, PCA is ahead on 4 of 5 folds of the anti-VEGF expression scan and angiopoietin-2 binding, and on 3 of 5 of $\Delta\Delta G$. With the DNN, PCA is ahead on 3 of 5 folds of the expression scan, and the hybrid on 4 of 5 of $\Delta\Delta G$, 0.354 against 0.239 Spearman.

Overfitting of the search

We must stop and ask the question: is this method considered overfitting?

Dataset	Model	Scaffolds	Molecules	Best single representation	Hybrid
RMSE ↓					
QM9	DNN	Frequent	2,085	0.639	0.558
		Rare	345	0.676	0.529
	LightGBM	Frequent	2,085	0.645	0.546
		Rare	345	0.590	0.576
HK_i	SVM	Frequent	840	0.632	0.588
		Rare	572	0.686	0.655
	DNN	Frequent	840	0.633	0.596
		Rare	572	0.696	0.667
Caco-2	DNN	Frequent	1,256	0.291	0.282
		Rare	745	0.367	0.327
	SVM	Frequent	1,256	0.295	0.280
		Rare	745	0.338	0.330
LogD	DNN	Frequent	1,351	**0.607**	**0.587**
		Rare	649	**0.670**	**0.573**
	SVM	Frequent	1,351	0.642	0.575
		Rare	649	0.696	0.620
AUROC ↑					
HB	LightGBM	Frequent	1,288	0.765	0.763
		Rare	712	0.791	0.810
	XGBoost	Frequent	1,288	0.753	0.757
		Rare	712	0.793	0.789

Table 5.17: Best single representation and hybrid on molecules from frequent and from rare scaffolds, each half searched separately, from about 2,000 molecules of each dataset. One row is one dataset, downstream model, and half. Held-out means over five outer folds, in the dataset's own metric. QM9 ran on three outer folds, and its rare half has held-out molecules on two of them. Buchwald–Hartwig yield, nanobody thermostability, EGFR binding, and $\Delta\Delta G$ also have a result, but they were split by their fold groups, which are not scaffolds, and are not shown. Dataset short names as in Table 5.8.

With budgets searched up to 8,000 features, and at half to twice the number of training samples, the selection bias is small on the larger small-molecule datasets. The folds also often pick a hybrid wider than their own training set, for example on every fold of EGFR binding with the SVM. The search's internal scores can choose between configurations with a few thousand molecules, but not with 150 to 540 training antibodies, where its choice is partly noise.

On permuted class labels, selecting features on the full dataset before CV gave Ambroise and McLachlan [185] an error close to zero, against close to chance with selection redone inside each fold. Apart from the search's own scores in Table 5.18, every hybrid score in this chapter is on samples the search never saw, so the gains

Dataset	Model	Score		Training samples	Budget picked on fold				
		Search	Held out		1	2	3	4	5
RMSE ↓									
QM9	DNN	0.322	0.331	7,982	2,000	1,000	1,000		
	LightGBM	0.353	0.367	7,982	1,000	2,000	1,000		
HK_i	SVM	0.584	0.559	1,129	564	250	564	564	500
	DNN	0.591	0.592	1,129	2,000	100	4,000	250	2,000
Caco-2	DNN	0.281	0.283	1,726	1,726	8,000	8,000	1,726	1,000
	SVM	0.288	0.286	1,726	500	500	500	500	1,000
LogD	DNN	0.426	0.413	4,022	8,044	4,022	2,000	5,028	3,016
	SVM	0.448	0.431	4,022	1,000	2,011	1,000	1,000	2,011
BH	RF	14.8	15.6	3,057	250	100	100	50	100
	LightGBM	15.3	15.6	3,057	1,000	1,000	100	250	50
AUROC ↑									
HB	LightGBM	0.862	0.863	6,112	7,640	6,112	3,056	1,000	2,000
	XGBoost	0.859	0.861	6,112	1,000	6,112	2,000	3,056	3,056
Hum	XGBoost	0.967	0.958	439	878	500	658		
	RF	0.968	0.964	439	250	100	549		
Spearman ↑									
Nb-Tm	DNN	0.678	0.623	537	1,000	1,000	1,000	1,000	403
	SVM	0.668	0.679	537	403	250	250	403	268
EGFR	SVM	0.397	0.327	517	4,000	8,000	4,000	2,000	8,000
	DNN	**0.371**	**0.152**	517	10	100	25	4,000	100
$\Delta\Delta G$	SVM	0.365	0.194	343	25	1,000	10	100	50
	DNN	0.340	0.177	343	172	172	257	4,000	514
SMAC	DNN	0.417	0.310	189	500	100	2,000	2,000	142
	SVM	0.377	0.366	189	500	1,000	500	2,000	4,000
AC-SINS	LightGBM	0.538	0.505	189	236	4,000	94	500	500
	GP	0.560	0.557	189	236	284	94	500	2,000
Titer	DNN	0.426	0.294	186	500	186	186	93	4,000
	RF	0.419	0.268	186	250	2,000	186	372	2,000
PR-CHO	GP	0.517	0.478	152	8,000	8,000	8,000	8,000	1,000
	DNN	0.500	0.405	152	4,000	25	8,000	2,000	500

Table 5.18: The search's own score of the hybrid it picked, on each fold's training samples, against the held-out score of the same hybrid, with budgets searched up to 8,000 features and at 0.5 to 2 times the number of training samples. One row is one dataset and downstream model. Scores are means over five outer folds (three for QM9 and antibody humanness), in the dataset's own metric. Training samples is the smallest training set of any fold; the last five columns give the budget (number of features) each fold picked. Dataset short names as in Table 5.8.

above are not this bias. The bias still exists inside the search, and the numbers above show where it is large: on small datasets, where many configurations are compared on few samples [250].

Leaving one more fold of training samples out of each fold's search makes the held-out score worse on every small-molecule dataset with a result, with both models. Table 5.19 gives the scores. The two searches differ in two ways at once, the training samples and the budget each one picks, so we cannot say which of the two causes the loss. In short, selection bias does not explain the gains in this chapter, since every reported hybrid score is on samples the search never saw. Within the search, however, the bias is large on small datasets, and giving the search fewer training samples makes the hybrid worse.

Table 5.19: Held-out score of each fold's own search and of the same search run with one more fold of training samples left out, and stability of the selected features across the five folds. One row is one dataset and downstream model. Scores are means over five held-out folds in the dataset's own metric. Stability is the index of Nogueira et al. [244]: 1 for the same features on every fold, about 0 for random selection. The last two columns give the fewest and most features selected on any fold. Dataset short names as in Table 5.8.

		Held-out score			Features selected	
Dataset	Model	Own search	One fold fewer	Stability	Fewest	Most
RMSE ↓						
QM9	DNN	0.323	0.334	0.471	891	1,854
	LightGBM	0.346	0.361	**0.577**	500	1,066
HK_i	SVM	0.560	0.593	0.323	250	1,000
	DNN	0.568	0.605	0.273	250	1,831
Caco-2	DNN	0.278	0.288	0.399	993	1,964
	SVM	0.286	0.295	0.424	500	1,000
LogD	DNN	0.417	0.440	0.420	2,000	2,000
	SVM	0.430	0.460	0.497	1,000	2,000
MDR1	SVM	0.437	0.446	0.391	500	1,955
	DNN	0.439	0.455	0.518	1,000	1,956
BH	RF	15.2	15.3	0.156	479	1,000
	LightGBM	15.6	16.0	0.144	500	1,994
AUROC ↑						
HB	LightGBM	0.867	0.854	0.447	1,000	1,940
	XGBoost	0.864	0.853	0.327	475	1,940
Hum	XGBoost	0.955	0.954	0.122	50	500
	RF	0.962	0.959	0.098	100	1,000

Dataset	Model	Held-out score		Stability	Features selected	
		Own search	One fold fewer		Fewest	Most
Spearman ↑						
VEGF-E	SVM	0.772	0.755	0.270	2,000	2,000
	DNN	0.745	0.735	0.199	2,000	2,000
Nb-Tm	DNN	0.615	0.602	0.081	250	1,000
	SVM	0.687	0.636	0.130	250	500
EGFR	SVM	0.316	0.223	0.075	224	1,824
	DNN	0.203	0.143	0.085	187	1,817
Ang2	GP	0.498	0.492	0.090	25	100
	SVM	0.512	0.506	0.094	25	100
$\Delta\Delta G$	SVM	0.213	0.240	0.113	100	498
	DNN	0.206	0.189	0.100	100	250
SMAC	DNN	0.317	0.253	0.099	128	2,000
	SVM	0.345	0.321	0.140	142	2,000
HIC-R	GP	0.551	0.495	0.063	250	500
	DNN	0.513	0.405	0.198	499	1,000
AC-SINS	LightGBM	0.524	0.483	0.100	100	2,000
	GP	0.557	0.512	0.122	250	2,000
Titer	DNN	0.318	0.281	0.169	500	1,683
	RF	0.271	0.292	0.115	250	2,000
PR-CHO	GP	0.443	0.447	0.090	500	2,000
	DNN	0.455	0.369	0.107	250	2,000

5.3.2 Sources of the gain

We call into question whether the performance gain we observe across hybrids is more than just due to cutting the number of features. Could it be that the importance is not a factor at all? We also examine its relationship with the downstream model. We run some smaller checks including tuning, testing the search with new embeddings, and calibration.

Selection at a fixed number of features

On most small-molecule datasets the hybrid beats other selections of the same number of features, but not on the reactions or on several antibody datasets. We compare it with recursive feature elimination and random features over all single representations joined. Recursive elimination ranks all joined features in one list, a pooled ranking the hybrid avoids, as explained in Section 5.2.2. Figure 5.3 shows the hybrid with the lower error at every budget on QM9, Caco-2, LogD, and MDR1–MDCK efflux. The exception is hERG K_i with the DNN, where random features beat the hybrid at 500,

1,000 and 2,000 features, and recursive elimination at 1,000. On the Buchwald–Hartwig reactions recursive elimination is ahead at most budgets. On antibodies, Figure 5.4 shows that the hybrid beats random features at most budgets. Recursive elimination, however, is ahead on some antibody datasets, for example on CHO polyreactivity at every budget. Additional file 24 also compares mRMR, which ran only at 100 features or fewer and gave mixed results against the hybrid.

Dependence on the downstream model

We now look at the budget, meaning the number of features, that each search picked. The dots under each panel of Figures 5.3 and 5.4 show the pick of each fold, and Additional file 25 lists them. We did not test whether these picks follow measurable properties of a dataset, so we only describe them.

Many differences between datasets were set in advance by each dataset's search grid, which Table 5.6 lists. For example, the QM9, LogD, and hERG blocking searches could only rank features by LightGBM gain. SMAC colloidal stability, HIC retention, expression titer, and CHO polyreactivity have fewer than 200 training antibodies, and their grids only held budgets larger than that. Dataset size also does not order the budgets. QM9, the largest dataset, mostly picked fewer features with the DNN than LogD did.

Where the grid left a choice, the downstream model moved the pick as much as the dataset did. On Caco-2 and nanobody thermostability the search picked more features for the DNN than for the SVM. On LogD the DNN took the largest budget in the grid on every fold. It also picked large budgets when budgets up to 8,000 features were allowed, as Table 5.18 lists. The SVM's smaller picks fit what Chapter 4 showed for distances. The SVM's kernel is computed over all features, so features that carry no information dilute it, while the DNN can likely learn to ignore them.

On angiopoietin-2 binding and $\Delta\Delta G$ the search picks at most a few hundred features. On expression titer the RF's pick varies widely between folds. These small and unstable picks on small antibody datasets are the overfitting of the search described in Section 5.3.1, "Overfitting of the search". No single budget or representation count would suit every dataset and model, which is why both are searched.

We then ask how much of the search's own score depends on the downstream model and how much on the budget. On most datasets the downstream model moves the score more than the number of features does. Table 5.20 splits the variance in score across a search's configurations into the downstream model, the budget, their interaction, and a residual. The residual is the variance between configurations that share the same

model and budget. It comes from the other settings of the grid, namely the importance method, the allocation method, the internal evaluator, and the representation count. On LogD, for example, the model explains about half of the variance and the budget almost none. The budget explains more than the model on Caco-2, MDR1–MDCK efflux, hERG blocking, nanobody thermostability, and antibody humanness. On Caco-2 most of the budget's effect sits in its interaction with the model, so it differs between the two models.

Additional file 26 shows that the budget does depend on the model's own settings. The SVM's setting C controls how closely it may fit the training data. With a small C of 0.01 or 0.1, the search picks the same budget on every fold of LogD, rat microsomal clearance, MDR1–MDCK efflux, and Caco-2. With $C = 10$ it picks larger budgets on LogD and rat microsomal clearance, likely because a model allowed to fit more closely can use more features. The size of the DNN's hidden layers does not move its pick in the same way. On LogD it takes the largest budget in the grid at every size, on all folds but one.

Finally, we check the internal evaluator, the model that scores each single representation during the search and so sets its share of the budget. On most small-molecule datasets the choice of evaluator makes almost no difference, as Additional file 27 reports. A cheaper model of another family ranks the single representations about the same way as the downstream model, so the search does not need the more expensive one. The exception is QM9 with LightGBM, where an SVM evaluator gives 0.363 RMSE against 0.346 for the matched one.

On some antibody datasets the evaluator matters. On EGFR binding, HIC retention, expression titer, and SMAC colloidal stability, the matched evaluator gives the best score with both models. Additional file 27 gives an example. On EGFR binding with the SVM, the matched evaluator gives 0.353 Spearman, against 0.299 with a LightGBM evaluator.

Using importance during training

We decided to repurpose the importance scores computed in the hybrid search for downstream training tasks. These importance scores have minimal impact on small molecules. Table 5.21 shows the best of the four conditions gains little over the plain hybrid on any small-molecule dataset, and feature scaling makes LogD with the SVM worse. The selection has already used the importance scores, so on small molecules there is little left for them to add. The impact is a bit more varied on antibodies. Feature scaling helps colloidal stability with the DNN most, and keeping the above-

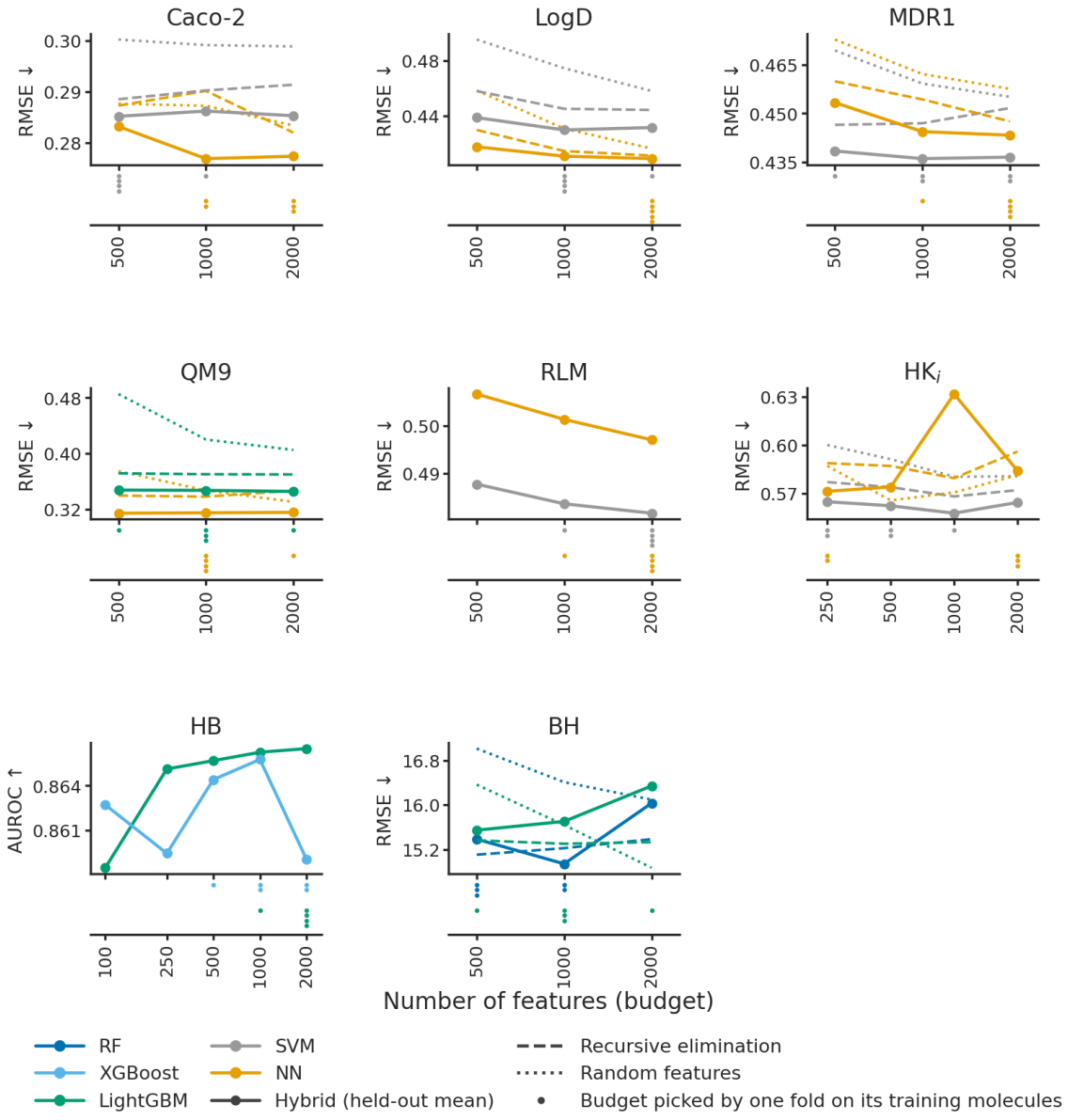


Figure 5.3: Small molecules and the Buchwald–Hartwig reactions. Held-out score against the number of features, one panel per dataset, one color per downstream model, in each dataset's own metric (arrow gives the better direction). Solid: the best hybrid of each fold's search. Dashed: recursive feature elimination. Dotted: randomly chosen features. Recursive elimination and random features were not run on RLM and HB. Lines are means over five outer folds. Dots under each panel: the budget each outer fold picked on its training molecules, one dot per fold. Dataset short names as in Table 5.8.

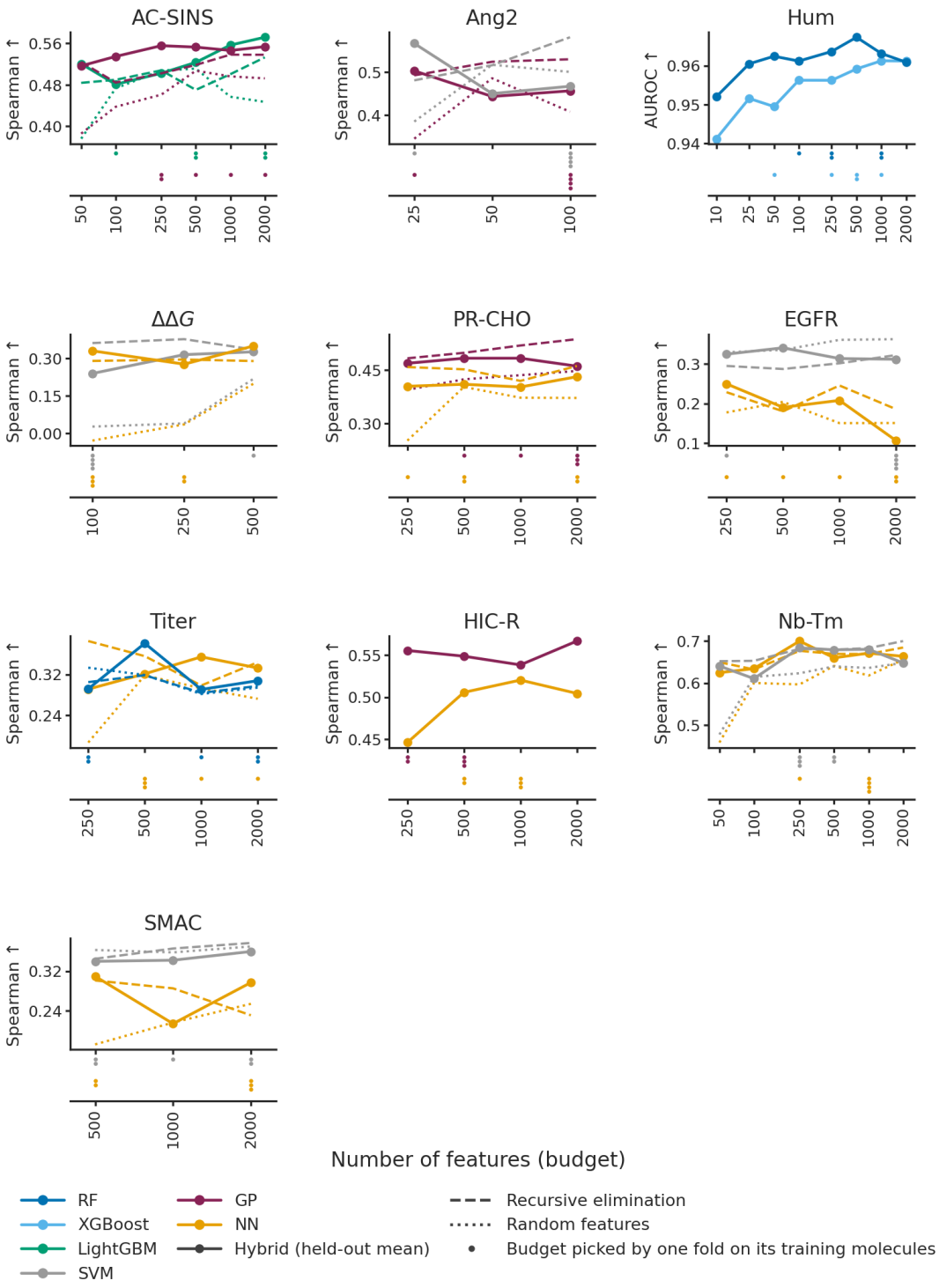


Figure 5.4: Antibodies. Held-out score against the number of features, one panel per dataset, one color per downstream model, in each dataset's own metric (arrow gives the better direction). Solid: the best hybrid of each fold's search. Dashed: recursive feature elimination. Dotted: randomly chosen features. Recursive elimination and random features were not run on Hum and HIC-R. Lines are means over five outer folds. Dots under each panel: the budget each outer fold picked on its training molecules, one dot per fold. Dataset short names as in Table 5.8.

Dataset	Downstream model	Budget	Interaction	Residual
Small molecules and reactions				
QM9	84.1	1.0	0.2	14.8
HK_i	59.7	1.0	2.4	37.0
Caco-2	0.0	6.6	43.6	49.9
LogD	**48.5**	**1.4**	1.6	48.5
RLM	77.4	14.3	2.3	6.0
MDR1	2.0	3.0	4.0	90.9
BH	82.6	1.6	0.2	15.6
HB	14.3	51.2	0.0	34.5
Antibodies				
Nb-Tm	7.2	25.4	1.9	65.5
EGFR	60.5	3.1	2.6	33.8
Ang2	18.3	13.8	0.0	67.8
$\Delta\Delta G$	14.2	6.1	0.4	79.3
SMAC	34.2	0.2	4.8	60.7
HIC-R	47.4	6.8	1.0	44.8
AC-SINS	37.6	17.0	0.2	45.1
Titer	14.5	1.0	1.7	82.7
PR-CHO	33.6	20.0	0.7	45.7
Hum	0.9	41.6	0.1	57.5

Table 5.20: Share of the variance in the search's own score on the training samples, across a search's configurations, explained by the downstream model, the budget, their interaction, and the residual (eta squared, in percent). One row is one dataset. Dataset short names as in Table 5.8.

median features also helps $\Delta\Delta G$ with the SVM. Both seemingly carry a regularization effect.

Table 5.21: Held-out score of the hybrid trained without weights, with features scaled by their importance, with molecules weighted, with both, and with only the features above the median importance. One row is one dataset and downstream model. Held-out means over five outer folds, in the dataset's own metric. The GP takes no sample weights. Dataset short names as in Table 5.8.

Dataset	Model	Un-weighted	Features scaled	Samples weighted	Both	Above median
RMSE ↓						
QM9	DNN	0.316	0.318	0.313	0.317	0.321
	LightGBM	0.346	0.344	0.3470	0.3472	0.345
HK_i	SVM	0.560	0.574	0.571	0.584	0.567
	DNN	0.5921	0.582	0.587	0.578	0.5915

Dataset	Model	Un-weighted	Features scaled	Samples weighted	Both	Above median
Caco-2	DNN	0.280	0.2762	0.2765	0.275	0.282
	SVM	0.286	0.2894	0.295	0.297	0.2887
LogD	DNN	0.409	0.416	0.408	0.41677	0.41685
	SVM	**0.430**	**0.462**	0.442	0.470	0.433
RLM	SVM	0.482	0.4883	0.4881	0.492	0.487
	DNN	0.499	0.500	0.497	0.498	0.503
MDR1	SVM	0.437	0.442	0.441	0.445	0.440
	DNN	0.444	0.451	0.4463	0.449	0.4458
BH	RF	15.21	15.24	15.4	15.3	15.5
	LightGBM	15.57	15.60	15.72	15.69	15.5
AUROC ↑						
HB	LightGBM	0.858	0.8603	0.857	0.856	0.8602
	XGBoost	0.863	0.860	0.856	0.858	0.851
Hum	XGBoost	0.9562	0.95649	0.9580	0.9579	0.95650
	RF	0.96115	0.96116	0.962	0.9606	0.959
Spearman ↑						
VEGF-E	SVM	0.772	0.7675	0.773	0.7678	0.770
	DNN	0.7348	0.738	0.740	0.743	0.7349
VEGF-B	RF	0.511	0.515	0.516	0.514	0.512
	SVM	0.507	0.505	0.509	0.506	0.508
Nb-Tm	DNN	0.628	0.607	0.632	0.592	0.594
	SVM	0.610	0.615	0.619	0.622	0.626
EGFR	SVM	0.314	0.243	0.327	0.249	0.290
	DNN	0.231	0.2219	0.2216	0.203	0.217
Ang2	GP	0.456	0.515			0.492
	SVM	0.467	0.456	0.469	0.455	0.434
$\Delta\Delta G$	SVM	0.239	0.261	0.246	0.262	0.312
	DNN	0.329	0.381	0.348	0.377	0.319
SMAC	DNN	**0.261**	**0.357**	0.309	0.355	0.311
	SVM	0.354	0.314	0.355	0.321	0.313
HIC-R	GP	0.551	0.537			0.499
	DNN	0.519	0.508	0.556	0.545	0.446
AC-SINS	LightGBM	0.481	0.4884	0.474	0.496	0.4882
	GP	0.535	0.501			0.527
Titer	DNN	0.344	0.355	0.349	0.363	0.340
	RF	0.149	0.150	0.161	0.165	0.225
PR-CHO	GP	0.425	0.421			0.461
	DNN	0.470	0.439	0.471	0.441	0.387

The hybrid process does not depend on existing representations; it can take in any. So we test what happens when we try to add a new small-molecule embedding to the search. A new embedding's features usually help on small molecules, but on several antibody datasets they add little. Additional file 28 gives the full results. We compare

the hybrid's selected features with Gaussian random features swapped into the same places in the same hybrid. No new search is run for the random features, so this is a weaker test than giving a search random features to choose from. On hERG K_i, Caco-2, MDR1–MDCK efflux, and LogD the embedding's features beat random features for every embedding or all but one. On hERG blocking a few embeddings lose to random features with one of the two models. On EGFR binding and $\Delta\Delta G$ they win for most embeddings. On several other antibody datasets and the Buchwald–Hartwig reactions they win for fewer than half, so another LM of the same sequence adds little.

With 90% conformal intervals, the hybrid's intervals on small molecules cover close to 90% of the held-out molecules, as the single representation's do. Additional file 29 gives the coverage. These intervals are also narrower, for example on LogD with the SVM. On several small antibody datasets the hybrid's intervals cover too few held-out antibodies, fewer than the single representation's, for example on angiopoietin-2 binding and $\Delta\Delta G$ with the SVM. The calibration set is part of a training fold of a few hundred antibodies, and the hybrid chosen on those antibodies is optimistic about them, the same selection bias as above.

5.3.3 Blueprints on specific datasets

We now consider the actual output of the search: the blueprint.

Figure 5.5 shows one fold of each dataset, where several hybrids draw on all ten representations in similar amounts, for example on LogD with the DNN and on the anti-VEGF expression scan with the SVM. However, for other hybrids, one or two representations carry the feature space. On EGFR binding almost all its features come from ESM-2 run on the joined chains.

With this format, we gain chemical insight into some of the results. LogD is a good example of this, as LogD's hybrids for SVM share 372 features on all five outer folds, 38 of them Mordred descriptors, 36 ECFP4 bits, and the rest from eight representations whose features carry no names. Among the descriptors are Crippen's calculated logP (SLogP), the number of basic groups (nBase), several surface areas binned by partial charge (PEOE_VSA), and counts of secondary amine and aromatic nitrogen environments. Among the bits are tertiary amine environments, in which a nitrogen is bonded to three carbons (`CN(C)C`), and aryl chloride environments. Unlike logP, which counts only the neutral form of a molecule, logD at pH 7.4 counts every charge state. At that pH a basic amine is largely protonated, which lowers logD below logP [116]. A model of LogD should therefore need both lipophilicity and basicity, and the hybrid

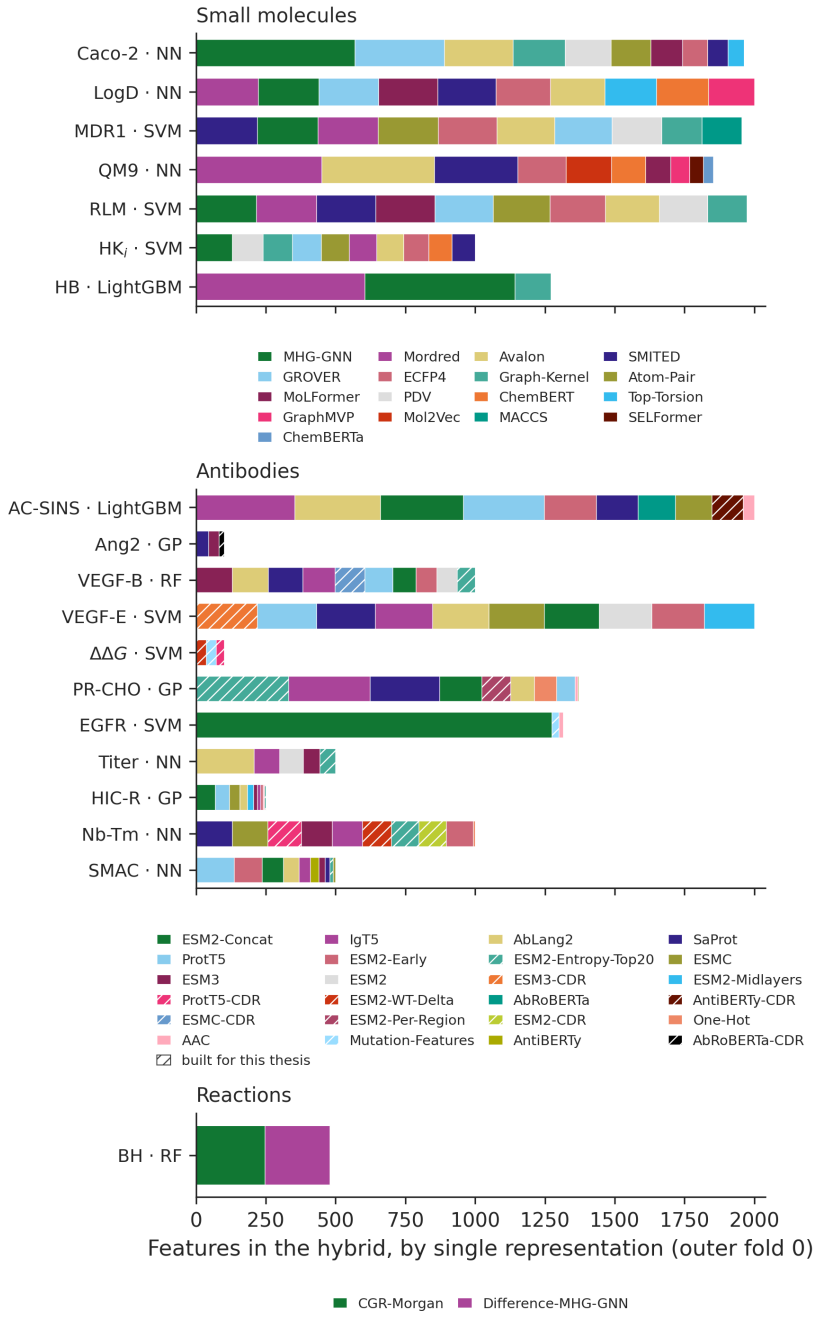


Figure 5.5: Number of features the hybrid took from each single representation, one bar per dataset, on the first outer fold with the downstream model named beside the bar. Antibody humanness has no bar. Hatched segments are representations built for this thesis. Dataset short names as in Table 5.8.

picked both on every fold. Lipophilicity comes from Mordred's SLogP, and basicity from Mordred's nBase and from ECFP4 amine bits.

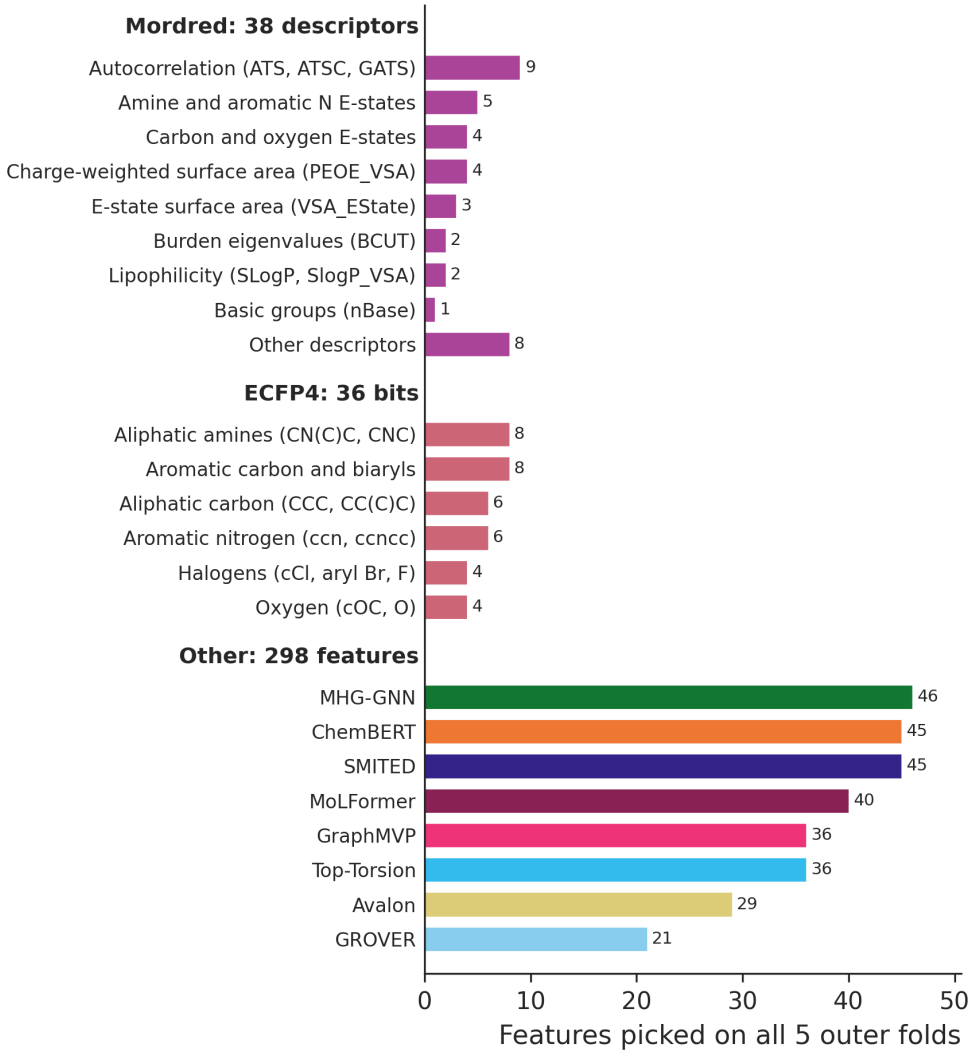


Figure 5.6: Number of features the SVM's LogD hybrid picked on all five outer folds. Mordred descriptors are grouped by descriptor family and ECFP4 bits by substructure. Every other representation is one bar.

The selected features vary between folds on the same search. On the stability index of Nogueira et al. [244] in Table 5.19, the small-molecule hybrids score higher than the antibody and Buchwald–Hartwig hybrids, and all score well below 1. Many features are correlated within and across representations, so a feature on one fold can stand in for a different one on another; an index well below 1 therefore need not mean the hybrid fits noise.

5.3.4 Extension studies

We begin turning our focus to more downstream applications, to the three AC targets of Dablander et al. [62]. This is a dataset the hybrid was not developed on. Table 5.22 compares the hybrid with the single representation the same search ranked first. For every target and model pair except factor Xa with LightGBM, the hybrid has the lower error. With the DNN on factor Xa it is lower on all five folds. These targets have many ACs, which is where a single similarity-based representation struggles most, as discussed in Chapter 4.

Target	Model	Top-ranked single representation	Hybrid
Dopamine D2	LightGBM	0.519	0.514
	DNN	0.523	0.499
Factor Xa	LightGBM	0.685	0.714
	DNN	**0.737**	**0.661**
SARS-CoV-2 main protease	LightGBM	0.465	0.452
	DNN	0.500	0.490

Table 5.22: Single representation the search ranked first and hybrid on the three AC targets of Dablander et al. [62]. One row is one target and downstream model. Held-out MAE in log units of activity, lower is better, mean over five outer folds.

On Caco-2, MACCS keys and the WL graph kernel both give some molecules a row that another molecule also has, as counted in Table 4.1. Amino-acid composition does the same to 432 of the 659 variants in the EGFR scan, which differ by single mutations. None of the hybrids built on these datasets leaves two samples with the same row.

On hERG K_i the graph kernel leaves some molecules sharing a row. Adding features from ECFP4 and MHG-GNN lowers its RMSE with the SVM on all five folds, and no two molecules share a row, as Table 5.23 shows. Adding ECFP4 and PDV instead improves it almost as much while some molecules still share a row. Most of the gain therefore comes from the information the added representations give every molecule, and little from breaking the ties.

Search settings found on LogD were reused on hERG K_i, with the features selected again on hERG K_i. In Table 5.24, LogD's feature set beat hERG K_i's best single representation with both models (with the SVM on all five folds). However, it stays behind a search run on hERG K_i itself. Reusing settings saves the search on the smaller dataset for some cost in accuracy.

Representations	RMSE		Molecules sharing a row
	SVM	DNN	
Graph kernel alone	**0.611**	0.624	105
Graph kernel, ECFP4, and PDV	0.575	0.580	56
Graph kernel, ECFP4, and MHG-GNN	**0.571**	0.581	0

Table 5.23: The WL graph kernel on hERG K_i alone and as a hybrid with features of two partner representations. One row is one set of representations. Held-out RMSE in pK_i units, lower is better, mean over five outer folds. The last column counts the molecules whose feature row another molecule also has.

Model	Best single representation	Search on hERG K_i	LogD settings, features reselected	LogD settings and features
SVM	**0.589**	0.563	**0.567**	0.569
DNN	0.604	0.567	0.589	0.593

Table 5.24: Search settings found on LogD reused on hERG K_i. One row is one downstream model. Columns: the best single representation on hERG K_i; a search run on hERG K_i; the LogD settings with features selected again on hERG K_i; the LogD settings and features reused unchanged. Held-out RMSE in pK_i units, lower is better, mean over five outer folds. These searches draw on the seven representations that hERG K_i shares with LogD, so the scores differ from Table 5.14, whose searches draw on all ten.

5.3.5 Features chosen across separate sites

We now return to the federated learning setting outlined in Section 5.2.11. There, data are held at separate sites and cannot be pooled, so feature selection has to run at each site. In our main experiment, every site holds the same molecules, but each computes only one family of representations, namely substructure fingerprints, physicochemical descriptors, or pretrained embeddings. We compare three ways to choose features. The first uses only the best family. The second runs one selection over all families together, as if they could be pooled. The third runs one selection at each site, each with an even share of the budget.

Selecting at each site does as well as one selection over all families, on Caco-2 and LogD with both models. Table 5.25 shows that both beat the best family and the best single representation. On LogD with the SVM, for example, one selection over all families gives 0.438 RMSE, against 0.460 for the best family. Selecting at each site can, however, end with a different number of features. As Section 5.2.11 describes, the search settings used here still come from one search on the training molecules of all sites together. Only the feature selection after that search runs at each site. Once the

settings are chosen, each site can select on its own and send only feature indices.

Dataset	Model	Best single representation	Best family	One selection, all families	Selection per family
Caco-2	DNN	0.305	0.288	0.284	0.283
	SVM	0.304	0.301	0.2847	0.2845
LogD	DNN	0.453	0.446	0.419	0.420
	SVM	0.493	**0.460**	**0.43823**	0.43822

Table 5.25: Feature selection when each site holds one family of representations of the same molecules. One row is one dataset and downstream model. Columns: the best single representation; the best single family; one selection over all families; one selection per family at an even share of the budget. Held-out RMSE, lower is better, mean over three seeds of the five-fold mean.

Additional file 30 reports three further experiments: one hybrid over sites that each hold one representation, selection at sites with flipped labels, and selection per cluster of Caco-2.

5.4 Conclusions

This chapter asked how to select the best set of features for a given dataset and model, and whether the method carries over from small molecules to antibodies and reactions. No fixed choice of features or settings works everywhere, but a search can find a good one for each task.

On small molecules the hybrid works. With both models, it beat the best single representation on held-out molecules on every dataset, for example from 0.450 to 0.409 RMSE on LogD with the DNN. It even beat the representation that turned out best on the held-out molecules, which no user could know in advance. The gain is not simply from using fewer features, since on most datasets the hybrid also beat other ways of selecting the same number. It comes from combining representations that carry different information. On LogD, the selected features describe lipophilicity and basicity, as a chemist would expect.

The best settings differed between datasets and models. They should therefore be searched for each new task rather than fixed in advance. Beyond small molecules the results are mixed. On four antibody datasets the hybrid beat every single representation, but on most of them an ensemble of one model per representation did better. On the Buchwald–Hartwig reactions the hybrid gained nothing.

Ranking features by importance helps to build the hybrid, since the top-ranked

features beat random features of the same number on most datasets. Using the same scores again to weight features during training adds little.

The main limitation is that the hybrid needs data. Its gains are consistent on a few thousand samples but not on a few hundred, where the search's own scores overstate its results. Most of our antibody datasets are that small, so their results say as much about size as about antibodies. The search is also costly, and must be rerun whenever the data, representations, or model change. The separate sites were simulated from two public datasets.

This chapter used the hybrid only as input to a model. Chapter 6 asks whether it also gives better neighbors, and Chapter 7 whether it helps with noisy labels.

Chapter 6

Similarity in an adaptive hybrid representation

6.1 Introduction

Chapter 5 set the stage for the adaptive hybrid representations. Taking a step back, Chapter 4 showed that in most standard representations the distance between two molecules barely rises with the difference in their labels. Any method that trusts a molecule's neighbors inherits this, such as predicting a label from the nearest neighbors or finding a noisy label by comparing it with its neighbors' labels. The first idea behind the hybrid was to compute distances on the features relevant to the task alone, and this chapter tests that idea.

A hybrid can be built for similarity in two ways. In the first, the search of Chapter 5 uses a kNN model as its downstream model, and we measure distances in the hybrid it selects. In the second, the search ranks each candidate hybrid by how well each molecule's five nearest neighbors in it predict its label, and fits no downstream model. We ask two research questions:

1. Do pairwise distances between molecules in a hybrid track their absolute label differences better than distances computed using the single representations?

2. Do the methods built on that distance work better in a hybrid? We test prediction from nearest neighbors, activity cliffs (ACs), Bayesian optimization, distance to the training molecules as a guide to prediction error, retrieval of neighbors, and clustering.

This chapter uses 14 datasets: six small-molecule, seven antibody, and one reaction dataset. Antibody datasets outnumber small-molecule datasets, so we give every result

by domain rather than pooled.

6.2 Methods

Datasets, folds (grouped by scaffold family for small molecules), and metrics are as in Chapter 5, with five outer folds. Of its 20 datasets, we leave out six: rat microsomal clearance, both anti-VEGF scans, angiopoietin-2 binding, antibody humanness, and HIC retention. Table 6.6 has no weighted values on Caco-2, hERG blocking, hERG K_i, expression titer, and SMAC colloidal stability, whose runs measured unweighted distances only. In each outer fold, everything an experiment compares is chosen from searches on that fold's training molecules alone, and then scored on its held-out molecules. Features are scaled on the training molecules: features that only take the values 0 and 1 are left as they are, and the rest are standardized. In the experiments, but not inside the searches, held-out values are cut to the training range as well, which stops a single extreme molecule from dominating a distance. Where a statement covers all 14 datasets, "molecule" also stands for an antibody or a reaction.

The fold split, the sample of molecule pairs, the label permutations, the random feature subsets, and the k-means starts all use fixed seeds. The spread we report is therefore the spread across the five outer folds of one split. We report the mean over the five folds, and the per-fold values where the spread matters, without significance tests. The simulation of choosing molecules to measure is repeated over three starting sets instead.

6.2.1 Feature spaces compared

Each outer fold runs three hybrid-style searches on its training molecules. (1) The general search is the standard hybrid search we've discussed thus far. (2) The kNN search, in which a kNN model is used as the downstream model with Euclidean distance and five equally weighted neighbors. Its budgets are capped at 500 features due to computation limitations. And finally (3), the similarity search, described in Section 6.2.2. From these three searches, we take the following feature spaces:

- the kNN model's best and second-best single representations, ranked by the kNN search;
- each downstream model's best and second-best single representations, ranked by its general search;

- each downstream model's two best hybrids, four in all: the predictive hybrids;
- the top hybrid of the kNN search: the kNN hybrid;
- the top hybrid of the similarity search: the similarity hybrid, on the datasets where that search is defined.

Unless stated otherwise, "the single representation" in the results means the kNN model's best single representation, and "the hybrid" means the kNN hybrid. Retrieval and clustering also score one control: the principal components of all representations joined, with as many components as the hybrid has features.

6.2.2 Similarity in a hybrid

We now describe the similarity search, which is the search of Chapter 5 with a different score. Each configuration θ of that search fixes an importance method, an allocation method, a number of representations, and a budget B, as described in Section 5.2.2. Given single representations $R_1, \ldots, R_m$, a configuration builds a feature set $S_\theta \subset R_1 \cup \cdots \cup R_m$ with $|S_\theta| \leq B$. The search returns S_{θ^*}, where $\theta^* = \arg\max_\theta s(S_\theta)$ and s is the search's score. In the general search, s is a downstream model's score on the search's own held-out folds. In the similarity search, $s(S) = q_5(S)$, namely how accurately each molecule's label is predicted from its five nearest neighbors in S, and no downstream model is fitted. All other settings, and the folds, are shared with the general search. When the budget is split between representations, this neighbor score also replaces the internal evaluator.

As discussed in Chapter 4, Zheng and Tropsha [21] and Itskowitz and Tropsha [22] chose descriptors to maximize nearest-neighbor prediction within one descriptor set. We modify this idea by choosing features across several representations. The similarity hybrid tests whether choosing features for how well the neighbors predict a label does better than choosing them for a model's prediction.

The score uses the Manhattan distance of Chapter 4, on features scaled as above. Each standardized feature adds a term of similar size to this distance. A representation's weight in the distance is therefore roughly its number of features divided by the budget. Features that only take the values 0 and 1, such as fingerprint bits, add less per feature. The neighbor score uses no importance weighting, and we did not compare its Manhattan distance and inverse-distance weights against alternatives.

We compute the score on the held-out molecules of each fold of the search, inside the outer training fold, in order to rank candidates. These molecules act as each other's

neighbors. Their groups (scaffold families, for small molecules) are split into up to 5 parts, and each molecule's 5 nearest neighbors are drawn from the other parts. A neighbor from the same scaffold family is an easy match and would inflate the score. This is the only place in the chapter where a molecule's own group is kept out of its neighbors. Each neighbor is weighted by the inverse of its distance [251]:

$$\hat{y}_i = \frac{\sum_{j \in N_5(i)} y_j / d_{ij}}{\sum_{j \in N_5(i)} 1 / d_{ij}}. \tag{6.1}$$

A neighbor at distance 0 receives all of the weight, shared equally among ties. On regression data, q_5 is the Spearman correlation $\rho(\hat{\mathbf{y}}, \mathbf{y})$. On binary data, $\hat{y}_i$ is the distance-weighted fraction of neighbors in class 1, and $q_5 = 2(\text{AUROC} - 0.5)$. Both scores put chance at 0, which the budget split needs, as it divides each representation's score by the sum of the scores. Negative scores are first shifted so that the lowest is 0. The related modelability index of Golbraikh et al. [133], discussed in Chapter 4, uses one nearest neighbor and a hard class match.

6.2.3 Nearest-neighbor settings

Every experiment that makes predictions based on its neighbors' labels or that measures retrieval scores each feature space at every combination of the settings in Table 6.1, instead of at one chosen setting. A held-out molecule's prediction is the weighted mean of its neighbors' labels,

$$\hat{y}_i = \frac{\sum_{j \in N_k(i)} w_{ij}\, y_j}{\sum_{j \in N_k(i)} w_{ij}}, \tag{6.2}$$

where $N_k(i)$ is the k training molecules nearest to i under distance d, and w_{ij} is the neighbor weight. On classification data, each class receives the summed weight of its neighbors. Unlike in the similarity score, a training molecule from the held-out molecule's own group (scaffold family, for small molecules) can be one of its neighbors. The one experiment that does not use these settings is the one on all representations joined, described in Section 6.2.4.

We also test importance weighting, a modified form of feature-weighted nearest neighbors [253]. Before the distance is computed, each feature is multiplied by a weight, $\mathbf{x}' = \mathbf{x} \odot \mathbf{w}$. The weights come from the importance scores of the hybrid search, with negative importances set to 0. For a single representation, the importance $\mathbf{s}$ is computed on the fold's training molecules, in most experiments with LightGBM gain. For a hybrid, it is the importance its search used to select the features. There are two exceptions. For distance against label difference, LightGBM gain is refitted on

Setting	Levels	Source	Status
Distance d	Manhattan; cosine; Minkowski with $p = 0.5$, $d(\mathbf{x}, \mathbf{y}) = \left(\sum_j \lvert x_j - y_j \rvert^{0.5}\right)^2$	Chapter 4 (Manhattan, cosine); this chapter (Minkowski)	Reused
Neighbors k	5, 10, 20	Standard	Reused
Neighbor weight w_{ij}	$1/r_{ij}$, where r_{ij} is the rank of j among the k neighbors of i, nearest first; $1/d_{ij}^2$	None found; Shepard [252]	New; reused
Importance weighting	None; $\mathbf{w} = (\mathbf{s}/\bar{s})^a$; $\mathbf{w} = \mathbf{s}^{1/2}$	Wettschereck et al. [253]	Modified

Table 6.1: Nearest-neighbor settings. One row is one setting, with its levels; every combination of levels is scored. In the last row, $\mathbf{s}$ is the vector of feature importances, $\bar{s}$ its mean, and a an exponent set by the distance. Minkowski distance is left out on datasets above 5,000 molecules (hERG blocking, LogD, and QM9).

all of the hybrid's features together. For retrieval, the search's importance method is rerun on the hybrid as a whole. We use two formulas for $\mathbf{w}$:

- for distance against label difference and the best number of features, $\mathbf{w} = (\mathbf{s}/\bar{s})^a$, with $a = 1$ under Manhattan distance, $a = 0.5$ under cosine, and $a = 2$ under Minkowski with $p = 0.5$;
- for every other experiment that weights, $\mathbf{w} = \mathbf{s}^{1/2}$ under every distance.

The two formulas weight features differently. In the first, the exponent a makes each feature's term in the distance scale with its relative importance, $s_f/\bar{s}$, under all three distances. In the second, under Manhattan distance, a feature four times as important as average counts only twice as much. The weighted distances of Sections 6.3.1 and 6.3.2 are therefore the same only under cosine.

6.2.4 Experiments

Table 6.2 lists every similarity experiment, and the paragraphs below define what each one estimates.

Distance against label difference

This experiment asks how well distance in a feature space follows the label. It uses the measure of Section 4.2.2, here on pairs of held-out molecules. For each pair, we compute the Spearman correlation between their distance and their absolute label difference. A positive value means that molecules further apart differ more in their

Question	Datasets	Varied	Held fixed	Score	Reported in
Does distance track label differences?	All 14	Feature space; distance; importance weighting	Up to 50,000 held-out pairs per fold	Spearman between pair distance and absolute label difference	Section 6.3.1
Best number of features	11 with a DNN	Budget	Each downstream model's top hybrid at each budget of its search	Dataset metric and Spearman as above, same hybrid	One line, Section 6.3.1
Prediction from nearest neighbors	All 14	Feature space; settings of Table 6.1	Comparison fixed before the run: Manhattan, 5 neighbors, rank	Dataset metric	Section 6.3.2
Importance weighting in four models	All 14; joined: 13 regression	Model; importance weighting; feature space	GP up to 5,000 molecules, SVM up to 10,000; joined: 10 neighbors, $1/d$, Manhattan	Dataset metric, weighted against unweighted	Section 6.3.2
Choosing molecules to measure	hERG K_i, QM9, Buchwald–Hartwig	Surrogate model; feature space	Three starting sets of at least 5% of the molecules; 100 picks by expected improvement	Area under the best-found-so-far curve	One line, Section 6.3.2
ACs	Six small-molecule	Feature space; downstream model	ECFP4 Tanimoto at least 0.55; gap set for 15% of training molecules; at least 10 cliff and 10 other molecules	Dataset metric on cliff and other held-out molecules	Section 6.3.3
Distance against prediction error	All 14	Feature space; distance; number of neighbors; importance weighting; model fit	Mean distance to the nearest training molecules	Spearman between distance and prediction error	Section 6.3.4
Retrieval and clustering	All 14	Feature space; settings of Table 6.1; 3, 5, or 10 clusters	k-means under squared Euclidean distance	Label gap to the nearest neighbors; label coherence of clusters	Section 6.3.4
Label information in the kept features	All 14	Kept features, random subset, principal components	At most 50 components; three label permutations	Summed mutual information, held-out molecules	One line, Section 6.4; Additional file 33

Table 6.2: Similarity experiments. One row is one experiment. Datasets are counted from the 14 of this chapter. All use the five outer folds, except choosing molecules to measure, which uses three starting sets.

labels. We measure it in each representation alone, unweighted, under every distance of Table 6.1. We then measure it in the feature spaces of Section 6.2.1, unweighted and weighted.

Prediction from nearest neighbors

We predict each held-out molecule's label from its nearest training molecules, as in Equation 6.2, and compare hybrid and single representation in two ways. The first comparison was fixed before the run. It uses Manhattan distance, 5 neighbors weighted by inverse rank, and no importance weighting. The second comparison picks a combination for each held-out fold from the held-out scores of the other four folds. The held-out fold's own scores are never used in this pick. In those four folds, however, the held-out fold's molecules were training molecules, so their labels served as neighbor labels. The hybrid side chooses among the four predictive hybrids, the kNN hybrid,

and the similarity hybrid, weighted or unweighted, at every setting of Table 6.1, up to 216 combinations. The single side chooses among the kNN model's two best single representations, unweighted, up to 36 combinations. The two sides therefore differ in two ways at once. The hybrid side has five or six times as many combinations, and only it can use importance weighting. The comparison fixed before the run has neither difference.

Importance weighting in four models

We score every feature space with and without importance weighting in four models that use a distance. These are prediction from nearest neighbors, a GP and an SVM with RBF kernels on regression data, and q-RASAR [254]. In q-RASAR, each downstream model receives four extra features from the nearest training molecules, namely their mean label and its spread, the distance to the nearest one, and their mean distance. The four extra features are computed under the three neighbor settings that rank best on the training molecules. A separate experiment weights all representations joined, on regression datasets, once per importance method.

Activity cliffs

An AC is a pair of structurally similar molecules with very different activity [63, 82]. Only the two hERG datasets measure an activity, but we use the term on all six small-molecule datasets. We define ACs as in Section 4.2.2. Two molecules are similar when their ECFP4 Tanimoto similarity is at least 0.55 [150]. We set the gap so that 15% of each outer fold's training molecules are cliff molecules. A training molecule is a cliff molecule if it is in a similar pair whose labels differ by more than the gap, or fall in different classes. A held-out molecule is one if at least two similar training molecules have labels spanning more than the gap, so its own label is never read. Each downstream model is fitted on its two best single representations and two best hybrids, and scored separately on cliff and other held-out molecules. The main comparison is the first hybrid against the best single representation.

Distance against prediction error

Sheridan et al. [255] found that a molecule's similarity to the training molecules tracks how well it is predicted. Distance to the training molecules is also a common way to define an applicability domain [152], as noted in Section 4.2.3. An applicability domain marks the molecules whose predictions can be trusted. We test whether distance

tracks error by fitting each downstream model on all representations joined and on its first hybrid. The error on a held-out molecule is the absolute error on regression data. On classification data, it is one minus the predicted probability of the true class. We correlate this error, by Spearman, with the mean distance to the 5, 10, or 20 nearest training molecules. We do this under every distance, in each feature space of Section 6.2.1, unweighted and weighted.

Retrieval and clustering

Retrieval scores each held-out molecule by its label gap, namely the weighted mean absolute difference between its label and those of its nearest training molecules, at every setting of Table 6.1. On classification data, the gap is one minus the weighted share of neighbors in its class. For clustering, we fit k-means [256] with 3, 5, and 10 clusters on the training molecules, and assign each held-out molecule to one. Label coherence is one minus the mean within-cluster SD of held-out labels, divided by their overall SD. On classification data, it is the mean share of the largest class per cluster. A lower label gap and a higher label coherence are better, and clusters with fewer than two held-out molecules are left out.

Three smaller checks

The number-of-features check finds the best number of features for prediction and for distance. At every budget of its search, we take each downstream model's top hybrid and record two scores, its prediction score and how well its distances track label differences. We then find where each peaks.

The Bayesian optimization check simulates choosing which molecules to measure next, on hERG K_i, QM9, and Buchwald–Hartwig only. The search looks for the lowest label on hERG K_i and QM9, and the highest yield on Buchwald–Hartwig. It starts from a set of whole groups holding at least 5% of the molecules. The groups are scaffold families on hERG K_i and QM9, and additives on Buchwald–Hartwig. A surrogate model then picks 100 molecules one at a time by expected improvement [257]. We repeat this from three starting sets. The score is the area under the curve of the best label found so far.

The label-information check asks whether the features a hybrid keeps carry more label information than other features of the same number. For each representation a predictive hybrid draws from, we compare the features it kept against principal components and against a random subset of the same size. Each block of features is

reduced to at most 50 principal components. We sum a mutual-information estimate over the components, and subtract the mean over three label permutations. We use the estimators of scikit-learn 1.6.1, which follow Kraskov et al. [258] for continuous labels and Ross [259] for classes. This sum counts information shared by two components twice.

6.3 Results

We first ask whether distance in a hybrid tracks label differences more closely than in a single representation, and then whether the methods built on that distance gain from it.

6.3.1 Distance against label difference

As in Chapter 4, distance in a single representation says little about how far apart two labels are. For each dataset, we score every pair of one single representation and one distance from Table 6.1, giving 20 to 30 pairs per dataset. Table 6.3 shows that even the best pair stays weak on every dataset, and varies widely between folds. This best pair is also picked with hindsight on the held-out molecules.

On five datasets, some pairs of representation and distance have a correlation at or below zero, so molecules further apart are not more different in their labels. These are MDR1–MDCK efflux, EGFR binding, CHO polyreactivity, AC-SINS self-association, and SMAC colloidal stability. SMAC colloidal stability is the weakest, with half of its 30 pairs at or below zero.

Table 6.4 shows that distance in the kNN hybrid tracks label differences more closely than in the single representation on all 14 datasets. On LogD the hybrid is higher in every fold, while on Caco-2 it is lower in two of the five. The two sides are not built the same way. The hybrid's features are chosen using the training labels. Every single representation is built without these labels, either as a fixed fingerprint or descriptor set or as a pretrained embedding. The weighted single representation of Table 6.6 also uses the training labels, through its feature importances, and is compared below.

Distance in the predictive hybrids, built for prediction rather than for distance, does not always track label differences more closely than in the single representation. Table 6.5 shows that on more than half the datasets, at least one predictive hybrid falls below its model's best single representation. On CHO polyreactivity all four do. A hybrid used for distance should therefore come from the kNN search.

	Best pair of representation and distance				All pairs	
Dataset	Pair	Mean	Lowest fold	Highest fold	Median	Lowest
Small molecules						
QM9	ChemBERTa-2, Manhattan	0.279	0.206	0.333	0.186	0.038
Caco-2	Topological torsions, Manhattan or Minkowski	0.229	0.170	0.338	0.133	0.004
MDR1-MDCK efflux	Atom pairs, Manhattan or Minkowski	**0.281**	0.258	0.305	0.127	−0.052
hERG K_i	SMI-TED, Manhattan	0.117	0.028	0.237	0.081	0.040
LogD	MoLFormer, Manhattan	0.147	0.104	0.193	0.085	0.050
hERG blocking	MoLFormer, cosine	0.041	0.021	0.054	0.021	0.003
Antibodies						
EGFR binding	AbLang-2, cosine	0.091	0.002	0.184	0.044	−0.022
Nanobody thermostability	One-hot, Manhattan or Minkowski	0.187	0.121	0.256	0.121	0.075
AC-SINS self-association	AbLang-2, cosine	0.050	0.015	0.077	0.022	−0.031
SMAC colloidal stability	AbLang-2, Minkowski	**0.016**	−0.074	0.104	**−0.0004**	−0.027
CHO polyreactivity	SaProt, cosine	0.133	0.034	0.256	0.088	−0.041
Expression titer	ESM-2 middle layers, Minkowski	0.142	0.044	0.266	0.099	0.005
Binding free energy ($\Delta\Delta G$)	ProtT5 CDR, Manhattan	0.168	−0.017	0.487	0.140	0.073
Reactions						
Buchwald–Hartwig yield	rxnfp, Manhattan	0.135	0.122	0.156	0.080	0.046

Table 6.3: Distance against label difference in single representations. One row is one dataset. Values are the Spearman correlation between the distance of two held-out molecules and their absolute label difference (higher is better), over pairs of one representation and one distance, unweighted. Best pair: picked on the held-out molecules; mean over five outer folds, and its lowest and highest fold. All pairs: median and lowest of the fold means.

Dataset	Best single representation	kNN hybrid
Spearman ↑		
Caco-2 efflux	0.1517	0.1969
LogD	0.0765	0.2687
MDR1-MDCK efflux	0.2286	0.2883
QM9 HOMO-LUMO gap	0.1298	0.3779
hERG Ki	0.0729	0.2625
hERG blocking	0.0257	0.0552
AC-SINS self-association	0.0210	0.0718
Binding free energy	0.1181	0.1919
CHO polyreactivity	0.1033	0.1276
EGFR binding	0.0378	0.0737
Expression titer	0.0097	0.1063
Nanobody thermostability	0.1201	0.2110
SMAC colloidal stability	0.0002	0.0501
Buchwald-Hartwig yield	0.1074	0.1431

Table 6.4: Spearman correlation between the distance between two samples and the difference in their labels, per dataset, mean over outer folds. Higher is better (arrow).

Weighting the single representation by feature importance closes most of its gap to the kNN hybrid on most datasets. Table 6.6 shows that on LogD, AC-SINS self-association, and Buchwald–Hartwig, it even passes the unweighted kNN hybrid. On EGFR binding, CHO polyreactivity, and $\Delta\Delta G$, weighting closes little of the gap.

We also compare against a stronger single representation. This is the pair of representation and distance whose distance best follows the label, picked with hindsight, as in Table 6.3. This pick beats the unweighted kNN hybrid of Table 6.6 on Caco-2, EGFR binding, expression titer, and CHO polyreactivity. On Caco-2, for example, it reaches 0.229 Spearman against 0.197. The similarity hybrid, at 0.277, beats it in turn.

The number of features at which the DNN's hybrid predicts best is often not the one where its distances best track label differences. As Table 6.7 shows, prediction usually peaks at more features than distance does, though on a few datasets the two peak together or prediction peaks at fewer. On LogD, for example, the hybrid predicts better at 2,000 features than at 500, while its distance correlation falls from 0.231 to 0.154 Spearman.

6.3.2 Prediction from nearest neighbors

Table 6.8 gives the results at the setting fixed before the run. There, the kNN hybrid predicts from the five nearest neighbors more accurately on all six small-molecule datasets. Scores for each fold are in Additional file 31.

In Table 6.9, the feature space and setting are picked again for each fold, and the

Dataset	Model	Best single representation	First hybrid	Second hybrid
Small molecules				
QM9	DNN	0.186	0.274	0.305
	LightGBM	0.186	0.215	0.212
Caco-2	DNN	0.147	0.196	0.206
	SVM	0.184	0.179	0.208
MDR1-MDCK efflux	SVM	0.229	0.240	0.243
	DNN	0.229	0.194	0.196
hERG K_i	SVM	0.093	0.215	0.212
	DNN	0.074	0.180	0.151
LogD	DNN	0.087	0.154	0.149
	SVM	0.099	0.1951	0.1948
hERG blocking	LightGBM	0.02444	0.02440	0.031
	XGBoost	0.024	0.026	0.028
Antibodies				
EGFR binding	SVM	0.039	0.035	0.054
	DNN	0.041	0.042	0.060
Nanobody thermostability	DNN	0.120	0.158	0.171
	SVM	0.120	0.163	0.162
AC-SINS self-association	LightGBM	−0.007	0.037	0.039
	GP	0.017	0.042	0.024
SMAC colloidal stability	DNN	0.007	−0.024	−0.0003
	SVM	−0.011	0.002	0.007
CHO polyreactivity	GP	0.124	**0.094**	**0.093**
	DNN	0.106	**0.064**	**0.096**
Expression titer	DNN	0.068	0.057	0.078
	RF	0.031	0.099	0.107
Binding free energy ($\Delta\Delta G$)	SVM	0.144	0.173	0.185
	DNN	0.132	0.218	0.197
Reactions				
Buchwald–Hartwig yield	RF	0.104	0.081	0.074
	LightGBM	0.067	0.0833	0.0831

Table 6.5: Distance against label difference in each downstream model's best single representation and its two predictive hybrids. One row is one dataset and model. Values are the Spearman correlation between the distance of two held-out molecules and their absolute label difference (higher is better), mean over five outer folds, Manhattan distance, unweighted.

Dataset	Single representation		kNN hybrid		Similarity hybrid	
	Unweighted	Weighted	Unweighted	Weighted	Unweighted	Weighted
Small molecules						
QM9	0.130	0.329	0.378	0.460	0.368	0.462
Caco-2	0.152		0.197		**0.277**	
MDR1-MDCK efflux	0.229	0.267	0.288	0.285	0.286	0.283
hERG K_i	0.073		0.263		0.234	
LogD	0.077	**0.276**	0.269	0.309	0.230	0.312
hERG blocking	0.026		0.055		0.064	
Antibodies						
EGFR binding	0.038	0.047	0.074	0.064	0.057	0.046
Nanobody thermostability	0.120	0.207	0.211	0.227	0.182	0.212
AC-SINS self-association	0.021	0.097	0.072	0.155		
SMAC colloidal stability	0.0002		0.050			
CHO polyreactivity	0.103	0.107	0.128	0.130		
Expression titer	0.010		0.106			
Binding free energy ($\Delta\Delta G$)	0.118	0.131	0.192	0.214		
Reactions						
Buchwald–Hartwig yield	0.1074	0.182	0.143	0.208	0.1069	0.189

Table 6.6: Distance against label difference in each feature space, with and without importance weighting. One row is one dataset. Values are the Spearman correlation between the distance of two held-out molecules and their absolute label difference (higher is better), mean over five outer folds, Manhattan distance. The single representation is the one the kNN model ranked first in each fold.

hybrid still predicts more accurately on all six small-molecule datasets. This second comparison favors the hybrid, however, as the hybrid picks from more combinations and alone can use importance weighting, as explained in Section 6.2.4.

On the antibody datasets and Buchwald–Hartwig, the two comparisons disagree about where the hybrid wins. At the fixed setting of Table 6.8, the hybrid loses on $\Delta\Delta G$, AC-SINS self-association, and Buchwald–Hartwig. Table 6.9 shows that with the setting picked again for each fold, it wins only on EGFR binding, $\Delta\Delta G$, and SMAC colloidal stability. SMAC colloidal stability gains in both, from 0.092 to 0.291 Spearman with the setting picked again.

Multiplying each feature by its importance mostly helps prediction from nearest

	Prediction score, by number of features						Spearman ↑, by number of features					
Dataset	50	100	250	500	1,000	2,000	50	100	250	500	1,000	2,000
Small molecules, prediction score in RMSE ↓												
QM9				0.316	0.315	0.312				0.323	0.282	0.254
Caco-2				0.284	0.279	0.278				0.216	0.205	0.211
MDR1-MDCK efflux				0.453	0.447	0.442				0.224	0.213	0.190
hERG K_i			0.569	0.571	0.580	0.575			0.235	0.200	0.155	0.135
LogD				0.415	0.413	**0.408**				**0.231**	0.187	0.154
Antibodies, prediction score in Spearman ↑												
EGFR binding			0.247	0.198	0.209	0.114			0.048	0.064	0.035	0.028
Nanobody thermostability	0.624	0.638	0.698	0.659	0.668	0.662	0.193	0.188	0.172	0.164	0.156	0.147
SMAC colloidal stability				0.309	0.216	0.299				−0.014	−0.006	−0.014
CHO polyreactivity			0.406	0.410	0.404	0.439			0.101	0.094	0.094	0.059
Expression titer			0.296	0.315	0.357	0.337			0.077	0.061	0.124	0.120
Binding free energy ($\Delta\Delta G$)		0.342	0.267	0.339				0.202	0.181	0.205		

Table 6.7: Prediction score and distance against label difference of the DNN's top hybrid at each number of features of its search. One row is one dataset; datasets without a DNN have no row. Prediction score: held-out score in the dataset's metric (direction in the group heading). Spearman: correlation between the distance of two held-out molecules and their absolute label difference in the same hybrid, Manhattan distance, unweighted. Means over five outer folds.

Dataset	Best single representation	Hybrid
RMSE ↓		
Caco-2 efflux	0.3386	0.3246
LogD	0.6766	0.5831
MDR1-MDCK efflux	0.5543	0.5202
QM9 HOMO-LUMO gap	0.6692	0.4205
hERG Ki	0.6280	0.5822
Buchwald-Hartwig yield	14.4572	**16.1811**
Spearman ↑		
AC-SINS self-association	0.3912	**0.3884**
Binding free energy	0.1424	**0.0899**
CHO polyreactivity	0.2059	0.3591
EGFR binding	0.1760	0.2126
Expression titer	0.2521	0.2622
Nanobody thermostability	0.5005	0.5458
SMAC colloidal stability	0.1268	0.2418
AUROC ↑		
hERG blocking	0.8143	0.8335

Table 6.8: Held-out score of predicting each molecule from its five nearest training molecules, per dataset, mean over outer folds, in each dataset's own metric. Bold: a dataset where the hybrid is worse.

neighbors. Table 6.12 marks the exceptions with †. In the other downstream models, weighting the single representation gains little or loses, as seen for the SVM in Table 6.10. Giving each model four extra features taken from a molecule's nearest neighbors helps or hurts depending on the setting. On LogD, the SVM improves with ten neighbors under cosine distance and gets worse with twenty.

Weighting also helps when all representations are joined into one feature set, as shown in Table 6.11. At least one importance method helps on every dataset. Weighting loses only with DeepLIFT on Caco-2, and with LightGBM gain and TreeSHAP on SMAC colloidal stability. Without a comparison against shuffled weights, we cannot say how much of the gain comes from the importance itself.

Whether the hybrid wins depends far more on the dataset than on how the neighbors are found, as Additional file 31 shows. Weighting both the single representation and the hybrid shrinks the gain wherever the hybrid wins, on LogD from 0.089 to 0.025 RMSE. Weighting accomplishes part of what the hybrid does. This makes sense, as it carries some mechanism of feature importance without a full re-selection of features. Where the hybrid loses, like on $\Delta\Delta G$ and AC-SINS self-association, that loss is smaller than the spread across folds. Some of these datasets have only a few hundred samples, so we cannot entirely separate dataset size from domain. This would require further

Dataset	Metric	Single representation	Hybrid
Small molecules			
QM9	RMSE ↓	0.617	0.422
Caco-2	RMSE ↓	0.326	0.319
MDR1-MDCK efflux	RMSE ↓	0.522	0.480
hERG K_i	RMSE ↓	0.598	0.564
LogD	RMSE ↓	0.638	**0.565**
hERG blocking	AUROC ↑	0.830	0.843
Antibodies			
EGFR binding	Spearman ↑	0.257	0.342
Nanobody thermostability	Spearman ↑	0.572	0.561
AC-SINS self-association	Spearman ↑	0.456	0.441
SMAC colloidal stability	Spearman ↑	0.092	**0.291**
CHO polyreactivity	Spearman ↑	0.350	0.260
Expression titer	Spearman ↑	0.322	0.248
Binding free energy ($\Delta\Delta G$)	Spearman ↑	0.113	0.145
Reactions			
Buchwald–Hartwig yield	RMSE ↓	13.8	14.0

Table 6.9: Prediction of each held-out molecule from its nearest training molecules, with the feature space and setting picked again for each outer fold. One row is one dataset; mean over five outer folds, in the dataset's own metric (arrow gives the direction). Single representation: the kNN model's two best, unweighted. Hybrid: up to six hybrids, unweighted or weighted. Buchwald–Hartwig RMSE is in percentage points of yield.

investigation on larger antibody datasets.

On the antibody datasets, the distance used changes how much closer the labels of the hybrid's neighbors are, which Section 6.3.4 also compares. Additional file 32 gives every setting. On $\Delta\Delta G$ the hybrid's neighbors have labels further apart under all three distances, though at the fixed setting by less than the spread across folds. On the other antibody datasets except EGFR binding, cosine distance gives the hybrid its smallest lead. On SMAC colloidal stability it reverses the result, from labels 0.028 closer under Manhattan distance to 0.057 further apart.

We also ran a simulated Bayesian optimization on hERG K_i, QM9, and Buchwald–Hartwig. Starting from a few known molecules, a model repeatedly picks the next molecule to measure, and we score how quickly it finds the best labels over 100 picks. This gives no clear advantage to the hybrid. On each dataset, the best of 20 hybrid simulations narrowly beat the single representation, which was tried only once. On QM9, for example, they scored 253.4 against 260.1, where lower is better, but the other 19 hybrid simulations all did worse than the single representation. The spread

Dataset	Unweighted	Weighted
Small molecules, RMSE ↓		
QM9	0.449	0.459
Caco-2	0.307	0.310
MDR1-MDCK efflux	0.468	0.466
hERG K_i	0.607	0.630
LogD	0.508	0.507
Antibodies, Spearman ↑		
EGFR binding	0.287	0.247
Nanobody thermostability	0.690	0.672
AC-SINS self-association	0.550	0.499
SMAC colloidal stability	0.282	0.158
CHO polyreactivity	0.3751	0.3749
Expression titer	0.376	0.296
Binding free energy ($\Delta\Delta G$)	0.209	0.206
Reactions, RMSE ↓		
Buchwald–Hartwig yield	14.9	14.8

Table 6.10: Importance weighting in an SVM with an RBF kernel, fitted on the kNN model's best single representation. One row is one regression dataset; mean over five outer folds, in the dataset's own metric (heading row gives metric and direction). Buchwald–Hartwig RMSE is in percentage points of yield.

between starting sets was also larger than the gap between the two.

6.3.3 Activity cliffs

When handling noise, we run into the issue of determining if a label that is presumably flagged as some kind of outlier is noise or an AC. If it's noise, you don't want any model fitting on it, and you want to either remove it, down-weight it, or correct the label. But if it's an AC, it has the potential to carry very critical chemical information. Because of this, in Chapter 7 we took two extra steps to try to protect the ACs from being affected by the noise methods: the Cliff-aware gap, a detector less likely to flag a molecule whose neighbors' labels differ widely, and Cliff put-back, which returns AC molecules after a cleanup.

We report ACs on the six small-molecule datasets only, using the definition of Section 6.2.4. The antibody datasets have too few cliff antibodies to score, for example 11 of the 429 held-out antibodies on $\Delta\Delta G$. Held-out molecules are flagged by the looser rule of Section 6.2.4, so on some datasets far more than 15% of them are labeled as cliffs. On LogD and hERG blocking, this is about a third to a half of the held-out molecules in each fold. QM9 has the fewest, at most 13 of about 1,996 per fold.

On LogD, Caco-2, hERG K_i, and hERG blocking, each model's first hybrid beats its

Dataset	Unweighted	Weighted by		
		LightGBM gain	DeepLIFT	TreeSHAP
Small molecules, RMSE ↓				
QM9	0.575	**0.484**		
Caco-2	0.3425	0.337	**0.3428**	0.338
MDR1-MDCK efflux	0.549	0.516	0.540	0.519
hERG K_i	0.642	0.620		0.624
LogD	0.696	0.669		
Antibodies, Spearman ↑				
EGFR binding	0.251	0.307		0.285
Nanobody thermostability	0.443	0.487		0.493
AC-SINS self-association	0.311	0.434	0.341	0.375
SMAC colloidal stability	0.134	**0.121**	0.143	**0.122**
CHO polyreactivity	0.240		0.302	0.413
Expression titer	0.233		0.244	0.257
Binding free energy ($\Delta\Delta G$)	0.050	0.146		0.066
Reactions, RMSE ↓				
Buchwald–Hartwig yield	19.1		17.9	16.5

Table 6.11: Prediction from the ten nearest neighbors in all representations joined, unweighted and with features weighted by each importance method. One row is one regression dataset; mean over five outer folds, in the dataset's own metric (heading row gives metric and direction). Empty cell: an importance method the dataset's search does not use (Table 5.6). Buchwald–Hartwig RMSE is in percentage points of yield.

best single representation on both cliff molecules and the other molecules in Table 6.13. On Caco-2, hERG K_i, and hERG blocking the gain is larger on cliff molecules. On LogD it is slightly smaller, but both models gain on cliff molecules in every fold.

QM9 and MDR1–MDCK efflux have too few cliff molecules to judge, 48 and 49 in all, and only two folds on each hold the 10 needed for a score. On both, Table 6.13 shows the hybrid is better on the other molecules. On cliff molecules it is worse or level, but each model's hybrid wins one of the two folds and loses the other.

6.3.4 Prediction error, retrieval, and clustering

A molecule's distance to its nearest training molecules rises only weakly with the error made on it. Table 6.14 shows that on the small-molecule datasets, distance in the hybrid tracks error better for both models, except on hERG blocking, where it is worse for both. The gain is largest on hERG K_i, from 0.200 to 0.276 Spearman for the SVM.

On the antibody datasets, the hybrid also tracks error better for both models, as Table 6.14 shows. The exceptions are EGFR binding, where the two models disagree,

		Single representation		kNN hybrid	
Dataset	Metric	Unweighted	Weighted	Unweighted	Weighted
Small molecules					
QM9	RMSE ↓	0.669	0.574	0.420	0.450†
Caco-2	RMSE ↓	0.339	0.334	0.325	0.3225
MDR1-MDCK efflux	RMSE ↓	0.554	0.532	0.520	0.513
hERG blocking	AUROC ↑	0.814	0.823	0.834	0.837
hERG K_i	RMSE ↓	0.628	0.597	0.582	0.580
LogD	RMSE ↓	0.677	0.6065	0.583	0.5765
Antibodies					
EGFR binding	Spearman ↑	0.176	0.207	0.213	0.223
Nanobody thermostability	Spearman ↑	0.500	0.533	0.546	0.552
AC-SINS self-association	Spearman ↑	0.391	0.416	0.388	0.422
SMAC colloidal stability	Spearman ↑	0.127	0.171	0.242	0.234†
CHO polyreactivity	Spearman ↑	0.206	0.271	0.359	0.357†
Expression titer	Spearman ↑	0.252	0.218†	0.262	0.229†
Binding free energy	Spearman ↑	0.142	0.124†	0.090	0.135
Reactions					
Buchwald–Hartwig yield	RMSE ↓	14.5	14.3	16.2	16.5†

Table 6.12: Prediction of each held-out molecule from its five nearest training molecules, with and without features multiplied by their importance, in the kNN model's best single representation and in the kNN hybrid. One row is one dataset; values are means over five outer folds, Manhattan distance, neighbors weighted by the inverse of their rank. The arrow gives the direction of the metric; Buchwald–Hartwig RMSE is in percentage points of yield. † marks a weighted value worse than the unweighted value beside it.

and SMAC colloidal stability, where it is worse for both. It is also worse for both on Buchwald–Hartwig. On Buchwald–Hartwig, $\Delta\Delta G$, and expression titer, distance in the single representation does not rise with error for either model.

Table 6.15 shows that, as the settings are picked each time per fold, the hybrid's neighbors have closer labels on all six small-molecule datasets. Its k-means clusters also hold more similar labels at 3, 5, and 10 clusters on hERG blocking, LogD, MDR1–MDCK efflux, and QM9, as Table 6.16 shows. On Caco-2 and hERG K_i, this changes with the number of clusters.

Table 6.15 shows that the hybrid's neighbors have closer labels across the antibody datasets except on $\Delta\Delta G$. On Buchwald–Hartwig the labels are further apart. Principal components of all representations do better than the hybrid only on EGFR binding, and tie with it on CHO polyreactivity. Table 6.16 shows that clustering is mixed. At every cluster count, the hybrid's clusters are more coherent on nanobody thermostability and CHO polyreactivity, and less coherent on EGFR binding and SMAC colloidal stability.

		Cliff molecules		Other molecules	
Dataset	Model	Single	Hybrid	Single	Hybrid
RMSE ↓					
QM9	DNN	0.242	**0.280**	0.358	0.313
	LightGBM	0.261	0.267	0.371	0.343
Caco-2	DNN	0.340	**0.288**	0.289	0.272
	SVM	0.331	0.308	0.290	0.275
MDR1-MDCK efflux	SVM	0.4175	0.4173	0.467	0.437
	DNN	0.455	0.465	0.471	0.440
hERG K_i	SVM	0.561	0.530	0.593	0.565
	DNN	0.570	0.527	0.610	0.579
LogD	DNN	0.430	0.388	0.465	0.417
	SVM	0.469	0.407	0.504	0.440
AUROC ↑					
hERG blocking	LightGBM	0.793	0.808	0.888	0.895
	XGBoost	0.791	0.811	0.887	0.888

Table 6.13: Prediction on held-out cliff molecules and on the other held-out molecules, by each downstream model's best single representation and its first hybrid. One row is one dataset and model; mean over the outer folds that hold at least 10 molecules of the kind: two folds for the QM9 and MDR1–MDCK efflux cliff molecules, five for every other cell.

6.4 Conclusions

Chapter 4 found that the distance computed using standard representations barely rises with a change in labels, as it should according to the SPP. Across the six small-molecule datasets, distance in a hybrid built for distance tracks label differences more closely, and its nearest neighbors predict better and have closer labels. Each model's first hybrid also predicts cliff molecules better on LogD, Caco-2, hERG K_i, and hERG blocking. The similarity hybrid has a value on nine datasets. On these, Table 6.6 shows it tracking label differences better than the unweighted kNN hybrid only on Caco-2 and hERG blocking. With both weighted, it is also slightly ahead on QM9 and LogD, for example 0.312 against 0.309 Spearman on LogD.

There is some benefit to weighting features by importance; it accomplishes a portion of what the hybrids do. On LogD, feature importance weights make the single representation track label differences better than the unweighted hybrid. It doesn't beat the weighted hybrid, though. However, the hybrid's distance tracks prediction error worse on hERG blocking. Except on QM9 and Buchwald–Hartwig, its features also carry no more label information than the same number of principal components.

On the antibody datasets and Buchwald–Hartwig, distance in the hybrid also

Dataset	Model	Single representation	kNN hybrid
Small molecules			
QM9	DNN	0.072	0.114
	LightGBM	0.137	0.209
Caco-2	DNN	0.116	0.123
	SVM	0.124	0.131
MDR1-MDCK efflux	SVM	0.230	0.288
	DNN	0.216	0.268
hERG K_i	SVM	0.200	**0.276**
	DNN	0.195	0.238
LogD	DNN	0.147	0.150
	SVM	0.216	0.220
hERG blocking	LightGBM	0.139	0.019
	XGBoost	0.146	0.041
Antibodies			
EGFR binding	SVM	−0.020	0.040
	DNN	0.105	0.067
Nanobody thermostability	DNN	0.0148	0.0149
	SVM	0.030	0.041
AC-SINS self-association	LightGBM	0.089	0.157
	GP	0.029	0.054
SMAC colloidal stability	DNN	0.136	**0.030**
	SVM	0.090	0.003
CHO polyreactivity	GP	0.082	0.092
	DNN	0.016	0.047
Expression titer	DNN	−0.095	0.110
	RF	−0.037	0.125
Binding free energy ($\Delta\Delta G$)	SVM	−0.052	0.100
	DNN	−0.067	0.047
Reactions			
Buchwald–Hartwig yield	RF	−0.002	−0.032
	LightGBM	−0.014	−0.044

Table 6.14: Distance to the nearest training molecules against prediction error. One row is one dataset and downstream model. Values are the Spearman correlation between a held-out molecule's mean distance to its 5 nearest training molecules and the model's error on it (higher is better), mean over five outer folds, Manhattan distance, unweighted. The model is fitted on all representations joined; distance is measured in the kNN model's best single representation and in the kNN hybrid.

Dataset	Single representation	Hybrid	Principal components
Small molecules			
QM9	0.601	**0.393**	0.570
Caco-2	0.302	0.286	0.300
MDR1-MDCK efflux	0.527	0.476	0.549
hERG K_i	0.528	0.512	0.543
LogD	0.593	0.543	0.579
hERG blocking	0.285	0.272	0.284
Antibodies			
EGFR binding	0.271	0.270	**0.261**
Nanobody thermostability	8.804	8.162	9.208
AC-SINS self-association	7.444	6.818	7.302
SMAC colloidal stability	0.459	0.424	0.474
CHO polyreactivity	0.164	0.1546	0.1546
Expression titer	117.424	115.030	115.474
Binding free energy ($\Delta\Delta G$)	1.439	**1.507**	1.567
Reactions			
Buchwald–Hartwig yield	12.0	12.1	16.3

Table 6.15: Label gap between each held-out molecule and its nearest training molecules (lower is better), with the feature space and setting picked again for each outer fold, unweighted. One row is one dataset; mean over five outer folds, in the dataset's label units (on hERG blocking, one minus the weighted share of neighbors in the molecule's class; on Buchwald–Hartwig, percentage points of yield). Single representation: the kNN model's two best. Hybrid: every hybrid of the dataset. Principal components: of all representations, as many as the hybrid has features.

tracks label differences more closely, but the methods built on it do not always gain. With the setting picked again, the hybrid predicts worse from nearest neighbors on nanobody thermostability, AC-SINS self-association, CHO polyreactivity, expression titer, and Buchwald–Hartwig. It retrieves neighbors with less close labels on $\Delta\Delta G$ and Buchwald–Hartwig.

Methods that find noisy labels by comparing each label with its neighbors' labels rest on the distance studied here. Chapter 7 tests whether the hybrid's distance makes noisy labels easier to find and to act on.

	3 clusters		5 clusters		10 clusters	
Dataset	Single	Hybrid	Single	Hybrid	Single	Hybrid
Small molecules						
QM9	0.114	0.330	0.100	0.348	0.130	0.357
Caco-2	0.082	0.056	0.081	0.093	0.107	0.114
MDR1-MDCK efflux	0.104	0.172	0.091	0.161	0.124	0.180
hERG K_i	0.334	0.280	0.292	0.302	0.224	0.223
LogD	0.053	0.139	0.063	0.149	0.100	**0.225**
hERG blocking	0.615	0.757	0.614	0.746	0.657	0.740
Antibodies						
EGFR binding	0.099	0.091	0.191	0.163	0.216	0.148
Nanobody thermostability	0.154	0.218	0.153	0.205	0.259	0.317
AC-SINS self-association	0.044	0.033	0.027	0.070	0.243	0.286
SMAC colloidal stability	0.241	0.180	0.467	0.350	0.537	**0.461**
CHO polyreactivity	0.044	0.094	0.084	0.152	0.112	0.327
Expression titer	0.080	0.071	0.241	0.212	0.231	0.244
Binding free energy ($\Delta\Delta G$)	0.028	0.148	0.134	0.200	0.224	0.218
Reactions						
Buchwald–Hartwig yield	0.036	0.072	0.092	0.075	0.083	0.104

Table 6.16: Label coherence of k-means clusters of the held-out molecules (higher is better), in the single representation and the hybrid picked again for each outer fold. One row is one dataset; mean over five outer folds. Coherence is one minus the mean within-cluster SD of the labels over their overall SD; on hERG blocking, the mean share of the largest class in each cluster.

Chapter 7

Detecting and handling label noise

7.1 Introduction

In Chapter 6, on all six small-molecule datasets, distance in the kNN hybrid, the hybrid built for prediction from nearest neighbors, tracked the label more closely than distance in a single representation, and nearest-neighbor prediction improved. Here we ask whether that helps with label noise. Can we find the noisy labels in a QSAR dataset, and does acting on them improve a model's held-out score? We ask this on labels damaged on purpose, where we know which labels were damaged and by how much. We also ask it on the labels as delivered, which carry an unknown amount of experimental error.

Every method here depends on a feature space, because a detector compares a molecule's label with its neighbors' labels or with a model's prediction. Features unrelated to the label still add to the distance, as explained in Chapter 4, and the adaptive hybrid representation of Chapter 5 leaves them out. For each method we therefore ask whether it improves the held-out score, and whether it improves the hybrid more than the best single representation.

We sort the methods into noise detection, noise mitigation, and noise-robust training, as defined in Table 2.3. We expand the sweep of twelve LNL methods from Chapter 4, as discussed in Section 4.2.3, "A small sweep of methods for learning with noisy labels". We also add methods that select molecules by a known flag, such as a label at an assay limit, which no published method uses. QM9, whose labels are computed rather than measured, is the clean-label control, as described in Section 7.2.5.

7.2 Methods

We use the datasets of Table 5.7 except seven, namely rat microsomal clearance, anti-VEGF scan expression and binding, EGFR binding, angiopoietin-2 binding, antibody humanness, and HIC retention. Folds and metrics are as in Chapter 5. We add hERG IC_{50}, shortened to HIC_{50} in the figures, as listed in Table 5.8. Its labels are hERG pIC_{50} values from ChEMBL, the negative base-10 logarithm of the molar concentration giving half of the maximal inhibition. hERG IC_{50} is the only dataset with real repeat measurements. In this chapter, "hERG" on its own means the hERG blocking dataset, whose labels are blocker or non-blocker.

In most experiments, the representations are chosen on clean labels. Within each outer fold, a search on the training molecules picks two single representations and two hybrids for each of the dataset's two downstream models. The search ranks them by their mean score over its inner folds. We call them the first and second single representation and the first and second hybrid. The second hybrid is the next one whose selected features differ from those of the first. The ranking uses the training molecules only, so the first need not score best on the held-out molecules. Every method is tested on all four. Only the hybrid's feature ranking and budget split are redone on the damaged labels. The rest of the search uses clean labels, which a real user would not have, so the reported scores may be too favorable. Section 7.3.6 measures this by searching on labels with Gaussian noise instead. For most datasets the two searches scored about the same. The largest loss was on hERG K_i with the DNN, 0.779 against 0.669 RMSE on one fold. Two experiments differ. The expanded LNL sweep repeats the whole search on the damaged labels for every damage kind, level, and draw. The experiment described in Section 7.2.3 compares cleaning before the feature search with cleaning after it.

We keep apart three tasks that are often treated as one, namely finding the damaged labels, sizing the damage, and improving the held-out score. A method can succeed at the first and fail at the third. Figure 7.1 shows how one method is run and scored.

7.2.1 Noise conditions

We inject label noise into a regression dataset in six ways, as listed in Table 7.1. Four are the Gaussian, outlier, grouped-shifted, and censoring conditions of Section 2.4.1. Censoring clips labels at a value that imitates an assay's reporting limit. We clip at the top of the label range, the rarer case, as real censoring of potency labels such as hERG K_i falls mostly at the low end. Under outlier noise, a random 5% of molecules

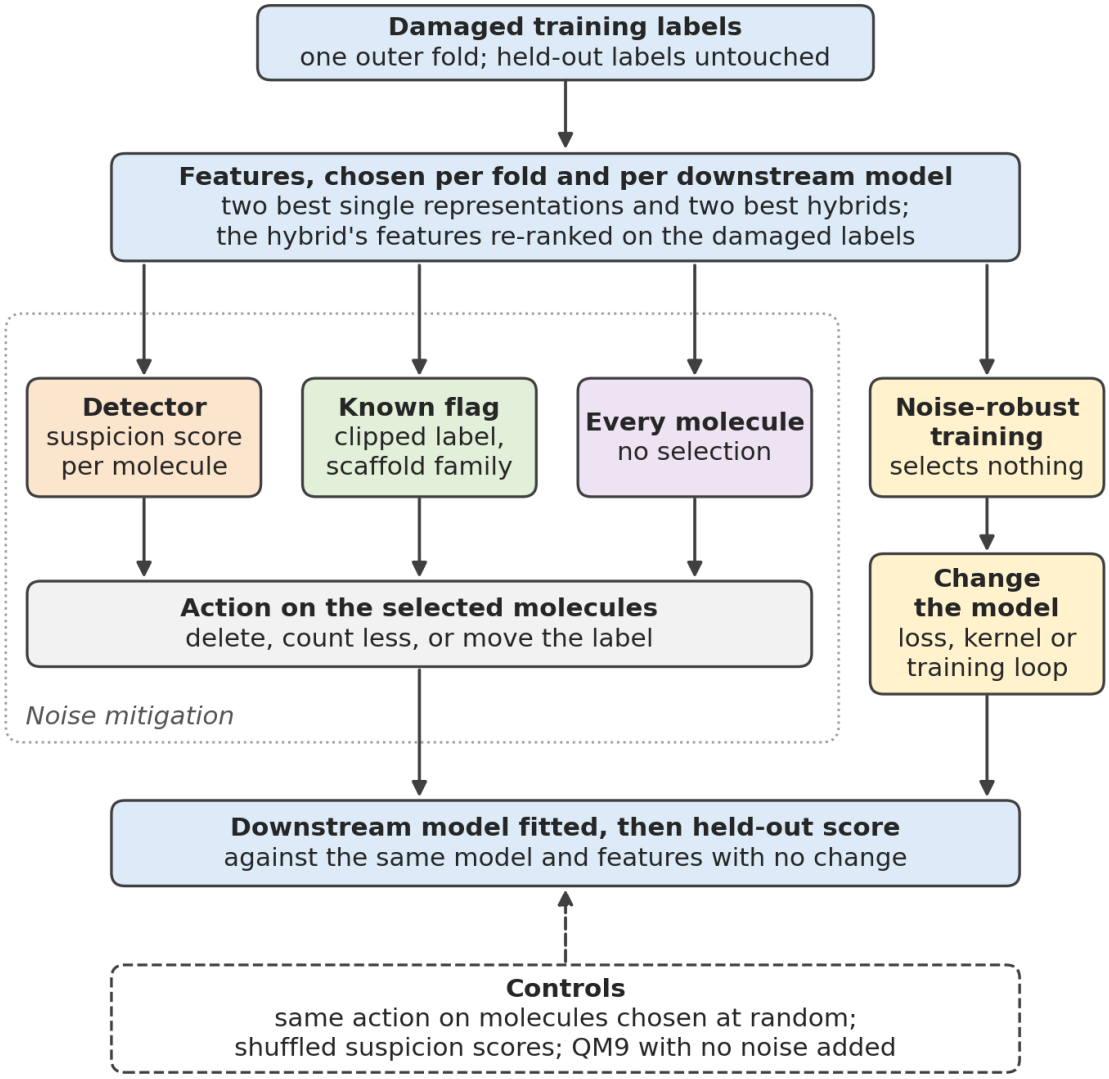


Figure 7.1: How a noise method is run and scored in one outer fold. Noise mitigation selects molecules by a detector, a known flag, or every molecule, then acts on them. Noise-robust training selects nothing and changes the model. Dashed: the controls.

get an error three times wider, against 10% in Chapter 3. The other labels keep the ordinary error, so the 5% carry about a third of the squared error. We leave out the grouped-wider condition and the Student-t and Laplace shapes of Chapter 3.

For classification tasks we take a different approach, flipping a share of the training labels. A label can flip uniformly, more often in the rarer classes, or more often in one direction. It can also flip at a rate that varies by molecule, in proportion to class frequency, or along a confusion matrix. With no matrix given, the last rule equals the first, leaving five distinct rules on a two-class dataset.

Most experiments first screen each method at the two levels given in Table 7.2. Experiments taken further run all six kinds of noise at every level, on datasets of 5,000

Damage kind	What is done to the training labels	Unit of the level	Labels moved
Gaussian	Every label gets Gaussian error (Equation 2.4)	SD of the clean training labels	All
Outlier	As Gaussian, but a random 5% get error three times wider	SD of the clean training labels	All
Grouped-shifted	Every scaffold family gets its own offset (Equation 2.5); small molecules only	SD of the clean training labels	All
Censoring	Labels above the $(1 - \text{level})$ quantile of the clean training labels are set to that quantile	Share of labels clipped	About the level
Shift in 10%	A random 10% are raised by one fixed amount	SD of the clean training labels	10%
Shift in 10%, sized by model error	As above, in a different unit	SD of the out-of-fold residual	10%

Table 7.1: The six damage kinds on a regression dataset. SD is standard deviation. The out-of-fold residual is that of a LightGBM model fitted to the clean training labels.

molecules or fewer, with three random draws per level.

Damage	Screen	Full run
Regression, other than censoring	None and 1.0	0, 0.5, 1.0, and 1.5
Censoring, share clipped	None and 25%	0, 10%, 25%, and 40%
Classification, share flipped	10%	Up to 40%

Table 7.2: Damage levels in the screen and in the full run. Regression levels are in the units of Table 7.1.

As in Chapter 3, only the training labels receive noise. Three settings use no injected damage: the labels as delivered, the real repeat measurements of hERG IC_{50}, and QM9 with no noise added. Three experiments add damage of their own, described alongside each experiment. One of them, on break-even precision, gives 5% of molecules wide errors and leaves the rest alone, as defined in Section 7.2.3.

7.2.2 Noise detection

A detector, as defined in Table 2.3, produces a suspicion score for each sample. The detectors, listed in Table 7.3, read an out-of-fold prediction, the neighbors' labels, or a model's predicted spread. Three spread detectors, the Heteroscedastic GP spread, the

Variance-head network, and the NGBoost spread, never actually read the molecule's label. Rather, they can only find a wrong label if the damage is tied to structure, meaning in theory they should only respond to systematic noise.

Detector	Reads	Suspicion score of molecule i	Source
Forest residual	Prediction	$\lvert y_i - \hat{y}_i \rvert / \sigma_y$, with $\hat{y}_i$ from an RF of 100 trees on 5 scaffold-group folds; on classification, $1 - \hat{p}_i(y_i)$	Brodley and Friedl [159]; Northcutt et al. [57]
LightGBM residual	Prediction	$\lvert y_i - \hat{y}_i \rvert$, with $\hat{y}_i$ from LightGBM on 5 scaffold-group folds	Brodley and Friedl [159]
Neighbor-label gap	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \sigma_y$; on classification, the share of neighbors in another class	Wilson [106], Martín et al. [161]
Gap over neighbor spread	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \max(s_{\mathcal{N}}, c)$, with $c = \max(0.01\,\sigma_y, 10^{-3})$	Båserud et al. [260]; Kordos and Blachnik [261]
Cliff-aware gap	Neighbors	$\dfrac{\lvert y_i - \bar{y}_{\mathcal{N}} \rvert}{\sigma_y} \cdot \dfrac{1}{1 + s_{\mathcal{N}}/\sigma_y}$	New; framing after Maggiora [81], Stumpfe and Bajorath [82]; related to Chawla and Sun [262]
GP spread and residual	Spread and prediction	Posterior SD plus $\lvert y_i - \hat{y}_i \rvert$, out of fold, each inner fold capped at 1,000 molecules	Rasmussen and Williams [122]; the sum is new
Heteroscedastic GP spread	Spread	Predicted noise SD, from a GP fitted jointly with a small network for the noise	After Kersting et al. [158]
Variance-head network	Spread	Predicted SD, trained with the β-NLL loss of Equation 7.1	Seitzer et al. [263]
NGBoost spread	Spread	Predicted SD, 50 boosting stages	Duan et al. [68]

Table 7.3: Detectors used in this chapter. The detectors from outside cheminformatics, used in one experiment, are described in the text. $\bar{y}_{\mathcal{N}}$ and $s_{\mathcal{N}}$ are the mean and SD of the labels of molecule i's k nearest training neighbors, and σ_y is the SD of the training labels. "New" means no published source was found.

Two of the neighbor detectors also take into account how much the neighbors disagree with each other, written $s_{\mathcal{N}}$ in Table 7.3. The Gap over neighbor spread divides a molecule's gap from its neighbors' mean by that disagreement, so a label far from neighbors that agree with each other scores highest. The Cliff-aware gap, which we introduce, lowers the score as the neighbors disagree more. A molecule near an activity

cliff (AC), whose neighbors' labels differ widely, is therefore less likely to be flagged. Similar ideas are used outside chemistry, to check weather-station readings against nearby stations [260], to filter regression labels [261], and to score outliers in spatial data [262]. We found no earlier use of the Gap over neighbor spread or the Cliff-aware gap on molecular labels.

The Variance-head network predicts a mean $\mu(x)$ and a variance $\sigma^2(x)$, and is trained with

$$\mathcal{L} = \frac{1}{2}\left(\frac{(y-\mu)^2}{\sigma^2} + \log\sigma^2\right)\cdot\sigma^{2\beta}, \tag{7.1}$$

where $\beta = 0.5$ keeps the variance from hiding the errors in the mean [263].

A neighbor detector needs a space, a neighbor count k, and a neighbor weighting. The space is a single representation, a hybrid, or a similarity space of Chapter 6. We try k of 5, 10, and 20, with neighbors weighted by 1/rank or $1/d^2$. To choose, we hold back 20% of the training scaffold families and predict each held-back label from its neighbors among the other training molecules. The three settings that predict best are kept.

Two experiments change the space in which the neighbors are found. The importance-scaling experiment multiplies each z-scored feature by the square root of its importance on the damaged training labels, rescaled to average 1, as in feature-weighted nearest neighbors [253]. The distance-family experiment compares four families of distance. These are the plain features under Euclidean and Manhattan distance, the features weighted by importance to the power 0.5 or 1.0, the hybrid's features, and forest proximity. The forest proximity of two molecules is the share of trees in which they share a leaf [264], and their distance is one minus this share. The forest is fitted to the damaged training labels.

The experiment on detectors from outside cheminformatics compares further detectors with the LightGBM residual. These are the Nearest-neighbor Shapley value, each molecule's Shapley value to a nearest-neighbor predictor [265, 266] and the Cleanlab score [267], which draws its internal folds at random rather than by scaffold family. The rest divide the LightGBM residual by a spread for each molecule, from NGBoost, a quantile forest, or a conformal interval.

Two experiments test the LightGBM residual under the shift sized by model error, grading the ranking by average precision. The first ranks molecules by the LightGBM residual. The second divides each residual by a second LightGBM model's estimate of that molecule's usual residual, and turns the result into conformal p-values [268]. The calibration set for these is a random 20% or 50% of the training molecules, or the

molecules with three or more agreeing repeat measurements. Molecules are flagged by the Benjamini–Hochberg rule [269]. It bounds the expected share of false flags, written q, at 5%, 10%, or 20% if the calibration labels are clean.

We can only grade a detector when some labels are damaged and others not. This leaves us with outlier noise, censoring, the two shifts in 10%, and label flips. The grade is the average precision of the suspicion scores against the damaged molecules. A detector that ranks at random scores the damaged share of the training molecules. Some experiments report the AUROC instead, where random chance is 0.5.

7.2.3 Noise mitigation

Table 7.4 describes each noise mitigation method by what selects the molecules and what is done to them, as defined in Table 2.3. A few ways of selecting are crossed with three actions.

Most methods read the suspicion score rescaled to $[0, 1]$, $\tilde{s}_i = (s_i - \min s)/(\max s - \min s)$. Deleting drops the molecules above a quantile of s, the filtering of Brodley and Friedl [159] that Zhao et al. [270] tested on QSAR modeling sets. Down-weighting gives molecule i the weight $1/(1 + \alpha\tilde{s}_i)$, divided by the mean weight. A proportional move takes the label part of the way to a target t_i,

$$y'_i = y_i + \alpha\, \tilde{s}_i\, (t_i - y_i), \tag{7.2}$$

so the most suspect label moves a share α of the way and the least suspect stays. The target is the weighted mean of the neighbors' labels [271, 272], or an out-of-fold LightGBM prediction, close to the target blending of Reed et al. [273]. The Score-scaled shrink moves a share $0.5\tilde{s}_i/(1 + 0.5\tilde{s}_i)$ of the way to that prediction. It is modeled on James–Stein shrinkage [274], which pulls an estimate toward a common value by an amount set by its variance. Here the variance cancels, so the share depends only on the score.

Selected by a detector

The main cleanup experiment pairs the Forest residual with each action above, and grades the Cliff-aware gap as a detector alongside it. A second experiment then asks what to do with that flagged label: do you delete it or keep it with a lower weight or a modified label? Its control deletes the same number of molecules at random, at five seeds. A third experiment swaps in five detectors, specifically the Cliff-aware gap, the

Method	Selected by	How many	Action	Target, how far
Selected by a detector				
Delete flagged	Forest residual or Neighbor-label gap	Top 5, 10, 20%	Delete	–
Five-detector delete or move	Five detectors (Section 7.2.2)	Top 5, 10, 20%	Delete or move	Detector's own prediction; α 0.25, 0.5, 1
Score down-weight	Forest residual or Neighbor-label gap	All	Count less	$1/(1+\alpha\tilde{s}_i)$
Neighbor-mean smoothing	As above	All	Move	Neighbors' mean; Equation 7.2, $\alpha = 0.25$
Move toward prediction	As above	All	Move	Out-of-fold LightGBM; Equation 7.2
Score-scaled shrink	As above	All	Move	Out-of-fold LightGBM; $0.5\tilde{s}_i/(1+0.5\tilde{s}_i)$
Neighbor weights	Neighbor-label gap, in a chosen distance	All	Count less	$1/(1+3\tilde{s}_i)$
Exponential down-weight	Gap over neighbor spread, GP spread and residual, or NGBoost spread	All	Count less	$\exp(-0.1\,\tilde{s}_i)$
Close-pair weights	Near-identical pairs whose labels disagree	Flagged pairs	Count less	Weight 0.25
Top-share down-weight	LightGBM residual	Top 5, 10, 20%	Count less	Weight 0.25
Cliff put-back	Any of the above, then a cliff test	Cliff molecules	Undo	Cliff molecules restored
Selected by a known flag				
Constant lift	Label at the assay limit	Clipped labels	Move	Limit plus one shared lift
Truncated-normal mean	As above	Clipped labels	Move	Equation 7.4
Censored EM	As above	Clipped labels	Move	As above, 3 rounds
Own lift	As above	Clipped labels	Move	Limit plus its own σm_i
Delete clipped	As above	Clipped labels	Delete	–
Scaffold-family offset	Scaffold family	All	Move	Minus the family's mean residual, or a random-intercept estimate
Source offset	Measuring source	All	Move	Minus the source's offset
Source weights	Measuring source	All	Count less	Inverse error variance
Count and spread weights	Repeat measurements	All	Count less	Count, or inverse error variance
Shrink by spread	Repeat measurements	All	Move	Out-of-fold LightGBM; further for a wider spread
Every molecule				
Huber blend	–	All	Move	Out-of-fold Huber LightGBM; halfway

Table 7.4: Noise mitigation methods, by what selects the molecules. $\tilde{s}_i$ is the suspicion score rescaled to $[0, 1]$ and α is the strength. Weights are divided by their mean. The four deleting methods of the LNL sweep are in Table 7.5.

GP spread and residual, the Heteroscedastic GP spread, the Variance-head network, and the NGBoost spread. Each then proceeds to delete its top 5%, 10%, or 20%. A detector that makes its own prediction also moves every label toward its associated predicted value. The share moved is given by Equation 7.2, or by $c_i = \alpha s_i/(1 + \alpha s_i)$, which uses the score in its own units.

Instead of deleting samples, four experiments weight them. Three use the weight $1/(1 + 3\tilde{s}_i)$, under which the most suspect molecule counts a quarter as much as the least suspect. They differ in where they look for neighbors, whether it be features scaled by importance on the damaged labels, or each of the four families of distance. The fourth uses the weight $\exp(-0.1\,\tilde{s}_i)$, under which the most suspect molecule still counts about 0.9 times as much. Its detector is the Gap over neighbor spread, the GP spread and residual, or the NGBoost spread.

Close-pair weights only look at near-identical pairs, defined as those whose pairwise distance is in the lowest 1% of training pairs. The squared label difference between two noisy copies of one value averages twice the noise variance. We therefore estimate the noise from the closest fifth of these pairs as $\hat{\sigma}^2 = \overline{\Delta y^2}/2$. Both molecules of a close pair whose labels differ by more than $3\sqrt{2}\,\hat{\sigma}$ get weight 0.25. A separate experiment asks how precise a detector must be before acting on its flags helps. It acts on sets of molecules of equal size whose share of truly damaged molecules runs from none to all. The share at which the gain crosses zero is the break-even precision.

The Cliff put-back method runs after a cleanup of the top 1%, 5%, or 20% of flagged molecules, and undoes it for AC molecules. A training molecule is designated to be a cliff if a partner with ECFP4 Tanimoto similarity of at least 0.55 [150] has a label far from its own [81, 82]. Far means more than twice the median absolute deviation of the labels. ECFP4 at this threshold is the standard definition of a similar pair, and we did not test other representations for this step, although the count of cliffs depends on the representation, as Chapter 4 showed. We keep the name on labels that are not a biological activity, such as LogD. The cliff set is rebuilt from the damaged training labels each time. Its control puts back as many molecules, drawn at random from the non-cliff molecules. We found no earlier method that puts examples back after a cleanup. Some cleaning methods exempt a whole class decided in advance [275, 276], and Brodley and Friedl [159] name the risk of discarding correct exceptions without giving a rule.

The hybrid's features then are chosen against the labels, such that the labels can be cleaned before the features are chosen or after. One experiment compares the two orders, under injected label damage and on labels as delivered, with nine cleanups

driven by the Forest residual. The search and the fit each use the labels as given or cleaned, giving four combinations. Down-weighting has only the two with an uncleaned search, because no weight enters the feature ranking or the budget split. We found no source that crosses the two. The nearest design ranks features on one version of the data and fits on another, but its second step is class-imbalance sampling, not label cleaning [277].

Nothing says which molecules definitively have noise on labels as delivered (except in the special case of QM9), so a gain can come from removing hard molecules rather than wrong ones. One experiment runs seven cleanups on delivered labels, with the Neighbor-label gap and the Forest residual as detectors. Its control shuffles each detector's scores across the training molecules, at three seeds, and reruns the cleanup. The shuffled cleanup touches as many molecules, but not the ones the Neighbor-label gap or the Forest residual picked.

Selected by a known flag

When labels are clipped to an assay limit, the detector is left with two things to guess: which labels are wrong and in what direction? We take the limit L as the largest clipped training label, if at least two molecules and 1% of the training molecules sit on it. An out-of-fold LightGBM model fitted to the clipped labels gives each molecule a prediction p_i. Let σ be the SD of $y_i - p_i$ over the molecules below the limit, and m_i the inverse Mills ratio,

$$m_i = \frac{\phi(a_i)}{1 - \Phi(a_i)}, \qquad a_i = \frac{L - p_i}{\sigma}, \tag{7.3}$$

where ϕ and Φ are the standard normal density and distribution function. The normal model for labels cut at a limit is that of Tobin [278]. The Truncated-normal mean replaces each clipped label by the mean of a normal distribution centered on its prediction and cut at the limit,

$$y'_i = \max(L,\ p_i + \sigma m_i)\,. \tag{7.4}$$

Censored EM repeats this for three rounds, after the iterated estimator of Buckley and James [279]. Each round refits the downstream model out of fold. The Constant lift raises every clipped label to $L + \overline{\sigma m}$, where $\overline{\sigma m}$ is the mean of σm_i over the clipped molecules. On average this shared lift exceeds the Truncated-normal mean's by $L - p_i$. Two further methods test what the flag is worth. One, Delete clipped, deletes the clipped molecules. The other, Own lift, raises each clipped label by its own σm_i, and

its control deals the same lifts to the wrong clipped molecules, at three seeds. The closest precedent is Svensson et al. [87], who fit censored regression to drug-discovery labels.

Another known flag is a scaffold family, as under grouped-shifted noise each family carries its own offset, and subtracting the family's mean out-of-fold residual undoes it. Three versions differ in how much they trust a small family. The first corrects every family and the second only families of three or more training molecules. The third is a random-intercept model, fitted by up to five rounds of EM as in mixed-effects random forests [280, 281]. It shrinks a small family's offset further toward zero.

One sign that your data has such noise is when several sources measured each molecule, resulting in a measuring source. We simulate this by creating eight "sources", such that each molecule is measured by one to four of them, and a number is drawn from hERG IC_{50}'s real counts or uniformly. Measurement s of molecule i is

$$y_{is} = y_i + b_s + \sigma_e \left(\sqrt{\rho}\, u_i + \sqrt{1-\rho}\, v_{is} \right), \tag{7.5}$$

where b_s is the source's offset, u_i an error shared by all of the molecule's measurements, and v_{is} an error of its own. The value of ρ, 0, 0.3, or 0.7, sets how much of the error the measurements share, and the offsets have SD 0 or $0.5\,\sigma_e$. The training label is then the mean of the molecule's measurements. The methods remove each source's offset, estimated by a mixed model with the source as a fixed effect and the molecule as a random intercept. They also weight each molecule by the inverse of its estimated error variance [282]. Two further versions use the true ρ and σ_e, which a user never has, so we treat them as bounds.

hERG IC_{50} carries its own flag, specifically how often each molecule was measured and how far apart the measurements are. A molecule is one canonical SMILES string exactly as ChEMBL gives it, with no further standardization. Stereoisomers or salt forms written as different strings therefore count as separate molecules. A molecule's count is its number of IC_{50} records, and its spread is the variance of their pIC_{50} values. One method weights each molecule by its count. A second weights it by $1/(\sigma_u^2 + \tilde{s}_i^2/n_i)$, the inverse of its estimated error variance [282]. Here n_i is the count and σ_u^2 is the error that remains however often a molecule is measured. Because a variance from two or three records is unreliable [283], $\tilde{s}_i^2$ is pulled toward the pooled spread, as if the pool were three more measurements. A third moves each label toward an out-of-fold LightGBM prediction by a share $v_i/(v_i+\tau^2)$, so a less precise label moves further. Here v_i is the spread divided by the count and τ^2 the variance of the out-of-fold residual.

Only held-out molecules with three or more measurements are graded.

Every molecule

One method moves every label halfway toward the out-of-fold prediction of a LightGBM model fitted with a Huber loss [102]. The Huber loss keeps outlying labels from pulling that prediction. Its threshold is 1.4826 times the median absolute out-of-fold residual.

7.2.4 Noise-robust training

Five experiments test noise-robust training, as defined in Table 2.3. They change the downstream model, rerun the twelve LNL methods, use a robust loss, exchange molecules between two learners, or use repeat measurements.

Changing the model instead of the labels

One important comparison point for every mitigation result is changing the model instead of the labels. One experiment does this in four ways. Retuning tries the default hyperparameters and 20 random settings on three scaffold-split inner folds of the damaged labels, and refits the best. Its control tunes on clean labels and fits the chosen setting to the damaged ones, which separates tuning against the noise from tuning at all. Seed averaging averages ten copies of LightGBM, XGBoost, or the DNN with different seeds, each tree model fitted on 80% of the molecules and features. Stacking combines the dataset's two models by non-negative least squares on their out-of-fold predictions [284]. The fourth, 1,000-tree RF, gives the RF 1,000 trees instead of 100. The damage is each regression kind of Table 7.1, plus Gaussian noise with variance 0.1, 0.3, or 1.0 times a LightGBM model's out-of-fold mean squared error on clean labels. This experiment ran on Buchwald–Hartwig, Caco-2, hERG K_i, LogD, and QM9.

The expanded sweep of methods for learning with noisy labels

If we recall, the LNL investigation began in Chapter 4 with 12 methods. All use the plain DNN, with hidden layers of 256, 128, and 64 units. Unlike the earlier sweep, this one covers many datasets instead of QM9 alone, every damage kind instead of Gaussian noise alone, and hybrids as well as single representations. It also scores each method against the plain DNN on the same damaged labels and features. Four of the twelve delete molecules a detector selects, so under Table 2.3 they get classified as noise mitigation.

Method	Group	What it does here	Source, and how it differs
Small-loss selection	Mitigation	After 10 epochs, keeps the 70% of molecules with the smallest loss and retrains	Han et al. [156]; one network, not two
Confident learning	Mitigation	Three networks; deletes molecules with error above the 85th percentile and ensemble spread below the 25th, then retrains	Northcutt et al. [57]; in-sample, no per-class thresholds
Distance-based selection	Mitigation	Deletes the 15% with the largest gap to their 10 neighbors' mean label, divided by those neighbors' spread (on classification, the share of neighbors in another class)	After Wilson [106]
Multistage cleaning	Mitigation	Three rounds, each deleting the 10% with the largest in-sample error	Adapted from Zhao et al. [285], which fits a mixture to loss histories and deletes once
Early learning	Robust training	Each epoch, molecules with above-median loss lose 5% of their weight	Adapted from Zhang et al. [286], which updates weights from a clean subset
Uncertainty curriculum	Robust training	After 20 epochs, trains on the lowest-loss 25%, then 50%, 75%, and 100%	Self-paced learning [287]
DivideMix	Robust training	Two networks; a two-component mixture on each network's loss picks the molecules the other trains on	Li et al. [155], without its semi-supervised half
Meta-Weight-Net	Robust training	A small network learns each molecule's weight from its loss, tuned on a held-back set of noisy labels	Shu et al. [243], which needs clean labels for that set
MentorNet	Robust training	A small network weights each molecule from its loss, loss history, and distance to its neighbors	After Jiang et al. [288]
Contrast-to-divide	Robust training	Down-weights molecules with high loss whose predictions are stable when the features are perturbed	Named after Zheltonozhskii et al. [289], which does not use perturbation
Heteroscedastic GP	Robust training	A GP's residuals train a network that predicts each molecule's noise; the plain DNN is then trained with weights $1/(1 + \text{noise})$	After Kersting et al. [158]
Evidential network	Robust training	Four output heads on the same layers predict the parameters of a normal-inverse-gamma distribution	Amini et al. [290]

Table 7.5: The twelve LNL methods of the expanded sweep. All use the same DNN. "Mitigation" marks the four that select molecules with a detector and delete them. Multistage cleaning, Early learning, and Contrast-to-divide are our adaptations of the named methods.

Huber loss

We explore alternative losses, specifically a Huber loss [102], with threshold c times 1.4826 times the median absolute out-of-fold residual, for c of 0.5, 1, 1.5, or 2. Under the shift sized by model error, each model gets its own robust loss at $c = 1.5$. We use this loss for LightGBM and the DNN, a pseudo-Huber for XGBoost, and absolute-error splits for the RF.

Co-teaching

Every molecule gets scored by LightGBM and an SVM by its out-of-fold error. For two rounds, each is refitted on the molecules the other finds easiest. Each then keeps its easiest 80%, 90%, or 95%, and the downstream model is fitted on the molecules both keep. This is the exchange of co-teaching [156], made between whole refits rather than inside one training loop. Its control deletes as many molecules at random, at three seeds.

Repeat measurements

On hERG IC_{50}, four methods change what the model receives. The first trains on one row per measurement instead of one per molecule. The second adds the log of the measurement count as a feature, and the third gives NGBoost that feature. The fourth fits an XGBoost accelerated-failure-time model [291] to the interval between each molecule's lowest and highest measurement.

7.2.5 Evaluation and controls

Each group of methods has its own question to answer, and Table 7.6 gives the test for each. A detector can only be graded where some labels were damaged and others were not, as Section 7.2.2 explains.

Each control is described alongside its experiment. QM9 acts as an entire control dataset due to its clean, computed labels [113]. A method that removes noise should therefore do nothing on QM9, or make it worse. A gain on QM9 comes from a change that would help on any labels, such as a better-trained model. The strongest evidence that a method found real noise is that it leaves QM9 unchanged and improves a measured dataset's labels as delivered. The shuffled-score control cannot make this distinction alone, because it asks whether the choice of molecules matters, not whether there is noise. Section 7.3.1 applies this test.

Group of methods	Compared with	Control
Noise detection	Damaged against undamaged molecules	On QM9 with no noise added, every flag is false
Mitigation, selected by a detector or flag	Same model and features, no action	Same action on randomly chosen molecules, where run
Mitigation on every molecule	No action	QM9 with no noise added only
Noise-robust training	Same model trained normally on the same labels	Gain on QM9 with no noise added, set beside the gain under damage

Table 7.6: How each group of noise-handling methods is tested. One row is one group, as defined in Table 2.3.

Where a clean score exists, we also report how much of the lost score a method wins back. Each model m is fitted to the clean, damaged, and treated training labels, giving held-out scores s^m_{clean}, s^m_{damaged}, and s^m_{treated} in the dataset's own metric. The recovery is

$$\text{Recovery}_m = 100 \times \frac{s^m_{\text{treated}} - s^m_{\text{damaged}}}{s^m_{\text{clean}} - s^m_{\text{damaged}}} \ \%, \tag{7.6}$$

which reads the same whether higher or lower values are better. It is undefined when the clean and damaged scores differ by 2% of the clean score or less. We report it for single representations only. A hybrid's features are re-ranked on the damaged labels, so its clean score comes from different features.

7.3 Results

Mitigation helps most often under injected damage and least often on the labels as delivered, as counted for each group in Additional file 34.

Three caveats hold for every result below. First, due to time and compute constraints, most results are on the first outer fold only, as explained in Section 5.2.6, and we give the fold count wherever more exist. Second, where a method has several settings, we report its best on the held-out score. The setting is picked on the same molecules that score it, so every gain is optimistic, for the single representation and the hybrid alike. The same choice is made on QM9 with no noise added. A gain there cannot come from removing noise, so Section 7.3.1 also shows how much the choice of setting and better training can add together. Third, the DNN's score changes between fits. On QM9, with the same fold and labels, its plain second hybrid scored

0.362 RMSE in one experiment and 0.364 in another, as Tables 7.12 and 7.34 show. LightGBM gives the same score every time.

7.3.1 The clean-label test: where noise is removed

We first sort the methods by what they do to QM9, as set out in Section 7.2.5. A method that also helps QM9 is, at least in part, a better way to train.

Deletion passes the test. On QM9 with no noise added, it made every model worse in every configuration of Table 7.12, and in the deletion row of Table 7.13. In other QM9 runs it gained at most 0.010 RMSE, about the DNN's spread between fits. On SMAC colloidal stability and expression titer, and on the weaker configurations of nanobody thermostability and AC-SINS self-association, deletion improved the labels as delivered and beat every shuffled control. These are the clearest cases in the chapter of a method finding noise already in the data. On the small-molecule labels as delivered, deletion found almost nothing to remove, as Section 7.3.2, "The labels as delivered", shows.

Neighbor-mean smoothing and weighting pass the test only in part. With the DNN they also improve QM9, while LightGBM barely moves, as Tables 7.11 and 7.13 and Figure 7.3 show. On Caco-2, LogD, and MDR1–MDCK efflux as delivered, the DNN gains about as much as on QM9, so those gains are not evidence of noise. The SVM gains almost nothing on these labels. The exception among small molecules is Neighbor-mean smoothing on hERG K_i. There the DNN's first single representation gains several times more than on QM9, relative to its score.

With the DNN, passing the shuffled-score control means little on its own. In Table 7.13, Neighbor-mean smoothing on QM9's first single representation beats all three shuffles, although there is no noise to find. The shuffles move about as much as the DNN does between fits.

Changing the model fails the test, so its gains come more from better training than from handling noise. On QM9 with no noise added, Stacking improves every configuration and Seed averaging improves most, as Table 7.31 shows. On hERG K_i and LogD as delivered, they help by about as much. Part of Seed averaging's gain does respond to noise, though. Under injected Gaussian noise it gains more than without noise in every configuration on hERG K_i, and with both models on QM9, as Table 7.30 shows.

The twelve LNL methods mostly pass the test, unlike changing the model. On QM9 with no noise added, each made the DNN worse in a first run, reported in Table 7.34. In a second run about half beat it, but only by about the DNN's own spread between

fits. A gain from one of them on a measured dataset is therefore more likely to be noise found. The clearest case is the Heteroscedastic GP on $\Delta\Delta G$, which raises the best single representation's Spearman and makes QM9 worse in the first run, as shown in Tables 7.33 and 7.34.

Noise is removed most clearly where deletion or an LNL method helps an antibody dataset as delivered and does nothing for QM9. On the small-molecule and reaction labels as delivered, three results clear this bar, namely Neighbor-mean smoothing on hERG K_i and on hERG blocking, and Delete flagged on Buchwald–Hartwig. Under injected damage, the Huber blend, which moves every label toward a Huber-fitted prediction, also clears it. It helped hERG K_i under three kinds of damage and made QM9 worse, as Section 7.3.4, "Gaussian noise, outliers, and shifts in a few labels", reports. The test rests on one computed property of one dataset, and a few methods have no QM9 result, such as those that use the repeat measurements of hERG IC_{50}.

7.3.2 Changing a label against deleting a molecule

On hERG blocking with 10% of labels flipped along a confusion matrix, every action helped and deletion helped most. Table 7.7 compares the actions using gradient-boosted trees, on the hybrid and the best single representation. Deletion, the only action with a control, also beat deleting the same number of molecules at random, as Table 7.8 shows. With the Neighbor-label gap as the detector, it took the hybrid to 0.838 AUROC, against 0.818 for the best of five random deletions. With the Forest residual, it beat random deletion on each of the three folds tested.

Deletion kept its lead over random deletion as the share of molecules flagged changed, as Figure 7.2 shows for hERG blocking with flipped labels. Under both kinds of flips, it beat the best random deletion at every share, on the hybrid and on the best single representation. This control is strict, since the best of the five random deletions is picked on the held-out score. Under the confusion-matrix flips, every action lifted the hybrid above the single representation, which had started ahead. The figure also varies the share of labels clipped on Caco-2. There Censored EM lowered the DNN's RMSE at every clipped share, while Delete clipped raised it. At 40% clipped, Censored EM took the hybrid from 0.365 to 0.317 RMSE.

The actions differ less on a regression dataset, as Table 7.9 shows for LogD with 5% outliers and the DNN. On the first single representation every action helped, most of all Delete flagged and the Score-scaled shrink toward an out-of-fold prediction. On the first hybrid, Score down-weight helped most and the Score-scaled shrink made it

Action	Fold 0, mean of 3 draws		Fold 1, draw 0	
	First single	First hybrid	First single	First hybrid
No cleaning	0.806	0.803	0.8155	0.831
Act on the 10% the forest residual flags				
Delete flagged	0.8253	**0.836**		
Neighbor-mean smoothing	0.8250	0.834		
Score down-weight	0.811	0.814		
Five-detector delete or move, top 10% deleted				
Cliff-aware gap	0.818	**0.838**	0.814	**0.833**
GP spread and residual	0.817	0.827	0.807	0.820
Variance-head network	0.797	0.813	0.795	0.815
NGBoost spread	0.804	0.807	0.804	0.829
Heteroscedastic GP spread	0.801	0.806	0.8147	0.823

Table 7.7: Actions and detectors under hERG label flips. hERG blocking with 10% of training labels flipped along a confusion matrix, XGBoost. One row is one action on the flagged molecules. Held-out AUROC, higher is better. Fold 1 was run for the detectors only, each at its best setting.

Configuration and detector	Share deleted	Fold	No cleaning	Delete flagged	Best random deletion
hERG, 10% confusion-matrix flips, XGBoost, first hybrid, AUROC					
Neighbor-label gap	10%	0	0.803	**0.838**	0.818
Forest residual	10%	0	0.803	0.836	0.813
Forest residual	10%	1	0.816	0.826	0.812
Forest residual	10%	2	0.819	0.835	0.828
LogD, 5% outliers, DNN, RMSE					
Second hybrid, forest residual	5%	0	0.585	**0.546**	0.558
First hybrid, forest residual	10%	0	0.588	0.569	0.565
Second hybrid, forest residual	5%	1	0.624	0.633	0.623
Buchwald–Hartwig, labels as delivered, LightGBM, RMSE					
First hybrid, neighbor-label gap	20%	0	20.1	**18.0**	19.9
Second single, neighbor-label gap	20%	0	20.2	18.3	19.9

Table 7.8: Deleting flagged molecules against random deletion. One row is one configuration, detector, and outer fold. The random control deletes the same number of molecules at five seeds; the best of the five is shown. The Neighbor-label gap row and the Cliff-aware gap row of Table 7.7 come from separate experiments that both reach 0.838. Scores are means over the noise draws. AUROC: higher is better. RMSE: lower is better; for Buchwald–Hartwig in percentage points of yield.

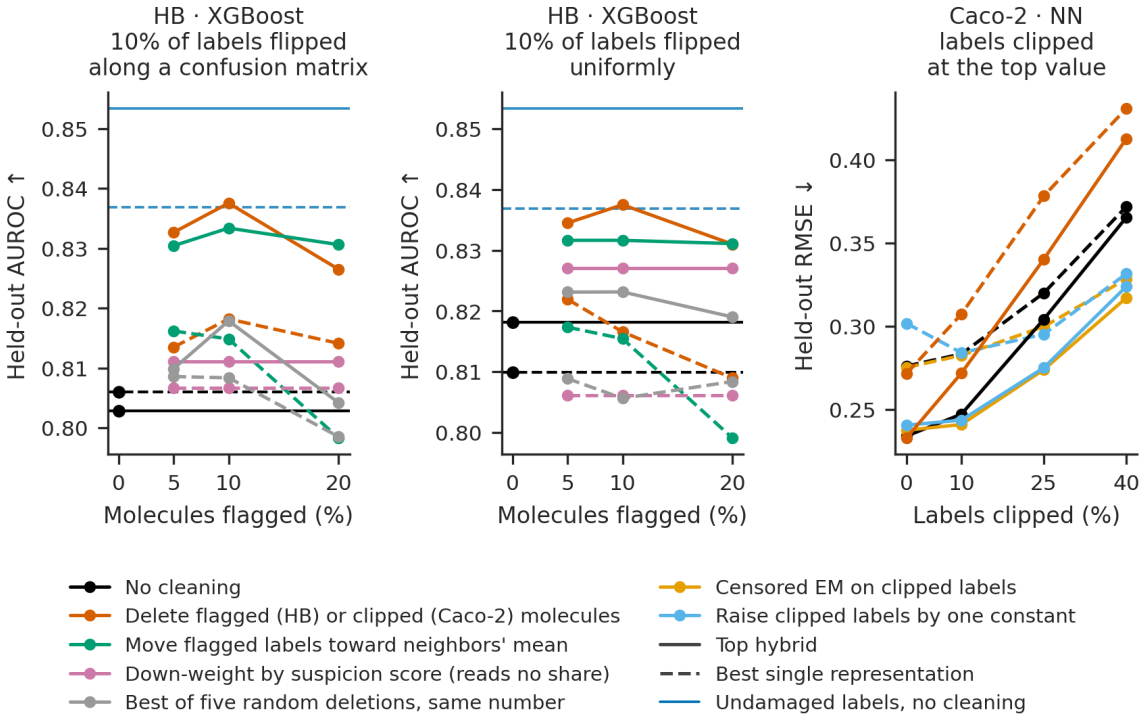


Figure 7.2: Left and middle: held-out AUROC (higher is better) of XGBoost on hERG blocking with 10% of training labels flipped, against the share of training molecules flagged by the Neighbor-label gap. The blue lines are the scores on undamaged labels. Right: held-out RMSE (lower is better) of the DNN on Caco-2 against the share of training labels clipped. Solid lines are the first hybrid and dashed lines the first single representation ("Top hybrid" and "Best single representation" in the legend). Outer fold 0. Dataset short names as in Table 5.8.

slightly worse. Against random deletion the result is mixed. Table 7.8 shows deletion beating it on the second hybrid and losing to it on the first.

The detector matters more than the action that follows. Under the same hERG flips, we deleted the top 10% by each of five detectors, as Table 7.7 lists. The Cliff-aware gap helped the hybrid most, followed by the GP spread and residual. The Variance-head network, NGBoost spread, and Heteroscedastic GP spread helped the hybrid least and made the single representation worse. They predict a spread from the features alone and never read a label, so they cannot see a flipped one, as explained in Section 7.2.2. On a second hERG fold, with one noise draw, the Cliff-aware gap improved the hybrid but not the single representation. The three detectors that never read a label made both worse. On LogD with outliers, the GP spread and residual was best for the DNN's hybrid, as Table 7.9 shows.

The gain depends on which molecules the action reaches, which we test with flagged sets whose content we control. On hERG K_i with wide errors in 5% of molecules, Top-share down-weight, which down-weights the 10% the LightGBM residual flags,

Action	First single	First hybrid
No cleaning	0.627	0.588
Act on the 5% the forest residual flags		
Delete flagged	0.603	0.572
Score down-weight	0.615	**0.563**
Neighbor-mean smoothing	0.612	0.580
Score-scaled shrink	**0.601**	0.590
Separate run: Five-detector delete or move, top 10%		
No cleaning	0.627	0.588
Cliff-aware gap	0.628	0.569
GP spread and residual	0.623	**0.558**
Variance-head network	0.636	0.584
NGBoost spread	0.657	0.580
Heteroscedastic GP spread	0.648	0.569

Table 7.9: Actions and detectors on LogD with 5% outliers. DNN, outer fold 0, mean of 3 draws. One row is one action. Held-out RMSE, lower is better.

improved the DNN on both configurations in Table 7.10. The first single representation went from 0.689 to 0.642 RMSE. Down-weighting the same number of molecules, none of them damaged, made both worse. On QM9 with no noise added, the same down-weighting made both slightly worse.

			10% with no damaged molecule	
Configuration	No cleaning	LightGBM residual's top 10%	Seed 0	Seed 1
hERG K_i, wide errors in 5% of molecules, mean of 3 draws				
First single	0.689	**0.642**	0.706	0.701
First hybrid	0.692	0.639	0.698	0.704
QM9, no noise added, RMSE in eV				
First single	0.396	0.402		
First hybrid	0.355	0.359		

Table 7.10: Down-weighting flags of known content on hERG K_i with wide errors in 5% of molecules, and the same down-weighting on QM9 with no noise added. DNN, outer fold 0. One row is one configuration. Held-out RMSE, lower is better. Empty: not defined, since QM9 has no damaged molecule. Bold: the values discussed in the text.

Weighting keeps every molecule and every label, and it helped on hERG, LogD, and Buchwald–Hartwig, as Table 7.11 shows. Under the hERG flips it helped the hybrid and barely moved the single representation. It also helped the DNN on QM9 with

no noise added, by about 0.01 RMSE with Neighbor weights or Exponential down-weight, while LightGBM barely moved. So a DNN gain of that size from weighting is not evidence that noise was removed. Close-pair weights, which down-weights near-identical pairs that disagree and helped Buchwald–Hartwig, also helped the DNN on QM9 but made LightGBM worse.

Configuration	Weighting	Unweighted	Weighted
hERG, 10% confusion-matrix flips, XGBoost, AUROC			
First single	Exponential down-weight, gap over neighbor spread	0.8060	0.8063
First hybrid	Exponential down-weight, gap over neighbor spread	0.803	**0.815**
LogD, 5% outliers, DNN, RMSE			
First single	Neighbor weights, best of four distances	0.627	0.591
First hybrid	Neighbor weights, best of four distances	0.588	**0.556**
Buchwald–Hartwig, Gaussian noise at 1.0, LightGBM, RMSE			
First hybrid	Close-pair weights	22.0	**20.7**
QM9, no noise added, RMSE			
DNN, second hybrid	Neighbor weights	0.362	**0.348**
DNN, second hybrid	Exponential down-weight		0.349
LightGBM, first hybrid	Neighbor weights	0.372	0.371
LightGBM, first hybrid	Exponential down-weight		0.372
DNN, second hybrid	Close-pair weights	0.362	0.350
LightGBM, first hybrid	Close-pair weights	0.372	0.390

Table 7.11: Weighting flagged molecules. One row is one configuration and weighting, at its best setting. Outer fold 0. AUROC: higher is better. RMSE: lower is better; for Buchwald–Hartwig in percentage points of yield. The Exponential down-weight run on QM9 stores no unweighted score.

Figure 7.3 shows how often each method improved on no cleaning when a detector selects the molecules. Figures 7.4 and 7.5 do the same for methods which act on a known flag and for noise-robust training.

Additional file 35 counts, for each broad noise mitigation method, the conditions where it improved the best single representation, the hybrid, both, or neither.

Deleting almost never helps on clean labels, which is the intention. On QM9 with no noise added, deletion made every model and configuration in Table 7.12 worse. The DNN's hybrid, for example, went from 0.355 to 0.393 RMSE. In two other QM9 experiments, deletion also made both models worse on the best single representation and the hybrid. In a third, deleting the top 10% by each of five detectors made the

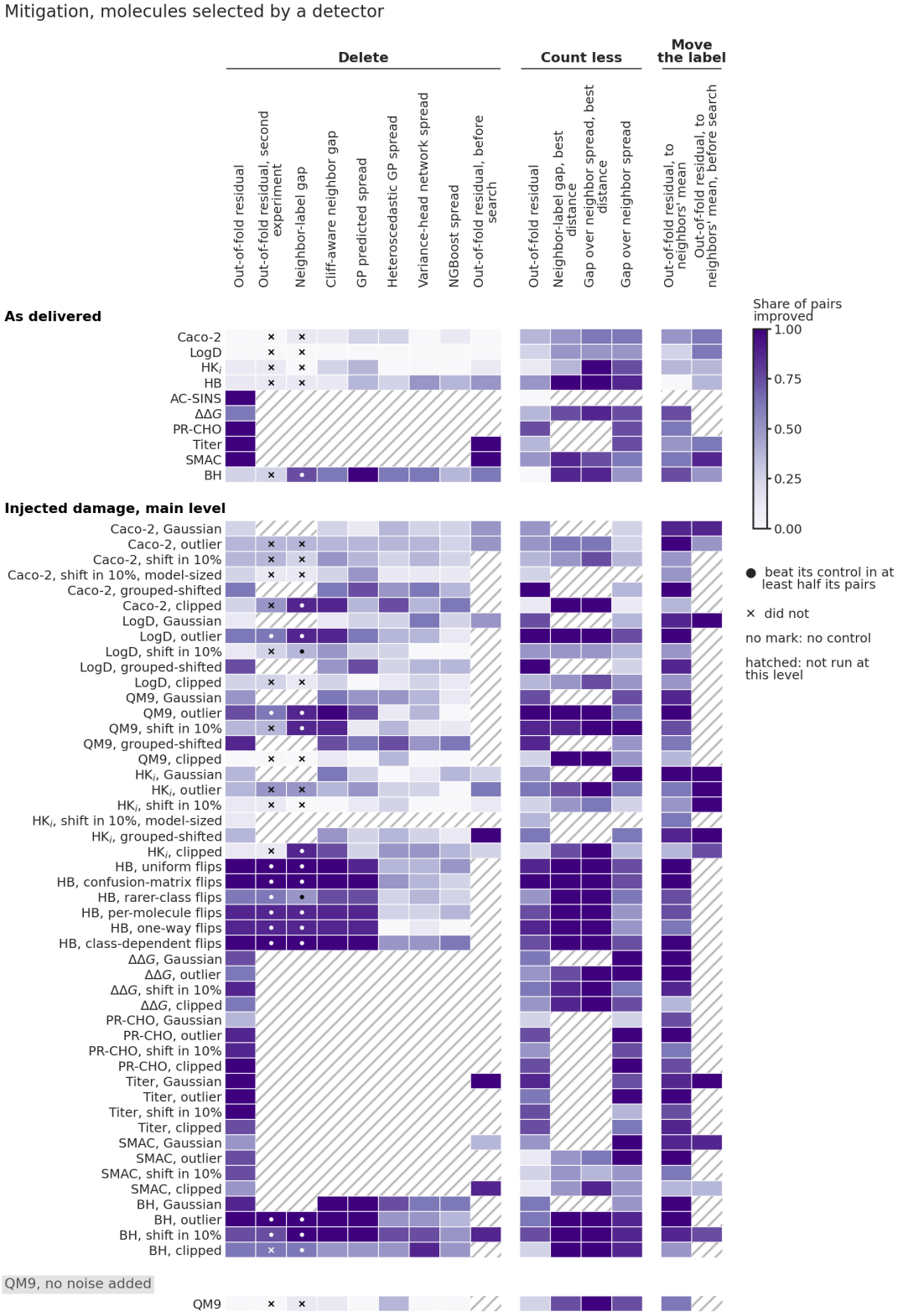


Figure 7.3: Noise mitigation with the molecules selected by a detector, first outer fold. Rows: labels as delivered, injected damage at the main level, and QM9 with no noise added. Columns: method, grouped by action. Shade: share of model-and-configuration pairs improved over no cleaning; darker is more. Dot: beat random deletion in at least half its pairs; cross: did not. Hatched: not run. Dataset short names as in Table 5.8.

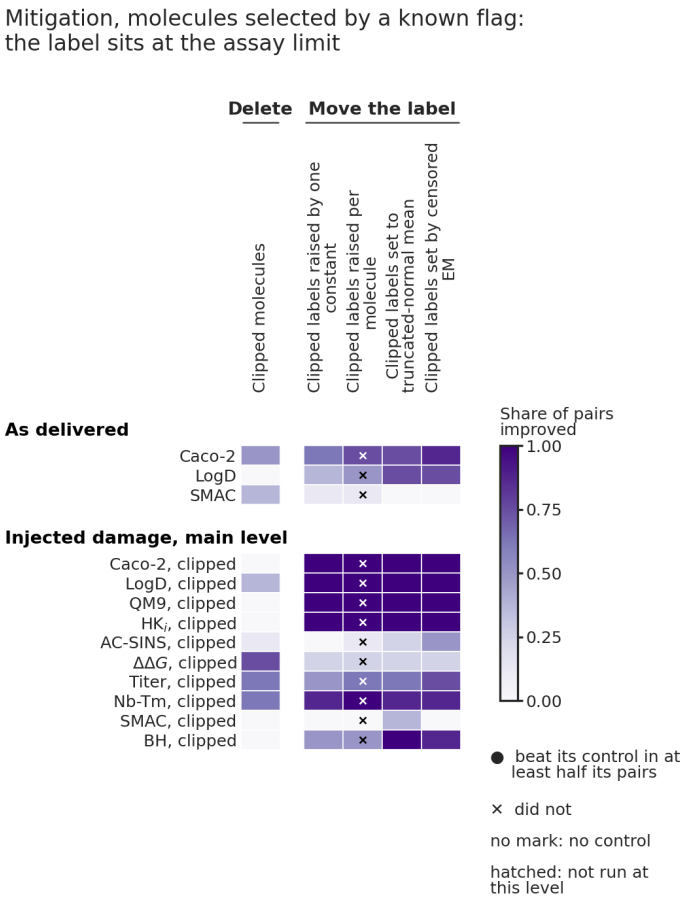


Figure 7.4: Noise mitigation with the molecules selected by a known flag, a label at the assay limit, first outer fold. Rows: labels as delivered and clipped labels at 25%. Columns: method, grouped by action. Shade: share of model-and-configuration pairs improved; darker is more. Cross: did not beat shuffled lifts in at least half its pairs. Dataset short names as in Table 5.8.

first hybrid worse with every model and detector. It improved single representations by at most 0.004 RMSE. It's clear that when deletion helps on a damaged dataset and beats random deletion, as under the hERG flips, the gain comes from removing noise.

The labels as delivered

On all five small-molecule datasets in Table 7.13, Neighbor-mean smoothing or Score down-weight beat shuffled detector scores on the labels as delivered, narrowly on Caco-2 and LogD. Here nothing marks which labels are wrong, so the control reruns each cleanup with the detector's scores shuffled across the training molecules. On hERG K_i, Neighbor-mean smoothing passed with both detectors, the Neighbor-label gap and the Forest residual. With the Neighbor-label gap, it took the DNN's first single representation from 0.638 to 0.578 RMSE, while every shuffle made it slightly worse. It

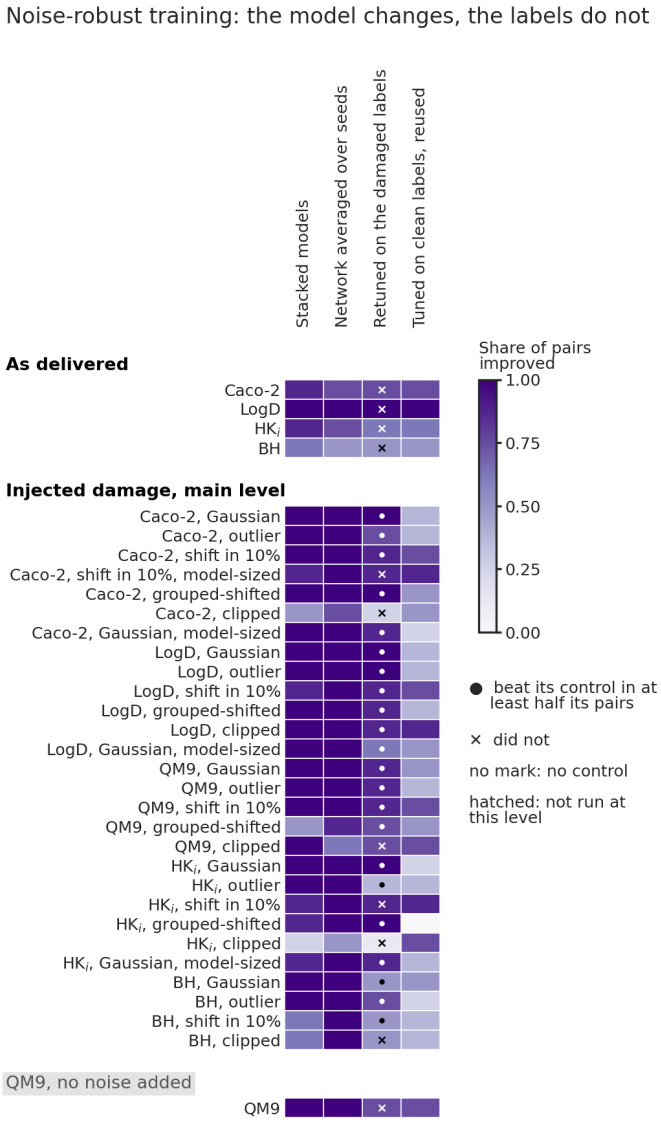


Figure 7.5: Noise-robust training, first outer fold. Rows: labels as delivered, injected damage at the main level, and QM9 with no noise added. Columns: change to the model. Shade: share of model-and-configuration pairs improved over the same model at default settings; darker is more. Dot: retuning beat tuning on clean labels in at least half its pairs; cross: did not. Dataset short names as in Table 5.8.

also improved the DNN's hybrid on each of three folds, as Table 7.14 shows. On hERG blocking, Neighbor-mean smoothing beat all its shuffles with gradient-boosted trees. On MDR1–MDCK efflux, Neighbor-mean smoothing and Score down-weight beat all shuffles with the DNN, but the SVM gained nothing from any cleanup. On Caco-2 and LogD, one shuffle recovers more than half of the gain from Score down-weight.

Model	Configuration	No deletion	Deleted
Forest residual, best share deleted			
DNN	First single	0.396	0.430
DNN	Second single	0.495	0.514
DNN	First hybrid	**0.355**	**0.393**
DNN	Second hybrid	0.362	0.388
LightGBM	First single	0.412	0.449
LightGBM	Second single	0.459	0.530
LightGBM	First hybrid	0.372	0.411
LightGBM	Second hybrid	0.376	0.412
Separate run: Five-detector delete or move, top 10%, first hybrid			
DNN	Cliff-aware gap	0.349	0.392
DNN	GP spread and residual	0.349	0.409
DNN	Variance-head network	0.349	0.423
DNN	NGBoost spread	0.349	0.396
DNN	Heteroscedastic GP spread	0.349	0.371
LightGBM	Cliff-aware gap	0.372	0.413
LightGBM	GP spread and residual	0.372	0.429
LightGBM	Variance-head network	0.372	0.452
LightGBM	NGBoost spread	0.372	0.466
LightGBM	Heteroscedastic GP spread	0.372	0.392

Table 7.12: Deleting flagged molecules on QM9 with no noise added. One row is one model and configuration, or one model and detector. First block: the best of 5, 10, and 20% deleted. Held-out RMSE in eV, lower is better. Outer fold 0. Bold: the values discussed in the text.

Table 7.13: Cleaning the labels as delivered against the same cleanup driven by shuffled detector scores. One row is one model, configuration, cleanup, and detector. QM9 has no noise to find. Outer fold 0.

Configuration, cleanup, and detector	No cleaning	Cleaned	Shuffled, seed 0	Shuffled, seed 1	Shuffled, seed 2
hERG K_i, DNN, RMSE (lower is better)					
First single, Neighbor-mean smoothing, Neighbor-label gap	0.638	0.578	0.642	0.642	0.645
First hybrid, Neighbor-mean smoothing, Forest residual	0.599	0.578	0.600	0.588	0.596
hERG blocking, XGBoost, AUROC (higher is better)					
First single, Neighbor-mean smoothing, Neighbor-label gap	0.837	0.845	0.835	0.831	0.833
Second hybrid, Neighbor-mean smoothing, Neighbor-label gap	0.854	0.860	0.848	0.848	0.855

Configuration, cleanup, and detector	No cleaning	Cleaned	Shuffled, seed 0	Shuffled, seed 1	Shuffled, seed 2
Caco-2, DNN, RMSE (lower is better)					
Second single, Score down-weight, Forest residual	0.266	0.255	0.268	0.262	0.258
LogD, DNN, RMSE (lower is better)					
Second single, Score down-weight, Neighbor-label gap	0.4212	0.414	0.436	0.417	0.4209
MDR1–MDCK efflux, DNN, RMSE (lower is better)					
First single, Neighbor-mean smoothing, Neighbor-label gap	0.4828	0.467	0.4831	0.484	0.481
First hybrid, Score down-weight, Neighbor-label gap	0.466	0.445	0.4530	0.4534	0.463
SMAC colloidal stability, Spearman (higher is better)					
SVM, first single, Delete flagged 20%, Forest residual	0.274	0.403	0.221	0.234	0.278
SVM, second hybrid, Delete flagged 20%, Forest residual	0.311	0.378	0.328	0.278	0.300
DNN, first single, Move toward prediction, Neighbor-label gap	0.065	0.319	0.133	0.073	0.064
DNN, first hybrid, Move toward prediction, Forest residual	0.227	0.442	0.220	0.217	0.223
$\Delta\Delta G$, DNN, Spearman (higher is better)					
First single, Neighbor-mean smoothing, Forest residual	0.341	0.484	0.277	0.352	0.460
First hybrid, Neighbor-mean smoothing, Forest residual	0.259	0.287	0.276	0.274	0.270
Expression titer, RF, Spearman (higher is better)					
Second single, Delete flagged, Forest residual	0.345	0.407	0.357	0.401	0.349
Second hybrid, Delete flagged, Forest residual	0.224	0.317	0.190	0.271	0.294
Nanobody thermostability, Spearman (higher is better)					
DNN, first single, Move toward prediction, Neighbor-label gap	0.770	0.775	0.773	0.769	0.762
DNN, second single, Delete flagged 20%, Forest residual	0.548	0.691	0.561	0.560	0.530
SVM, second single, Delete flagged 20%, Forest residual	0.581	0.737	0.606	0.671	0.524
DNN, first hybrid, Delete flagged 20%, Forest residual	0.724	0.761	0.635	0.780	0.737
AC-SINS self-association, Spearman (higher is better)					
LightGBM, first single, Delete flagged 5%, Neighbor-label gap	0.437	0.539	0.419	0.422	0.522
GP, first single, Delete flagged 20%, Forest residual	0.522	0.540	0.608	0.505	0.607

Configuration, cleanup, and detector	No cleaning	Cleaned	Shuffled, seed 0	1	2
GP, second single, Delete flagged 20%, Forest residual	0.429	0.598	0.398	0.543	0.459
LightGBM, second hybrid, Delete flagged 1%, Neighbor-label gap	0.509	0.590	0.566	0.572	0.531
QM9, no noise added, RMSE in eV (lower is better)					
DNN, first single, Neighbor-mean smoothing, Neighbor-label gap	0.3999	0.3927	0.3996	0.3967	0.4005
DNN, first hybrid, Neighbor-mean smoothing, Forest residual	0.3491	0.3487	0.3568	0.3502	0.3456
DNN, first single, Delete flagged 20%, Forest residual	0.400	0.461	0.433	0.429	0.412
LightGBM, first single, Neighbor-mean smoothing, Neighbor-label gap	0.412	0.414	0.417	0.418	0.417

Fold	First hybrid		Second single	
	No cleaning	Neighbor-mean smoothing	No cleaning	Neighbor-mean smoothing
0	0.599	**0.567**	0.589	0.596
1	0.550	**0.547**	0.610	0.600
2	0.550	**0.537**	0.645	0.629

Table 7.14: Neighbor-mean smoothing on hERG K_i's labels as delivered, by outer fold. DNN. One row is one outer fold. Held-out RMSE, lower is better. First hybrid: Forest residual, flag share 5%. Second single: the best setting of the Neighbor-label gap or the Forest residual on each fold.

On the small-molecule labels as delivered, deletion does not pass its control. On Caco-2, hERG K_i, and LogD, no deletion setting beat both no cleaning and every random deletion. On MDR1–MDCK efflux, deleting the 5% with the largest Neighbor-label gap did, but only by 0.004 RMSE over the best random deletion. That margin is inside the DNN's spread between fits. Buchwald–Hartwig is different. There, deleting the 20% of reactions with the largest Neighbor-label gap improved LightGBM and beat random deletion on the hybrid and the second single representation, as Table 7.8 shows. Random deletion alone also lowered the RMSE a little.

Putting AC molecules back after Delete flagged with the Neighbor-label gap gave mixed results, as Table 7.15 shows. The control puts back a random set of the same size. On Caco-2, hERG blocking, and LogD this control cannot be run, as more than

half of the training molecules count as cliff molecules. On hERG K_i, putting the cliff molecules back gave a lower RMSE than putting random ones back on the first and third folds, but not on the second. On QM9 with no noise added, only 19 of 7,982 training molecules count as cliff molecules. Putting them back recovered part of the loss from deletion, while random molecules recovered none of it.

Dataset and fold	No cleaning	Delete flagged	Cliff molecules put back	Random molecules put back
hERG K_i, fold 0	0.597	0.596	0.579	0.595
hERG K_i, fold 1	0.551	0.5681	0.5684	0.557
hERG K_i, fold 2	0.550	0.575	0.537	0.545
QM9, fold 0	0.349	0.382	0.372	0.382

Table 7.15: Putting AC molecules back after deleting the 1% the Neighbor-label gap flags, on hERG K_i's labels as delivered and on QM9 with no noise added. DNN on the first hybrid. One row is one dataset and outer fold. Held-out RMSE, lower is better; for QM9 in eV.

7.3.3 Detection of the suspect labels

The Forest residual finds most kinds of damage well above chance, but not clipped labels. We grade the detectors on injected damage alone, since only there do we know which labels are wrong, as explained in Section 7.2.2. Table 7.16 shows the Forest residual well above chance under outliers, shifted labels, and flipped classes on every dataset. On Caco-2 under outliers, for example, its average precision is 0.338 against a chance level of 0.051. With 25% of labels clipped, it is near or below chance on the small molecules and reactions, and above chance on the antibody datasets. A model fitted to clipped labels predicts values near the limit for the clipped molecules, so their residuals are small. Chapter 3 found the same in the models' own uncertainty, which was lowest on the clipped labels.

The Cliff-aware gap beats the Forest residual when only some labels are shifted, and no detector is best everywhere. Under the shift in 10% on Caco-2, its median neighbor setting reaches an average precision of 0.269, against 0.202 for the Forest residual. Under outliers its best setting is ahead of the Forest residual and its median setting behind. Under censoring it trails the Forest residual on Caco-2, hERG K_i, and LogD at every setting, as Table 7.16 shows. A neighbor detector is better at a shift that moves a few labels a long way, and a model residual is as good or better at outliers.

Noisy labels were not easier to find with neighbors in the hybrid's features than in

Dataset	Damage	Chance	Forest residual	Cliff-aware, best setting	Cliff-aware, median setting
QM9	Clipped 25%	0.250	0.157	0.227	0.161
	Outliers	0.051	0.391	0.388	0.358
	Shift in 10%	0.101	0.618	0.724	0.599
Caco-2	Clipped 25%	0.250	0.306	0.292	0.253
	Outliers	0.051	**0.338**	0.347	0.294
	Shift in 10%	0.099	0.202	0.296	**0.269**
	Shift in 10%, model error	0.100	0.137	0.180	0.170
HK_i	Clipped 25%	0.244	0.264	0.248	0.212
	Outliers	0.056	0.370	0.379	0.356
	Shift in 10%	0.098	0.226	0.373	0.340
	Shift in 10%, model error	0.096	0.149	0.230	0.209
LogD	Clipped 25%	0.237	0.196	0.195	0.169
	Outliers	0.051	0.398	0.402	0.357
	Shift in 10%	0.099	0.339	0.473	0.439
HB	Uniform flips	0.093	0.397	0.493	0.406
	One-way flips	0.095	0.349	0.430	0.360
	Class-dependent flips	0.100	0.400	0.478	0.403
	Class-frequency flips	0.094	0.276	0.397	0.338
	Instance flips	0.106	0.403	0.494	0.411
	Confusion-matrix flips	0.100	0.412	0.498	0.415
BH	Clipped 25%	0.250	**0.193**	0.335	0.249
	Outliers	0.051	0.352	0.386	0.342
	Shift in 10%	0.099	0.285	0.765	0.497
SMAC	Clipped 25%	0.243	0.506	0.573	0.407
	Outliers	0.055	0.306	0.401	0.340
	Shift in 10%	0.095	0.233	0.470	0.373
$\Delta\Delta G$	Clipped 25%	0.251	0.289	0.414	0.328
	Outliers	0.055	0.397	0.508	0.390
	Shift in 10%	0.098	0.148	0.295	0.249
Titer	Clipped 25%	0.253	0.362	0.356	0.311
	Outliers	0.056	0.378	0.429	0.378
	Shift in 10%	0.104	0.163	0.193	0.162
PR-CHO	Clipped 25%	0.250	0.346	0.428	0.321
	Outliers	0.048	0.380	0.420	0.358
	Shift in 10%	0.101	0.201	0.258	0.193

Table 7.16: Detection by the Forest residual and the Cliff-aware gap. One row is one dataset and damage kind at its main level (25% clipped, 10% flipped, otherwise 1.0). Average precision of the damaged molecules, higher is better; chance is the share damaged. Outer fold 0. Bold: the values discussed in the text. Dataset short names as in Table 5.8.

other spaces, at least with each detector at its best setting. We group the spaces into four families, namely plain features, importance-weighted features, forest proximity, and the hybrid's own features. Table 7.17 gives the best setting in each family, and the space changes detection little. On Caco-2 under outliers, for example, the four families reach 0.335 to 0.352 average precision. The importance-weighted features were best on most datasets. The hybrid's own features were best on no dataset overall, and won only single damage kinds, such as outliers on the change in binding free energy ($\Delta\Delta G$) and a few others shown in the table. Averaged over the detector settings instead, the hybrid's features come out ahead more often, so this ranking depends on how a detector's setting is chosen.

Scaling each feature by its importance before finding the neighbors raised detection under the shift in 10% on most datasets, and lowered it under censoring on every dataset. Table 7.18 lists the scores. The largest gain was on QM9, from 0.538 to 0.634 average precision. On hERG K_i and CHO polyreactivity it was unchanged or slightly lower, and under outliers scaling barely changed detection. Importance is computed against the damaged labels. Under censoring it likely rewards the features that predict the clipped values, so the clipped molecules look like their neighbors.

Only one model-uncertainty detector could be graded, namely the GP spread and residual, a GP's own predicted spread plus its residual, since the other three predict a spread from the features alone. It was slightly ahead of the Cliff-aware gap under outliers on Caco-2 and LogD, and behind it under every shift and hERG flip kind. Detectors from research fields outside cheminformatics did better where the Forest residual and the LightGBM residual are weakest, and Additional file 36 gives these results. Under censoring on Buchwald–Hartwig, the Nearest-neighbor Shapley value reached an average precision of 0.391, against 0.195 for the LightGBM residual, close to the 0.193 in Table 7.16. The Nearest-neighbor Shapley value was also ahead on SMAC colloidal stability, and the Cleanlab score was well ahead on Buchwald–Hartwig under the shift in 10%. These numbers are means over each detector's settings, and the Nearest-neighbor Shapley value has far more settings than the LightGBM residual.

The LightGBM residual can rank damaged molecules, but it cannot reliably say which ones to flag. It ranked the shifted molecules above chance on every dataset; on expression titer, for example, its average precision was 0.20 to 0.25, against 0.106 by chance. Turning that ranking into flags with conformal p-values, as described in Section 7.2.2, almost never flagged a molecule on the measured datasets. It flagged molecules regularly only on QM9 with outliers or a shift injected, and acting on those flags barely changed the held-out score.

Dataset	Damage	Chance	Plain	Importance-weighted	Forest proximity	Hybrid's features
QM9	Clipped 25%	0.250	0.191	0.192	0.226	0.195
	Outliers	0.051	0.403	0.429	0.416	0.434
	Shift in 10%	0.100	0.508	0.729	0.700	0.717
Caco-2	Clipped 25%	0.250	0.329	0.335	0.208	0.324
	Outliers	0.051	0.337	**0.352**	0.336	0.335
	Shift in 10%	0.099	0.346	0.374	0.316	0.367
HK_i	Clipped 25%	0.244	0.284	0.285	0.218	0.252
	Outliers	0.056	0.386	0.393	0.368	0.389
	Shift in 10%	0.098	0.411	0.434	0.370	0.425
LogD	Clipped 25%	0.237	0.223	0.234	0.194	0.211
	Outliers	0.051	0.397	0.407	0.400	0.395
	Shift in 10%	0.099	0.455	0.503	0.464	0.495
MDR1	Clipped 25%	0.250	0.686	0.731	0.211	0.551
	Outliers	0.050	0.356	0.364	0.343	0.357
	Shift in 10%	0.101	0.326	0.343	0.227	0.378
HB	Uniform flips	0.093	0.451	0.469	0.492	0.473
	One-way flips	0.095	0.403	0.414	0.437	0.402
	Class-dependent flips	0.100	0.451	0.474	0.466	0.470
	Class-frequency flips	0.094	0.359	0.400	0.393	0.397
	Instance flips	0.106	0.455	0.483	0.488	0.477
	Confusion-matrix flips	0.100	0.467	0.484	0.494	0.482
BH	Clipped 25%	0.250	0.355	0.374	0.225	0.324
	Outliers	0.051	0.389	0.411	0.404	0.391
	Shift in 10%	0.099	0.677	0.783	0.778	0.672
$\Delta\Delta G$	Clipped 25%	0.251	0.444	0.408	0.347	0.391
	Outliers	0.055	0.485	0.488	0.471	**0.507**
	Shift in 10%	0.098	0.357	0.329	0.327	0.328
SMAC	Clipped 25%	0.243	0.576	0.578	0.212	**0.613**
	Outliers	0.055	0.365	0.441	0.253	0.381
	Shift in 10%	0.095	0.494	0.633	0.394	0.525

Table 7.17: Detection by the neighbor detectors in four families of distance. One row is one dataset and damage kind at its main level. Each cell is the best average precision over the detector settings and spaces in that family; higher is better. Chance is the share damaged. Outer fold 0. Dataset short names as in Table 5.8.

A detector can rank the damaged labels above chance and still make the model worse. The break-even precision is the share of truly damaged molecules a flagged set needs before acting on it helps, as explained in Section 7.2.3. Table 7.19 compares it with the precision of the LightGBM residual's flags. With wide errors in 5% of molecules, the LightGBM residual's precision is above the break-even on Caco-2 and hERG K_i. Under the shift sized by model error, it falls below the break-even for deletion. On Caco-2, for example, the top 5% has a precision of 0.171, while deleting

	Clipped 25%		Outliers		Shift in 10%	
Dataset	Unscaled	Scaled	Unscaled	Scaled	Unscaled	Scaled
QM9	0.1682	0.1678	0.344	0.349	**0.538**	**0.634**
BH	0.253	0.237	0.305	0.312	**0.465**	**0.512**
Caco-2	0.256	0.253	0.258	0.259	0.268	0.271
HK_i	0.212	0.207	0.304	0.305	0.3259	0.3257
LogD	0.179	0.174	0.311	0.313	0.412	0.442
MDR1	0.373	0.356	0.284	0.286	0.3046	0.3052
$\Delta\Delta G$	0.331	0.321	0.359	0.357	0.243	0.250
Nb-Tm	0.188	0.185	0.272	0.271	0.261	0.269
AC-SINS	0.450	0.415	0.375	0.373	0.236	0.241
SMAC	0.415	0.393	0.293	0.297	0.331	0.368
Titer	0.315	0.312	0.342	0.345	0.161	0.162
PR-CHO	0.333	0.330	0.333	0.334	0.198	0.197

Table 7.18: Neighbor detection with and without scaling each feature by its importance. One row is one dataset; the damage is at its main level. Average precision of the damaged molecules, mean over matched settings; higher is better. Outer fold 0. Bold: the values discussed in the text. Dataset short names as in Table 5.8.

them pays off only above 0.616.

Deleting the molecules that two learners flag made the model worse under the shift in 10% on hERG K_i and Caco-2. Figure 7.6 compares the precision of the deleted molecules with the break-even precision for the two learners, LightGBM and an SVM, that exchange molecules during training. Each flags the training molecules with the largest loss. We delete every molecule that either one flags, so slightly more than the flagged share is deleted. On hERG K_i and Caco-2, the precision stayed below the break-even at every share. LogD has no break-even precision. There, deleting 5% under outliers, with a precision of 0.36, lowered the RMSE from 0.587 to 0.557.

7.3.4 Effect by noise type

Labels clipped at an assay limit

Clipping labels at an assay limit is the damage we could most often undo, and undoing it needs no detector. Figure 7.7 gives the held-out RMSE with a quarter of the training labels clipped. There, Censored EM lowers it for every model and configuration, though not to the level before clipping. On LogD, the DNN on the best single representation wins back about three quarters of the lost RMSE, as defined in Equation 7.6. QM9 shows the same for the DNN and LightGBM, and Caco-2 a smaller gain.

Table 7.20 shows that the gain holds on the other outer folds. On all five LogD

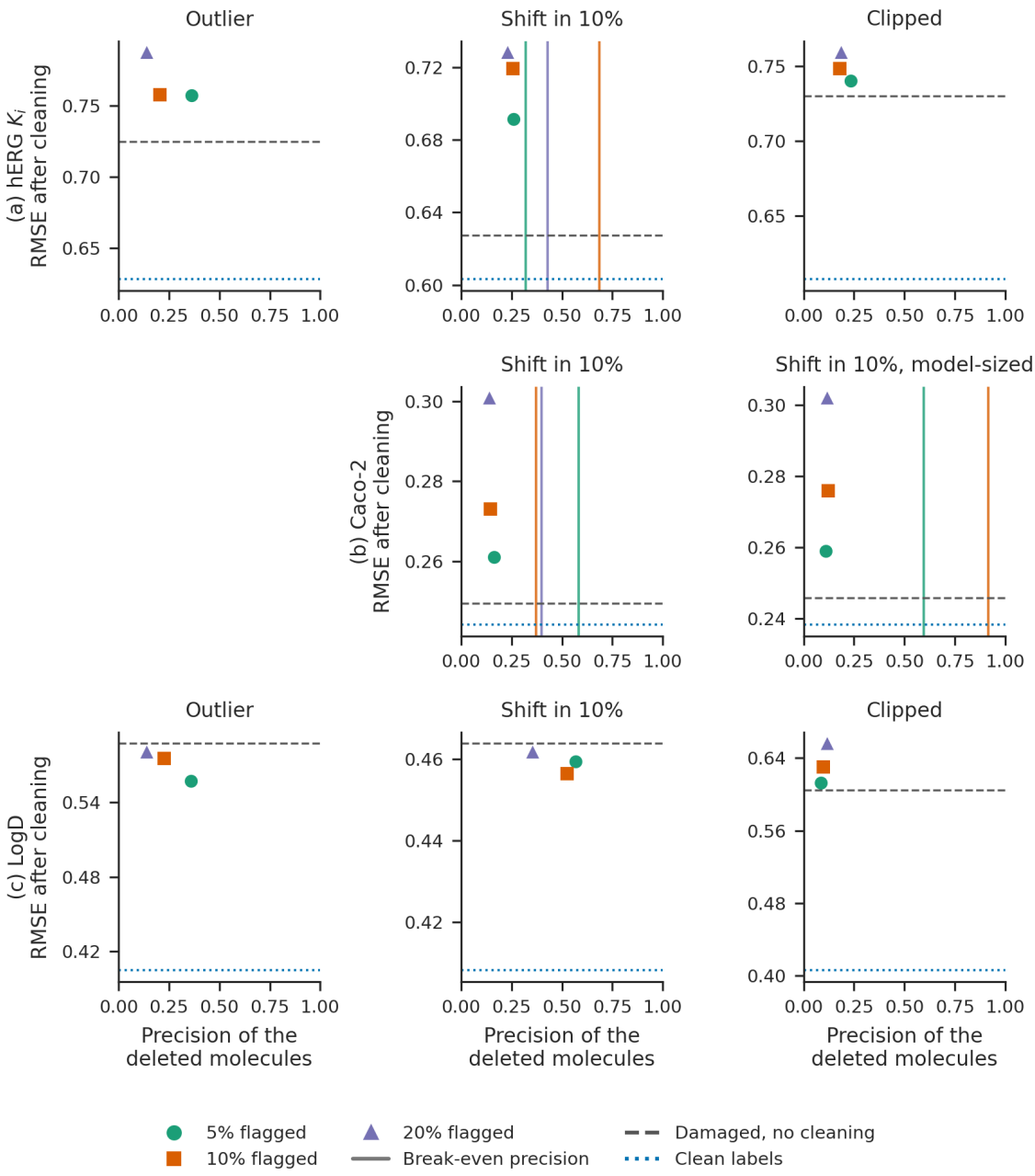


Figure 7.6: Held-out RMSE (lower is better) after deleting the molecules flagged by two learners, against the precision of the deleted molecules: the share whose label the damage moved. Rows: (a) hERG K_i, (b) Caco-2, (c) LogD. Columns: damage kind. One marker per flagged share. Vertical lines: break-even precision for the same share. Dashed: damaged labels, no cleaning. Dotted: clean labels. DNN on the first hybrid, outer fold 0. Dataset short names as in Table 5.8.

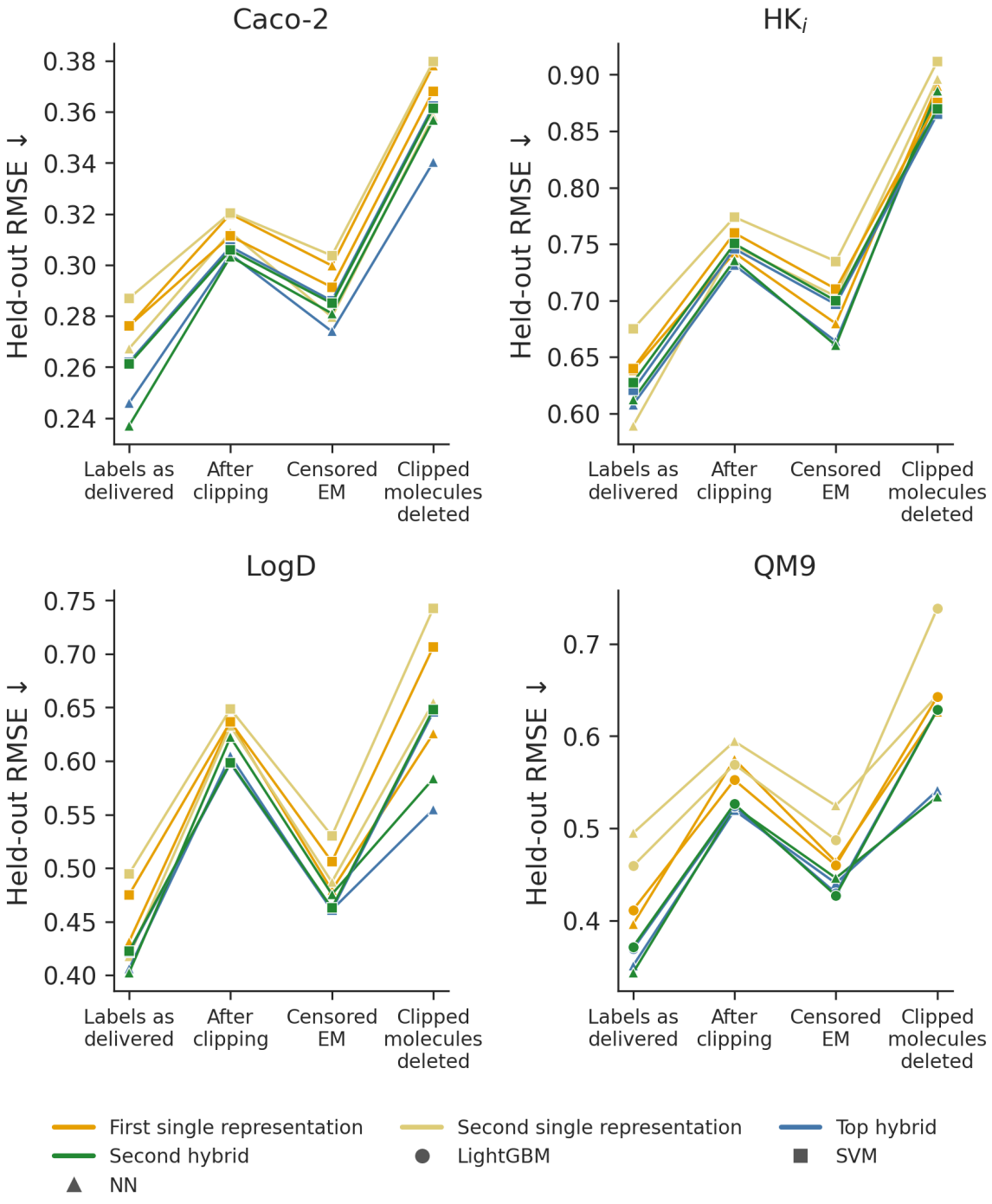


Figure 7.7: Held-out RMSE (lower is better) at four steps: the training labels before clipping, after a quarter of them are clipped at the assay limit, after Censored EM raises the clipped labels, and with the clipped molecules deleted instead. One line per model and configuration; color is the configuration and marker shape the model. Outer fold 0. Dataset short names as in Table 5.8.

Dataset	Damage	Share flagged	Precision	Break-even precision			
				Delete	Down-weight	Relabel	Shrink
Caco-2	Wide errors in 5%	5%	0.627	0.275	0.239	0.134	
	Wide errors in 5%	10%	0.370	0.222	0.214	0.119	
	Wide errors in 5%	20%	0.206	0.095	0.134	0.091	
	Shift in 10%	5%	0.271	0.239	0.201	0.331	0.406
	Shift in 10%	10%	0.218	0.168	0.155	0.299	0.529
	Shift in 10%	20%	0.200	0.292	0.266	0.244	
	Shift in 10%, model error	5%	**0.171**	**0.616**	0.563	0.575	0.074
	Shift in 10%, model error	10%	0.144	0.405	0.242	0.366	0.290
	Shift in 10%, model error	20%	0.128	0.420	0.422	0.502	0.111
HK_i	Wide errors in 5%	5%	0.620	0.417	0.270	0.156	
	Wide errors in 5%	10%	0.355	0.268	0.223	0.184	
	Wide errors in 5%	20%	0.192	0.121	0.071	0.090	
	Shift in 10%	5%	0.294	0.484	0.441	0.313	0.204
	Shift in 10%	10%	**0.262**	**0.684**	0.213	0.105	0.106
	Shift in 10%	20%	0.224	0.450	0.298	0.402	0.319
	Shift in 10%, model error	5%	0.195	0.456	0.026	0.116	
	Shift in 10%, model error	10%	0.160	0.914	0.120	0.422	0.109
	Shift in 10%, model error	20%	0.141	0.393		0.154	0.064

Table 7.19: Precision of the LightGBM residual's flags against the break-even precision of each action. Delete drops the flagged molecules; Down-weight gives them weight 0.25, as in Top-share down-weight; Relabel replaces their labels by an out-of-fold LightGBM prediction; Shrink moves them part of the way toward it. One row is one dataset, damage kind at level 1.0 and flagged share. Precision is the share of flagged molecules that were damaged, mean over configurations and models; break-even is the median over configurations and models. An action helps when the precision is above its break-even. Empty: no break-even defined. Outer fold 0. Bold: the values discussed in the text. Dataset short names as in Table 5.8.

folds and on two further Caco-2 folds, Censored EM improves the DNN's hybrid over the clipped labels, without reaching the score before clipping.

No one way of raising the clipped labels is best on every dataset. We compare three, namely the constant lift, the truncated-normal mean, and censored EM. Table 7.21 gives one row per dataset, and Additional file 37 gives every model and configuration. Censored EM is best in most configurations on LogD, QM9, and AC-SINS self-association. On Buchwald–Hartwig it brings the RF back to about its score before

Dataset	Fold	Before clipping	Clipped	Censored EM
LogD	0	0.404	0.613	0.480
	1	0.418	0.596	0.472
	2	0.411	0.572	0.470
	3	0.413	0.570	0.475
	4	0.398	0.515	0.437
Caco-2	1	0.261	0.322	0.291
	2	0.281	0.400	0.348

Table 7.20: Censored EM on further outer folds, with 25% of training labels clipped. DNN on the first hybrid. One row is one outer fold. Held-out RMSE, lower is better. The run is separate from that of Figure 7.7. Dataset short names as in Table 5.8.

clipping. The constant lift is best in most configurations on hERG K_i and MDR1–MDCK efflux, and wins back most of the loss for the DNN's hybrid on nanobody thermostability. The truncated-normal mean trails both on LogD, hERG K_i, and Caco-2.

Dataset	Before clipping	Clipped	Constant lift	Truncated-normal mean	Censored EM
DNN, RMSE in the label's units (lower is better)					
LogD	0.406	0.605	0.460	0.507	0.461
QM9	0.351	0.520	0.450	0.444	0.441
hERG K_i	0.608	0.731	0.668	0.693	0.664
Caco-2	0.246	0.304	0.275	0.288	0.274
MDR1–MDCK efflux	0.456	0.511	0.461	0.483	0.466
RF, RMSE in yield points (lower is better)					
Buchwald–Hartwig	17.6	18.7	18.0	18.1	**17.6**
DNN, Spearman (higher is better)					
Nanobody thermostability	0.749	0.610	**0.7164**	0.7156	0.699
SMAC colloidal stability	0.162	0.1469	0.141	0.1467	0.129
$\Delta\Delta G$	0.305	0.351	0.302	0.341	0.346
Expression titer	0.348	0.3687	0.396	0.367	0.3694
LightGBM, Spearman (higher is better)					
AC-SINS self-association	0.476	0.464	0.424	0.364	0.407

Table 7.21: Three ways to raise clipped training labels, with 25% of training labels clipped. One row is one dataset, on the first hybrid. Outer fold 0. Every model and configuration is in Additional file 37.

Deleting the clipped molecules instead (delete clipped) is worse than keeping their clipped labels on Caco-2, hERG K_i, and QM9, also shown in Figure 7.7. It is also worse on MDR1–MDCK efflux, with both models in every configuration. On LogD,

it beats the clipped labels with the DNN's hybrid but stays far behind raising them. Without the clipped molecules, the top of the label range is missing from training.

The size of each molecule's own lift does not matter. We raised each clipped label by its own estimated amount (own lift), and compared this with dealing the same lifts to the clipped molecules at random, three times over. In Table 7.22, every shuffle beats the own lift in every configuration on LogD and Caco-2. The same holds on hERG K_i for the second single representation, and on MDR1–MDCK efflux for both hybrids. What matters is knowing which labels are clipped and raising them by about the right average amount, which is what the constant lift does.

Dataset	Configuration	Own lift	Shuffled seed 0	Shuffled seed 1	Shuffled seed 2
LogD	First single	0.527	0.505	0.507	0.507
	Second single	0.510	0.482	0.477	0.483
	First hybrid	**0.499**	0.471	0.467	**0.463**
	Second hybrid	0.509	0.484	0.482	0.483
HK_i	First single	0.691	0.681	0.708	0.687
	Second single	0.702	0.678	0.694	0.670
	First hybrid	0.677	0.666	0.696	0.680
	Second hybrid	0.667	0.708	0.692	0.669
Caco-2	First single	0.300	0.296	0.293	0.295
	Second single	0.295	0.277	0.280	0.282
	First hybrid	0.288	0.275	0.273	0.271
	Second hybrid	0.282	0.269	0.274	0.270
QM9	First single	0.494	0.492	0.492	0.481
	Second single	0.533	0.518	0.508	0.527
	First hybrid	0.448	0.450	0.440	0.446
	Second hybrid	0.458	0.464	0.446	0.442
MDR1	First single	0.499	0.485	0.501	0.489
	Second single	0.513	0.515	0.516	0.503
	First hybrid	0.481	0.467	0.468	0.469
	Second hybrid	0.480	0.452	0.469	0.464

Table 7.22: Each clipped label raised by its own estimated lift, against the same lifts dealt to the clipped molecules at random. DNN, 25% of training labels clipped. One row is one dataset and configuration. Held-out RMSE, lower is better. Outer fold 0. Bold: the values discussed in the text. Dataset short names as in Table 5.8.

On three antibody datasets the clipping did no harm, so there was nothing to fix. Clipping a quarter of the training labels raised the held-out Spearman in most configurations, as listed in Additional file 37. With nothing lost, censored EM moved the score back down in most configurations on SMAC and $\Delta\Delta G$, and raised it a bit on expression titer.

We clipped a quarter of QM9's training labels again, this time on all five outer folds rather than the single fold of Table 7.21. As a control, we applied the same treatment to the same number of unclipped labels. Moving the clipped labels lowered the RMSE with both models in every feature space, while moving the unclipped labels raised it, which was the intended behavior. With the DNN on the first hybrid, the mean RMSE went from 0.5072 before treatment to 0.3980 after censored EM.

A shared offset per scaffold group

Let's assume you know you have some form of systematic, grouped-shifted noise. Then, you can estimate a per-scaffold offset (scaffold-family offset). Subtracting this offset helped on Caco-2, hERG K_i, LogD, and QM9. We estimate the offset either as the family's mean out-of-fold residual or with a random-intercept model. Table 7.23 shows that on Caco-2, LogD, and QM9, both improved the DNN on the hybrid on every noise draw, without reaching the score on clean labels. The random-intercept model was usually the better of the two. On MDR1–MDCK efflux, the random-intercept model improved the hybrid by less, while subtracting family means made one of the two draws worse.

Dataset	Draw	Damaged	Family mean	Random intercept	Clean labels
Caco-2	0	0.321	0.297	**0.280**	0.239
	1	0.344	0.298	0.294	0.242
	2	0.361	0.277	0.267	0.231
LogD	0	0.684	0.562	0.546	0.398
	1	0.690	0.606	0.573	0.402
	2	0.672	0.570	0.589	0.395
HK_i	0	0.804	0.691	0.693	0.610
	1	0.834	0.708	0.723	0.611
	2	0.743	0.719	0.666	0.591
QM9	0	0.909	0.574	0.507	0.346
	1	0.844	0.683	0.571	0.347
	2	1.295	0.806	0.638	0.341
MDR1	0	0.536	0.515	0.513	0.459
	1	0.511	0.516	0.499	0.460

Table 7.23: Removing a shared offset per scaffold family under grouped-shifted noise at level 1.0. Both treated columns are scaffold-family offset, with the offset estimated as the family's mean out-of-fold residual (family mean) or by a random-intercept model. DNN on the first hybrid. One row is one dataset and noise draw. Held-out RMSE, lower is better. Outer fold 0. Dataset short names as in Table 5.8.

Applied to the labels as delivered, neither method found a shared offset per scaffold

family, as Table 7.24 shows. Both made every dataset worse, except that the random-intercept model lowered the RMSE on MDR1–MDCK efflux by less than the DNN's spread between fits. Caco-2, LogD, and MDR1–MDCK efflux each come from one company's measurements, where such offsets are less likely. The test is also weak on these datasets, as their scaffold families hold about two molecules or fewer on average. A family's mean is then mostly its own molecules' labels. On QM9 with no noise added, subtracting family means made every configuration of both models worse, because it removed real differences between families.

Dataset	As delivered	Family mean	Random intercept
Caco-2	**0.233**	**0.267**	0.244
LogD	0.398	0.487	0.437
HK_i	0.599	0.625	0.603
MDR1	0.466	0.486	0.453
QM9	0.355	0.400	0.391

Table 7.24: Removing a shared offset per scaffold family from the labels as delivered; QM9 with no noise added. Both treated columns are scaffold-family offset, estimated as in Table 7.23. DNN on the first hybrid. One row is one dataset. Held-out RMSE, lower is better. Outer fold 0. Bold: the values discussed in the text. Dataset short names as in Table 5.8.

Several sources per molecule

When each molecule has several simulated sources, each with its own offset, removing the offsets (source offset) made little difference on small molecules. On hERG K_i, the DNN on the hybrid scored 0.742 RMSE with the plain mean of the measurements and 0.748 after removing the offsets. Removing the offsets helped most often on Buchwald–Hartwig, where the offsets were large compared to the spread of the labels. Even there, the RF on the hybrid stayed far from its score on clean labels. On the other datasets, QM9 among them, it helped in about half the settings or fewer.

Gaussian noise, outliers, and shifts in a few labels

Table 7.25 covers Huber blend, a method without any detector, which moves every label halfway toward a Huber-loss model's prediction. This helped the DNN on hERG K_i under all three kinds of damage. Under Gaussian noise at level 1.0, it took the second hybrid from 0.718 to 0.681 RMSE, and the gains held on every noise draw.

On QM9 with no noise added, Huber blend made every configuration worse with both models (the table gives two of the DNN's), so its gains on hERG K_i are noise removed.

Damage and configuration	Draw	Damaged labels	Huber blend	Clean labels
Gaussian noise, second hybrid	0	0.693	0.660	0.612
	1	0.755	0.678	0.588
	2	0.708	0.705	0.622
Scaffold-family shift, second hybrid	0	0.776	0.700	0.613
	1	0.821	0.733	0.605
	2	0.739	0.684	0.595
Outliers, first single	0	0.753	0.727	0.638
	1	0.723	0.660	0.638
	2	0.748	0.707	0.638
QM9, no noise added, second hybrid	0	0.362	0.368	
QM9, no noise added, first single	0	0.396	0.405	

Table 7.25: Moving every label halfway toward a Huber-fitted prediction on hERG K_i, damage at level 1.0, and on QM9 with no noise added. DNN, outer fold 0. One row is one damage kind, configuration, and noise draw. Held-out RMSE, lower is better; for QM9 in eV, where the first column is the untreated score.

7.3.5 Effect by dataset

We now turn to the labels as delivered, dataset by dataset. On five antibody datasets, cleaning these labels passes the shuffled-score control in some configurations, and where it passes it gives the largest gains in this chapter.

The antibody datasets

SMAC colloidal stability gives the strongest controlled result on any labels as delivered. Table 7.13 lists four configurations where delete flagged helps the SVM and moving flagged labels toward an out-of-fold prediction (move toward prediction) helps the DNN. In all four, every shuffle of the forest residual or neighbor-label gap scores does worse. Deleting the 20% of antibodies flagged by the forest residual, for example, raises the SVM's first single representation from 0.274 to 0.403 Spearman. The held-out fold has only 53 antibodies, but the shuffles were fitted on the same fold and stayed low.

$\Delta\Delta G$ and expression titer also pass the shuffled control on their labels as delivered, in all four of their configurations in Table 7.13. On both datasets, the cleaned single representation still beats the cleaned hybrid. On nanobody thermostability and AC-SINS self-association, delete flagged passed the control mainly where the starting score

was low. Where the starting score was highest, as for the GP on AC-SINS, the gain was small or a shuffle did as well.

Table 7.26 shows that weighting each antibody by its agreement with its neighbors (neighbor-agreement weights) helped the DNN on SMAC by about as much as deleting the flagged antibodies. This method has no control, though.

Configuration	Unweighted	Weighted
First single	**0.065**	**0.209**
First hybrid	0.227	0.370

Table 7.26: Weighting by agreement with the neighbors on SMAC colloidal stability's labels as delivered. DNN, outer fold 0. One row is one configuration. Held-out Spearman, higher is better. Bold: the values discussed in the text.

The held-out folds of the antibody datasets contain only 44 to 135 antibodies, and several start off from weak scores, so large gains come easily and count for less. We still count SMAC, $\Delta\Delta G$, and expression titer as real results because they pass the shuffled controls, alongside nanobody thermostability and AC-SINS on their weaker configurations.

Small molecules and the reaction set

On the small-molecule labels as delivered, two neighbor-mean smoothing gains pass their control and are not explained by the DNN's spread between fits. One is for the DNN on hERG K_i and the other for XGBoost on hERG blocking, as described in Section 7.3.2, "The labels as delivered". Buchwald–Hartwig is the one dataset where deleting flagged reactions from the labels as delivered beats random deletion.

Repeat measurements on hERG IC_{50}

hERG IC_{50} is the one dataset whose noise was measured rather than injected, as explained in Section 7.2. As such, we do not add any noise, and in each draw a held-out molecule's label is a fresh, random choice of its measurements. Which use of the repeat measurements helps depends on the model, as Table 7.27 shows. Weighting each molecule by its number of measurements (count and spread weights) improves the SVM on the hybrid on all three draws, but makes the DNN worse. The DNN gained instead from the log of the measurement count as a feature (log-count feature), and from shrinking each label toward a prediction by its spread (shrink by spread). Training on one row per measurement instead of the mean barely changed either model.

ChEMBL curators flagged 301 of the 12,008 records, most of them as outside the typical range. We graded the detectors against these flags on the first outer fold. The neighbor-label gap and the forest residual ranked the flagged records above chance in every configuration. Their average precisions were 0.14 to 0.23, against a chance level of 0.036.

Model and method	Draw 0	Draw 1	Draw 2
SVM			
No weighting	0.760	0.766	0.761
Weight by measurement count	**0.728**	**0.722**	**0.707**
Weight by inverse error variance	0.735	0.738	0.738
DNN			
No weighting	0.712	0.701	0.699
Weight by measurement count	0.765	0.739	0.716
Log-count feature	0.656	0.665	0.632
Shrink by spread	0.673	0.668	0.659
NGBoost			
No extra feature	0.786	0.823	0.811
Log-count feature	0.763	0.805	0.797
XGBoost, accelerated failure time			
Point model	0.724	0.728	0.725
Interval model	0.736	0.747	0.730
Separate run, one fit: mean label against one row per measurement			
SVM, mean label		0.767	
SVM, one row per measurement		0.765	
DNN, mean label		0.726	
DNN, one row per measurement		0.734	

Table 7.27: Using the repeat measurements on hERG IC_{50}. First hybrid. One row is one model and method. Held-out RMSE on molecules with three or more measurements, lower is better. Outer fold 0. Weighting by measurement count and by inverse error variance are the two versions of count and spread weights.

7.3.6 Cleaning inside the feature search

We can clean the labels used by the hybrid search, the labels used to fit the final model, both, or neither, as outlined in Section 7.2.3. Cleaning both does best in about half the rows of Table 7.28. On expression titer's labels as delivered, the forest residual flags 20% of the antibodies. Deleting them from both the search and the fit takes the DNN on the hybrid from 0.155 to 0.562 Spearman, and cleaning only one of the two helps by less. On SMAC colloidal stability with a quarter of the labels clipped, deleting the

flagged 5% also does best in both. There, cleaning the search alone does worse than no cleaning.

The largest gain in Table 7.28 comes from cleaning the search alone, but cleaning before the search can also hurt. Shrinking SMAC colloidal stability's labels as delivered toward an out-of-fold prediction (score-scaled shrink), for the search only, lifts the DNN's second hybrid from −0.014 to 0.560 Spearman. That gain starts near zero, on a held-out fold of 53 antibodies. Deleting flagged antibodies before the search made the DNN's first hybrid on SMAC worse. On hERG K_i, cleaning before the search helped on some noise draws and folds and hurt on others. This experiment has no random-flag control.

Dataset, labels, configuration and cleanup	No cleaning	Fit only	Search and fit	Search only
Spearman (higher is better)				
Titer, as delivered, first hybrid, delete flagged 20%	0.155	0.390	**0.562**	0.423
SMAC, 25% clipped, first hybrid, delete flagged 5%	0.237	0.416	**0.489**	0.221
SMAC, as delivered, second hybrid, score-scaled shrink	−0.014	−0.021	0.484	**0.560**
SMAC, as delivered, first hybrid, delete flagged 5%	0.227	0.241	0.085	0.107
hERG K_i, RMSE (lower is better)				
Gaussian 1.0, second hybrid, delete flagged 5%, draw 0	0.718	0.749	0.649	0.676
Gaussian 1.0, second hybrid, delete flagged 5%, draw 1	0.758	0.760	0.776	0.770
Gaussian 1.0, second hybrid, delete flagged 5%, draw 2	0.723	0.715	0.713	0.714
As delivered, first hybrid, neighbor-mean smoothing, fold 0	0.599	0.593	0.607	0.613
As delivered, first hybrid, neighbor-mean smoothing, fold 1	0.551	0.546	0.529	0.540
As delivered, first hybrid, neighbor-mean smoothing, fold 2	0.550	0.538	0.5444	0.5438

Table 7.28: Cleaning the fit, the hybrid search, or both. DNN; the flags are computed in a single representation. One row is one dataset, labels, configuration, and cleanup. Outer fold 0 unless a fold is named. Dataset short names as in Table 5.8.

For most datasets, Gaussian noise does little damage to the search itself. We searched the hybrid on clean labels or on labels injected with Gaussian noise, fitted both, and scored them on held-out labels as delivered. Table 7.29 shows the two

searches scoring almost the same on most datasets and folds. The largest loss was on hERG K_i with the DNN, 0.669 against 0.779 RMSE on one fold. On SMAC colloidal stability and $\Delta\Delta G$, the two searches swap places from fold to fold.

7.3.7 Changing the model instead of the labels

On hERG K_i under Gaussian noise, changing the model helped as much as the best cleaning of the labels, with no detector and no label changed. Averaging ten DNN seeds, for example, takes the DNN's second hybrid from 0.718 to 0.656 RMSE at level 1.0, as Table 7.30 shows. It wins on each of the three noise draws. Stacking and Retuning help that configuration by less, and none of the three reaches the clean-label score. On QM9, Tables 7.30 and 7.31 show that Seed averaging helps the DNN's second hybrid more under Gaussian noise than with none.

Table 7.31 shows that on the labels as delivered, Seed averaging, Stacking, and Retuning also help, but not everywhere. They help QM9 with no noise added by about as much, so their gains on the labels as delivered are better training rather than noise found, as explained in Section 7.3.1. On hERG K_i, the DNN's best single representation does not improve with any of the three, and Retuning makes both hybrids worse. Stacking combines both models, so its gains over the SVM compare two models with the weaker one alone.

Table 7.30 shows that under Gaussian noise on hERG K_i, Retuning on the damaged labels mostly beats reusing settings tuned on clean labels. Only on the DNN's first hybrid are the clean-label settings slightly ahead. With a quarter of the labels clipped, the clean-label settings did better in every configuration, and Retuning on the clipped labels rarely beat the plain model.

The other noise-robust training methods are in Additional file 38, and each helped only in some settings. A Huber loss for the DNN helped under outliers on hERG K_i and made Gaussian noise worse at every threshold. Each model's own robust loss helped in some configurations under the shift sized by model error, though by little. On LogD's first hybrid, for example, it improved the DNN by less than 0.01 RMSE, about half the gap to clean labels. Two-learner selection beat random deletion with tree models. On Buchwald–Hartwig under the shift in 10%, it took the RF's first single representation from 20.3 to 18.6 RMSE in yield points. It beat random deletion at all three seeds. The tree models and the DNN ran on different datasets, so we cannot say whether the model or the dataset is the reason.

Model	Searched on	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4
Caco-2, RMSE (lower is better)						
DNN	clean labels	0.298	0.328	0.328	0.368	0.346
DNN	damaged labels	0.303	0.329	0.334	0.365	0.340
SVM	clean labels	0.303	0.309	0.327	0.342	0.336
SVM	damaged labels	0.298	0.310	0.328	0.344	0.332
LogD, RMSE (lower is better)						
DNN	clean labels	0.582	0.613	0.571		
DNN	damaged labels	0.575	0.602	0.594		
SVM	clean labels	0.5726	0.579	0.5938		
SVM	damaged labels	0.5732	0.580	0.5942		
hERG K_i, RMSE (lower is better)						
DNN	clean labels	0.718	**0.669**	0.699		
DNN	damaged labels	0.733	**0.779**	0.708		
SVM	clean labels	0.675	0.620	0.651		
SVM	damaged labels	0.676	0.617	0.646		
SMAC colloidal stability, Spearman (higher is better)						
DNN	clean labels	−0.045	0.255	0.057	0.110	0.041
DNN	damaged labels	0.043	0.125	0.107	0.135	0.038
SVM	clean labels	0.078	0.131	−0.031	0.186	0.121
SVM	damaged labels	0.093	0.049	−0.012	0.116	0.091
Buchwald–Hartwig, RMSE in yield points (lower is better)						
LightGBM	clean labels	21.8	18.2	17.2		
LightGBM	damaged labels	21.6	17.8	17.1		
RF	clean labels	19.4	17.4	16.5		
RF	damaged labels	19.6	17.5	17.0		
$\Delta\Delta G$, Spearman (higher is better)						
DNN	clean labels	0.328	0.110	0.164	0.506	0.239
DNN	damaged labels	0.225	0.141	0.095	0.516	0.005
SVM	clean labels	0.06994	0.178	0.202	0.038	0.131
SVM	damaged labels	0.06987	0.200	0.171	0.332	0.190
QM9, RMSE in eV (lower is better)						
DNN	clean labels	0.544	0.450	0.4654		
DNN	damaged labels	0.553	0.460	0.4651		
LightGBM	clean labels	0.563	0.471	0.500		
LightGBM	damaged labels	0.556	0.458	0.507		

Table 7.29: The second hybrid searched on clean labels or on labels with Gaussian noise at level 1.0, both fitted on the damaged labels and scored on the held-out labels as delivered. One row is one model and search labels. Mean over three noise draws (two for SVM on SMAC fold 3, the draws with both arms); empty cells were not run. QM9 is scored on its held-out labels with no noise added. Buchwald–Hartwig RMSE is in percentage points of yield. Bold: the values discussed in the text.

Configuration	Plain	Seed averaging	Stacking	Retuning	Clean-label settings	Plain on clean labels
hERG K_i, Gaussian noise at 1.0, DNN						
First single	0.701	0.678	0.681	0.688	0.704	0.638
Second single	0.735	0.675	0.685	0.734	0.735	0.589
First hybrid	0.720	0.664	0.663	0.712	0.709	0.607
Second hybrid	0.718	**0.656**	0.671	**0.691**	**0.703**	0.608
hERG K_i, Gaussian noise at 1.0, SVM						
First single	0.699		0.681	0.682	0.724	0.640
Second single	0.708		0.685	0.689	0.759	0.675
First hybrid	0.668		0.663	0.660	0.702	0.616
Second hybrid	0.675		0.671	0.673	0.702	0.617
hERG K_i, 25% clipped, DNN						
First single	0.742	0.758	0.764	0.7634	0.7633	0.638
Second single	0.749	0.750	0.761	0.768	0.749	0.589
First hybrid	0.731	0.719	0.751	0.735	0.727	0.608
Second hybrid	0.736	0.724	0.748	0.746	0.728	0.612
hERG K_i, 25% clipped, SVM						
First single	0.760		0.764	0.769	0.749	0.640
Second single	0.774		0.761	0.766	0.765	0.675
First hybrid	0.746		0.751	0.749	0.727	0.621
Second hybrid	0.750		0.748	0.753	0.736	0.628
QM9, Gaussian noise at 1.0, DNN, RMSE in eV						
First single	0.595	0.535	0.528	0.560	0.579	0.400
Second single	0.641	0.597	0.514	0.626	0.641	0.487
First hybrid	0.521	0.479	0.494	0.515	0.518	0.357
Second hybrid	0.547	0.476	0.505	0.511	0.529	0.358
QM9, Gaussian noise at 1.0, LightGBM, RMSE in eV						
First single	0.582	0.533	0.528	0.542	0.550	0.412
Second single	0.565	0.540	0.514	0.574	0.703	0.459
First hybrid	0.564	0.504	0.494	0.506	0.611	0.378
Second hybrid	0.563	0.506	0.505	0.499	0.567	0.382

Table 7.30: Changing the model instead of the labels, on damaged hERG K_i and QM9 labels. One row is one configuration. Held-out RMSE, lower is better. Retuning: settings tuned on the damaged labels. Stacking combines both models and has one score per configuration. Outer fold 0, means over three noise draws. Bold: the values discussed in the text.

Configuration	Plain	Seed averaging	Stacking	Retuning
hERG K_i, DNN				
First single	0.638	0.618	0.632	0.628
Second single	0.589	0.596	0.606	0.589
First hybrid	0.597	0.596	0.582	0.623
Second hybrid	0.612	0.598	0.589	0.639
hERG K_i, SVM				
First single	0.640		0.632	0.618
Second single	0.675		0.606	0.640
First hybrid	0.607		0.582	0.573
Second hybrid	0.612		0.589	0.579
LogD, DNN				
First single	0.431	0.417	0.430	0.428
Second single	0.417	**0.405**	0.408	0.409
First hybrid	0.397	0.379	0.389	0.394
Second hybrid	0.394	**0.376**	0.387	0.390
LogD, SVM				
First single	0.475		0.430	0.421
Second single	0.495		0.408	0.474
First hybrid	0.421		0.389	0.387
Second hybrid	0.421		0.387	0.387
QM9, no noise added, DNN, RMSE in eV				
First single	0.400	0.391	0.375	0.389
Second single	0.487	0.463	0.410	0.487
First hybrid	0.3491	0.3493	0.331	0.356
Second hybrid	0.357	0.346	0.340	0.358
QM9, no noise added, LightGBM, RMSE in eV				
First single	0.412	0.403	0.375	0.403
Second single	0.459	0.458	0.410	0.455
First hybrid	0.372	0.371	0.331	0.3705
Second hybrid	0.376	0.370	0.340	0.370

Table 7.31: Changing the model instead of the labels, on the labels as delivered and on QM9 with no noise added. One row is one configuration. Held-out RMSE, lower is better. Retuning: settings tuned on these labels. Stacking combines both models and has one score per configuration. Outer fold 0.

The expanded sweep of methods for learning with noisy labels

Methods whose primary mechanism was deleting molecules based on large training losses did not have success on small molecules. Both Small-loss selection and Multistage cleaning work this way. Table 7.32 shows them helping more often on Buchwald–Hartwig and the antibodies, though Buchwald–Hartwig is a single dataset.

Only two of the twelve methods helped in a specific case, and each only a little.

Method	Small molecules, of 37	Buchwald–Hartwig, of 5	Antibodies, of 25
Noise mitigation: selected by the DNN's loss or a neighbor gap, then deleted			
Small-loss selection	2	5	12
Confident learning	17	1	12
Distance-based selection	10	4	10
Multistage cleaning	1	2	5
Noise-robust training			
Early learning	16	2	11
Uncertainty curriculum	15	3	14
DivideMix	8	2	14
Meta-Weight-Net	12	3	11
MentorNet	14	4	12
Contrast-to-divide	10	2	12
Heteroscedastic GP[a]	18	2	14
Evidential network[a]	15	4	13

Table 7.32: Conditions where each method of the expanded sweep beat the plain DNN on both the best single representation and the best hybrid. A condition is one dataset, damage kind and level, or the labels as delivered. The DNN only; outer fold 0. [a]Regression only, so 30 small-molecule conditions.

Heteroscedastic GP was the only method to help on $\Delta\Delta G$'s labels as delivered, for some of the configurations listed in Table 7.33. It was also one of the methods that held up under Gaussian noise on QM9 in Chapter 4. Distance-based selection was the only method to help both the single representation and the hybrid on hERG blocking with 10% of labels flipped one way, by less than 0.01 AUROC.

Method	Configuration	Plain DNN	With the method
$\Delta\Delta G$, labels as delivered, Spearman (higher is better)			
Heteroscedastic GP	First single	**0.341**	**0.433**
Heteroscedastic GP	Second hybrid	0.250	0.364
Heteroscedastic GP	First hybrid	0.305	0.288
hERG blocking, 10% of labels flipped one way, AUROC (higher is better)			
Distance-based selection	First single	0.815	0.817
Distance-based selection	First hybrid	0.821	0.825

Table 7.33: Two results of the expanded sweep. One row is one method and configuration. Outer fold 0. Bold: the values discussed in the text.

On QM9 with no noise added, every one of the twelve methods made the DNN worse, on both the best single representation and the hybrid, as Table 7.34 shows. In a second run of eleven of the methods on the same labels, about half of them beat the

plain DNN on its second hybrid. The difference between the runs is the DNN's own spread from one fit to the next, which we come back to in Section 7.3.8.

Method	First run		Second run
	First single	First hybrid	Second hybrid
Plain DNN	**0.396**	**0.355**	0.364
Small-loss selection	0.465	0.422	0.423
Confident learning	0.412	0.371	0.383
Distance-based selection	0.413	0.368	
Multistage cleaning	0.455	0.419	0.382
Early learning	0.420	0.370	0.356
Uncertainty curriculum	0.444	0.380	0.354
DivideMix	0.499	0.452	0.405
Meta-Weight-Net	0.426	0.380	0.353
MentorNet	0.425	0.392	**0.343**
Contrast-to-divide	0.419	0.393	0.350
Heteroscedastic GP	0.417	0.372	0.363
Evidential network	0.427	0.383	0.372

Table 7.34: The expanded sweep on QM9 with no noise added, in two runs of the same DNN. One row is one method. Held-out RMSE, lower is better; bold is the lowest in each column. Outer fold 0. Empty: not run.

As is a trend in this thesis, everything is conditional. The method that works the best with hybrids is rarely the one that works best on single representations. On the first hybrid there are three exceptions, namely Caco-2 under the shift in 10% (Distance-based selection), hERG under class-dependent flips (Confident learning), and hERG under instance-dependent flips (Distance-based selection). A ranking of noise-handling methods made on one representation just does not carry over to another.

7.3.8 Effect by model and representation

Cleaning methods are definitely benefited by hybrid representations on small molecules; the hybrid beats the best single representation in almost every small-molecule condition prior to noise injection, and still does even after. On LogD with outliers in Table 7.11, for example, the DNN's hybrid cleaned with Neighbor weights scores 0.556 RMSE against 0.591 for the cleaned single representation. Cleaning rarely changes which of the two wins, and when it does, it more often than not moves the win towards the hybrid. On antibodies the hybrid is ahead in about half the conditions either way, and on the reaction set it's on par.

We now return to the question this thesis began with: does a better feature set make it easier to train under noise? The closest test (Neighbor weights) down-weights each molecule by how far its label is from its neighbors' labels. The neighbors are found either in the model's own single representation or in the hybrid. Neither choice is reliably better. The hybrid's neighbors helped slightly more often, but the largest differences go both ways, as Table 7.35 shows. On LogD under a scaffold-family shift, for example, the hybrid's neighbors helped the DNN while its own neighbors made it worse. This comparison also has no control.

Dataset and damage	Configuration	No weighting	Own neighbors	Hybrid's neighbors
RMSE (lower is better)				
LogD, scaffold-family shift	First single	**0.705**	0.753	**0.663**
Spearman (higher is better)				
Nb-Tm, outliers	Second single	0.3336	0.3344	0.518
Titer, Gaussian noise	Second single	−0.050	0.150	−0.012

Table 7.35: Neighbor weights with neighbors found in the model's own single representation or in the hybrid, from the Neighbor-label gap (LogD, Titer) or the gap over the neighbors' spread (Nb-Tm). DNN, damage at level 1.0, one noise draw, outer fold 0. One row is one dataset, damage, and configuration; the three largest differences. Bold: the values discussed in the text. Dataset short names as in Table 5.8.

The same cleanup on the same labels helps one model and hurts another. On hERG IC_{50}, for example, weighting each molecule by how often it was measured (Count and spread weights) improved the SVM but made the DNN worse, so you can't say whether a noise method works without saying which model it feeds. On QM9 with clean labels, Neighbor-mean smoothing and weighting improved the DNN while LightGBM barely moved, as Section 7.3.1 found. Across the injected damage, no one model gained most often everywhere. The DNN gained most often on Caco-2, hERG K_i, and LogD, the SVM on $\Delta\Delta G$, and the RF on expression titer. Part of the DNN's lead may come from its own variation between fits, since a model that gives the same score every time never counts as helped by chance. Therefore, a noise-handling method should be chosen alongside the model, not on its own.

On small molecules, hybrids maintained their lead over single representations under cleaning. Cleaning can still make a model worse, as deletion did on QM9 with no noise added. Which method did best also changed between the hybrid and the single representations, as Section 7.3.7, "The expanded sweep of methods for learning with noisy labels", describes. So whether a noise-handling method works depends on the

feature space it runs in, as Chapter 4 found for distance.

7.4 Conclusions

We were able to successfully identify noisy labels in small-molecule data and correct them in several settings. QM9, the one dataset with computed labels, tests whether a method removes noise or only trains a better model, as described in Section 7.3.1. Deletion and the LNL methods helped QM9's clean labels by roughly the same amount as the DNN's own spread. However, we were not able to run as many iterations as we would have liked, due to time constraints.

Where deletion or an LNL method helped an antibody dataset's labels as delivered, it therefore most likely removed noise that was already in the data. Neighbor-mean smoothing, weighting, and changes to the model, by contrast, also lowered the held-out error on QM9 with no noise added, where there is no noise to remove. Most of what they gain on measured labels therefore comes from training a better model rather than from removing noise. The exceptions are Neighbor-mean smoothing on hERG K_i and on hERG blocking. Moving every label toward a Huber-fitted prediction (Huber blend) also behaved like noise removal, as it helped hERG K_i under damage and made QM9 worse.

Corrections worked best when the damaged labels were flagged in the data, as with labels clipped at an assay limit. Raising these labels won back about half to over four fifths of the lost RMSE on the small-molecule datasets, with no detector. With a detector, deleting or relabeling flagged molecules (Delete flagged or Neighbor-mean smoothing) helped under label flips and outliers. Cleanups driven by a detector passed random or shuffled controls under hERG's label flips. They also passed on the labels as delivered of hERG K_i, Buchwald–Hartwig, and five antibody datasets. On two of these they passed only in their weaker configurations. QM9 shows, though, that the shuffled control alone is weak evidence with the DNN.

The hybrid stayed ahead of the single representation on small molecules in almost every condition, before and after treatment. Finding the neighbors for Neighbor weights in the hybrid, rather than in the model's own single representation, helped slightly more often, with large differences in both directions. Which noise-handling method works best thus depends on the feature space it runs in.

The majority of results in this chapter rely on only the first of the five outer folds. Where an experiment was also run on the other outer folds, those results mostly agree with the first. Cleaning before the search gave large gains. However, on QM9 with no

noise added only the fit was cleaned, never the search, so those gains cannot be split into noise removed and a better model. Chapter 8 uses these results to answer the research questions of Chapter 1, and to set out how to choose a noise-handling method for a new dataset.

Chapter 8

Conclusions and further thoughts

This chapter returns to the original research questions posed in Chapter 1 regarding label noise in molecular data. It then discusses how to choose a noise-handling method based on your actual data.

8.1 Summary of findings

Chapter 3 asked what makes a QSAR model robust to label noise. We found that noise robustness depended more on the model family than on the representation. On QM9 under Gaussian noise, model family explained 47.2% of the variance in AUC_{norm}, and the representation 8.1%. The representation mattered more for predictive performance, as expected, since label noise leaves the features untouched. Among the models, RFs were the most consistent under noise. LightGBM and XGBoost displayed strong predictive performance on clean labels yet were unable to retain that edge as the injected label noise rose. Therefore, a clean score doesn't say much about that model's ability to withstand noise. The type of noise mattered more than the shape of its distribution. Systematic noise did the most damage, specifically noise shared within a scaffold family or in the form of clipped labels at an assay limit. Finally, models that estimate their own uncertainty reported that uncertainty rose with injected noise, yet no model's uncertainty was able to identify which individual labels were corrupt. The exception to this was censoring, where dividing a molecule's error by its uncertainty picked out the clipped labels better than the error alone, as the model was more certain on the clipped labels.

In Chapter 4, we asked why certain LNL methods did poorly. All 20 configurations with the lowest retention of R^2 under noise used a pairwise distance between molecules. Some methods were not designed with a distance, yet we tried those meth-

ods with and without the influence of distance. We then considered the individual features by which the distance is computed. Within one dataset, most features of a standard representation barely vary, and the ones that do may be irrelevant. Features that effectively function as noise carry over into the distance measure and even further into downstream applications that use distance, like LNL techniques. Several representations also give different molecules, mostly stereoisomers, the same feature vector. The adaptive hybrid representation was built to address both problems.

The second half of this thesis asked how we can adapt the representation to the task at hand. Chapter 5 showed that the best set of features depends on the dataset and the downstream model, and that a search can find it. The hybrid representation beat the best single representation on every small-molecule dataset with both downstream models. These wins carried over to antibodies, although there the gain depended more on the dataset. On most antibody datasets, a late fusion ensemble of one model per representation outperformed the hybrids. On the Buchwald–Hartwig reactions, the hybrid gained nothing beyond the spread across folds. Chapter 6 showed that distance in a hybrid tracked the label better than distance in a single representation, on all six small-molecule datasets and on antibodies and reactions. Hybrids improved nearest-neighbor-based predictions for small molecules, yet made them worse for four antibody datasets and for Buchwald–Hartwig.

The second half also asked whether feature importance helps. Ranking features by importance helped to build the hybrid, which beat recursive feature elimination and random features of the same number on most datasets. Reusing the same scores to weight features during training added little on small molecules.

Finally, Chapter 7 loops back around to noise, demonstrating that noisy labels can be found and corrected. Unsurprisingly, corrections worked best when the damaged labels were flagged in the data in some way. This mimicked the application of known data collection conditions to an ML pipeline. Correcting labels clipped at an assay limit won back about half to over four fifths of the lost RMSE on the small-molecule datasets, with no detector. Where nothing flagged the damaged labels, cleaning guided by a detector still beat random deletion and shuffled detector scores under label flips on hERG blocking. It also beat them on the labels as delivered of hERG K_i, Buchwald–Hartwig, and five antibody datasets. Not every gain was noise removed, though. Neighbor-mean smoothing, weighting, and changes to the model also improved QM9, whose labels are computed and carry no assay error. Most of their gains on measured labels therefore come from training a better model, not from removing noise. The exceptions were Neighbor-mean smoothing on hERG K_i and on hERG blocking. On small molecules,

the hybrid stayed ahead of the single representation in almost every noise condition, though not always with the DNN on hERG K_i.

8.2 Choosing a noise-handling method

No single method handled label noise well on every dataset, and a method that helped in one setting often hurt in another. The clearest pattern is that the right method depends on whether the noise leaves a trace in the data. When it does, as with labels clipped at an assay limit, a correction that uses that trace works better than a general detector. It doesn't even have to be an explicit statistical trace, it could be information about the data collection mechanism that gives you a clue as to what kind of noise you're working with. Under clipping, for example, the Forest residual did worse than chance on LogD, QM9, and Buchwald–Hartwig, while the methods that used the clipping did best. A method built for noise the data do not carry can in fact do harm, as Scaffold-family offset made Caco-2's labels as delivered worse. The feature space matters too, since the ranking of methods changed between the hybrid and the single representations. Simple methods were often enough. On hERG K_i, raising every clipped label by one shared amount (Constant lift) beat Censored EM in most configurations.

8.3 Limitations

We were limited first by the data available. QM9 is the only dataset whose labels carry no measurement error, and as in Chapter 3 we used only one of its properties, the HOMO–LUMO gap. The assay datasets are smaller, and their labels already carry an unknown amount of noise, so injected noise sits on top of it. Most antibody datasets hold only a few hundred antibodies, so their results say as much about size as about antibodies, as discussed in Section 5.4. Only one dataset holds reactions. The separate sites, described in Section 5.2.11, were simulated from two public datasets, and in the main experiment every site held the same molecules. As Section 5.2.7 describes, the LogD and Caco-2 labels are clipped at their 1st and 99th percentiles, so on Caco-2, 22 molecules share the highest label.

Our evaluation was limited by time and compute. Not every experiment ran on every fold or every dataset, and most noise experiments ran on the first outer fold only, as Section 5.2.6 explains. Where a method has several settings, Chapter 7 reports its best on the held-out score, so every gain is optimistic. Each dataset's grid came from

an earlier search of the whole dataset, which included the samples that the outer folds later hold out. Section 5.3.1, "Overfitting of the search", showed that on a few hundred samples, the search's own scores overstate the hybrid's results, and Section 5.3.2 that its conformal intervals cover too few held-out antibodies. The DNN's score also changes between fits. On QM9, one hybrid scored 0.362 and 0.364 RMSE on the same fold and labels in two experiments. Several design choices in Section 5.2.2 rest on preliminary runs, such as scoring features within each representation rather than in one pool.

Some comparisons are weaker than others. When the nearest-neighbor setting is picked again for each fold, the hybrid has five or six times as many combinations to choose from. Only the hybrid can also use importance weighting, described in Section 6.2. The test of a new embedding in Section 5.3.2 swapped random features into the same places of one hybrid, with no new search.

As Section 5.4 notes, the search is costly, and must be rerun whenever the data, representations, or model change. To save compute, each dataset's search space was hand-selected from the full search, as Section 5.2.5 describes, so it may find a less informative hybrid. QM9 and hERG blocking were searched over a shorter fixed list of settings. Adaptive pruning, described in Section 5.2.4, exists but was not used. The kNN search of Section 6.2.1 capped its budgets at 500 features. As noted in Section 5.3.2, we have not tested whether the best settings follow measurable properties of a dataset.

Our noise experiments cover only some kinds of noise. Most are on regression data. A noise level is a fraction of the spread of the clean training labels, so it cannot be mapped onto a measurement error. The robustness score $\mathrm{AUC}_{\mathrm{norm}}$ of Chapter 3 is a ratio to clean R^2, so a model with a low clean score has less to lose. There, the heavy-tailed, outlier, and censored conditions ran on only some pairings of model and representation, set out in Section 3.2. As Section 7.2.2 explains, a detector can be graded only where some labels were damaged and others not, which excludes Gaussian and grouped-shifted noise. The exploratory sweep of Chapter 4 added a distance to some methods designed without one, so poor adaptation may explain part of its result.

The controls for noise handling are also limited. QM9 with no noise added is the only clean control, and it rests on one computed property. A few methods have no QM9 result, such as those that use the repeat measurements of hERG IC_{50}, and some have no control at all, such as neighbor-agreement weights on SMAC colloidal stability (Section 7.3.5). With no ground truth for the labels as delivered, a control can show that a gain beats chance, but not that a label was corrected. Section 7.4 notes that on QM9, cleaning before the search cleaned only the fit, so those gains cannot be split

into noise removed and a better model. We did not study in depth why the methods that worked do so, or how each interacts with each kind of noise.

Finally, the representations and models were fixed in advance. As Section 4.2.1, "Molecules with identical feature vectors", showed, ECFP4 was built with stereochemistry off, and PDV and Mordred ignore it, so these merge stereoisomers. As Section 5.2.2 describes, the single representations of each dataset were chosen before the search, by their scores and the kind of information they carry. A sparse feature, such as one position of a one-hot sequence, scores low importance on its own. Model hyperparameters were kept the same across datasets. Tuning was tested only on one outer fold of five datasets, as Section 5.2.9 describes.

8.4 Future work

The most immediate and obvious next step is to finish what time did not allow. Because of time and compute constraints, not every experiment ran on every fold or every dataset, and the majority of the results in Chapter 7 come from the first outer fold only. While we do observe consistent gains in the same directions and can draw preliminary conclusions from that, running the remaining folds would show which of these results carry significance. The searches themselves could also be more expansive. Each dataset searched a grid cut down from an earlier search, and QM9 and hERG blocking were limited to the fast tier. A full search on every dataset would show whether those cuts cost any accuracy. The upside to this setup is that the results in this thesis if anything found worse-performing hybrids than what we could have found had a full search been run.

The noise-handling methods that worked deserve a closer look. Correcting labels clipped at an assay limit, Neighbor-mean smoothing on the two hERG datasets, and Delete flagged with the Forest residual on SMAC colloidal stability and expression titer all passed their controls and showed promise on the tested datasets. We did not have the time at the end of this work to study these methods in as much depth as we would have liked, in particular why they work and how each one interacts with the different types of noise. A systematic comparison of each method under each noise type, with clean controls throughout, is the next step. QM9 offers twelve computed properties, and we used only the HOMO–LUMO gap, so it can supply more of those controls. Cliff put-back should also be tested with activity cliffs (ACs) defined in other representations, including the hybrid itself, since the ACs found depend on the representation.

Classification also needs more work. Most of our noise experiments cover regression tasks. In QSAR, a class label usually comes from cutting a continuous measurement at a threshold, so measurement error mostly flips molecules near that threshold. The detectors of noisy labels should next be tested on flips of that kind, on more classification datasets. We have the code in place; however, no in-depth analysis was conducted.

The search could also be sped up. Summed over five folds, it took up to 28.2 hours on a small-molecule dataset, and on the largest datasets the full grid was out of reach. This is counterintuitive, considering the hybrids performed well on larger datasets. Adaptive pruning already exists in the framework but was not used for the results here due to time constraints. The most logical next step would be to further calibrate adaptive pruning on more datasets. Another option is reusing the importance scores and fitted models shared between configurations.

Finally, the hybrid search needs only a supervised task and several feature matrices for the same samples. It can be applied to any domain, from gene-expression profiles to financial time series, since each has many competing representations. A hybrid could also be the input to the model that scores molecules for a generative model, and a deployment across real sites would test the results of Chapter 5 on data held at separate sites.

8.5 Concluding remarks

Label noise in molecular data poses some complex problems. But when it comes to selecting methods of handling that, whether it be choosing a model that is inherently robust to noise, conducting specific noise-robust training methods, or attempting to mitigate the label noise itself, it is important to remember that there is no one-size-fits-all solution. The "right" method depends on the task, the data, the model, and the type of noise.

Knowing what type of noise is in your data ahead of time, through statistical tests or through knowledge of the source of the data, can help guide you towards a solution. And you can successfully train with noise. Most studies just assume label noise is Gaussian, but that's not a sure bet given how experimental assay data is collected. And non-Gaussian noise, specifically systematic noise, has the potential to do far more damage. It is also the easiest to correct.

One way around the conditional nature of the methods in this thesis is adaptivity, specifically within the feature set. Most features within a standard molecular representation barely vary on a given dataset and have little relation to the actual pre-

diction task. Yet, each one still contributes to the pairwise distance measure between molecules. And any method that uses such distance metrics, such as neighbor-based comparison methods, inherits this flaw. The adaptive hybrid representation selects features that are relevant to the dataset and model given. On small molecules, it predicted better than every single representation tested, its distances tracked label differences more closely, and it outperformed standard representations across almost every noise condition.

When working with noisy molecular data, we recommend that the first step, which is often overlooked, should be to consider what kind of noise the labels carry. Then, be prepared to adapt to the problem, considering the differences brought on by the data and the models chosen. Label noise, which is pervasive in cheminformatics data, is not something to be ignored, but rather something to be understood. And hopefully, with the right features, something we can learn from.

Summary of Research Contributions

Research Papers

In preparation: Punt, A., Hanser, T., Werner, S., & Morris, G.M. (2026). NoiseInject: A Benchmarking Framework to Study Noise Robustness in QSAR Models. To be submitted to the *Journal of Cheminformatics*. Chapter 3.

In preparation: Punt, A., Morris, G.M., Werner, S., & Hanser, T. (2026). Adaptive hybrid representations: selecting features from several molecular representations for each dataset and model (working title). Chapters 5 and 6.

In preparation: Punt, A., Morris, G.M., Werner, S., & Hanser, T. (2026). Detecting, removing, and training through label noise in molecular property prediction. Chapter 7.

Software

NoiseInject, a Python package for benchmarking noise robustness and uncertainty quantification: `https://github.com/adpunt/noise_inject`. Chapter 3.

KIRBy, the adaptive hybrid representation search: `https://github.com/adpunt/KIRBy`. Chapters 5 to 7.

Conference Presentations

Punt, A., Morris, G.M., Werner, S., & Hanser, T. (2024). Investigating the impact and mitigation of noise in QSAR models. Presented at the Autumn UK-QSAR 2024 Meeting.

Bibliography

[1] James P. Hughes, Stephen Rees, S. Barrett Kalindjian, and Karen L. Philpott. Principles of early drug discovery. *British Journal of Pharmacology*, 162(6):1239–1249, 2011.

[2] Duxin Sun, Wei Gao, Hongxiang Hu, and Simon Zhou. Why 90% of clinical drug development fails and how to improve it? *Acta Pharmaceutica Sinica B*, 12(7): 3049–3062, 2022.

[3] Bruno J. Neves, Rodolpho C. Braga, Cleber C. Melo-Filho, José Teófilo Moreira-Filho, Eugene N. Muratov, and Carolina Horta Andrade. QSAR-based virtual screening: Advances and applications in drug discovery. *Frontiers in Pharmacology*, 9:1275, 2018.

[4] María Virginia Sabando, Ignacio Ponzoni, Evangelos E. Milios, and Axel J. Soto. Using molecular embeddings in QSAR modeling: does it make a difference? *Briefings in Bioinformatics*, 23(1):bbab365, 2022.

[5] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010.

[6] Laurianne David, Amol Thakkar, Rocío Mercado, and Ola Engkvist. Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1):56, 2020.

[7] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Sort & Slice: a simple and superior alternative to hash-based folding for extended-connectivity fingerprints. *Journal of Cheminformatics*, 16(1):135, 2024.

[8] Peter Gedeck, Bernhard Rohde, and Christian Bartels. QSAR – how good is it in practice? Comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5):1924–1936, 2006.

[9] Akihiro Kishimoto, Hiroshi Kajino, Masataka Hirose, Junta Fuchiwaki, Indra Priyadarsini, Lisa Hamada, Hajime Shinohara, Daiju Nakano, and Seiji Takeda. MHG-GNN: Combination of molecular hypergraph grammar with graph neural network. arXiv:2309.16374, 2023.

[10] Scott S. Kolmar and Christopher M. Grulke. The effect of noise on the predictive limit of QSAR models. *Journal of Cheminformatics*, 13(1):92, 2021.

[11] Kjell Jorner, Tore Brinck, Per-Ola Norrby, and David Buttar. Machine learning meets mechanistic modelling for accurate prediction of experimental activation energies. *Chemical Science*, 12(3):1163–1175, 2021.

[12] Christian Kramer, Tuomo Kalliokoski, Peter Gedeck, and Anna Vulpetti. The experimental uncertainty of heterogeneous public K_{i} data. *Journal of Medicinal Chemistry*, 55(11):5165–5173, 2012.

[13] Tuomo Kalliokoski, Christian Kramer, Anna Vulpetti, and Peter Gedeck. Comparability of mixed IC50 data – a statistical analysis. *PLoS ONE*, 8(4):e61007, 2013.

[14] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and QSAR modeling research. *Journal of Chemical Information and Modeling*, 50(7):1189–1204, 2010.

[15] W. Patrick Walters. We need better benchmarks for machine learning in drug discovery. Practical Cheminformatics blog, `https://practicalcheminformatics.blogspot.com/2023/08/we-need-better-benchmarks-for-machine.html`, 2023.

[16] Gregory A. Landrum and Sereina Riniker. Combining IC_{50} or K_i values from different sources is a source of significant noise. *Journal of Chemical Information and Modeling*, 64(5):1560–1567, 2024.

[17] Isidro Cortés-Ciriano, Andreas Bender, and Thérèse E. Malliavin. Comparing the influence of simulated experimental errors on 12 machine learning algorithms in bioactivity modeling using 12 diverse data sets. *Journal of Chemical Information and Modeling*, 55(7):1413–1425, 2015.

[18] Esther Heid, Charles J. McGill, Florence H. Vermeire, and William H. Green. Characterizing uncertainty in machine learning for chemistry. *Journal of Chemical Information and Modeling*, 63(13):4012–4029, 2023.

[19] Viktoriia Turkina, Melanie R. W. Messih, Etienne Kant, Jelle T. Gringhuis, Annemieke Petrignani, Garry Corthals, Jake W. O'Brien, and Saer Samanipour. Enhancing predictive modeling with molecular fingerprint fusion strategies. *Digital Discovery*, 5(5):2232–2242, 2026.

[20] Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(1):49–67, 2006.

[21] Weifan Zheng and Alexander Tropsha. Novel variable selection quantitative structure–property relationship approach based on the k-nearest-neighbor principle. *Journal of Chemical Information and Computer Sciences*, 40(1):185–194, 2000.

[22] Peter Itskowitz and Alexander Tropsha. k nearest neighbors QSAR modeling as a variational problem: Theory and applications. *Journal of Chemical Information and Modeling*, 45(3):777–785, 2005.

[23] Davide Bassani, Alessandro Brigo, and Andrea Andrews-Morger. Federated learning in computational toxicology: an industrial perspective on the Effiris hackathon. *Chemical Research in Toxicology*, 36(9):1503–1517, 2023.

[24] Thierry Hanser. Federated learning for molecular discovery. *Current Opinion in Structural Biology*, 79:102545, 2023.

[25] David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

[26] Michael Backenköhler, Paula Linh Kramer, Joschka Groß, Gerrit Großmann, Roman Joeres, Azat Tagirdzhanov, Dominique Sydow, Hamza Ibrahim, Floriane Odje, Verena Wolf, et al. TeachOpenCADD goes deep learning: Open-source teaching platform exploring molecular DL applications. ChemRxiv, 2023.

[27] Alexandre V. Fassio, Laura Shub, Luca Ponzoni, Jessica McKinley, Matthew J. O'Meara, Rafaela S. Ferreira, Michael J. Keiser, and Raquel C. de Melo Minardi.

Prioritizing virtual screening with interpretable interaction fingerprints. *Journal of Chemical Information and Modeling*, 62(18):4300–4318, 2022.

[28] Liangxu Xie, Lei Xu, Ren Kong, Shan Chang, and Xiaojun Xu. Improvement of prediction performance with conjoint molecular fingerprint in deep learning. *Frontiers in Pharmacology*, 11:606668, 2020.

[29] Isidro Cortés-Ciriano, Ctibor Škuta, Andreas Bender, and Daniel Svozil. QSAR-derived affinity fingerprints (part 2): modeling performance for potency prediction. *Journal of Cheminformatics*, 12(1):41, 2020.

[30] Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018.

[31] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. ChemBERTa: Large-scale self-supervised pretraining for molecular property prediction. arXiv:2010.09885, 2020.

[32] Benedek Fabian, Thomas Edlich, Héléna Gaspar, Marwin Segler, Joshua Meyers, Marco Fiscato, and Mohamed Ahmed. Molecular representation learning with language models and domain-relevant auxiliary tasks. arXiv:2011.13230, 2020.

[33] Walid Ahmad, Elana Simon, Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. ChemBERTa-2: Towards chemical foundation models. arXiv:2209.01712, 2022.

[34] Roberto Todeschini and Viviana Consonni. *Molecular Descriptors for Chemoinformatics*. Wiley-VCH, Weinheim, 2nd edition, 2009.

[35] Andrea Mauri, Viviana Consonni, Manuela Pavan, and Roberto Todeschini. DRAGON software: An easy approach to molecular descriptor calculations. *MATCH Communications in Mathematical and in Computer Chemistry*, 56(2): 237–248, 2006.

[36] Artem Cherkasov, Eugene N. Muratov, Denis Fourches, Alexandre Varnek, Igor I. Baskin, Mark Cronin, John Dearden, Paola Gramatica, Yvonne C. Martin, Roberto Todeschini, et al. QSAR modeling: Where have you been? Where are you going to? *Journal of Medicinal Chemistry*, 57(12):4977–5010, 2014.

[37] Greg Landrum. RDKit: Open-source cheminformatics. `https://www.rdkit.org`, 2023. Version 2023.09.1.

[38] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003.

[39] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016.

[40] Zhichun Guo, Kehan Guo, Bozhao Nan, Yijun Tian, Roshni G. Iyer, Yihong Ma, Olaf Wiest, Xiangliang Zhang, Wei Wang, Chuxu Zhang, et al. Graph-based molecular representation learning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, pages 6638–6646, 2023.

[41] Sophia M. N. Hönig, Christian Lemmen, and Matthias Rarey. Small molecule superposition: A comprehensive overview on pose scoring of the latest methods. *WIREs Computational Molecular Science*, 13(2):e1640, 2023.

[42] Ge Yan, Huaijin Wu, and Junchi Yan. Quantum 3D graph learning with applications to molecule embedding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 39126–39137, 2023.

[43] Ziduo Yang, Weihe Zhong, Qiujie Lv, Tiejun Dong, and Calvin Yu-Chian Chen. Geometric interaction graph neural network for predicting protein–ligand binding affinities from 3D structures (GIGN). *The Journal of Physical Chemistry Letters*, 14(8):2020–2033, 2023.

[44] Wenjie Du, Xiaoting Yang, Di Wu, FenFen Ma, Baicheng Zhang, Chaochao Bao, Yaoyuan Huo, Jun Jiang, Xin Chen, and Yang Wang. Fusing 2D and 3D molecular graphs as unambiguous molecular descriptors for conformational and chiral stereoisomers. *Briefings in Bioinformatics*, 24(1):bbac560, 2023.

[45] Denis Kuzminykh, Daniil Polykovskiy, Artur Kadurin, Alexander Zhebrak, Ivan Baskov, Sergey Nikolenko, Rim Shayakhmetov, and Alex Zhavoronkov. 3D molecular representations based on the wave transform for convolutional neural networks. *Molecular Pharmaceutics*, 15(10):4378–4385, 2018.

[46] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting.

IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(1):657–668, 2023.

[47] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties. *Nature Machine Intelligence*, 4(12):1256–1264, 2022.

[48] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth International Group, Belmont, CA, 1984.

[49] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[50] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[51] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[52] Junshui Ma, Robert P. Sheridan, Andy Liaw, George E. Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure–activity relationships. *Journal of Chemical Information and Modeling*, 55(2):263–274, 2015.

[53] Igor I. Baskin, Vladimir A. Palyulin, and Nikolai S. Zefirov. Neural networks in building QSAR models. In David J. Livingstone, editor, *Artificial Neural Networks: Methods and Applications*, volume 458 of *Methods in Molecular Biology*, pages 133–154. Humana Press, Totowa, NJ, 2009.

[54] Alexios Koutsoukas, Keith J. Monaghan, Xiaoli Li, and Jun Huan. Deep-learning: investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *Journal of Cheminformatics*, 9(1):42, 2017.

[55] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations*, 2017.

[56] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: a survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023.

[57] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. Confident learning: estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021.

[58] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations*, 2018.

[59] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2818–2826, 2016.

[60] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[61] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[62] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Exploring QSAR models for activity-cliff prediction. *Journal of Cheminformatics*, 15(1):47, 2023.

[63] Derek van Tilborg, Alisa Alenicheva, and Francesca Grisoni. Exposing the limitations of molecular machine learning with activity cliffs. *Journal of Chemical Information and Modeling*, 62(23):5938–5951, 2022.

[64] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[65] Ismail Babajide Mustapha and Faisal Saeed. Bioactive molecule prediction using extreme gradient boosting. *Molecules*, 21(8):983, 2016.

[66] Hao Tian, Rajas Ketkar, and Peng Tao. ADMETboost: A web server for accurate ADMET prediction. *Journal of Molecular Modeling*, 28(12):408, 2022.

[67] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting

decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[68] Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Ng, and Alejandro Schuler. NGBoost: Natural gradient boosting for probabilistic prediction. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 2690–2700, 2020.

[69] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995.

[70] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004.

[71] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[72] Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3):1–27, 2011.

[73] Ryan-Rhys Griffiths, Leo Klarner, Henry Moss, Aditya Ravuri, Sang Truong, Yuanqi Du, Samuel Stanton, Gary Tom, Bojana Rankovic, Arian Jamasb, et al. GAUCHE: A library for Gaussian processes in chemistry. In *Advances in Neural Information Processing Systems*, volume 36, pages 76923–76946, 2023.

[74] Liva Ralaivola, Sanjay J. Swamidass, Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural Networks*, 18(8):1093–1110, 2005.

[75] Henry B. Moss and Ryan-Rhys Griffiths. Gaussian process molecule property prediction with FlowMO. arXiv:2010.01118, 2020.

[76] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, pages 6402–6413, 2017.

[77] Glenn Palmer, Siqi Du, Alexander Politowicz, Joshua Paul Emory, Xiyu Yang, Anupraas Gautam, Grishma Gupta, Zhelong Li, Ryan Jacobs, and Dane Morgan. Calibration after bootstrap for accurate uncertainty quantification in regression models. *npj Computational Materials*, 8(1):115, 2022.

[78] Mark A. Johnson and Gerald M. Maggiora, editors. *Concepts and Applications of Molecular Similarity*. John Wiley & Sons, New York, 1990.

[79] Gerald Maggiora, Martin Vogt, Dagmar Stumpfe, and Jürgen Bajorath. Molecular similarity in medicinal chemistry. *Journal of Medicinal Chemistry*, 57(8): 3186–3204, 2014.

[80] Dávid Bajusz, Anita Rácz, and Károly Héberger. Why is Tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7(1):20, 2015.

[81] Gerald M. Maggiora. On outliers and activity cliffs—why QSAR often disappoints. *Journal of Chemical Information and Modeling*, 46(4):1535, 2006.

[82] Dagmar Stumpfe and Jürgen Bajorath. Exploring activity cliffs in medicinal chemistry. *Journal of Medicinal Chemistry*, 55(7):2932–2942, 2012.

[83] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is "nearest neighbor" meaningful? In *Database Theory – ICDT'99*, volume 1540 of *Lecture Notes in Computer Science*, pages 217–235, 1999.

[84] Felix A. Kruger and John P. Overington. Global analysis of small molecule binding to related protein targets. *PLoS Computational Biology*, 8(1):e1002333, 2012.

[85] Ben Lambert, Chon Lok Lei, Martin Robinson, Michael Clerx, Richard Creswell, Sanmitra Ghosh, Simon Tavener, and David J. Gavaghan. Autocorrelated measurement processes and inference for ordinary differential equation models of biological systems. *Journal of the Royal Society Interface*, 20:20220725, 2023.

[86] Joe Bentz, Michael P. O'Connor, Dallas Bednarczyk, JoAnn Coleman, Caroline Lee, Johan Palm, Y. Anne Pak, Elke S. Perloff, Eric Reyner, Praveen Balimane, et al. Variability in P-glycoprotein inhibitory potency (IC50) using various in vitro experimental systems: Implications for universal digoxin drug-drug interaction risk assessment decision criteria. *Drug Metabolism and Disposition*, 41(7): 1347–1366, 2013.

[87] Emma Svensson, Hannah Rosa Friesacher, Susanne Winiwarter, Lewis Mervin, Adam Arany, and Ola Engkvist. Enhancing uncertainty quantification in drug discovery with censored regression labels. *Artificial Intelligence in the Life Sciences*, 7:100128, 2025.

[88] Vladimir Makarenkov, Pablo Zentilli, Dmytro Kevorkov, Andrei Gagarin, Nathalie Malo, and Robert Nadon. An efficient method for the detection and elimination of systematic error in high-throughput screening. *Bioinformatics*, 23 (13):1648–1657, 2007.

[89] James Inglese, Douglas S. Auld, Ajit Jadhav, Ronald L. Johnson, Anton Simeonov, Adam Yasgar, Wei Zheng, and Christopher P. Austin. Quantitative high-throughput screening: A titration-based approach that efficiently identifies biological activities in large chemical libraries. *Proceedings of the National Academy of Sciences*, 103(31):11473–11478, 2006.

[90] Ji-Hu Zhang, Thomas D. Y. Chung, and Kevin R. Oldenburg. A simple statistical parameter for use in evaluation and validation of high throughput screening assays. *Journal of Biomolecular Screening*, 4(2):67–73, 1999.

[91] Anton Simeonov, Ajit Jadhav, Craig J. Thomas, Yuhong Wang, Ruili Huang, Noel T. Southall, Paul Shinn, Jeremy Smith, Christopher P. Austin, Douglas S. Auld, et al. Fluorescence spectroscopic profiling of compound libraries. *Journal of Medicinal Chemistry*, 51(8):2363–2371, 2008.

[92] Christos Hatzis, Philippe L. Bedard, Nicolai J. Birkbak, Andrew H. Beck, Hugo J. W. L. Aerts, David F. Stern, Leming Shi, Robert Clarke, John Quackenbush, and Benjamin Haibe-Kains. Enhancing reproducibility in cancer drug screening: How do we move forward? *Cancer Research*, 74(15):4016–4023, 2014.

[93] Elena Postnikova, Yu Cong, Lisa Evans DeWald, Julie Dyall, Shuiqing Yu, Brit J. Hart, Huanying Zhou, Robin Gross, James Logue, Yingyun Cai, et al. Testing therapeutics in cell-based assays: Factors that influence the apparent potency of drugs. *PLOS ONE*, 13(3):e0194880, 2018.

[94] Anna Gaulton, Anne Hersey, Michał Nowotka, A. Patrícia Bento, Jon Chambers, David Mendez, Prudence Mutowo, Francis Atkinson, Louisa J. Bellis, Elena Cibrián-Uhalte, et al. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017.

[95] Michael K. Gilson, Tiqing Liu, Michael Baitaluk, George Nicola, Linda Hwang, and Jenny Chong. BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1):D1045–D1053, 2016.

[96] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016.

[97] Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Paul Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11-12):1337–1345, 2008.

[98] A. Patrícia Bento, Anne Hersey, Eloy Félix, Greg Landrum, Anna Gaulton, Francis Atkinson, Louisa J. Bellis, Marleen De Veij, and Andrew R. Leach. An open source chemical structure curation pipeline using RDKit. *Journal of Cheminformatics*, 12(1):51, 2020.

[99] Insha Ullah and A. H. Welsh. On the effect of noise on fitting linear regression models. *Computational Statistics & Data Analysis*, 224:108421, 2026.

[100] Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen J. Roberts, and Chris C. Holmes. Explicit regularisation in Gaussian noise injections. In *Advances in Neural Information Processing Systems*, volume 33, pages 16603–16614, 2020.

[101] Guy W. Bemis and Mark A. Murcko. The properties of known drugs. 1. Molecular frameworks. *Journal of Medicinal Chemistry*, 39(15):2887–2893, 1996.

[102] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[103] Frank R. Hampel. Robust statistics: a brief introduction and overview. Research Report 94, Seminar für Statistik, Eidgenössische Technische Hochschule (ETH) Zürich, 2001.

[104] Alex Avdeef. Multi-lab intrinsic solubility measurement reproducibility in CheqSol and shake-flask methods. *ADMET and DMPK*, 7(3):210–219, 2019.

[105] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996.

[106] Dennis L. Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-2(3):408–421, 1972.

[107] Adelaide Punt. NoiseInject: A framework for label noise robustness testing. Version 0.3.0, Zenodo, `https://doi.org/10.5281/zenodo.20532710`, 2026.

[108] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48, pages 1050–1059, 2016.

[109] Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, et al. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019.

[110] Izhar Wallach and Abraham Heifets. Most ligand-based classification benchmarks reward memorization rather than generalization. *Journal of Chemical Information and Modeling*, 58(5):916–932, 2018.

[111] Gregory A. Landrum, Maximilian Beckers, Jessica Lanini, Nadine Schneider, Nikolaus Stiefl, and Sereina Riniker. SIMPD: an algorithm for generating simulated time splits for validating machine learning approaches. *Journal of Cheminformatics*, 15:119, 2023.

[112] Jianyuan Deng, Zhibo Yang, Hehe Wang, Iwao Ojima, Dimitris Samaras, and Fusheng Wang. A systematic study of key elements underlying molecular property prediction. *Nature Communications*, 14(1):6395, 2023.

[113] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1(1):140022, 2014.

[114] Open ADMET Consortium. OpenADMET-ExpansionRx challenge dataset. `https://huggingface.co/datasets/openadmet/openadmet-expansionrx-challenge-data`, 2026.

[115] Barbara Zdrazil, Eloy Felix, Fiona Hunter, Emma J. Manners, James Blackshaw, Sybilla Corbett, Marleen de Veij, Harris Ioannidis, David Mendez Lopez, Juan F. Mosquera, et al. The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 2024.

[116] Sanjivanjit K. Bhal, Karim Kassam, Ian G. Peirson, and Greg M. Pearl. The rule of five revisited: Applying log D in place of log P in drug-likeness filters. *Molecular Pharmaceutics*, 4(4):556–560, 2007.

[117] Ina Hubatsch, Eva G. E. Ragnarsson, and Per Artursson. Determination of drug permeability and prediction of drug absorption in Caco-2 monolayers. *Nature Protocols*, 2(9):2111–2119, 2007.

[118] Michael C. Sanguinetti and Martin Tristani-Firouzi. hERG potassium channels and cardiac arrhythmia. *Nature*, 440(7083):463–469, 2006.

[119] Vladimir Svetnik, Andy Liaw, Christopher Tong, J. Christopher Culberson, Robert P. Sheridan, and Bradley P. Feuston. Random forest: A classification and regression tool for compound classification and QSAR modeling. *Journal of Chemical Information and Computer Sciences*, 43(6):1947–1958, 2003.

[120] Nicolai Meinshausen. Quantile regression forests. *Journal of Machine Learning Research*, 7:983–999, 2006.

[121] Olga Obrezanova, Gábor Csányi, Joelle M. R. Gola, and Matthew D. Segall. Gaussian processes: a method for automatic QSAR modeling of ADME properties. *Journal of Chemical Information and Modeling*, 47(5):1847–1857, 2007.

[122] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, Cambridge, MA, 2005.

[123] Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate Gaussian process regression. *Journal of Machine Learning Research*, 6(65):1939–1959, 2005.

[124] Jacob R. Gardner, Geoff Pleiss, Kilian Q. Weinberger, David Bindel, and Andrew Gordon Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[125] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[126] Alex Kendall and Yarin Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[127] James Harrison, John Willes, and Jasper Snoek. Variational Bayesian last layers. In *International Conference on Learning Representations*, 2024.

[128] Jessica Lanini, Minh Tam Davide Huynh, Gaetano Scebba, Nadine Schneider, and Raquel Rodríguez-Pérez. UNIQUE: A framework for uncertainty quantification benchmarking. *Journal of Chemical Information and Modeling*, 64(22): 8379–8386, 2024.

[129] Iurie Caraus, Abdulaziz A. Alsuwailem, Robert Nadon, and Vladimir Makarenkov. Detecting and overcoming systematic bias in high-throughput screening technologies: a comprehensive review of practical issues and methodological solutions. *Briefings in Bioinformatics*, 16(6):974–986, 2015.

[130] Daniel Crusius, Flaviu Cipcigan, and Philip C. Biggin. Are we fitting data or noise? Analysing the predictive power of commonly used datasets in drug-, materials-, and molecular-discovery. *Faraday Discussions*, 256:304–321, 2025.

[131] Markus Orsi and Jean-Louis Reymond. One chiral fingerprint to find them all. *Journal of Cheminformatics*, 16(1):53, 2024.

[132] Lien Ai Nguyen, Hua He, and Chuong Pham-Huy. Chiral drugs: An overview. *International Journal of Biomedical Science*, 2(2):85–100, 2006.

[133] Alexander Golbraikh, Eugene Muratov, Denis Fourches, and Alexander Tropsha. Data set modelability by QSAR. *Journal of Chemical Information and Modeling*, 54(1):1–4, 2014.

[134] Robert P. Sheridan, Prabha Karnachi, Matthew Tudor, Yuting Xu, Andy Liaw, Falgun Shah, Alan C. Cheng, Elizabeth Joshi, Meir Glick, and Juan Alvarez. Experimental error, kurtosis, activity cliffs, and methodology: What limits the predictivity of quantitative structure–activity relationship models? *Journal of Chemical Information and Modeling*, 60(4):1969–1982, 2020.

[135] José L. Medina-Franco. Activity cliffs: Facts or artifacts? *Chemical Biology & Drug Design*, 81(5):553–556, 2013.

[136] Sereina Riniker and Gregory A. Landrum. Open-source platform to benchmark fingerprints for ligand-based virtual screening. *Journal of Cheminformatics*, 5 (1):26, 2013.

[137] Mateusz Praski, Jakub Adamczyk, and Wojciech Czech. Benchmarking pretrained molecular embedding models for molecular representation learning. arXiv:2508.06199, 2025.

[138] Florian Huber and Julian Pollmann. Count your bits: fingerprint benchmarking to assess broad chemical space representation. *Journal of Cheminformatics*, 18 (1):83, 2026.

[139] Pawel Dabrowski-Tumanski, Bartosz Topolski, Dariusz Plewczynski, and Tomasz Jetka. The geometry of activity cliffs: Representation dependence and multi-scale characterization of activity landscapes. arXiv:2605.30831, 2026.

[140] Damien E. Coupry and Peter Pogány. Application of deep metric learning to molecular graph similarity. *Journal of Cheminformatics*, 14(1):11, 2022.

[141] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

[142] Cheng Fang, Ye Wang, Richard Grater, Sudarshan Kapadnis, Cheryl Black, Patrick Trapa, and Simone Sciabola. Prospective validation of machine learning algorithms for absorption, distribution, metabolism, and excretion prediction: An industrial perspective. *Journal of Chemical Information and Modeling*, 63 (11):3263–3274, 2023.

[143] Rajarshi Guha and John H. Van Drie. Structure–activity landscape index: Identifying and quantifying activity cliffs. *Journal of Chemical Information and Modeling*, 48(3):646–658, 2008.

[144] Kenneth López-Pérez, Juan F. Avellaneda-Tamayo, Lexin Chen, Edgar López-López, K. Eurídice Juárez-Mercado, José L. Medina-Franco, and Ramón Alain Miranda-Quintana. Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2(2):100077, 2024.

[145] Peter Willett. Similarity-based virtual screening using 2D fingerprints. *Drug Discovery Today*, 11(23–24):1046–1053, 2006.

[146] Alice Capecchi, Daniel Probst, and Jean-Louis Reymond. One molecular fingerprint to rule them all: drugs, biomolecules, and the metabolome. *Journal of Cheminformatics*, 12(1):43, 2020.

[147] Noel M. O'Boyle and Roger A. Sayle. Comparing structural fingerprints using a literature-based similarity benchmark. *Journal of Cheminformatics*, 8(1):36, 2016.

[148] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *Database Theory — ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434, 2001.

[149] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11:2487–2531, 2010.

[150] Ye Hu, Norbert Furtmann, Dagmar Stumpfe, and Jürgen Bajorath. Comprehensive knowledge base of two- and three-dimensional activity cliffs for medicinal and computational chemistry. *F1000Research*, 4:168, 2015.

[151] David E. Patterson, Richard D. Cramer, Allan M. Ferguson, Robert D. Clark, and Laurence E. Weinberger. Neighborhood behavior: A useful concept for validation of "molecular diversity" descriptors. *Journal of Medicinal Chemistry*, 39(16): 3049–3059, 1996.

[152] Alexander Tropsha. Best practices for QSAR model development, validation, and exploitation. *Molecular Informatics*, 29(6-7):476–488, 2010.

[153] Darko Butina. Unsupervised data base clustering based on Daylight's fingerprint and Tanimoto similarity: A fast and automated way to cluster small and large data sets. *Journal of Chemical Information and Computer Sciences*, 39(4):747–750, 1999.

[154] Thierry Hanser, Ernst Ahlberg, Alexander Amberg, Lennart T. Anger, Chris Barber, Richard J. Brennan, Alessandro Brigo, Annie Delaunois, Susanne Glowienke, Nigel Greene, et al. Data-driven federated learning in drug discovery with knowledge distillation. *Nature Machine Intelligence*, 7(3):423–436, 2025.

[155] Junnan Li, Richard Socher, and Steven C. H. Hoi. DivideMix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*, 2020.

[156] Bo Han, Quanming Yao, Xingrui Yu, Gang Niu, Miao Xu, Weihua Hu, Ivor Tsang, and Masashi Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[157] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.

[158] Kristian Kersting, Christian Plagemann, Patrick Pfaff, and Wolfram Burgard. Most likely heteroscedastic Gaussian process regression. In *Proceedings of the 24th International Conference on Machine Learning*, pages 393–400, 2007.

[159] Carla E. Brodley and Mark A. Friedl. Identifying mislabeled training data. *Journal of Artificial Intelligence Research*, 11:131–167, 1999.

[160] Benoît Frénay and Michel Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5): 845–869, 2014.

[161] Juan Martín, José A. Sáez, and Emilio Corchado. On the regressand noise problem: Model robustness and synergy with regression-adapted noise filters. *IEEE Access*, 9:145800–145816, 2021.

[162] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. Could graph neural networks learn better molecular representation for drug discovery? A comparison study of descriptor-based and graph-based models. *Journal of Cheminformatics*, 13(1):12, 2021.

[163] Andreas Mayr, Günter Klambauer, Thomas Unterthiner, Marvin Steijaert, Jörg K. Wegner, Hugo Ceulemans, Djork-Arné Clevert, and Sepp Hochreiter. Large-scale comparison of machine learning methods for drug target prediction on ChEMBL. *Chemical Science*, 9(24):5441–5451, 2018.

[164] Jessica Lanini, Gianluca Santarossa, Finton Sirockin, Richard Lewis, Nikolas Fechner, Hubert Misztela, Sarah Lewis, Krzysztof Maziarz, Megan Stanley, Marwin Segler, et al. PREFER: A new predictive modeling framework for molecular discovery. *Journal of Chemical Information and Modeling*, 63(15):4497–4504, 2023.

[165] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024.

[166] Peter Neidlinger, Omar S. M. El Nahhas, Hannah Sophie Muti, Tim Lenz, Michael Hoffmeister, Hermann Brenner, Marko van Treeck, Rupert Langer, Bastian Dislich, Hans Michael Behrens, et al. Benchmarking foundation models as feature extractors for weakly supervised computational pathology. *Nature Biomedical Engineering*, 10(6):1113–1123, 2026.

[167] Gordon F. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968.

[168] Richard E. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton, NJ, 1961.

[169] Sören Richard Stahlschmidt, Benjamin Ulfenborg, and Jane Synnergren. Multimodal deep learning for biomedical data fusion: a review. *Briefings in Bioinformatics*, 23(2):bbab569, 2022.

[170] Sandra Steyaert, Marija Pizurica, Divya Nagaraj, Priya Khandelwal, Tina Hernandez-Boussard, Andrew J. Gentles, and Olivier Gevaert. Multimodal data fusion for cancer biomarker discovery with deep learning. *Nature Machine Intelligence*, 5(4):351–362, 2023.

[171] Tongxin Wang, Wei Shao, Zhi Huang, Haixu Tang, Jie Zhang, Zhengming Ding, and Kun Huang. MOGONET integrates multi-omics data using graph convolutional networks allowing patient classification and biomarker identification. *Nature Communications*, 12(1):3445, 2021.

[172] Aurélien Beaude, Franck Augé, Farida Zehraoui, and Blaise Hanczar. CrossAttOmics: multiomics data integration with cross-attention. *Bioinformatics*, 41(6): btaf302, 2025.

[173] Mehmet Gönen and Ethem Alpaydın. Multiple kernel learning algorithms. *Journal of Machine Learning Research*, 12:2211–2268, 2011.

[174] Anna Cichonska, Tapio Pahikkala, Sandor Szedmak, Heli Julkunen, Antti Airola, Markus Heinonen, Tero Aittokallio, and Juho Rousu. Learning with multiple pairwise kernels for drug bioactivity prediction. *Bioinformatics*, 34(13):i509–i518, 2018.

[175] Oliver Laufkötter, Noé Sturm, Jürgen Bajorath, Hongming Chen, and Ola Engkvist. Combining structural and bioactivity-based fingerprints improves prediction performance and scaffold hopping capability. *Journal of Cheminformatics*, 11(1):54, 2019.

[176] Ian T. Jolliffe. A note on the use of principal components in regression. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 31(3):300–303, 1982.

[177] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58(2): 109–130, 2001.

[178] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005.

[179] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2):273–324, 1997.

[180] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996.

[181] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46(1–3):389–422, 2002.

[182] Cecile Valsecchi, Francesca Grisoni, Viviana Consonni, and Davide Ballabio. Consensus versus individual QSARs in classification: Comparison on a large-scale case study. *Journal of Chemical Information and Modeling*, 60(3):1215–1223, 2020.

[183] José Jiménez-Luna, Francesca Grisoni, and Gisbert Schneider. Drug discovery with explainable artificial intelligence. *Nature Machine Intelligence*, 2(10):573–584, 2020.

[184] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, and Hongming Chen. Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1):48, 2017.

[185] Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002.

[186] Andrius Vabalas, Emma Gowen, Ellen Poliakoff, and Alexander J. Casson. Machine learning algorithm validation with a limited sample size. *PLOS ONE*, 14 (11):e0224365, 2019.

[187] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7(1):91, 2006.

[188] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010.

[189] Jiayun Dong and Cynthia Rudin. Exploring the cloud of variable importance for the set of all good models. *Nature Machine Intelligence*, 2(12):810–824, 2020.

[190] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8(1):25, 2007.

[191] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3145–3153, 2017.

[192] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020.

[193] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations*, 2015.

[194] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, et al. Captum: A unified and generic model interpretability library for PyTorch. arXiv:2009.07896, 2020.

[195] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. In *International Conference on Learning Representations*, 2018.

[196] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. QM9 dataset. Figshare, `https://doi.org/10.6084/m9.figshare.978904`, 2014.

[197] Polaris contributors. polaris-hub/polaris. Zenodo, `https://doi.org/10.5281/zenodo.13652587`, 2025.

[198] Patrick Koenig, Chingwei V. Lee, Benjamin T. Walters, Vasantharajan Janakiraman, Jeremy Stinson, Thomas W. Patapoff, and Germaine Fuh. Mutational landscape of antibody variable domains reveals a switch modulating the interdomain conformational dynamics and antigen binding. *Proceedings of the National Academy of Sciences*, 114(4):E486–E495, 2017.

[199] Michael Chungyoun, Jeffrey Ruffolo, and Jeffrey Gray. FLAb: Benchmarking deep learning methods for antibody fitness prediction. bioRxiv, 2024.

[200] Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023:baad021, 2023.

[201] Gabriel Cia, Dong Li, Simón Poblete, Marianne Rooman, and Fabrizio Pucci. AbAgym: a well-curated dataset for the mutational analysis of antibody–antigen complexes. *mAbs*, 17(1):2592421, 2025.

[202] David Prihoda, Jad Maamary, Andrew Waight, Veronica Juan, Laurence Fayadat-Dilman, Daniel Svozil, and Danny A. Bitton. BioPhi: A platform for antibody design, humanization, and humanness evaluation based on natural antibody repertoires and deep learning. *mAbs*, 14(1):2020203, 2022.

[203] Justina Jankauskaitė, Brian Jiménez-García, Justas Dapkūnas, Juan Fernández-Recio, and Iain H. Moal. SKEMPI 2.0: an updated benchmark of changes in protein–protein binding energy, kinetics and thermodynamics upon mutation. *Bioinformatics*, 35(3):462–469, 2019.

[204] Sarah Sirin, James R. Apgar, Eric M. Bennett, and Amy E. Keating. AB-Bind: Antibody binding mutational database for computational affinity predictions. *Protein Science*, 25(2):393–409, 2016.

[205] Ammar Arsiwala, Rebecca Bhatt, Lood van Niekerk, Porfirio Quintero-Cadena, Xiang Ao, Adam Rosenbaum, Aanal Bhatt, Alexander Smith, Yaoyu Yang, KC Anderson, et al. A high-throughput platform for biophysical antibody developability assessment to enable AI/ML model training. *mAbs*, 17(1):2593055, 2025.

[206] Lood van Niekerk, Joshua Moller, Seth Ritter, Porfirio Quintero-Cadena, Rich Cohen, Georgia Channing, Michael Chungyoun, Laura Rand, Alexander Smith, Aanal Bhatt, et al. 2025 Ginkgo Datapoints antibody developability competition outcomes: limited model performance and a call for data standardization. *mAbs*, 18(1):2634216, 2026.

[207] Derek T. Ahneman, Jesús G. Estrada, Shishi Lin, Spencer D. Dreher, and Abigail G. Doyle. Predicting reaction performance in C–N cross-coupling using machine learning. *Science*, 360(6385):186–190, 2018.

[208] Luiz C. Vieira, Sophia Lin, and Claus O. Wilke. Intrinsic dataset features drive mutational effect prediction by protein language models. bioRxiv, 2026.

[209] Alexandre Varnek, Denis Fourches, Frank Hoonakker, and Vitaly P. Solov'ev. Substructural fragments: an universal language to encode reactions, molecular and supramolecular structures. *Journal of Computer-Aided Molecular Design*, 19 (9–10):693–703, 2005.

[210] Frank Hoonakker, Nicolas Lachiche, Alexandre Varnek, and Alain Wagner. A representation to apply usual data mining techniques to chemical reactions — illustration on the rate constant of S_N2 reactions in water. *International Journal on Artificial Intelligence Tools*, 20(2):253–270, 2011.

[211] Philippe Schwaller, Benjamin Hoover, Jean-Louis Reymond, Hendrik Strobelt, and Teodoro Laino. Extraction of organic chemistry grammar from unsupervised learning of chemical reactions. *Science Advances*, 7(15):eabe4166, 2021.

[212] Philippe Schwaller, Daniel Probst, Alain C. Vaucher, Vishnu H. Nair, David Kreutter, Teodoro Laino, and Jean-Louis Reymond. Mapping the space of chemical reactions using attention-based neural networks. *Nature Machine Intelligence*, 3(2):144–152, 2021.

[213] Tatsuya Sagawa and Ryosuke Kojima. ReactionT5: a pre-trained transformer model for accurate chemical reaction prediction with limited data. *Journal of Cheminformatics*, 17(1):126, 2025.

[214] Kangway V. Chuang and Michael J. Keiser. Comment on "Predicting reaction performance in C–N cross-coupling using machine learning". *Science*, 362(6416): eaat8603, 2018.

[215] Steven Henikoff and Jorja G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992.

[216] Marie-Paule Lefranc, Christelle Pommié, Manuel Ruiz, Véronique Giudicelli, Elodie Foulquier, Lisa Truong, Valérie Thouvenin-Contet, and Gérard Lefranc. IMGT unique numbering for immunoglobulin and T cell receptor variable domains and Ig superfamily V-like domains. *Developmental & Comparative Immunology*, 27(1):55–77, 2003.

[217] James Dunbar and Charlotte M. Deane. ANARCI: antigen receptor numbering and receptor classification. *Bioinformatics*, 32(2):298–300, 2016.

[218] Raymond E. Carhart, Dennis H. Smith, and R. Venkataraghavan. Atom pairs as molecular features in structure-activity studies: definition and applications. *Journal of Chemical Information and Computer Sciences*, 25(2):64–73, 1985.

[219] Ramaswamy Nilakantan, Norman Bauman, J. Scott Dixon, and R. Venkataraghavan. Topological torsion: a new molecular descriptor for SAR applications. Comparison with other descriptors. *Journal of Chemical Information and Computer Sciences*, 27(2):82–85, 1987.

[220] Joseph L. Durant, Burton A. Leland, Douglas R. Henry, and James G. Nourse. Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*, 42(6):1273–1280, 2002.

[221] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research*, 12(77):2539–2561, 2011.

[222] Hirotomo Moriwaki, Yu-Shi Tian, Norihito Kawashita, and Tatsuya Takagi. Mordred: a molecular descriptor calculator. *Journal of Cheminformatics*, 10(1):4, 2018.

[223] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. Self-supervised graph transformer on large-scale molecular

data. In *Advances in Neural Information Processing Systems*, volume 33, pages 12559–12571, 2020.

[224] Shengchao Liu, Hanchen Wang, Weiyang Liu, Joan Lasenby, Hongyu Guo, and Jian Tang. Pre-training molecular graph representation with 3D geometry. In *International Conference on Learning Representations*, 2022.

[225] Eduardo Soares, Emilio Vital Brazil, Victor Shirasuna, Dmitry Zubarev, Renato Cerqueira, and Kristin Schmidt. An open-source family of large encoder-decoder foundation models for chemistry. *Communications Chemistry*, 8(1):193, 2025.

[226] Xiao-Chen Zhang, Cheng-Kun Wu, Jia-Cai Yi, Xiang-Xiang Zeng, Can-Qun Yang, Ai-Ping Lu, Ting-Jun Hou, and Dong-Sheng Cao. Pushing the boundaries of molecular property prediction for drug discovery with multitask learning BERT enhanced by SMILES enumeration. *Research*, 2022:0004, 2022.

[227] Atakan Yüksel, Erva Ulusoy, Atabey Ünlü, and Tunca Doğan. SELFormer: Molecular representation learning via SELFIES language models. *Machine Learning: Science and Technology*, 4(2):025035, 2023.

[228] Pedro J. Ballester and W. Graham Richards. Ultrafast shape recognition to search compound databases for similar molecular shapes. *Journal of Computational Chemistry*, 28(10):1711–1723, 2007.

[229] Adrian M. Schreyer and Tom Blundell. USRCAT: real-time ultrafast shape recognition with pharmacophoric constraints. *Journal of Cheminformatics*, 4(1):27, 2012.

[230] Matthias Rupp, Alexandre Tkatchenko, Klaus-Robert Müller, and O. Anatole von Lilienfeld. Fast and accurate modeling of molecular atomization energies with machine learning. *Physical Review Letters*, 108(5):058301, 2012.

[231] Yuyang Wang, Jianren Wang, Zhonglin Cao, and Amir Barati Farimani. Molecular contrastive learning of representations via graph neural networks. *Nature Machine Intelligence*, 4(3):279–287, 2022.

[232] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.

[233] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alex Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. In *Advances in Neural Information Processing Systems*, volume 34, pages 29287–29303, 2021.

[234] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, et al. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025.

[235] Salvatore Candido, Thomas Hayes, Alexander Derry, Roshan Rao, Zeming Lin, Robert Verkuil, Bryan Z. Wu, Jin Sub Lee, Elise S. Bruguera, Jehan A. Keval, et al. Language modeling materializes a world model of protein biology. bioRxiv, 2026.

[236] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, et al. ProtTrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022.

[237] Jin Su, Chenchen Han, Yuyang Zhou, Junjie Shan, Xibin Zhou, and Fajie Yuan. SaProt: Protein language modeling with structure-aware vocabulary. In *International Conference on Learning Representations*, 2024.

[238] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. Addressing the antibody germline bias and its effect on language models for improved antibody design. *Bioinformatics*, 40(11):btae618, 2024.

[239] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning. arXiv:2112.07782, 2021.

[240] Eunna Huh, Hyeonsu Lee, and Hyunjin Shin. Antibody foundational model: Ab-RoBERTa. arXiv:2506.13006, 2025.

[241] Brennan Abanades, Wing Ki Wong, Fergus Boyles, Guy Georges, Alexander Bujotzek, and Charlotte M. Deane. ImmuneBuilder: Deep-learning models for predicting the structures of immune proteins. *Communications Biology*, 6(1):575, 2023.

[242] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, second edition, 2009.

[243] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-Weight-Net: Learning an explicit mapping for sample weighting. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[244] Sarah Nogueira, Konstantinos Sechidis, and Gavin Brown. On the stability of feature selection algorithms. *Journal of Machine Learning Research*, 18(174): 1–54, 2018.

[245] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[246] Ivan Olier, Noureddin Sadawi, G. Richard Bickerton, Joaquin Vanschoren, Crina Grosan, Larisa Soldatova, and Ross D. King. Meta-QSAR: a large-scale application of meta-learning to drug design and discovery. *Machine Learning*, 107(1): 285–311, 2018.

[247] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282, 2017.

[248] Wouter Heyndrickx, Lewis Mervin, Tobias Morawietz, Noé Sturm, Lukas Friedrich, Adam Zalewski, Anastasia Pentina, Lina Humbeck, Martijn Oldenhof, Ritsuya Niwayama, et al. MELLODDY: Cross-pharma federated learning at unprecedented scale unlocks benefits in QSAR without compromising proprietary information. *Journal of Chemical Information and Modeling*, 64(7):2331–2344, 2024.

[249] Pietro Cassarà, Alberto Gotta, and Lorenzo Valerio. Federated feature selection for cyber-physical systems of systems. *IEEE Transactions on Vehicular Technology*, 71(9):9937–9950, 2022.

[250] Ryan J. Tibshirani. Degrees of freedom and model search. *Statistica Sinica*, 25 (3):1265–1296, 2015.

[251] Sahibsingh A. Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-6(4):325–327, 1976.

[252] Donald Shepard. A two-dimensional interpolation function for irregularly-spaced data. In *Proceedings of the 1968 23rd ACM National Conference*, pages 517–524, 1968.

[253] Dietrich Wettschereck, David W. Aha, and Takao Mohri. A review and empirical evaluation of feature weighting methods for a class of lazy learning algorithms. *Artificial Intelligence Review*, 11(1–5):273–314, 1997.

[254] Arkaprava Banerjee and Kunal Roy. First report of q-RASAR modeling toward an approach of easy interpretability and efficient transferability. *Molecular Diversity*, 26(5):2847–2862, 2022.

[255] Robert P. Sheridan, Bradley P. Feuston, Vladimir N. Maiorov, and Simon K. Kearsley. Similarity to molecules in the training set is a good discriminator for prediction accuracy in QSAR. *Journal of Chemical Information and Computer Sciences*, 44(6):1912–1928, 2004.

[256] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[257] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.

[258] Alexander Kraskov, Harald Stögbauer, and Peter Grassberger. Estimating mutual information. *Physical Review E*, 69(6):066138, 2004.

[259] Brian C. Ross. Mutual information between discrete and continuous data sets. *PLoS ONE*, 9(2):e87357, 2014.

[260] Line Båserud, Cristian Lussana, Thomas N. Nipen, Ivar A. Seierstad, Louise Oram, and Trygve Aspelien. TITAN automatic spatial quality control of meteorological in-situ observations. *Advances in Science and Research*, 17:153–163, 2020.

[261] Mirosław Kordos and Marcin Blachnik. Instance selection with neural networks for regression problems. In *Artificial Neural Networks and Machine Learning – ICANN 2012*, volume 7553 of *Lecture Notes in Computer Science*, pages 263–270, 2012.

[262] Sanjay Chawla and Pei Sun. SLOM: a new measure for local spatial outliers. *Knowledge and Information Systems*, 9(4):412–429, 2006.

[263] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks. In *International Conference on Learning Representations*, 2022.

[264] Andy Liaw and Matthew Wiener. Classification and regression by randomForest. *R News*, 2(3):18–22, 2002.

[265] Amirata Ghorbani and James Zou. Data Shapley: Equitable valuation of data for machine learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 2242–2251, 2019.

[266] Ruoxi Jia, David Dao, Boxin Wang, Frances Ann Hubis, Nezihe Merve Gurel, Bo Li, Ce Zhang, Costas Spanos, and Dawn Song. Efficient task-specific data valuation for nearest neighbor algorithms. *Proceedings of the VLDB Endowment*, 12(11):1610–1623, 2019.

[267] Hang Zhou, Jonas Mueller, Mayank Kumar, Jane-Ling Wang, and Jing Lei. Detecting errors in a numerical response via any regression model. *Journal of Data-centric Machine Learning Research*, 1(6):1–25, 2024.

[268] Stephen Bates, Emmanuel Candès, Lihua Lei, Yaniv Romano, and Matteo Sesia. Testing for outliers with conformal p-values. *The Annals of Statistics*, 51(1): 149–178, 2023.

[269] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300, 1995.

[270] Linlin Zhao, Wenyi Wang, Alexander Sedykh, and Hao Zhu. Experimental errors in QSAR modeling sets: What we can do and what we cannot do. *ACS Omega*, 2(6):2805–2812, 2017.

[271] E. A. Nadaraya. On estimating regression. *Theory of Probability & Its Applications*, 9(1):141–142, 1964.

[272] Geoffrey S. Watson. Smooth regression analysis. *Sankhyā: The Indian Journal of Statistics, Series A*, 26:359–372, 1964.

[273] Scott Reed, Honglak Lee, Dragomir Anguelov, Christian Szegedy, Dumitru Erhan, and Andrew Rabinovich. Training deep neural networks on noisy labels with bootstrapping. In *International Conference on Learning Representations, Workshop Track*, 2015.

[274] W. James and Charles Stein. Estimation with quadratic loss. In *Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 361–379, Berkeley, CA, 1961. University of California Press.

[275] Miroslav Kubat and Stan Matwin. Addressing the curse of imbalanced training sets: one-sided selection. In *Proceedings of the Fourteenth International Conference on Machine Learning*, pages 179–186, 1997.

[276] Jerzy Stefanowski and Szymon Wilk. Selective pre-processing of imbalanced data for improving classification performance. In *Data Warehousing and Knowledge Discovery: 10th International Conference, DaWaK 2008*, volume 5182 of *Lecture Notes in Computer Science*, pages 283–292, 2008.

[277] Ahmad Abu Shanab, Taghi M. Khoshgoftaar, Randall Wald, and Jason Van Hulse. Evaluation of the importance of data pre-processing order when combining feature selection and data sampling. *International Journal of Business Intelligence and Data Mining*, 7(1/2):116–134, 2012.

[278] James Tobin. Estimation of relationships for limited dependent variables. *Econometrica*, 26(1):24–36, 1958.

[279] Jonathan Buckley and Ian James. Linear regression with censored data. *Biometrika*, 66(3):429–436, 1979.

[280] Ahlem Hajjem, François Bellavance, and Denis Larocque. Mixed-effects random forest for clustered data. *Journal of Statistical Computation and Simulation*, 84 (6):1313–1328, 2014.

[281] Fabio Sigrist. Gaussian process boosting. *Journal of Machine Learning Research*, 23(232):1–46, 2022.

[282] Larry V. Hedges and Ingram Olkin. *Statistical Methods for Meta-Analysis*. Academic Press, Orlando, FL, 1985.

[283] Gordon K. Smyth. Linear models and empirical Bayes methods for assessing differential expression in microarray experiments. *Statistical Applications in Genetics and Molecular Biology*, 3(1):1–25, 2004.

[284] Leo Breiman. Stacked regressions. *Machine Learning*, 24(1):49–64, 1996.

[285] Zixiao Zhao, Fatemeh H. Fard, and Jie JW Wu. MANTRA: a framework for multi-stage adaptive noise treatment during training. arXiv:2512.04319, 2025.

[286] Yiming Zhang, Chester Holtz, Gal Mishne, and Alex Cloninger. Revisiting meta-learning with noisy labels: Reweighting dynamics and theoretical guarantees. arXiv:2510.12209, 2025.

[287] M. Pawan Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. In *Advances in Neural Information Processing Systems*, volume 23, pages 1189–1197, 2010.

[288] Lu Jiang, Zhengyuan Zhou, Thomas Leung, Li-Jia Li, and Li Fei-Fei. MentorNet: Learning data-driven curriculum for very deep neural networks on corrupted labels. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80, pages 2304–2313, 2018.

[289] Evgenii Zheltonozhskii, Chaim Baskin, Avi Mendelson, Alex M Bronstein, and Or Litany. Contrast to divide: Self-supervised pre-training for learning with noisy labels. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 387–397, 2022.

[290] Alexander Amini, Wilko Schwarting, Ava Soleimany, and Daniela Rus. Deep evidential regression. In *Advances in Neural Information Processing Systems*, volume 33, pages 14927–14937, 2020.

[291] Avinash Barnwal, Hyunsu Cho, and Toby Hocking. Survival regression with accelerated failure time model in XGBoost. *Journal of Computational and Graphical Statistics*, 31(4):1292–1302, 2022.